

SALTO GESTAO

Sistema de Gestao v3.0

Codigo Fonte Completo

82 arquivos

Backend: Python (FastAPI + SQLAlchemy + SQLite/PostgreSQL)

Frontend: Next.js (React + TypeScript)

Desktop: Flet (Python)

Autor: Corumba Sistemas

Data: 27/05/2026

INDICE

- [auth.py](#) (10698B)
- [backendPy/alembic/env.py](#) (1813B)
- [backendPy/database.py](#) (3321B)
- [backendPy/dev.py](#) (4615B)
- [backendPy/historico.py](#) (5939B)
- [backendPy/licenca.py](#) (1888B)
- [backendPy/main.py](#) (3861B)
- [backendPy/models.py](#) (8870B)
- [backendPy/models_core.py](#) (6591B)
- [backendPy/models_historico.py](#) (2820B)
- [backendPy/requirements.txt](#) (251B)
- [backendPy/routers/__init__.py](#) (18B)
- [backendPy/routers/admin.py](#) (11847B)
- [backendPy/routers/admin_auth.py](#) (3432B)
- [backendPy/routers/auth.py](#) (7862B)
- [backendPy/routers/caixa.py](#) (9463B)
- [backendPy/routers/clientes.py](#) (6964B)
- [backendPy/routers/dashboard.py](#) (3916B)
- [backendPy/routers/fornecedores.py](#) (3528B)
- [backendPy/routers/historico.py](#) (7470B)
- [backendPy/routers/login_operacional.py](#) (3514B)
- [backendPy/routers/painel.py](#) (4308B)
- [backendPy/routers/produtos.py](#) (6678B)
- [backendPy/routers/usuarios.py](#) (4323B)
- [backendPy/routers/vendas.py](#) (15595B)
- [backendPy/schemas.py](#) (8174B)
- [backendPy/security.py](#) (4947B)
- [backendPy/seed_admin.py](#) (1668B)
- [backendPy/seed_clientes.py](#) (4331B)
- [backendPy/seed_inicial.py](#) (5965B)
- [backendPy/seed_produtos.py](#) (2414B)
- [backendPy/seed_test.py](#) (31137B)
- [backendPy/seed_test_local.py](#) (12365B)
- [backendPy/templates/painel.html](#) (5520B)
- [clientes.py](#) (6977B)
- [dashboard.py](#) (3916B)
- [desktop/README.md](#) (988B)
- [desktop/design_system.py](#) (24126B)
- [desktop/main.py](#) (30996B)
- [desktop/main_flet.py](#) (36016B)
- [desktop/main_v2.py](#) (36754B)
- [desktop/requirements.txt](#) (30B)
- [desktop/run.sh](#) (888B)
- [desktop/testar_100_clientes.py](#) (13639B)
- [desktop/utils/__init__.py](#) (38B)
- [desktop/utils/api_client.py](#) (9508B)
- [desktop/utils/config.py](#) (3624B)
- [documentos/README.md](#) (727B)
- [frontend-web/AGENTS.md](#) (327B)
- [frontend-web/app/admin/clientes/criar/page.tsx](#) (9929B)
- [frontend-web/app/admin/clientes/listar/page.tsx](#) (8056B)
- [frontend-web/app/admin/dashboard/page.tsx](#) (6450B)
- [frontend-web/app/admin/login/page.tsx](#) (3884B)
- [frontend-web/app/api/admin/auth/login/route.ts](#) (738B)
- [frontend-web/app/api/admin/clientes/criar/route.ts](#) (736B)
- [frontend-web/app/api/admin/clientes/route.ts](#) (1479B)
- [frontend-web/app/api/admin/dashboard/route.ts](#) (877B)
- [frontend-web/app/api/admin/produtos/route.ts](#) (510B)
- [frontend-web/app/api/login/route.ts](#) (961B)
- [frontend-web/app/api/painel/produtos/route.ts](#) (907B)
- [frontend-web/app/api/seed/route.ts](#) (1535B)
- [frontend-web/app/api/usuarios/route.ts](#) (2770B)
- [frontend-web/app/dashboard/page.tsx](#) (12753B)
- [frontend-web/app/dashboard/salto-gestao/metricas/route.ts](#) (1782B)
- [frontend-web/app/dashboard/salto-gestao/page.tsx](#) (20378B)
- [frontend-web/app/globals.css](#) (738B)
- [frontend-web/app/layout.tsx](#) (719B)
- [frontend-web/app/login/page.tsx](#) (3782B)
- [frontend-web/app/page.tsx](#) (256B)
- [frontend-web/next-env.d.ts](#) (251B)
- [frontend-web/next.config.ts](#) (198B)
- [frontend-web/package.json](#) (710B)
- [frontend-web/tsconfig.json](#) (656B)
- [iniciar.sh](#) (1112B)
- [licenca.py](#) (1888B)
- [produtos.py](#) (6678B)
- [rodar-backend.bat](#) (499B)
- [sites/README.md](#) (761B)
- [testar_100_clientes_api.py](#) (13971B)
- [update.sh](#) (1542B)
- [web/app.py](#) (33032B)
- [web/index.html](#) (25954B)

```

1 """
2 routers/auth.py - Autenticação por licença + login de colaboradores
3
4 FLUXO ADMIN (primeiro acesso / reativacao):
5 1. Admin insere a CHAVE DE LICENCA (ex: CORUMBA-A1B2-C3D4-E5F6)
6 2. Sistema valida a chave, cria/ativa a licença
7 3. Licença ATIVA = acesso total, sistema abre direto sem pedir senha
8 4. Licença EXPIRADA/SUSPENSA = pede nova chave para reativar
9
10 FLUXO COLABORADOR (dia a dia):
11 1. Colaborador insere CODIGO + SENHA
12 2. Sistema verifica se a licença da empresa dele está ATIVA
13 3. Se ATIVA: login normal, acesso aos dados da empresa dele
14 4. Se INATIVA: acesso bloqueado, pede admin reativar com nova chave
15
16 ISOLAMENTO MULTI-TENANT:
17 - Cada CNPJ tem sua própria licença
18 - Usuários de empresas diferentes podem ter mesmos códigos (1001, 1002, etc.)
19 - Dados de um CNPJ NUNCA se misturam com outro
20 """
21 from datetime import datetime
22 from fastapi import APIRouter, Depends, HTTPException
23 from sqlalchemy.orm import Session
24 from pydantic import BaseModel
25
26 from database import get_db_core
27 from models_core import Usuario, Empresa, Licenca
28 from schemas import LoginResponse
29 from security import hash_senha, verificar_senha, criar_token
30 from licenca import validar_formato_chave, verificar_licenca_ativa, licenca_expirou
31
32 router = APIRouter(prefix="/auth", tags=["Autenticacao"])
33
34 # ===== SCHEMAS =====
35
36 class LoginAdminRequest(BaseModel):
37     """Login do admin com chave de licença (sem senha)."""
38     chave_licenca: str # Ex: CORUMBA-A1B2-C3D4-E5F6
39
40
41 class LoginColaboradorRequest(BaseModel):
42     """Login do colaborador com código + senha."""
43     codigo: str # Código do colaborador (ex: 1001, V01)
44     senha: str
45
46
47 class AtivarLicencaRequest(BaseModel):
48     """Ativa uma licença existente com a chave correta."""
49     chave_licenca: str
50
51
52 class CriarLicencaRequest(BaseModel):
53     """Cria uma nova licença (uso interno/admin master)."""
54     empresa_documento: str
55     chave_licenca: str = None # Se None, gera automaticamente
56     plano: str = "BÁSICO"
57     dias_validade: int = None # None = sem expiração
58
59
60 # ===== ENDPOINTS =====
61
62 @router.post("/admin/login")
63 def login_admin(req: LoginAdminRequest, db: Session = Depends(get_db_core)):
64     """
65     Login do administrador via CHAVE DE LICENÇA.
66     Se a licença está ATIVA, abre direto sem pedir senha.
67     Se EXPIRADA/SUSPENSA, retorna erro com mensagem para reativar.
68     """
69     # Validar formato da chave
70     if not validar_formato_chave(req.chave_licenca):
71         raise HTTPException(status_code=400, detail="Formato de chave inválido. Use: CORUMBA-XXXX-XXXX-XXXX")
72
73     # Buscar licença pela chave
74     licenca = db.query(Licenca).filter(Licenca.chave == req.chave_licenca).first()
75     if not licenca:
76         raise HTTPException(status_code=404, detail="Chave de licença não encontrada.")
77
78     # Verificar se expirou
79     if licenca_expirou(licenca.data_expiracao):
80         licenca.status = "EXPIRADA"
81         db.commit()
82         raise HTTPException(status_code=403, detail="Licença expirada. Insira uma nova chave para reativar.")
83
84     # Verificar status
85     if not verificar_licenca_ativa(licenca.status):
86         raise HTTPException(status_code=403, detail=f"Licença {licenca.status}. Insira uma nova chave para reativar.")
87
88     # Buscar o admin da empresa
89     admin = db.query(Usuario).filter(
90         Usuario.empresa_documento == licenca.empresa_documento,
91         Usuario.papel == "ADMIN",
92         Usuario.ativo == True
93     ).first()
94
95     if not admin:
96         raise HTTPException(status_code=404, detail="Administrador não encontrado para esta licença.")
97
98     # Atualizar verificação
99     licenca.data_ultima_verificacao = datetime.utcnow()
100     admin.ultimo_acesso = datetime.utcnow()
101     db.commit()
102
103     # Gerar token JWT
104     token = criar_token({
105         "sub": admin.username,
106         "empresa_documento": admin.empresa_documento,
107         "papel": admin.papel,
108         "usuario_id": admin.id,
109         "nome": admin.nome,
110         "licenca_ativa": True
111     })
112
113     return {
114         "access_token": token,
115         "token_type": "bearer",
116         "nome": admin.nome,
117         "username": admin.username,
118         "papel": admin.papel,
119         "empresa_documento": admin.empresa_documento,
120         "licenca_status": licenca.status,
121         "plano": licenca.plano,
122         "message": "Licença validada com sucesso. Acesso liberado."
123     }
124
125
126 @router.post("/colaborador/login")
127 def login_colaborador(req: LoginColaboradorRequest, db: Session = Depends(get_db_core)):
128     """
129     Login do colaborador com CODIGO + SENHA.
130     Verifica se a licença da empresa dele está ATIVA antes de permitir acesso.
131     """
132     # Buscar colaborador por código (em qualquer empresa)
133     user = db.query(Usuario).filter(

```

```

135     Usuario.codigo == req.codigo,
136     Usuario.ativo == True
137 ).first()
138
139 if not user:
140     raise HTTPException(status_code=401, detail="Codigo de colaborador nao encontrado.")
141
142 # Verificar senha
143 if not verificar_senha(req.senha, user.senha_hash):
144     raise HTTPException(status_code=401, detail="Senha incorreta.")
145
146 # Verificar se a licenca da empresa esta ativa
147 licenca = db.query(Licenca).filter(
148     Licenca.empresa_documento == user.empresa_documento
149 ).first()
150
151 if not licenca:
152     raise HTTPException(status_code=403, detail="Nenhuma licenca encontrada para esta empresa. Contate o administrador.")
153
154 if licenca.expirou(licenca.data_expiracao):
155     licenca.status = "EXPIRADA"
156     db.commit()
157     raise HTTPException(status_code=403, detail="Licenca da empresa expirada. Contate o administrador para reativar.")
158
159 if not verificar_licenca_ativa(licenca.status):
160     raise HTTPException(status_code=403, detail=f"Licenca da empresa {licenca.status}. Contate o administrador.")
161
162 # Atualizar acessos
163 licenca.data_ultima_verificacao = datetime.utcnow()
164 user.ultimo_acesso = datetime.utcnow()
165 db.commit()
166
167 # Gerar token JWT
168 token = criar_token({
169     "sub": user.username,
170     "empresa_documento": user.empresa_documento,
171     "papel": user.papel,
172     "usuario_id": user.id,
173     "nome": user.nome,
174     "licenca_ativa": True
175 })
176
177 return {
178     "access_token": token,
179     "token_type": "bearer",
180     "nome": user.nome,
181     "username": user.username,
182     "papel": user.papel,
183     "empresa_documento": user.empresa_documento,
184     "licenca_status": licenca.status,
185     "plano": licenca.plano
186 }
187
188 @router.post("/licenca/ativar")
189 def ativar_licenca(req: AtivarLicencaRequest, db: Session = Depends(get_db_core)):
190     """
191     Ativa ou reativa uma licenca com a chave.
192     Usado quando a licenca expirou ou foi suspensa.
193     """
194     if not validar_formato_chave(req.chave_licenca):
195         raise HTTPException(status_code=400, detail="Formato de chave invalido.")
196
197     licenca = db.query(Licenca).filter(Licenca.chave == req.chave_licenca).first()
198     if not licenca:
199         raise HTTPException(status_code=404, detail="Chave de licenca nao encontrada.")
200
201     # Reativar
202     licenca.status = "ATIVA"
203     licenca.data_ativacao = datetime.utcnow()
204     licenca.data_ultima_verificacao = datetime.utcnow()
205     db.commit()
206
207     return {
208         "status": "Licenca ativada com sucesso",
209         "empresa_documento": licenca.empresa_documento,
210         "plano": licenca.plano,
211         "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else "Sem expiracao"
212     }
213
214
215
216 @router.post("/licenca/criar")
217 def criar_licenca(req: CriarLicencaRequest, db: Session = Depends(get_db_core)):
218     """
219     Cria uma nova licenca para uma empresa.
220     Se nao informar chave, gera automaticamente.
221     """
222     # Verificar se empresa existe
223     empresa = db.query(Empresa).filter(Empresa.documento == req.empresa_documento).first()
224     if not empresa:
225         raise HTTPException(status_code=404, detail="Empresa nao encontrada.")
226
227     # Gerar chave se nao informada
228     chave = req.chave_licenca
229     if not chave:
230         from licenca import gerar_chave_licenca
231         chave = gerar_chave_licenca()
232
233     # Verificar se chave ja existe
234     existing = db.query(Licenca).filter(Licenca.chave == chave).first()
235     if existing:
236         raise HTTPException(status_code=400, detail="Chave de licenca ja existe.")
237
238     # Calcular expiracao
239     data_expiracao = None
240     if req.dias_validade:
241         data_expiracao = datetime.utcnow() + timedelta(days=req.dias_validade)
242
243     licenca = Licenca(
244         empresa_documento=req.empresa_documento,
245         chave=chave,
246         status="ATIVA",
247         plano=req.plano,
248         data_expiracao=data_expiracao,
249         ativo=True
250     )
251     db.add(licenca)
252     db.commit()
253     db.refresh(licenca)
254
255     return {
256         "status": "Licenca criada",
257         "chave": licenca.chave,
258         "empresa": req.empresa_documento,
259         "plano": licenca.plano,
260         "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else "Sem expiracao"
261     }
262
263
264 @router.get("/licenca/verificar/{empresa_documento}")
265 def verificar_licenca(empresa_documento: str, db: Session = Depends(get_db_core)):
266     """Verifica o status da licenca de uma empresa."""
267     licenca = db.query(Licenca).filter(
268         Licenca.empresa_documento == empresa_documento
269     ).first()
270
271     if not licenca:
272         return {"status": "SEM_LICENCA", "ativa": False}

```

```

273
274 # Auto-expirar se passou da data
275 if licenca_expirou(licenca.data_expiracao) and licenca.status == "ATIVA":
276     licenca.status = "EXPIRADA"
277     db.commit()
278
279 return {
280     "status": licenca.status,
281     "ativa": verificar_licenca_ativa(licenca.status),
282     "plano": licenca.plano,
283     "data_ativacao": str(licenca.data_ativacao),
284     "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else None,
285     "max_usuarios": licenca.max_usuarios,
286     "max_produtos": licenca.max_produtos
287 }
288
289
290 @router.get("/licenca/status")
291 def status_licenca(chave: str, db: Session = Depends(get_db_core)):
292     """Consulta status de uma licenca pela chave."""
293     licenca = db.query(Licenca).filter(Licenca.chave == chave).first()
294     if not licenca:
295         raise HTTPException(status_code=404, detail="Licenca nao encontrada.")
296
297     return {
298         "chave": licenca.chave,
299         "empresa_documento": licenca.empresa_documento,
300         "status": licenca.status,
301         "ativa": verificar_licenca_ativa(licenca.status),
302         "plano": licenca.plano,
303         "data_ativacao": str(licenca.data_ativacao),
304         "data_expiracao": str(licenca.data_expiracao) if licenca.data_expiracao else None,
305         "ultima_verificacao": str(licenca.data_ultima_verificacao) if licenca.data_ultima_verificacao else None
306     }
307

```

```

1 from logging.config import fileConfig
2
3 from sqlalchemy import engine_from_config
4 from sqlalchemy import pool
5
6 from alembic import context
7
8 # this is the Alembic Config object, which provides
9 # access to the values within the .ini file in use.
10 config = context.config
11
12 # Interpret the config file for Python logging.
13 # This line sets up loggers basically.
14 if config.config_file_name is not None:
15     fileConfig(config.config_file_name)
16
17 # add your model's MetaData object here
18 # for 'autogenerate' support
19 from database import BaseSalto
20 import models
21 # º Corrigido: adicionado o "t" inicial
22 target_metadata = BaseSalto.metadata
23
24 # other values from the config, defined by the needs of env.py,
25 # can be acquired:
26 # my_important_option = config.get_main_option("my_important_option")
27 # ... etc.
28
29 def run_migrations_offline() -> None:
30     """Run migrations in 'offline' mode."""
31     # º Corrigido: Ele apenas procura a chave, não recebe a URL direta aqui
32     url = config.get_main_option("sqlalchemy.url")
33     context.configure(
34         url=url,
35         target_metadata=target_metadata,
36         literal_binds=True,
37         dialect_opts={"paramstyle": "named"},
38     )
39
40     with context.begin_transaction():
41         context.run_migrations()
42
43 def run_migrations_online() -> None:
44     """Run migrations in 'online' mode."""
45     connectable = engine_from_config(
46         config.get_section(config.config_ini_section, {}),
47         prefix="sqlalchemy.",
48         poolclass=pool.NullPool,
49     )
50
51     with connectable.connect() as connection:
52         context.configure(
53             connection=connection, target_metadata=target_metadata
54         )
55
56     with context.begin_transaction():
57         context.run_migrations()
58
59 if context.is_offline_mode():
60     run_migrations_offline()
61 else:
62     run_migrations_online()

```

```

1  """
2  database.py - Configuração do banco de dados e sessões SQLAlchemy
3
4  Suporta 3 ambientes automaticamente:
5  - development (sua máquina) → SQLite local
6  - production (VPS dedicada) → PostgreSQL centralizado
7  - testing (CI/automação) → SQLite em memória
8  """
9  import os
10 import sys
11 from sqlalchemy import create_engine, Column, Integer, String, event
12 from sqlalchemy.orm import sessionmaker, declarative_base
13
14
15 def _build_engine(url: str):
16     """Cria engine compatível com SQLite e PostgreSQL."""
17     if url.startswith("sqlite"):
18         eng = create_engine(url, connect_args={"check_same_thread": False})
19         # Otimizações SQLite para PDV
20         @event.listens_for(eng, "connect")
21         def _sqlite_pragma(dbapi_conn, connection_record):
22             dbapi_conn.execute("PRAGMA journal_mode=WAL")
23             dbapi_conn.execute("PRAGMA foreign_keys=ON")
24             dbapi_conn.execute("PRAGMA synchronous=NORMAL")
25             dbapi_conn.execute("PRAGMA cache_size=10000")
26         return eng
27     else:
28         return create_engine(
29             url,
30             pool_pre_ping=True,
31             pool_size=20,
32             max_overflow=30,
33             pool_timeout=30,
34             pool_recycle=3600,
35             echo=False,
36         )
37
38
39 # =====
40 # AMBIENTE E URL DO BANCO
41 # =====
42 APP_ENV = os.getenv("APP_ENV", "development")
43
44 if "DATABASE_URL" in os.environ:
45     DB_URL = os.environ["DATABASE_URL"]
46 elif APP_ENV == "production":
47     DB_URL = os.getenv("DATABASE_URL", "postgresql+psycopg2://corumba:corumba2025@localhost:5432/corumba")
48 elif APP_ENV == "testing":
49     DB_URL = "sqlite:///file::memory:?cache=shared&mode=memory"
50 else: # development
51     DB_URL = "sqlite:///./salto_principal.db"
52
53 print(f"📦 [database] Ambiente: {APP_ENV} | Banco: {DB_URL.split(':')[0]}")
54
55 engine = _build_engine(DB_URL)
56 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
57
58 # =====
59 # BASES DECLARATIVAS (metadados separados)
60 # =====
61 # BaseCore: models do sistema Corumbá (Cliente, Usuario, AdminCorumba, Produto catálogo, etc.)
62 # BaseSalto: models operacionais do SaltoGestão (clientes, produtos, vendas, etc.)
63 # Em dev SQLite ambas usam o mesmo engine/arquivo; em produção PostgreSQL usariam bancos separados
64 BaseCore = declarative_base()
65 BaseSalto = declarative_base()
66
67
68 # =====
69 # MIXIN MULTI-TENANT (para models operacionais)
70 # =====
71 class TenantMixin:
72     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
73     empresa_documento = Column(String(20), index=True, nullable=False)
74
75
76 # =====
77 # DEPENDENCIES PARA FASTAPI
78 # =====
79 def get_db():
80     """Sessão do banco (core)."""
81     db = SessionLocal()
82     try:
83         yield db
84     finally:
85         db.close()
86
87
88 def get_db_salto():
89     """Sessão do banco operacional SaltoGestão."""
90     db = SessionLocal()
91     try:
92         yield db
93     finally:
94         db.close()
95
96
97 def get_db_core():
98     """Sessão do banco core (Corumbá)."""
99     db = SessionLocal()
100    try:
101        yield db
102    finally:
103        db.close()
104

```

```

1 #!/usr/bin/env python3
2 """
3 dev.py - Script para iniciar o SaltoGestão
4 Uso:
5   Desenvolvimento: python dev.py
6   Produção:       APP_ENV=production python dev.py
7 """
8 import os
9 import sys
10
11 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
12
13 # Detectar ambiente
14 APP_ENV = os.getenv("APP_ENV", "development")
15
16 # Carregar .env apropriado
17 env_dir = os.path.dirname(os.path.abspath(__file__))
18 env_files = [
19     os.path.join(env_dir, f".env.{APP_ENV}"), # .env.development ou .env.production
20     os.path.join(env_dir, ".env"),           # .env.local (override)
21 ]
22 for env_path in env_files:
23     if os.path.exists(env_path):
24         print(f"📁 Carregando: {env_path}")
25         with open(env_path) as f:
26             for line in f:
27                 line = line.strip()
28                 if line and not line.startswith("#") and "=" in line:
29                     key, _, value = line.partition("=")
30                     key = key.strip()
31                     value = value.strip().strip('\'').strip('"')
32                     if key not in os.environ:
33                         os.environ[key] = value
34
35 # Defaults para desenvolvimento
36 os.environ.setdefault("DATABASE_URL", "sqlite:///./salto_principal.db")
37 os.environ.setdefault("SECRET_KEY", "dev-local-key-change-in-production")
38 os.environ.setdefault("CORS_ORIGINS", "http://localhost:3000,http://localhost:5173")
39
40 # --- Instalar driver PostgreSQL automaticamente se necessário ---
41 db_url = os.environ["DATABASE_URL"]
42 if db_url.startswith("postgresql"):
43     try:
44         import psycopg2 # noqa: F401
45     except ImportError:
46         print("🔧 Instalando psycopg2-binary (driver PostgreSQL)...")
47         import subprocess
48         subprocess.check_call([sys.executable, "-m", "pip", "install", "psycopg2-binary", "-q"])
49         print("✓ psycopg2-binary instalado!")
50
51 print("=" * 50)
52 print(f"🏢 Corumbá Sistemas API v3.0 - {APP_ENV.upper()}")
53 print("=" * 50)
54 print(f"🗄️ DB: {os.environ['DATABASE_URL'].split(':///')[0]}")
55 print(f"🌐 CORS: {os.environ['CORS_ORIGINS']}")
56 print("=" * 50)
57
58 # =====
59 # SEED AUTOMÁTICO (na primeira execução)
60 # =====
61 from database import SessionLocal, engine, BaseCore, BaseSalto
62 import models_core # Registra models do banco core (Cliente, Usuario, AdminCorumba, etc.)
63 import models      # Registra models operacionais (Cliente, Produto, Venda, etc.)
64
65 BaseCore.metadata.create_all(bind=engine)
66 BaseSalto.metadata.create_all(bind=engine)
67 print("✓ Tabelas criadas/atualizadas (core + salto)")
68
69 _db = SessionLocal()
70
71 # 1) Seed Admin (se não existir)
72 from models_core import AdminCorumba
73 from security import hash_senha
74 if not _db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumbasistemas.com.br").first():
75     _db.add(AdminCorumba(nome="Admin Corumbá", email="admin@corumbasistemas.com.br",
76                           senha_hash=hash_senha("corumba2026"), papel="admin", ativo=True))
77     print("✓ Admin criado: admin@corumbasistemas.com.br / corumba2026")
78 if not _db.query(AdminCorumba).filter(AdminCorumba.email == "vendedor@corumbasistemas.com.br").first():
79     _db.add(AdminCorumba(nome="Vendedor Corumbá", email="vendedor@corumbasistemas.com.br",
80                           senha_hash=hash_senha("vendedor123"), papel="vendedor", ativo=True))
81     print("✓ Vendedor criado: vendedor@corumbasistemas.com.br / vendedor123")
82
83 # 2) Seed Produtos (se não existir)
84 from models_core import Produto
85 if _db.query(Produto).count() == 0:
86     _db.add_all([
87         Produto(nome="SaltoGestão", descricao="Sistema de gestão empresarial completo",
88                 preco_mensal=149.90, preco_anual=1499.90, ativo=True, ordem=1, icone="🏢"),
89         Produto(nome="MesaPronta", descricao="Sistema de gestão para restaurantes e lanchonetes",
90                 preco_mensal=249.90, preco_anual=2499.90, ativo=True, ordem=2, icone="🍽️"),
91         Produto(nome="ConnectFood", descricao="Integração com iFood e 99Food",
92                 preco_mensal=49.90, preco_anual=499.90, ativo=True, ordem=3, icone="🚗"),
93     ])
94     print("✓ 3 produtos criados (SaltoGestão, MesaPronta, ConnectFood)")
95
96 _db.commit()
97 _db.close()
98 print("✓ Seed automático concluído")
99
100 from main import app
101 import uvicorn
102
103 if __name__ == "__main__":
104     print(f"🚀 Iniciando FastAPI em http://localhost:8080")
105     print(f"📖 Documentação: http://localhost:8080/docs")
106     print(f"🛑 Para parar: CTRL+C\n")
107     uvicorn.run(
108         "main:app",
109         host="0.0.0.0",
110         port=8080,
111         reload=True,
112         reload_dirs=[os.path.dirname(os.path.abspath(__file__))],
113     )
114

```



```

1 """
2 historico.py - Módulo de Histórico de Vendas
3
4 Registra TODA operação de venda para auditoria e rastreabilidade.
5 Chamado automaticamente pelo router de vendas em cada operação.
6 """
7 import json
8 import datetime
9 from typing import Optional, List, Dict, Any
10 from sqlalchemy.orm import Session
11
12 from database import get_db_salto
13 from models import HistoricoVenda
14
15
16 def gerar_numero_pedido(empresa_documento: str) -> str:
17     """Gera número de pedido único: PED-YYYYMMDD-XXXXX."""
18     agora = datetime.datetime.utcnow()
19     prefixo = f"PED-{agora.strftime('%Y%m%d')}"
20     # Os últimos 5 chars do CNPJ + timestamp para unicidade
21     sufixo = f"{empresa_documento[-5:]}{agora.strftime('%H%M%S%f')[:8]}"
22     return f"{prefixo}-{sufixo}"
23
24
25 def registrar_venda(
26     db: Session,
27     empresa_documento: str,
28     usuario_id: Optional[int],
29     usuario_nome: Optional[str],
30     usuario_papel: Optional[str],
31     venda_id: Optional[int],
32     itens: List[Dict[str, Any]],
33     pagamentos: List[Dict[str, Any]],
34     cliente_id: Optional[int] = None,
35     cliente_nome: Optional[str] = None,
36     valor_total: float = 0,
37     valor_desconto: float = 0,
38     valor_recebido: float = 0,
39     troco: float = 0,
40     forma_pagamento: str = "",
41     numero_pedido: Optional[str] = None,
42     origem: str = "PDV",
43     observacao: Optional[str] = None,
44     ip_origem: Optional[str] = None,
45 ) -> HistoricoVenda:
46     """Registra uma venda no histórico."""
47     if not numero_pedido:
48         numero_pedido = gerar_numero_pedido(empresa_documento)
49
50     # Calcular totais se não informados
51     if valor_total == 0 and itens:
52         valor_total = sum(
53             float(item.get('preco_unitario', 0)) * float(item.get('quantidade', 1))
54             for item in itens
55         )
56
57     historico = HistoricoVenda(
58         empresa_documento=empresa_documento,
59         venda_id=venda_id,
60         numero_pedido=numero_pedido,
61         usuario_id=usuario_id,
62         usuario_nome=usuario_nome,
63         usuario_papel=usuario_papel,
64         cliente_id=cliente_id,
65         cliente_nome=cliente_nome,
66         valor_total=valor_total,
67         valor_desconto=valor_desconto,
68         valor_recebido=valor_recebido,
69         troco=troco,
70         forma_pagamento=forma_pagamento,
71         itens_json=json.dumps(itens, default=str),
72         status="CONCLUIDA",
73         tipo_operacao="VENDA",
74         origem=origem,
75         observacao=observacao,
76         ip_origem=ip_origem,
77         data_criacao=datetime.datetime.utcnow(),
78     )
79     db.add(historico)
80     db.flush()
81     return historico
82
83
84 def registrar_cancelamento(
85     db: Session,
86     empresa_documento: str,
87     venda_id: int,
88     usuario_id: Optional[int],
89     usuario_nome: Optional[str],
90     motivo: Optional[str] = None,
91 ) -> Optional[HistoricoVenda]:
92     """Registra o cancelamento de uma venda no histórico."""
93     # Buscar o histórico original
94     original = db.query(HistoricoVenda).filter(
95         HistoricoVenda.empresa_documento == empresa_documento,
96         HistoricoVenda.venda_id == venda_id,
97         HistoricoVenda.status == "CONCLUIDA"
98     ).order_by(HistoricoVenda.data_criacao.desc()).first()
99
100     if original:
101         # Atualizar status do original
102         original.status = "CANCELADA"
103         original.data_cancelamento = datetime.datetime.utcnow()
104         original.motivo_cancelamento = motivo
105
106         # Registrar log de cancelamento
107         cancelamento = HistoricoVenda(
108             empresa_documento=empresa_documento,
109             venda_id=venda_id,
110             numero_pedido=original.numero_pedido,
111             usuario_id=usuario_id,
112             usuario_nome=usuario_nome,
113             usuario_papel=original.usuario_papel,
114             cliente_id=original.cliente_id,
115             cliente_nome=original.cliente_nome,
116             valor_total=original.valor_total,
117             valor_desconto=original.valor_desconto,
118             forma_pagamento=original.forma_pagamento,
119             itens_json=original.itens_json,
120             status="CANCELADA",
121             tipo_operacao="CANCELAMENTO",
122             origem=original.origem,
123             observacao=f"Cancelamento: {motivo or 'Sem motivo informado'}",
124             data_cancelamento=datetime.datetime.utcnow(),
125             motivo_cancelamento=motivo,
126         )
127         db.add(cancelamento)
128         db.flush()
129         return cancelamento
130     return None
131
132
133 def registrar_erro_venda(
134     db: Session,

```

```

135     empresa_documento: str,
136     usuario_id: Optional[int],
137     usuario_nome: Optional[str],
138     erro: str,
139     dados_tentativa: Optional[Dict] = None,
140 ) -> HistoricoVenda:
141     """Registra uma tentativa de venda que falhou."""
142     historico = HistoricoVenda(
143         empresa_documento=empresa_documento,
144         numero_pedido=gerar_numero_pedido(empresa_documento),
145         usuario_id=usuario_id,
146         usuario_nome=usuario_nome,
147         erro_detalhes=erro,
148         status="ERRO",
149         tipo_operacao="ERRO",
150         metadados_json=json.dumps(dados_tentativa, default=str) if dados_tentativa else None,
151         data_criacao=datetime.datetime.utcnow(),
152     )
153     db.add(historico)
154     db.flush()
155     return historico
156
157
158 def registrar_edicao_venda(
159     db: Session,
160     empresa_documento: str,
161     venda_id: int,
162     usuario_id: int,
163     usuario_nome: str,
164     campo_alterado: str,
165     valor_anterior: str,
166     valor_novo: str,
167 ) -> HistoricoVenda:
168     """Registra uma edição em uma venda existente."""
169     historico = HistoricoVenda(
170         empresa_documento=empresa_documento,
171         venda_id=venda_id,
172         numero_pedido=f"EDIT-{gerar_numero_pedido(empresa_documento)}",
173         usuario_id=usuario_id,
174         usuario_nome=usuario_nome,
175         status="CONCLUIDA",
176         tipo_operacao="EDICAO",
177         observacao=f"Campo '{campo_alterado}' alterado de '{valor_anterior}' para '{valor_novo}'",
178         data_alteracao=datetime.datetime.utcnow(),
179         versao=2,
180     )
181     db.add(historico)
182     db.flush()
183     return historico
184

```

```

1 """
2 licenca.py - Gerenciamento de licencas do Corumbá Sistemas
3
4 Fluxo de licenciamento:
5 1. Admin recebe chave de licenca (ex: CORUMBA-A1B2-C3D4-E5F6)
6 2. Na primeira execucao, sistema pede a chave
7 3. Chave validada -> licenca ATIVA -> acesso total sem senha
8 4. Em cada login, verifica se licenca ainda esta ativa
9 5. Se EXPIRADA/SUSPENSA -> pede nova chave para reativar
10 6. Isolamento: cada CNPJ tem sua propria licenca e seus proprios dados
11 """
12 import hashlib
13 import secrets
14 import string
15 from datetime import datetime, timedelta
16 from typing import Optional
17
18
19 def gerar_chave_licenca() -> str:
20     """Gera uma chave de licenca unica no formato CORUMBA-XXXX-XXXX-XXXX."""
21     partes = []
22     for _ in range(3):
23         parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
24         partes.append(parte)
25     return f"CORUMBA-{'-'.join(partes)}"
26
27
28 def gerar_hash_chave(chave: str) -> str:
29     """Gera hash da chave para armazenamento seguro (nao guarda a chave pura)."""
30     return hashlib.sha256(chave.encode()).hexdigest()
31
32
33 def validar_formato_chave(chave: str) -> bool:
34     """Valida se a chave esta no formato correto: CORUMBA-XXXX-XXXX-XXXX."""
35     if not chave or not chave.startswith("CORUMBA-"):
36         return False
37     partes = chave.split("-")
38     if len(partes) != 4:
39         return False
40     for parte in partes[1:]:
41         if len(parte) != 4 or not all(c in string.ascii_uppercase + string.digits for c in parte):
42             return False
43     return True
44
45
46 def verificar_licenca_ativa(status: str) -> bool:
47     """Verifica se a licenca permite acesso."""
48     return status == "ATIVA"
49
50
51 def licenca_expirou(data_expiracao: Optional[datetime]) -> bool:
52     """Verifica se a licenca expirou."""
53     if data_expiracao is None:
54         return False # Sem expiracao = valida para sempre
55     return datetime.utcnow() > data_expiracao
56

```

```

1 """
2 main.py - Ponto de entrada da API FastAPI
3 """
4 from fastapi import FastAPI, Request
5 from fastapi.middleware.cors import CORSMiddleware
6 from fastapi.staticfiles import StaticFiles
7 from fastapi.responses import HTMLResponse
8 import os
9
10 # Importar modelos para garantir que as tabelas sejam criadas
11 import models_core
12
13 # Importar routers
14 from routers import auth, painel, admin, admin_auth, login_operacional
15 from routers import clientes, produtos, vendas, caixa, dashboard, fornecedores, historico, usuarios
16
17 app = FastAPI(
18     title="Corumbá Sistemas API",
19     description="API REST para gestão de clientes e assinaturas",
20     version="3.0.0"
21 )
22
23 # Arquivos estáticos (pasta prints para testes de conexão)
24 PRINTS_DIR = os.path.join(os.path.dirname(__file__), "..", "prints")
25 os.makedirs(PRINTS_DIR, exist_ok=True)
26 app.mount("/prints", StaticFiles(directory=PRINTS_DIR), name="prints")
27
28 # CORS
29 CORS_ORIGINS = os.getenv("CORS_ORIGINS", "http://localhost:3000,http://localhost:5173")
30 app.add_middleware(
31     CORSMiddleware,
32     allow_origins=CORS_ORIGINS.split(","),
33     allow_credentials=True,
34     allow_methods=["*"],
35     allow_headers=["*"],
36 )
37
38 # Registrar routers
39 # Login operacional (colaboradores SaltoGestão) - registrado antes do auth de cliente
40 app.include_router(login_operacional.router, prefix="/api")
41 app.include_router(auth.router, prefix="/api")
42 app.include_router(painel.router, prefix="/api")
43 app.include_router(admin.router, prefix="/api")
44 app.include_router(admin_auth.router, prefix="/api")
45 # Routers operacionais (SaltoGestão desktop)
46 app.include_router(clientes.router, prefix="/api")
47 app.include_router(produtos.router, prefix="/api")
48 app.include_router(vendas.router, prefix="/api")
49 app.include_router(caixa.router, prefix="/api")
50 app.include_router(dashboard.router, prefix="/api")
51 app.include_router(fornecedores.router, prefix="/api")
52 app.include_router(historico.router, prefix="/api")
53 app.include_router(usuarios.router, prefix="/api")
54
55
56 @app.get("/")
57 def root():
58     return {"message": "Corumbá Sistemas API v3.0", "status": "operante"}
59
60
61 @app.get("/health")
62 def health():
63     return {"status": "ok"}
64
65
66 # =====
67 # PAINEL WEB (alternativa ao desktop tkinter)
68 # =====
69 import os as _os
70 _WEB_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "web")
71 _web_index = None
72
73 def load_web_index():
74     global _web_index
75     web_html_path = _os.path.join(_WEB_DIR, "index.html")
76     if _os.path.exists(web_html_path):
77         with open(web_html_path) as f:
78             _web_index = f.read()
79     return _web_index
80
81 @app.get("/painel", response_class=HTMLResponse)
82 def painel_web():
83     """Painel web - SaltoGestão (funciona no navegador, sem tkinter)."""
84     html = load_web_index()
85     if html:
86         return HTMLResponse(content=html)
87     return HTMLResponse(content=_PAINEL_HTML, status_code=200)
88
89
90 # HTML inline como fallback
91 from fastapi.responses import HTMLResponse as _HTMLResp
92 _PAINEL_HTML = """<!DOCTYPE html>
93 <html><head><meta charset="UTF-8"><title>SaltoGestão Web</title></head>
94 <body style="font-family:monospace; max-width:800px; margin:40px auto; padding:20px;">
95 <h1> SaltoGestão - Painel Web</h1>
96 <p> O painel web está funcionando! A API está operacional.</p>
97 <h3>Endpoints disponíveis:</h3>
98 <ul>
99 <li><a href="/docs">/docs</a> - Documentação Swagger</li>
100 <li><a href="/api/clientes/">/api/clientes/</a> - Clientes</li>
101 <li><a href="/api/produtos/">/api/produtos/</a> - Produtos</li>
102 <li><a href="/api/vendas/">/api/vendas/</a> - Vendas</li>
103 <li><a href="/api/fornecedores/">/api/fornecedores/</a> - Fornecedores</li>
104 <li><a href="/api/dashboard/">/api/dashboard/</a> - Dashboard</li>
105 </ul>
106 <p style="color:#666;">Para usar o painel completo, acesse via arquivo index.html na pasta web</p>
107 </body></html>"""
108

```

```

1 """
2 models.py - Modelos SQLAlchemy do banco operacional (BaseSalto)
3 Todas as tabelas herdam de TenantMixin para isolamento multi-tenant por CNPJ.
4
5 Modelos do banco core (Usuario, Empresa) estao em models_core.py
6 """
7 import datetime
8 from sqlalchemy import (
9     Column, Integer, String, Numeric, Boolean, ForeignKey,
10     DateTime, Text, Enum as SAEnum
11 )
12 from sqlalchemy.orm import relationship
13 from database import BaseSalto, TenantMixin
14
15
16 # =====
17 # PRODUTOS
18 # =====
19 class Produto(BaseSalto, TenantMixin):
20     __tablename__ = "op_produtos"
21
22     nome = Column(String(255), nullable=False)
23     descricao = Column(Text, nullable=True)
24     sku = Column(String(50), index=True)
25     codigo_barras = Column(String(50), index=True)
26     preco_venda = Column(Numeric(10, 2), nullable=False, default=0)
27     preco_custo = Column(Numeric(10, 2), default=0)
28     estoque_atual = Column(Numeric(10, 3), default=0)
29     estoque_minimo = Column(Numeric(10, 3), default=0)
30     unidade = Column(String(10), default="UN") # UN, KG, LT, CX, etc.
31     ncm = Column(String(8))
32     cfop = Column(String(4))
33     cest = Column(String(7))
34     ativo = Column(Boolean, default=True)
35
36     # Relacoes
37     movimentacoes = relationship("EstoqueMovimentacao", back_populates="produto", cascade="all, delete-orphan")
38     venda_itens = relationship("VendaItem", back_populates="produto")
39
40
41 # =====
42 # CLIENTES
43 # =====
44 class Cliente(BaseSalto, TenantMixin):
45     __tablename__ = "op_clientes"
46
47     nome = Column(String(255), nullable=False)
48     telefone = Column(String(20), index=True)
49     email = Column(String(255), index=True)
50     cpf_cnpj = Column(String(20), index=True)
51     endereco = Column(String(500))
52     cidade = Column(String(100))
53     estado = Column(String(2))
54     cep = Column(String(10))
55     data_nascimento = Column(String(10))
56     pontos_fidelidade = Column(Integer, default=0)
57     ativo = Column(Boolean, default=True)
58
59     # Relacoes
60     vendas = relationship("Venda", back_populates="cliente")
61
62
63 # =====
64 # FORNECEDORES
65 # =====
66 class Fornecedor(BaseSalto, TenantMixin):
67     __tablename__ = "op_fornecedores"
68
69     nome_fantasia = Column(String(255), nullable=False)
70     razao_social = Column(String(255))
71     cnpj_cpf = Column(String(20), index=True)
72     telefone = Column(String(20))
73     email = Column(String(255))
74     endereco = Column(String(500))
75     ativo = Column(Boolean, default=True)
76
77
78 # =====
79 # VENDAS E ITENS
80 # =====
81 class Venda(BaseSalto, TenantMixin):
82     __tablename__ = "op_vendas"
83
84     cliente_id = Column(Integer, ForeignKey("op_clientes.id"), nullable=True)
85     vendedor_id = Column(Integer, nullable=True)
86     vendedor_nome = Column(String(255), nullable=True)
87     operador = Column(String(255), nullable=True)
88     total = Column(Numeric(10, 2), nullable=False, default=0)
89     desconto = Column(Numeric(10, 2), default=0)
90     valor_recebido = Column(Numeric(10, 2), default=0)
91     troco = Column(Numeric(10, 2), default=0)
92     forma_pagamento = Column(String(50))
93     status = Column(String(20), default="CONCLUIDA") # CONCLUIDA, CANCELADA
94     observacao = Column(Text, nullable=True)
95     data_venda = Column(DateTime, default=datetime.datetime.utcnow)
96
97     # Relacoes
98     itens = relationship("VendaItem", back_populates="venda", cascade="all, delete-orphan", lazy="joined")
99     cliente = relationship("Cliente", back_populates="vendas", lazy="joined")
100
101
102 class VendaItem(BaseSalto, TenantMixin):
103     __tablename__ = "op_venda_itens"
104
105     venda_id = Column(Integer, ForeignKey("op_vendas.id"), nullable=False)
106     produto_id = Column(Integer, ForeignKey("op_produtos.id"), nullable=False)
107     produto_nome = Column(String(255)) # snapshot do nome no momento da venda
108     quantidade = Column(Numeric(10, 3), nullable=False)
109     preco_unitario = Column(Numeric(10, 2), nullable=False)
110     subtotal = Column(Numeric(10, 2), nullable=False)
111
112     venda = relationship("Venda", back_populates="itens")
113     produto = relationship("Produto", back_populates="venda_itens", lazy="joined")
114
115
116 # =====
117 # ESTOQUE - MOVIMENTACOES
118 # =====
119 class EstoqueMovimentacao(BaseSalto, TenantMixin):
120     __tablename__ = "op_estoque_movimentacoes"
121
122     produto_id = Column(Integer, ForeignKey("op_produtos.id"), nullable=False)
123     tipo = Column(String(20), nullable=False) # ENTRADA, SAIDA, AJUSTE, INVENTARIO
124     quantidade = Column(Numeric(10, 3), nullable=False)
125     motivo = Column(String(255), nullable=True)
126     usuario = Column(String(255), nullable=True)
127     data_movimentacao = Column(DateTime, default=datetime.datetime.utcnow)
128
129     produto = relationship("Produto", back_populates="movimentacoes")
130
131
132 # =====
133 # CAIXA - SESSOES E TRANSACOES
134 # =====

```

```

135 class CaixaSessao(BaseSalto, TenantMixin):
136     __tablename__ = "op_caixa_sesoes"
137
138     operador = Column(String(255), nullable=False)
139     usuario_id = Column(Integer, nullable=True)
140     status = Column(String(20), default="ABERTO") # ABERTO, FECHADO
141     saldo_inicial = Column(Numeric(10, 2), default=0)
142     saldo_final = Column(Numeric(10, 2), nullable=True)
143     total_vendas = Column(Numeric(10, 2), default=0)
144     total_entradas = Column(Numeric(10, 2), default=0)
145     total_saidas = Column(Numeric(10, 2), default=0)
146     data_abertura = Column(DateTime, default=datetime.datetime.utcnow)
147     data_fechamento = Column(DateTime, nullable=True)
148
149     transacoes = relationship("CaixaTransacao", back_populates="sessao", cascade="all, delete-orphan")
150
151
152 class CaixaTransacao(BaseSalto, TenantMixin):
153     __tablename__ = "op_caixa_transacoes"
154
155     sessao_id = Column(Integer, ForeignKey("op_caixa_sesoes.id"), nullable=False)
156     tipo = Column(String(20), nullable=False) # VENDA, ENTRADA, SAIDA, SANGRIA, DESCONTO, CANCELAMENTO
157     valor = Column(Numeric(10, 2), nullable=False)
158     descricao = Column(String(255), nullable=True)
159     venda_id = Column(Integer, nullable=True)
160     operador = Column(String(255), nullable=True)
161     autorizado_por = Column(String(255), nullable=True) # Para descontos/sangrias
162     data_transacao = Column(DateTime, default=datetime.datetime.utcnow)
163
164     sessao = relationship("CaixaSessao", back_populates="transacoes")
165
166
167 # =====
168 # FLUXO DE CAIXA (resumo diario consolidado)
169 # =====
170 class FluxoCaixa(BaseSalto, TenantMixin):
171     __tablename__ = "op_fluxo_caixa"
172
173     data_referencia = Column(String(10), nullable=False) # YYYY-MM-DD
174     total_vendas = Column(Numeric(10, 2), default=0)
175     total_entradas = Column(Numeric(10, 2), default=0)
176     total_saidas = Column(Numeric(10, 2), default=0)
177     saldo_dia = Column(Numeric(10, 2), default=0)
178     quantidade_vendas = Column(Integer, default=0)
179     ticket_medio = Column(Numeric(10, 2), default=0)
180     atualizado_em = Column(DateTime, default=datetime.datetime.utcnow)
181
182
183 # =====
184 # HISTÓRICO DE VENDAS - Auditoria completa
185 # =====
186 class HistoricoVenda(BaseSalto, TenantMixin):
187     __tablename__ = "op_historico_vendas"
188
189     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
190     venda_id = Column(Integer, nullable=True)
191     numero_pedido = Column(String(50), index=True)
192     usuario_id = Column(Integer, nullable=True)
193     usuario_nome = Column(String(255), nullable=True)
194     usuario_papel = Column(String(30), nullable=True)
195     cliente_id = Column(Integer, ForeignKey("op_clientes.id"), nullable=True)
196     cliente_nome = Column(String(255), nullable=True)
197     valor_total = Column(Numeric(10, 2), default=0)
198     valor_desconto = Column(Numeric(10, 2), default=0)
199     valor_recebido = Column(Numeric(10, 2), default=0)
200     troco = Column(Numeric(10, 2), default=0)
201     forma_pagamento = Column(String(50), nullable=True)
202     itens_json = Column(Text, nullable=True)
203     status = Column(String(20), default="CONCLUIDA")
204     tipo_operacao = Column(String(30), default="VENDA")
205     origem = Column(String(50), default="PDV")
206     observacao = Column(Text, nullable=True)
207     data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
208     data_alteracao = Column(DateTime, nullable=True)
209     data_cancelamento = Column(DateTime, nullable=True)
210     motivo_cancelamento = Column(Text, nullable=True)
211     erro_detalhes = Column(Text, nullable=True)
212     ip_origem = Column(String(50), nullable=True)
213     sessao_id = Column(Integer, nullable=True)
214     versao = Column(Integer, default=1)
215     metadados_json = Column(Text, nullable=True)
216

```

```

1 """
2 models_core.py - Modelos SQLAlchemy do banco CORE
3
4 Arquitetura:
5 - Cliente: pessoa/empresa que compra serviços da Corumbá Sistemas
6 - Assinatura: vincula um cliente aos produtos que ele adquiriu
7 - Produto: catálogo de serviços/produtos oferecidos pela Corumbá (Salto PDV, MesaPronta, etc.)
8 - ChaveAtivacao: chave única usada no download para validar assinatura
9 """
10 import datetime
11 from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, Numeric, ForeignKey, Table
12 from sqlalchemy.orm import relationship
13 from database import BaseCore
14
15
16 # =====
17 # CLIENTE (quem compra da Corumbá Sistemas)
18 # =====
19 class Cliente(BaseCore):
20     __tablename__ = "clientes"
21
22     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
23     nome = Column(String(255), nullable=False)
24     email = Column(String(255), unique=True, index=True, nullable=False)
25     telefone = Column(String(20))
26     cpf_cnpj = Column(String(20), unique=True, index=True, nullable=False)
27     senha_hash = Column(String(255), nullable=False)
28     ativo = Column(Boolean, default=True)
29     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow())
30     ultimo_acesso = Column(DateTime, nullable=True)
31
32     # Relacionamentos
33     assinaturas = relationship("Assinatura", back_populates="cliente", cascade="all, delete-orphan")
34     chaves = relationship("ChaveAtivacao", back_populates="cliente", cascade="all, delete-orphan")
35
36
37 # =====
38 # PRODUTO/SERVIÇO (catálogo da Corumbá Sistemas)
39 # =====
40 class Produto(BaseCore):
41     __tablename__ = "produtos_catalogo"
42
43     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
44     nome = Column(String(255), nullable=False) # Ex: "Salto PDV", "MesaPronta"
45     descricao = Column(Text, nullable=True)
46     preco_mensal = Column(Numeric(10, 2), nullable=False, default=0)
47     preco_anual = Column(Numeric(10, 2), nullable=True)
48     ativo = Column(Boolean, default=True)
49     icone = Column(String(50), nullable=True) # Emoji ou classe CSS
50     ordem = Column(Integer, default=0) # Ordem de exibição no painel
51     data_criacao = Column(DateTime, default=datetime.datetime.utcnow())
52
53
54 # =====
55 # ASSINATURA (liga cliente aos produtos adquiridos)
56 # =====
57 class Assinatura(BaseCore):
58     __tablename__ = "assinaturas"
59
60     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
61     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
62     produto_id = Column(Integer, ForeignKey("produtos_catalogo.id"), nullable=False)
63     status = Column(String(20), default="ATIVA") # ATIVA, INADIMIENTE, CANCELADA, EXPIRADA
64     data_aquisicao = Column(DateTime, default=datetime.datetime.utcnow())
65     data_expiracao = Column(DateTime, nullable=True) # NULL = sem expiracao
66     valor_mensal = Column(Numeric(10, 2), nullable=False, default=0)
67     dia_vencimento = Column(Integer, nullable=True) # Dia do mês de vencimento (1-31)
68
69     # Relacionamentos
70     cliente = relationship("Cliente", back_populates="assinaturas")
71     produto = relationship("Produto")
72
73
74 # =====
75 # CHAVE DE ATIVAÇÃO (usada no download do desktop)
76 # =====
77 class ChaveAtivacao(BaseCore):
78     __tablename__ = "chaves_ativacao"
79
80     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
81     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=False)
82     chave = Column(String(50), unique=True, index=True, nullable=False)
83     produto_id = Column(Integer, ForeignKey("produtos_catalogo.id"), nullable=True) # NULL = vale para todos
84     usada = Column(Boolean, default=False)
85     data_criacao = Column(DateTime, default=datetime.datetime.utcnow())
86     data_uso = Column(DateTime, nullable=True)
87
88     # Relacionamentos
89     cliente = relationship("Cliente", back_populates="chaves")
90     produto = relationship("Produto")
91
92
93 # =====
94 # EMPRESA (dados da empresa do cliente, preenchidos após ativação)
95 # =====
96 class Empresa(BaseCore):
97     __tablename__ = "empresas"
98
99     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
100     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=True)
101     razao_social = Column(String(255), nullable=False)
102     nome_fantasia = Column(String(255))
103     documento = Column(String(20), unique=True, index=True, nullable=False) # CNPJ
104     email = Column(String(255), index=True)
105     telefone = Column(String(20))
106     endereco = Column(String(500))
107     ativo = Column(Boolean, default=True)
108     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow())
109
110
111 # =====
112 # USUÁRIO (colaboradores da empresa do cliente)
113 # =====
114 class Usuario(BaseCore):
115     __tablename__ = "usuarios"
116
117     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
118     empresa_id = Column(Integer, ForeignKey("empresas.id"), nullable=True)
119     cliente_id = Column(Integer, ForeignKey("clientes.id"), nullable=True)
120     empresa_documento = Column(String(20), index=True, nullable=False, default="00.000.000/0001-00")
121     nome = Column(String(255), nullable=False)
122     username = Column(String(100), index=True, nullable=False)
123     email = Column(String(255), index=True, nullable=True)
124     senha_hash = Column(String(255), nullable=False)
125     papel = Column(String(30), default="OPERADOR")
126     ativo = Column(Boolean, default=True)
127     codigo = Column(String(20), nullable=True)
128     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow())
129     ultimo_acesso = Column(DateTime, nullable=True)
130
131
132 # =====
133 # ADMINISTRADOR CORUMBÁ (vendedor/funcionário da Corumbá Sistemas)
134 # =====

```

```
135 class AdminCorumba(BaseCore):
136     __tablename__ = "admins_corumba"
137
138     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
139     nome = Column(String(255), nullable=False)
140     email = Column(String(255), unique=True, index=True, nullable=False)
141     senha_hash = Column(String(255), nullable=False)
142     papel = Column(String(30), default="VENDEDOR") # ADMIN, VENDEDOR, SUPORTE
143     ativo = Column(Boolean, default=True)
144     data_cadastro = Column(DateTime, default=datetime.datetime.utcnow)
145     ultimo_acesso = Column(DateTime, nullable=True)
146
```



```

1 """
2 models.py - Adicionar modelo de HistoricoVendas
3
4 Registra TODA operação de venda para auditoria e rastreabilidade.
5 Cada venda gera múltiplos registros de histórico:
6 - Criação da venda
7 - Cada item adicionado
8 - Alterações de status
9 - Cancelamentos
10 - Edições
11 - Logs de erro
12 """
13 # Adicionar ao final de models.py
14
15 import datetime
16 from sqlalchemy import Column, Integer, String, Numeric, Boolean, ForeignKey, DateTime, Text, JSON
17 from sqlalchemy.orm import relationship
18 from database import BaseSalto, TenantMixin
19
20
21 class HistoricoVenda(BaseSalto, TenantMixin):
22     __tablename__ = "historico_vendas"
23
24     id = Column(Integer, primary_key=True, index=True, autoincrement=True)
25
26     # Referência à venda
27     venda_id = Column(Integer, nullable=True) # NULL se a venda falhou antes de ser criada
28     numero_pedido = Column(String(50), index=True) # Número legível (ex: PED-20260601-0001)
29
30     # Dados da empresa e usuário
31     empresa_documento = Column(String(20), index=True, nullable=False)
32     usuario_id = Column(Integer, nullable=True)
33     usuario_nome = Column(String(255), nullable=True)
34     usuario_papel = Column(String(30), nullable=True)
35
36     # Dados do cliente
37     cliente_id = Column(Integer, nullable=True)
38     cliente_nome = Column(String(255), nullable=True)
39
40     # Dados financeiros
41     valor_total = Column(Numeric(10, 2), default=0)
42     valor_desconto = Column(Numeric(10, 2), default=0)
43     valor_recebido = Column(Numeric(10, 2), default=0)
44     troco = Column(Numeric(10, 2), default=0)
45     forma_pagamento = Column(String(50), nullable=True)
46
47     # Itens da venda (JSON para flexibilidade)
48     itens_json = Column(Text, nullable=True) # JSON com produtos, quantidades, preços
49
50     # Status e tipo de operação
51     status = Column(String(20), default="CONCLUIDA") # CONCLUIDA, CANCELADA, PENDENTE, ERRO
52     tipo_operacao = Column(String(30), default="VENDA") # VENDA, CANCELAMENTO, EDICAO, TROCA, DEVOLUCAO
53
54     # Rastreamento
55     origem = Column(String(50), default="PDV") # PDV, WEB, API, MOBILE
56     observacao = Column(Text, nullable=True)
57
58     # Dados de controle
59     data_criacao = Column(DateTime, default=datetime.datetime.utcnow)
60     data_alteracao = Column(DateTime, nullable=True)
61     data_cancelamento = Column(DateTime, nullable=True)
62     motivo_cancelamento = Column(Text, nullable=True)
63
64     # Logs de erro (se houve falha)
65     erro_detalhes = Column(Text, nullable=True)
66
67     # IP e sessão (para auditoria)
68     ip_origem = Column(String(50), nullable=True)
69     sessao_id = Column(String(100), nullable=True)
70
71     # Versão dos dados (para controle de concorrência)
72     versao = Column(Integer, default=1)
73
74     # Dados extras flexíveis
75     metadados_json = Column(Text, nullable=True) # JSON com dados adicionais
76

```

backendPy/requirements.txt

```
1 fastapi>=0.110.0
2 uvicorn[standard]>=0.29.0
3 sqlalchemy>=2.0.0
4 pydantic>=2.0.0
5 python-jose[cryptography]>=3.3.0
6 passlib>=1.7.4
7 python-dotenv>=1.0.0
8 alembic>=1.13.0
9 psycopg2-binary>=2.9.0
10 bcrypt>=4.0.0
11 python-multipart>=0.0.9
12 Jinja2>=3.1.0
13 httpx>=0.27.0
14
```

```
1 # routers package
2
```

```

1 """
2 routers/admin.py - Área administrativa do vendedor Corumbá Sistemas
3 - Cadastrar clientes
4 - Gerar chaves de ativação
5 - Gerenciar assinaturas
6 - Ver status de inadimplência
7 """
8 import secrets
9 import string
10 from datetime import datetime
11 from fastapi import APIRouter, Depends, HTTPException
12 from sqlalchemy.orm import Session
13 from pydantic import BaseModel
14
15 from database import get_db
16 from models_core import Cliente, Produto, Assinatura, ChaveAtivacao
17 from security import hash_senha
18
19 router = APIRouter(prefix="/admin", tags=["Administração Corumbá"])
20
21
22 # ===== PRODUTOS DO CATÁLOGO =====
23
24 @router.get("/produtos")
25 def listar_produtos_catalogo(db: Session = Depends(get_db)):
26     """Lista todos os produtos do catálogo ativos."""
27     produtos = db.query(Produto).filter(Produto.ativo == True).order_by(Produto.ordem).all()
28     return [
29         {
30             "id": p.id,
31             "nome": p.nome,
32             "descricao": p.descricao,
33             "preco_mensal": float(p.preco_mensal) if p.preco_mensal else 0,
34             "preco_anual": float(p.preco_anual) if p.preco_anual else None,
35             "icone": p.icone,
36             "ordem": p.ordem,
37         }
38         for p in produtos
39     ]
40
41
42 # ===== CLIENTES =====
43
44 class CadastrarClienteRequest(BaseModel):
45     """Vendedor da Corumbá cadastra um novo cliente."""
46     nome: str
47     email: str
48     telefone: str
49     cpf_cnpj: str
50     senha: str
51     produto_ids: list[int] = [] # IDs dos produtos que o cliente está adquirindo agora
52
53
54 class CriarChaveRequest(BaseModel):
55     """Gera uma ou mais chaves de ativação para um cliente."""
56     cliente_id: int
57     produto_ids: list[int] = [] # Lista de produtos. Vazio = chave genérica para todos
58     quantidade_chaves: int = 1
59
60
61 class AlterarStatusAssinaturaRequest(BaseModel):
62     """Altera o status de uma assinatura (ATIVA, INADIMIENTE, CANCELADA)."""
63     novo_status: str
64
65
66 # ===== ENDPOINTS =====
67
68 @router.post("/clientes/criar")
69 def criar_cliente(req: CadastrarClienteRequest, db: Session = Depends(get_db)):
70     """
71     Vendedor da Corumbá cadastra um novo cliente no sistema.
72     """
73     # Validações
74     existing = db.query(Cliente).filter(Cliente.email == req.email).first()
75     if existing:
76         raise HTTPException(status_code=400, detail="E-mail já cadastrado.")
77
78     existing_doc = db.query(Cliente).filter(Cliente.cpf_cnpj == req.cpf_cnpj).first()
79     if existing_doc:
80         raise HTTPException(status_code=400, detail="CPF/CNPJ já cadastrado.")
81
82     cliente = Cliente(
83         nome=req.nome,
84         email=req.email,
85         telefone=req.telefone,
86         cpf_cnpj=req.cpf_cnpj,
87         senha_hash=hash_senha(req.senha),
88         ativo=True,
89     )
90     db.add(cliente)
91     db.flush()
92
93     # Criar assinaturas iniciais
94     assinaturas_criadas = []
95     for pid in req.produto_ids:
96         produto = db.query(Produto).filter(Produto.id == pid, Produto.ativo == True).first()
97         if produto:
98             assinatura = Assinatura(
99                 cliente_id=cliente.id,
100                 produto_id=produto.id,
101                 status="ATIVA",
102                 valor_mensal=produto.preco_mensal,
103             )
104             db.add(assinatura)
105             assinaturas_criadas.append(produto.nome)
106
107     db.commit()
108     db.refresh(cliente)
109
110     return {
111         "status": "Cliente cadastrado com sucesso",
112         "cliente_id": cliente.id,
113         "nome": cliente.nome,
114         "email": cliente.email,
115         "cpf_cnpj": cliente.cpf_cnpj,
116         "produtos_ativos": assinaturas_criadas,
117         "proximo_passo": "Gere chaves de ativação para o cliente fazer download do desktop",
118     }
119
120
121 @router.get("/clientes")
122 def listar_clientes(
123     busca: str = None,
124     status: str = None, # ATIVO, INADIMIENTE
125     db: Session = Depends(get_db)
126 ):
127     """Lista todos os clientes cadastrados."""
128     query = db.query(Cliente)
129
130     if busca:
131         query = query.filter(
132             Cliente.nome.ilike(f"%{busca}%")
133             | Cliente.email.ilike(f"%{busca}%")
134             | Cliente.cpf_cnpj.ilike(f"%{busca}%")

```

```

135     )
136
137     clientes = query.order_by(Cliente.nome).all()
138
139     resultado = []
140     for c in clientes:
141         assinaturas = db.query(Assinatura).filter(Assinatura.cliente_id == c.id).all()
142         produtos_info = []
143         tem_inadimplencia = False
144         for a in assinaturas:
145             produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
146             produtos_info.append({
147                 "nome": produto.nome if produto else "Desconhecido",
148                 "status": a.status,
149                 "valor": float(a.valor_mensal),
150             })
151             if a.status == "INADIMIENTE":
152                 tem_inadimplencia = True
153
154         resultado.append({
155             "id": c.id,
156             "nome": c.nome,
157             "email": c.email,
158             "telefone": c.telefone,
159             "cpf_cnpj": c.cpf_cnpj,
160             "ativo": c.ativo,
161             "tem_inadimplencia": tem_inadimplencia,
162             "produtos": produtos_info,
163             "total_produtos": len(produtos_info),
164         })
165
166     # Filtrar por status se solicitado
167     if status == "INADIMIENTE":
168         resultado = [r for r in resultado if r["tem_inadimplencia"]]
169     elif status == "ATIVO":
170         resultado = [r for r in resultado if not r["tem_inadimplencia"]]
171
172     return resultado
173
174
175 @router.get("/clientes/{cliente_id}")
176 def detalhe_cliente(cliente_id: int, db: Session = Depends(get_db)):
177     """Detalhes de um cliente específico."""
178     cliente = db.query(Cliente).filter(Cliente.id == cliente_id).first()
179     if not cliente:
180         raise HTTPException(status_code=404, detail="Cliente não encontrado.")
181
182     assinaturas = db.query(Assinatura).filter(Assinatura.cliente_id == cliente_id).all()
183     produtos_info = []
184     for a in assinaturas:
185         produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
186         produtos_info.append({
187             "produto_id": a.produto_id,
188             "nome": produto.nome if produto else "Desconhecido",
189             "status": a.status,
190             "valor": float(a.valor_mensal),
191             "data_aquisicao": str(a.data_aquisicao),
192             "data_expiracao": str(a.data_expiracao) if a.data_expiracao else None,
193         })
194
195     chaves = db.query(ChaveAtivacao).filter(ChaveAtivacao.cliente_id == cliente_id).all()
196     chaves_info = []
197     for k in chaves:
198         produto = db.query(Produto).filter(Produto.id == k.produto_id).first() if k.produto_id else None
199         chaves_info.append({
200             "chave": k.chave,
201             "produto": produto.nome if produto else "Genérica",
202             "usada": k.usada,
203             "data_criacao": str(k.data_criacao),
204             "data_uso": str(k.data_uso) if k.data_uso else None,
205         })
206
207     return {
208         "id": cliente.id,
209         "nome": cliente.nome,
210         "email": cliente.email,
211         "telefone": cliente.telefone,
212         "cpf_cnpj": cliente.cpf_cnpj,
213         "ativo": cliente.ativo,
214         "data_cadastro": str(cliente.data_cadastro),
215         "ultimo_acesso": str(cliente.ultimo_acesso) if cliente.ultimo_acesso else None,
216         "produtos": produtos_info,
217         "chaves": chaves_info,
218     }
219
220
221 @router.post("/chaves/gerar")
222 def gerar_chaves(req: CriarChaveRequest, db: Session = Depends(get_db)):
223     """
224     Gera chaves de ativação para um cliente.
225     O cliente usa a chave UMA VEZ no download do desktop.
226     """
227     cliente = db.query(Cliente).filter(Cliente.id == req.cliente_id).first()
228     if not cliente:
229         raise HTTPException(status_code=404, detail="Cliente não encontrado.")
230
231     chaves_criadas = []
232     for _ in range(req.quantidade_chaves):
233         for pid in (req.produto_ids or [None]):
234             # Gerar chave única
235             partes = []
236             for _ in range(3):
237                 parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
238                 partes.append(parte)
239             chave_str = f"CORUMBA-{'-'.join(partes)}"
240
241             chave = ChaveAtivacao(
242                 cliente_id=req.cliente_id,
243                 chave=chave_str,
244                 produto_id=pid,
245                 usada=False,
246             )
247             db.add(chave)
248             chaves_criadas.append(chave_str)
249
250             # Se tem produto específico, garantir que existe assinatura
251             if pid:
252                 existing = db.query(Assinatura).filter(
253                     Assinatura.cliente_id == req.cliente_id,
254                     Assinatura.produto_id == pid,
255                 ).first()
256                 if not existing:
257                     produto = db.query(Produto).filter(Produto.id == pid).first()
258                     if produto:
259                         assinatura = Assinatura(
260                             cliente_id=req.cliente_id,
261                             produto_id=pid,
262                             status="ATIVA",
263                             valor_mensal=produto.preco_mensal,
264                         )
265                         db.add(assinatura)
266
267     db.commit()
268
269     return {
270         "status": f"{len(chaves_criadas)} chave(s) gerada(s)",
271         "chaves": chaves_criadas,
272         "cliente": cliente.nome,

```

```

273     }
274
275 @router.put("/assinaturas/{assinatura_id}/status")
276 def alterar_status_assinatura(
277     assinatura_id: int,
278     req: AlterarStatusAssinaturaRequest,
279     db: Session = Depends(get_db)
280 ):
281     """Altera o status de uma assinatura."""
282     assinatura = db.query(Assinatura).filter(Assinatura.id == assinatura_id).first()
283     if not assinatura:
284         raise HTTPException(status_code=404, detail="Assinatura não encontrada.")
285
286     if req.novo_status not in ("ATIVA", "INADIMIENTE", "CANCELADA", "EXPIRADA"):
287         raise HTTPException(status_code=400, detail="Status inválido. Use: ATIVA, INADIMIENTE, CANCELADA, EXPIRADA.")
288
289     assinatura.status = req.novo_status
290     db.commit()
291
292     produto = db.query(Produto).filter(Produto.id == assinatura.produto_id).first()
293     cliente = db.query(Cliente).filter(Cliente.id == assinatura.cliente_id).first()
294
295     return {
296         "status": "Alterado com sucesso",
297         "cliente": cliente.nome if cliente else "?",
298         "produto": produto.nome if produto else "?",
299         "novo_status": req.novo_status,
300     }
301
302
303 @router.get("/dashboard")
304 def dashboard_admin(db: Session = Depends(get_db)):
305     """Dashboard administrativo com métricas gerais."""
306     total_clientes = db.query(Cliente).count()
307     clientes_ativos = db.query(Cliente).filter(Cliente.ativo == True).count()
308
309     total_assinaturas = db.query(Assinatura).count()
310     assinaturas_ativas = db.query(Assinatura).filter(Assinatura.status == "ATIVA").count()
311     assinaturas_inadimplentes = db.query(Assinatura).filter(Assinatura.status == "INADIMIENTE").count()
312     assinaturas_canceladas = db.query(Assinatura).filter(Assinatura.status == "CANCELADA").count()
313
314     total_chaves = db.query(ChaveAtivacao).count()
315     chaves_usadas = db.query(ChaveAtivacao).filter(ChaveAtivacao.usada == True).count()
316     chaves_pendentes = db.query(ChaveAtivacao).filter(ChaveAtivacao.usada == False).count()
317
318     # Receita mensal estimada
319     from sqlalchemy import func
320     receita_mensal = db.query(func.sum(Assinatura.valor_mensal)).filter(
321         Assinatura.status == "ATIVA"
322     ).scalar()
323
324     return {
325         "clientes": {
326             "total": total_clientes,
327             "ativos": clientes_ativos,
328         },
329         "assinaturas": {
330             "total": total_assinaturas,
331             "ativas": assinaturas_ativas,
332             "inadimplentes": assinaturas_inadimplentes,
333             "canceladas": assinaturas_canceladas,
334         },
335         "chaves_ativacao": {
336             "total": total_chaves,
337             "usadas": chaves_usadas,
338             "pendentes": chaves_pendentes,
339         },
340         "receita_mensal_estimada": float(receita_mensal) if receita_mensal else 0,
341     }
342
343

```

```

1 """
2 routers/admin_auth.py - Autenticação do vendedor/administrador Corumbá Sistemas
3 """
4 import secrets
5 import string
6 from datetime import datetime
7 from fastapi import APIRouter, Depends, HTTPException
8 from sqlalchemy.orm import Session
9 from pydantic import BaseModel
10
11 from database import get_db
12 from models_core import AdminCorumba, Cliente, Produto, Assinatura
13 from security import hash_senha, verificar_senha, criar_token
14
15 router = APIRouter(prefix="/admin/auth", tags=["Autenticação Admin"])
16
17
18 # ===== SCHEMAS =====
19
20 class AdminLoginRequest(BaseModel):
21     email: str
22     senha: str
23
24
25 class AdminCadastroRequest(BaseModel):
26     """Cadastro de novo vendedor/administrador (apenas ADMIN pode criar)."""
27     nome: str
28     email: str
29     senha: str
30     papel: str = "VENDEDOR"
31
32
33 # ===== ENDPOINTS =====
34
35 @router.post("/login")
36 def admin_login(req: AdminLoginRequest, db: Session = Depends(get_db)):
37     """Login do vendedor/administrador Corumbá."""
38     admin = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
39
40     if not admin:
41         raise HTTPException(status_code=401, detail="E-mail não encontrado.")
42
43     if not admin.ativo:
44         raise HTTPException(status_code=403, detail="Conta desativada.")
45
46     if not verificar_senha(req.senha, admin.senha_hash):
47         raise HTTPException(status_code=401, detail="Senha incorreta.")
48
49     admin.ultimo_acesso = datetime.utcnow()
50     db.commit()
51
52     token = criar_token({
53         "sub": admin.email,
54         "admin_id": admin.id,
55         "nome": admin.nome,
56         "papel": admin.papel,
57         "tipo": "admin_corumba",
58     })
59
60     return {
61         "access_token": token,
62         "token type": "bearer",
63         "admin": {
64             "id": admin.id,
65             "nome": admin.nome,
66             "email": admin.email,
67             "papel": admin.papel,
68         },
69     }
70
71
72 @router.post("/cadastro")
73 def admin_cadastro(req: AdminCadastroRequest, db: Session = Depends(get_db)):
74     """
75     Cadastro de novo administrador/vendedor.
76     Em produção, isso deveria ser restrito a admins já logados.
77     """
78     existing = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
79     if existing:
80         raise HTTPException(status_code=400, detail="E-mail já cadastrado.")
81
82     if req.papel not in ("ADMIN", "VENDEDOR", "SUPORTE"):
83         raise HTTPException(status_code=400, detail="Papel inválido. Use: ADMIN, VENDEDOR, SUPORTE.")
84
85     admin = AdminCorumba(
86         nome=req.nome,
87         email=req.email,
88         senha_hash=hash_senha(req.senha),
89         papel=req.papel,
90         ativo=True,
91     )
92     db.add(admin)
93     db.commit()
94     db.refresh(admin)
95
96     return {
97         "status": "Administrador cadastrado com sucesso",
98         "admin_id": admin.id,
99         "nome": admin.nome,
100         "email": admin.email,
101         "papel": admin.papel,
102     }
103
104
105 @router.get("/me")
106 def admin_me(
107     email: str = None,
108     db: Session = Depends(get_db)
109 ):
110     """Retorna dados do admin logado (para verificação de token)."""
111     if not email:
112         raise HTTPException(status_code=401, detail="Não autenticado.")
113
114     admin = db.query(AdminCorumba).filter(AdminCorumba.email == email).first()
115     if not admin:
116         raise HTTPException(status_code=404, detail="Admin não encontrado.")
117
118     return {
119         "id": admin.id,
120         "nome": admin.nome,
121         "email": admin.email,
122         "papel": admin.papel,
123     }
124

```

```

1 """
2 routers/auth.py - Autenticação do cliente Corumbá Sistemas
3
4 FLUXO:
5 1. Vendedor da Corumbá cadastra o cliente (nome, email, telefone, CPF/CNPJ, senha)
6 2. Cliente faz login com email + senha
7 3. No painel, cliente vê produtos (adquiridos = ativos, não adquiridos = bloqueados)
8 4. Ao clicar em produto não adquirido, abre informações + botão de compra
9 5. Se inadimplente, produtos ficam bloqueados até inserir nova chave de ativação
10 6. Chave de ativação é usada UMA VEZ no download do desktop
11 """
12 import secrets
13 import string
14 from datetime import datetime
15 from fastapi import APIRouter, Depends, HTTPException
16 from sqlalchemy.orm import Session
17 from pydantic import BaseModel
18
19 from database import get_db
20 from models_core import Cliente, Produto, Assinatura, ChaveAtivacao
21 from security import hash_senha, verificar_senha, criar_token, get_current_cliente
22
23 router = APIRouter(prefix="/auth", tags=["Autenticação"])
24
25
26 # ===== SCHEMAS =====
27
28 class CadastroClienteRequest(BaseModel):
29     """Dados para o vendedor da Corumbá cadastrar um novo cliente."""
30     nome: str
31     email: str
32     telefone: str
33     cpf_cnpj: str
34     senha: str
35
36
37 class LoginClienteRequest(BaseModel):
38     """Login do cliente com email + senha."""
39     email: str
40     senha: str
41
42
43 class AtivarChaveRequest(BaseModel):
44     """Cliente insere chave de ativação (uma única vez, no download)."""
45     chave: str
46
47
48 class CriarChaveRequest(BaseModel):
49     """Vendedor/administrador gera uma chave de ativação para o cliente."""
50     cliente_id: int
51     produto_id: int = None # None = vale para todos os produtos
52
53
54 # ===== ENDPOINTS =====
55
56 @router.post("/cliente/cadastro")
57 def cadastro_cliente(req: CadastroClienteRequest, db: Session = Depends(get_db)):
58     """
59     Cadastra um novo cliente (feito pelo vendedor da Corumbá Sistemas).
60     """
61     # Verificar se email já existe
62     existing = db.query(Cliente).filter(Cliente.email == req.email).first()
63     if existing:
64         raise HTTPException(status_code=400, detail="E-mail já cadastrado.")
65
66     # Verificar se CPF/CNPJ já existe
67     existing_doc = db.query(Cliente).filter(Cliente.cpf_cnpj == req.cpf_cnpj).first()
68     if existing_doc:
69         raise HTTPException(status_code=400, detail="CPF/CNPJ já cadastrado.")
70
71     cliente = Cliente(
72         nome=req.nome,
73         email=req.email,
74         telefone=req.telefone,
75         cpf_cnpj=req.cpf_cnpj,
76         senha_hash=hash_senha(req.senha),
77         ativo=True,
78     )
79     db.add(cliente)
80     db.commit()
81     db.refresh(cliente)
82
83     return {
84         "status": "Cliente cadastrado com sucesso",
85         "cliente_id": cliente.id,
86         "nome": cliente.nome,
87         "email": cliente.email,
88     }
89
90
91 @router.post("/cliente/login")
92 def login_cliente(req: LoginClienteRequest, db: Session = Depends(get_db)):
93     """
94     Login do cliente com email + senha.
95     Retorna JWT token para acesso ao painel.
96     """
97     cliente = db.query(Cliente).filter(Cliente.email == req.email).first()
98     if not cliente:
99         raise HTTPException(status_code=401, detail="E-mail não encontrado.")
100
101     if not cliente.ativo:
102         raise HTTPException(status_code=403, detail="Conta desativada. Contate a Corumbá Sistemas.")
103
104     if not verificar_senha(req.senha, cliente.senha_hash):
105         raise HTTPException(status_code=401, detail="Senha incorreta.")
106
107     # Atualizar último acesso
108     cliente.ultimo_acesso = datetime.utcnow()
109     db.commit()
110
111     # Verificar status das assinaturas
112     assinaturas = db.query(Assinatura).filter(
113         Assinatura.cliente_id == cliente.id
114     ).all()
115
116     produtos_adquiridos = []
117     tem_inadimplente = False
118     for a in assinaturas:
119         if a.status == "ATIVA":
120             produtos_adquiridos.append(a.produto_id)
121         elif a.status == "INADIMIENTE":
122             tem_inadimplente = True
123
124     token = criar_token({
125         "sub": cliente.email,
126         "cliente_id": cliente.id,
127         "nome": cliente.nome,
128     })
129
130     return {
131         "access token": token,
132         "token type": "bearer",
133         "cliente": {
134             "id": cliente.id,

```



```

135         "nome": cliente.nome,
136         "email": cliente.email,
137         "telefone": cliente.telefone,
138         "cpf_cnpj": cliente.cpf_cnpj,
139     },
140     "produtos_adquiridos": produtos_adquiridos,
141     "tem_inadimplencia": tem_inadimplente,
142 }
143
144
145 @router.get("/cliente/me")
146 def get_me(cliente: Cliente = Depends(get_current_cliente)):
147     """Retorna dados do cliente logado."""
148     return {
149         "id": cliente.id,
150         "nome": cliente.nome,
151         "email": cliente.email,
152         "telefone": cliente.telefone,
153         "cpf_cnpj": cliente.cpf_cnpj,
154     }
155
156
157 @router.post("/chave/ativar")
158 def ativar_chave(
159     req: AtivarChaveRequest,
160     cliente: Cliente = Depends(get_current_cliente),
161     db: Session = Depends(get_db)
162 ):
163     """
164     Cliente insere chave de ativação (uma única vez, no download do desktop).
165     Se a assinatura estiver inadimplente, a chave reativa o acesso.
166     """
167     chave = db.query(ChaveAtivacao).filter(
168         ChaveAtivacao.chave == req.chave,
169         ChaveAtivacao.cliente_id == cliente.id,
170     ).first()
171
172     if not chave:
173         raise HTTPException(status_code=404, detail="Chave de ativação não encontrada.")
174
175     if chave.usada:
176         raise HTTPException(status_code=400, detail="Esta chave já foi utilizada.")
177
178     # Marcar chave como usada
179     chave.usada = True
180     chave.data_uso = datetime.utcnow()
181
182     # Ativar/reactivar assinatura do produto
183     if chave.produto_id:
184         assinatura = db.query(Assinatura).filter(
185             Assinatura.cliente_id == cliente.id,
186             Assinatura.produto_id == chave.produto_id,
187         ).first()
188         if assinatura:
189             assinatura.status = "ATIVA"
190     else:
191         # Chave genérica – ativar todas as assinaturas inadimplentes
192         assinaturas = db.query(Assinatura).filter(
193             Assinatura.cliente_id == cliente.id,
194             Assinatura.status == "INADIMIENTE",
195         ).all()
196         for a in assinaturas:
197             a.status = "ATIVA"
198
199     db.commit()
200
201     return {
202         "status": "Chave ativada com sucesso!",
203         "message": "Seus produtos foram liberados. Você já pode fazer o download.",
204     }
205
206
207 @router.post("/chave/gerar")
208 def gerar_chave(
209     req: CriarChaveRequest,
210     db: Session = Depends(get_db)
211 ):
212     """
213     Administrador/vendedor gera uma chave de ativação para o cliente.
214     """
215     cliente = db.query(Cliente).filter(Cliente.id == req.cliente_id).first()
216     if not cliente:
217         raise HTTPException(status_code=404, detail="Cliente não encontrado.")
218
219     # Gerar chave aleatória
220     partes = []
221     for _ in range(3):
222         parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
223         partes.append(parte)
224     chave_str = f"CORUMBA-{'-'.join(partes)}"
225
226     chave = ChaveAtivacao(
227         cliente_id=req.cliente_id,
228         chave=chave_str,
229         produto_id=req.produto_id,
230         usada=False,
231     )
232     db.add(chave)
233
234     # Criar assinatura se não existir
235     if req.produto_id:
236         existing = db.query(Assinatura).filter(
237             Assinatura.cliente_id == req.cliente_id,
238             Assinatura.produto_id == req.produto_id,
239         ).first()
240         if not existing:
241             produto = db.query(Produto).filter(Produto.id == req.produto_id).first()
242             assinatura = Assinatura(
243                 cliente_id=req.cliente_id,
244                 produto_id=req.produto_id,
245                 status="ATIVA",
246                 valor_mensal=produto.preco_mensal if produto else 0,
247             )
248             db.add(assinatura)
249
250     db.commit()
251     db.refresh(chave)
252
253     return {
254         "status": "Chave gerada com sucesso",
255         "chave": chave_str,
256         "cliente": cliente.nome,
257         "produto_id": req.produto_id,
258     }
259

```

```

1 """
2 routers/caixa.py - Gestao de caixa: abertura, fechamento, transacoes, sangria
3 Regras de seguranca:
4 - Sangria/Saida exige senha admin
5 - Venda automaticamente registra transacao no caixa aberto
6 """
7 from fastapi import APIRouter, Depends, HTTPException, Query
8 from sqlalchemy.orm import Session
9 from datetime import datetime
10 from typing import Optional
11
12 from database import get_db_salto, get_db_core
13 from models import CaixaSessao, CaixaTransacao
14 from schemas import CaixaAbrirRequest, CaixaFecharRequest, CaixaTransacaoRequest
15 from models_core import Usuario
16 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
17
18 router = APIRouter(prefix="/caixa", tags=["Caixa"])
19
20
21 @router.post("/abrir")
22 def abrir_caixa(
23     dados: CaixaAbrirRequest,
24     empresa_documento: str = Depends(get_empresa_from_user),
25     current_user: Usuario = Depends(get_current_user),
26     db: Session = Depends(get_db_salto)
27 ):
28     caixa_aberto = db.query(CaixaSessao).filter(
29         CaixaSessao.empresa_documento == empresa_documento,
30         CaixaSessao.status == "ABERTO"
31     ).first()
32     if caixa_aberto:
33         raise HTTPException(status_code=400, detail=f"Ja existe um caixa aberto por {caixa_aberto.operador}.")
34
35     nova_sessao = CaixaSessao(
36         empresa_documento=empresa_documento,
37         operador=dados.operador,
38         usuario_id=current_user.id,
39         saldo_inicial=dados.saldo_inicial,
40         status="ABERTO",
41         data_abertura=datetime.utcnow()
42     )
43     db.add(nova_sessao)
44     db.commit()
45     db.refresh(nova_sessao)
46
47     if dados.saldo_inicial > 0:
48         transacao = CaixaTransacao(
49             empresa_documento=empresa_documento,
50             sessao_id=nova_sessao.id,
51             tipo="ENTRADA",
52             valor=dados.saldo_inicial,
53             descricao="Troco inicial (fundo de caixa)",
54             operador=dados.operador
55         )
56         db.add(transacao)
57         db.commit()
58
59     return {"status": "Caixa aberto", "sessao_id": nova_sessao.id, "operador": dados.operador}
60
61
62 @router.post("/fechar")
63 def fechar_caixa(
64     dados: CaixaFecharRequest,
65     empresa_documento: str = Depends(get_empresa_from_user),
66     current_user: Usuario = Depends(get_current_user),
67     db: Session = Depends(get_db_salto)
68 ):
69     caixa_aberto = db.query(CaixaSessao).filter(
70         CaixaSessao.empresa_documento == empresa_documento,
71         CaixaSessao.status == "ABERTO"
72     ).first()
73     if not caixa_aberto:
74         raise HTTPException(status_code=400, detail="Nenhum caixa aberto encontrado.")
75
76     transacoes = db.query(CaixaTransacao).filter(
77         CaixaTransacao.sessao_id == caixa_aberto.id,
78         CaixaTransacao.empresa_documento == empresa_documento
79     ).all()
80
81     total_vendas = sum(float(t.valor) for t in transacoes if t.tipo == "VENDA")
82     total_entradas = sum(float(t.valor) for t in transacoes if t.tipo == "ENTRADA")
83     total_saidas = sum(float(t.valor) for t in transacoes if t.tipo in ("SAIDA", "SANGRIA"))
84     saldo_esperado = float(caixa_aberto.saldo_inicial) + total_entradas + total_vendas - total_saidas
85     diferenca = float(dados.saldo_final) - saldo_esperado
86
87     caixa_aberto.status = "FECHADO"
88     caixa_aberto.saldo_final = dados.saldo_final
89     caixa_aberto.total_vendas = total_vendas
90     caixa_aberto.total_entradas = total_entradas
91     caixa_aberto.total_saidas = total_saidas
92     caixa_aberto.data_fechamento = datetime.utcnow()
93     db.commit()
94
95     resultado = {
96         "status": "Caixa fechado",
97         "sessao_id": caixa_aberto.id,
98         "operador": caixa_aberto.operador,
99         "saldo_inicial": float(caixa_aberto.saldo_inicial),
100         "total_vendas": total_vendas,
101         "total_entradas": total_entradas,
102         "total_saidas": total_saidas,
103         "saldo_esperado": saldo_esperado,
104         "saldo_informado": float(dados.saldo_final),
105         "diferenca": diferenca
106     }
107     if abs(diferenca) > 0.01:
108         resultado["alerta"] = f"DIFERENCA DE R$ {diferenca:.2f} {'SOBRA' if diferenca > 0 else 'FALTA'}"
109     return resultado
110
111
112 @router.get("/status")
113 def status_caixa(
114     empresa_documento: str = Depends(get_empresa_from_user),
115     current_user: Usuario = Depends(get_current_user),
116     db: Session = Depends(get_db_salto)
117 ):
118     caixa_aberto = db.query(CaixaSessao).filter(
119         CaixaSessao.empresa_documento == empresa_documento,
120         CaixaSessao.status == "ABERTO"
121     ).first()
122
123     if not caixa_aberto:
124         return {"status": "FECHADO"}
125
126     transacoes = db.query(CaixaTransacao).filter(
127         CaixaTransacao.sessao_id == caixa_aberto.id,
128         CaixaTransacao.empresa_documento == empresa_documento
129     ).order_by(CaixaTransacao.data_transacao.desc()).all()
130
131     total_vendas = sum(float(t.valor) for t in transacoes if t.tipo == "VENDA")
132     total_entradas = sum(float(t.valor) for t in transacoes if t.tipo == "ENTRADA")
133     total_saidas = sum(float(t.valor) for t in transacoes if t.tipo in ("SAIDA", "SANGRIA"))
134     saldo_esperado = float(caixa_aberto.saldo_inicial) + total_entradas + total_vendas - total_saidas

```

```

135
136
137     "status": "ABERTO",
138     "sessao_id": caixa_aberto.id,
139     "operador": caixa_aberto.operador,
140     "saldo_inicial": float(caixa_aberto.saldo_inicial),
141     "total_vendas": total_vendas,
142     "total_entradas": total_entradas,
143     "total_saídas": total_saídas,
144     "saldo_esperado": saldo_esperado,
145     "data_abertura": str(caixa_aberto.data_abertura),
146     "transacoes": [
147         {
148             "id": t.id,
149             "tipo": t.tipo,
150             "valor": float(t.valor),
151             "descricao": t.descricao,
152             "operador": t.operador,
153             "data_transacao": str(t.data_transacao)
154         }
155         for t in transacoes
156     ]
157 }
158
159
160 @router.post("/transacao")
161 def registrar_transacao(
162     dados: CaixaTransacaoRequest,
163     empresa_documento: str = Depends(get_empresa_from_user),
164     current_user: Usuario = Depends(get_current_user),
165     db_core: Session = Depends(get_db_core),
166     db: Session = Depends(get_db_salto)
167 ):
168     """
169     Registra entrada, saída ou sangria no caixa.
170     SAÍDA e SANGRIA exigem senha de admin.
171     """
172     caixa_aberto = db.query(CaixaSessao).filter(
173         CaixaSessao.empresa_documento == empresa_documento,
174         CaixaSessao.status == "ABERTO"
175     ).first()
176     if not caixa_aberto:
177         raise HTTPException(status_code=400, detail="Caixa fechado.")
178
179     # Verificar senha admin para saída/sangria
180     if dados.tipo in ("SAÍDA", "SANGRIA"):
181         if not dados.senha_admin:
182             raise HTTPException(status_code=403, detail=f"Senha de administrador obrigatória para {dados.tipo}.")
183
184         admin_user = db_core.query(Usuario).filter(
185             Usuario.empresa_documento == empresa_documento,
186             Usuario.papel.in_([ "ADMIN", "GERENTE" ]),
187             Usuario.ativo == True
188         ).all()
189
190         senha_valida = False
191         for u in admin_user:
192             if verificar_senha(dados.senha_admin, u.senha_hash):
193                 senha_valida = True
194                 break
195
196         if not senha_valida:
197             raise HTTPException(status_code=403, detail="Senha de administrador inválida.")
198
199     transacao = CaixaTransacao(
200         empresa_documento=empresa_documento,
201         sessao_id=caixa_aberto.id,
202         tipo=dados.tipo,
203         valor=dados.valor,
204         descricao=dados.descricao or f"{dados.tipo} - {current_user.nome}",
205         operador=current_user.nome,
206         autorizado_por="SENHA ADMIN" if dados.tipo in ("SAÍDA", "SANGRIA") else None,
207         data_transacao=datetime.utcnow()
208     )
209     db.add(transacao)
210
211     if dados.tipo == "ENTRADA":
212         caixa_aberto.total_entradas += dados.valor
213     elif dados.tipo in ("SAÍDA", "SANGRIA"):
214         caixa_aberto.total_saídas += dados.valor
215
216     db.commit()
217     return {"status": f"{dados.tipo} registrada", "valor": dados.valor}
218
219
220 @router.get("/historico")
221 def historico_caixa(
222     empresa_documento: str = Depends(get_empresa_from_user),
223     data_inicio: Optional[str] = None,
224     data_fim: Optional[str] = None,
225     limite: int = 20,
226     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
227     db: Session = Depends(get_db_salto)
228 ):
229     """Histórico de sessões de caixa fechadas."""
230     query = db.query(CaixaSessao).filter(
231         CaixaSessao.empresa_documento == empresa_documento,
232         CaixaSessao.status == "FECHADO"
233     )
234
235     if data_inicio:
236         try:
237             di = datetime.strptime(data_inicio, "%Y-%m-%d")
238             query = query.filter(CaixaSessao.data_abertura >= di)
239         except ValueError:
240             pass
241
242     if data_fim:
243         try:
244             df = datetime.strptime(data_fim, "%Y-%m-%d")
245             query = query.filter(CaixaSessao.data_abertura <= df)
246         except ValueError:
247             pass
248
249     sessoes = query.order_by(CaixaSessao.data_abertura.desc()).limit(limite).all()
250     return [
251         {
252             "sessao_id": s.id,
253             "operador": s.operador,
254             "saldo_inicial": float(s.saldo_inicial),
255             "saldo_final": float(s.saldo_final) if s.saldo_final else None,
256             "total_vendas": float(s.total_vendas),
257             "total_entradas": float(s.total_entradas),
258             "total_saídas": float(s.total_saídas),
259             "data_abertura": str(s.data_abertura),
260             "data_fechamento": str(s.data_fechamento) if s.data_fechamento else None
261         }
262         for s in sessoes
263     ]
264

```

```

1 """
2 routers/clientes.py - CRUD de clientes + historico de compras + fidelidade
3 Multi-tenant: todas as operacoes filtram por empresa_documento.
4 """
5 from fastapi import APIRouter, Depends, HTTPException, Query
6 from sqlalchemy.orm import Session
7 from sqlalchemy import func
8 from typing import Optional
9 from datetime import datetime, timedelta
10
11 from database import get_db_salto
12 from models import Cliente, Venda, VendaItem
13 from schemas import ClienteCreate, ClienteUpdate, ClienteResponse, ClienteHistoricoResponse
14 from models_core import Usuario
15 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
16
17 router = APIRouter(prefix="/clientes", tags=["Clientes"])
18
19
20 @router.post("/", response_model=ClienteResponse)
21 def criar_cliente(
22     cliente: ClienteCreate,
23     empresa_documento: str = Depends(get_empresa_from_user),
24     current_user: Usuario = Depends(get_current_user),
25     db: Session = Depends(get_db_salto)):
26     obj = Cliente(empresa_documento=empresa_documento, **cliente.model_dump())
27     db.add(obj)
28     db.commit()
29     db.refresh(obj)
30     return obj
31
32
33 @router.get("/", response_model=list[ClienteResponse])
34 def listar_clientes(
35     empresa_documento: str = Depends(get_empresa_from_user),
36     busca: Optional[str] = None,
37     ativo: Optional[bool] = True,
38     ordem: str = "nome", # nome, pontos, recente
39     current_user: Usuario = Depends(get_current_user),
40     db: Session = Depends(get_db_salto)
41 ):
42     query = db.query(Cliente).filter(Cliente.empresa_documento == empresa_documento)
43     if ativo is not None:
44         query = query.filter(Cliente.ativo == ativo)
45     if busca:
46         query = query.filter(
47             Cliente.nome.ilike(f"%{busca}%") |
48             Cliente.telefone.ilike(f"%{busca}%") |
49             Cliente.cpf_cnpj.ilike(f"%{busca}%") |
50             Cliente.email.ilike(f"%{busca}%")
51         )
52     if ordem == "pontos":
53         query = query.order_by(Cliente.pontos_fidelidade.desc())
54     elif ordem == "recente":
55         query = query.order_by(Cliente.id.desc())
56     else:
57         query = query.order_by(Cliente.nome)
58     return query.all()
59
60
61 @router.get("/{cliente_id}", response_model=ClienteResponse)
62 def get_cliente(
63     cliente_id: int,
64     empresa_documento: str = Depends(get_empresa_from_user),
65     current_user: Usuario = Depends(get_current_user),
66     db: Session = Depends(get_db_salto)
67 ):
68     obj = db.query(Cliente).filter(
69         Cliente.id == cliente_id,
70         Cliente.empresa_documento == empresa_documento
71     ).first()
72     if not obj:
73         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
74     return obj
75
76
77 @router.put("/{cliente_id}", response_model=ClienteResponse)
78 def atualizar_cliente(
79     cliente_id: int,
80     dados: ClienteUpdate,
81     empresa_documento: str = Depends(get_empresa_from_user),
82     current_user: Usuario = Depends(get_current_user),
83     db: Session = Depends(get_db_salto)
84 ):
85     obj = db.query(Cliente).filter(
86         Cliente.id == cliente_id,
87         Cliente.empresa_documento == empresa_documento
88     ).first()
89     if not obj:
90         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
91     for key, value in dados.model_dump(exclude_unset=True).items():
92         setattr(obj, key, value)
93     db.commit()
94     db.refresh(obj)
95     return obj
96
97
98 @router.delete("/{cliente_id}")
99 def desativar_cliente(
100     cliente_id: int,
101     empresa_documento: str = Depends(get_empresa_from_user),
102     current_user: Usuario = Depends(get_current_user),
103     db: Session = Depends(get_db_salto)
104 ):
105     obj = db.query(Cliente).filter(
106         Cliente.id == cliente_id,
107         Cliente.empresa_documento == empresa_documento
108     ).first()
109     if not obj:
110         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
111     obj.ativo = False
112     db.commit()
113     return {"status": "Cliente desativado"}
114
115
116 # =====
117 # HISTORICO DE COMPRAS + FIDELIDADE
118 # =====
119
120 @router.get("/{cliente_id}/historico")
121 def historico_compras(
122     cliente_id: int,
123     empresa_documento: str = Depends(get_empresa_from_user),
124     limite: int = 30,
125     current_user: Usuario = Depends(get_current_user),
126     db: Session = Depends(get_db_salto)
127 ):
128     """Retorna historico de compras do cliente com detalhes."""
129     cliente = db.query(Cliente).filter(
130         Cliente.id == cliente_id,
131         Cliente.empresa_documento == empresa_documento
132     ).first()
133     if not cliente:
134         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")

```

```

135
136 vendas = db.query(Venda).filter(
137     Venda.cliente_id == cliente_id,
138     Venda.empresa_documento == empresa_documento,
139     Venda.status == "CONCLUIDA"
140 ).order_by(Venda.data_venda.desc()).limit(limite).all()
141
142 total_gasto = db.query(func.sum(Venda.total)).filter(
143     Venda.cliente_id == cliente_id,
144     Venda.empresa_documento == empresa_documento,
145     Venda.status == "CONCLUIDA"
146 ).scalar() or 0
147
148 return {
149     "cliente": cliente.nome,
150     "total_gasto": float(total_gasto),
151     "pontos_fidelidade": cliente.pontos_fidelidade,
152     "compras": [
153         {
154             "venda_id": v.id,
155             "data": str(v.data_venda),
156             "total": float(v.total),
157             "forma_pagamento": v.forma_pagamento,
158             "itens": [
159                 {
160                     "produto": item.produto_nome,
161                     "quantidade": float(item.quantidade),
162                     "preco": float(item.preco_unitario),
163                     "subtotal": float(item.subtotal)
164                 }
165                 for item in v.itens
166             ]
167         }
168         for v in vendas
169     ]
170 }
171
172
173 @router.post("/{cliente_id}/pontos")
174 def adicionar_pontos(
175     cliente_id: int,
176     pontos: int,
177     empresa_documento: str = Depends(get_empresa_from_user),
178     current_user: Usuario = Depends(get_current_user),
179     db: Session = Depends(get_db_salto)
180 ):
181     """Adiciona pontos de fidelidade ao cliente."""
182     cliente = db.query(Cliente).filter(
183         Cliente.id == cliente_id,
184         Cliente.empresa_documento == empresa_documento
185     ).first()
186     if not cliente:
187         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
188     cliente.pontos_fidelidade += pontos
189     db.commit()
190     return {"pontos_totais": cliente.pontos_fidelidade}
191
192
193 @router.get("/fidelidade/ranking")
194 def ranking_fidelidade(
195     empresa_documento: str = Depends(get_empresa_from_user),
196     limite: int = 20,
197     current_user: Usuario = Depends(get_current_user),
198     db: Session = Depends(get_db_salto)
199 ):
200     """Top clientes por pontuacao de fidelidade."""
201     return db.query(Cliente).filter(
202         Cliente.empresa_documento == empresa_documento,
203         Cliente.ativo == True,
204         Cliente.pontos_fidelidade > 0
205     ).order_by(Cliente.pontos_fidelidade.desc()).limit(limite).all()
206

```

```

1 """
2 routers/dashboard.py - Metricas e informacoes consolidadas
3 """
4 from fastapi import APIRouter, Depends, HTTPException, Query
5 from sqlalchemy.orm import Session
6 from sqlalchemy import func
7 from datetime import datetime, timedelta
8
9 from database import get_db_salto, get_db_core
10 from models import Venda, Produto, Cliente, EstoqueMovimentacao
11 from models_core import Usuario
12 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
13
14 router = APIRouter(prefix="/dashboard", tags=["Dashboard"])
15
16
17 @router.get("/")
18 def get_dashboard(
19     empresa_documento: str = Depends(get_empresa_from_user),
20     current_user: Usuario = Depends(get_current_user),
21     db: Session = Depends(get_db_salto)
22 ):
23     hoje = datetime.utcnow().date()
24     ontem = hoje - timedelta(days=1)
25     inicio_mes = hoje.replace(day=1)
26
27     def _date_str(d):
28         return d.isoformat()
29
30     # ---- Vendas hoje ----
31     vendas_hoje_q = db.query(func.sum(Venda.total), func.count(Venda.id), func.avg(Venda.total)).filter(
32         Venda.empresa_documento == empresa_documento,
33         Venda.status == "CONCLUIDA",
34         func.date(Venda.data_venda) == hoje
35     ).first()
36     vendas_hoje = float(vendas_hoje_q[0] or 0)
37     qtd_vendas_hoje = int(vendas_hoje_q[1] or 0)
38     ticket_medio_hoje = float(vendas_hoje_q[2] or 0)
39
40     # ---- Vendas ontem ----
41     vendas_ontem_q = db.query(func.sum(Venda.total)).filter(
42         Venda.empresa_documento == empresa_documento,
43         Venda.status == "CONCLUIDA",
44         func.date(Venda.data_venda) == ontem
45     ).first()
46     vendas_ontem = float(vendas_ontem_q[0] or 0)
47
48     # ---- Vendas mes ----
49     vendas_mes_q = db.query(func.sum(Venda.total)).filter(
50         Venda.empresa_documento == empresa_documento,
51         Venda.status == "CONCLUIDA",
52         func.date(Venda.data_venda) >= inicio_mes
53     ).first()
54     vendas_mes = float(vendas_mes_q[0] or 0)
55
56     # ---- Estoque baixo ----
57     produtos_estoque_baixo = db.query(func.count(Produto.id)).filter(
58         Produto.empresa_documento == empresa_documento,
59         Produto.ativo == True,
60         Produto.estoque_atual <= Produto.estoque_minimo
61     ).scalar() or 0
62
63     # ---- Clientes ativos ----
64     clientes_ativos = db.query(func.count(Cliente.id)).filter(
65         Cliente.empresa_documento == empresa_documento,
66         Cliente.ativo == True
67     ).scalar() or 0
68
69     # ---- Vendedores do dia (who made sales today) ----
70     vendedores_dia = db.query(
71         Venda.vendedor_nome,
72         func.count(Venda.id).label("qtd_vendas"),
73         func.sum(Venda.total).label("total_vendido")
74     ).filter(
75         Venda.empresa_documento == empresa_documento,
76         Venda.status == "CONCLUIDA",
77         func.date(Venda.data_venda) == hoje
78     ).group_by(Venda.vendedor_nome).order_by(func.sum(Venda.total).desc()).all()
79
80     # ---- Ultimas vendas ----
81     ultimas = db.query(Venda).filter(
82         Venda.empresa_documento == empresa_documento,
83         Venda.status == "CONCLUIDA"
84     ).order_by(Venda.data_venda.desc()).limit(10).all()
85
86     return {
87         "vendas_hoje": vendas_hoje,
88         "vendas_ontem": vendas_ontem,
89         "vendas_mes": vendas_mes,
90         "quantidade_vendas_hoje": qtd_vendas_hoje,
91         "ticket_medio_hoje": round(ticket_medio_hoje, 2),
92         "produtos_estoque_baixo": produtos_estoque_baixo,
93         "clientes_ativos": clientes_ativos,
94         "vendedores_do_dia": [
95             {
96                 "nome": v[0] or "Sem vendedor",
97                 "qtd_vendas": int(v[1]),
98                 "total_vendido": float(v[2])
99             }
100             for v in vendedores_dia
101         ],
102         "ultimas_vendas": [
103             {
104                 "id": v.id,
105                 "total": float(v.total),
106                 "vendedor": v.vendedor_nome,
107                 "cliente_id": v.cliente_id,
108                 "data": str(v.data_venda)
109             }
110             for v in ultimas
111         ]
112     }
113

```

```

1 """
2 routers/fornecedores.py - CRUD de fornecedores
3 """
4 from fastapi import APIRouter, Depends, HTTPException, Query
5 from sqlalchemy.orm import Session
6 from typing import Optional
7
8 from database import get_db_salto, get_db_core
9 from models import Fornecedor
10 from schemas import FornecedorCreate, FornecedorResponse
11 from models_core import Usuario
12 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
13
14 router = APIRouter(prefix="/fornecedores", tags=["Fornecedores"])
15
16
17 @router.post("/", response_model=FornecedorResponse)
18 def criar_fornecedor(
19     fornecedor: FornecedorCreate,
20     empresa_documento: str = Depends(get_empresa_from_user),
21     current_user: Usuario = Depends(get_current_user),
22     db: Session = Depends(get_db_salto)
23 ):
24     obj = Fornecedor(empresa_documento=empresa_documento, **fornecedor.model_dump())
25     db.add(obj)
26     db.commit()
27     db.refresh(obj)
28     return obj
29
30
31 @router.get("/", response_model=list[FornecedorResponse])
32 def listar_fornecedores(
33     empresa_documento: str = Depends(get_empresa_from_user),
34     busca: Optional[str] = None,
35     ativo: Optional[bool] = True,
36     current_user: Usuario = Depends(get_current_user),
37     db: Session = Depends(get_db_salto)
38 ):
39     query = db.query(Fornecedor).filter(Fornecedor.empresa_documento == empresa_documento)
40     if ativo is not None:
41         query = query.filter(Fornecedor.ativo == ativo)
42     if busca:
43         query = query.filter(
44             Fornecedor.nome_fantasia.ilike(f"%{busca}%") |
45             Fornecedor.razao_social.ilike(f"%{busca}%") |
46             Fornecedor.cnpj_cpf.ilike(f"%{busca}%")
47         )
48     return query.order_by(Fornecedor.nome_fantasia).all()
49
50
51 @router.get("/{fornecedor_id}", response_model=FornecedorResponse)
52 def get_fornecedor(
53     fornecedor_id: int,
54     empresa_documento: str = Depends(get_empresa_from_user),
55     current_user: Usuario = Depends(get_current_user),
56     db: Session = Depends(get_db_salto)
57 ):
58     obj = db.query(Fornecedor).filter(
59         Fornecedor.id == fornecedor_id,
60         Fornecedor.empresa_documento == empresa_documento
61     ).first()
62     if not obj:
63         raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
64     return obj
65
66
67 @router.put("/{fornecedor_id}", response_model=FornecedorResponse)
68 def atualizar_fornecedor(
69     fornecedor_id: int,
70     dados: FornecedorCreate,
71     empresa_documento: str = Depends(get_empresa_from_user),
72     current_user: Usuario = Depends(get_current_user),
73     db: Session = Depends(get_db_salto)
74 ):
75     obj = db.query(Fornecedor).filter(
76         Fornecedor.id == fornecedor_id,
77         Fornecedor.empresa_documento == empresa_documento
78     ).first()
79     if not obj:
80         raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
81     for key, value in dados.model_dump(exclude_unset=True).items():
82         setattr(obj, key, value)
83     db.commit()
84     db.refresh(obj)
85     return obj
86
87
88 @router.delete("/{fornecedor_id}")
89 def desativar_fornecedor(
90     fornecedor_id: int,
91     empresa_documento: str = Depends(get_empresa_from_user),
92     current_user: Usuario = Depends(get_current_user),
93     db: Session = Depends(get_db_salto)
94 ):
95     obj = db.query(Fornecedor).filter(
96         Fornecedor.id == fornecedor_id,
97         Fornecedor.empresa_documento == empresa_documento
98     ).first()
99     if not obj:
100         raise HTTPException(status_code=404, detail="Fornecedor nao encontrado.")
101     obj.ativo = False
102     db.commit()
103     return {"status": "Fornecedor desativado"}
104

```

```

1 """
2 routers/historico.py - API de Histórico de Vendas
3
4 Endpoints para consultar o historico completo de vendas por empresa.
5 """
6 from fastapi import APIRouter, Depends, HTTPException, Query
7 from sqlalchemy.orm import Session
8 from sqlalchemy import func, and_, or_
9 from datetime import datetime, timedelta
10 from typing import Optional
11
12 from database import get_db_salto, get_db_core
13 from models import HistoricoVenda, Venda
14 from models_core import Usuario
15 from security import get_empresa_from_user, get_current_user, require_role
16
17 router = APIRouter(prefix="/historico", tags=["Historico de Vendas"])
18
19
20 @router.get("/vendas")
21 def listar_historico_vendas(
22     empresa_documento: str = Depends(get_empresa_from_user),
23     data_inicio: Optional[str] = None,
24     data_fim: Optional[str] = None,
25     usuario_id: Optional[int] = None,
26     cliente_id: Optional[int] = None,
27     status: Optional[str] = None,
28     tipo_operacao: Optional[str] = None,
29     busca: Optional[str] = None,
30     limite: int = Query(50, ge=1, le=200),
31     offset: int = Query(0, ge=0),
32     current_user: Usuario = Depends(get_current_user),
33     db: Session = Depends(get_db_salto),
34 ):
35     """Lista o historico de vendas da empresa com filtros."""
36     query = db.query(HistoricoVenda).filter(
37         HistoricoVenda.empresa_documento == empresa_documento
38     )
39
40     if data_inicio:
41         try:
42             di = datetime.strptime(data_inicio, "%Y-%m-%d")
43             query = query.filter(HistoricoVenda.data_criacao >= di)
44         except ValueError:
45             pass
46
47     if data_fim:
48         try:
49             df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
50             query = query.filter(HistoricoVenda.data_criacao < df)
51         except ValueError:
52             pass
53
54     if usuario_id:
55         query = query.filter(HistoricoVenda.usuario_id == usuario_id)
56     if cliente_id:
57         query = query.filter(HistoricoVenda.cliente_id == cliente_id)
58     if status:
59         query = query.filter(HistoricoVenda.status == status)
60     if tipo_operacao:
61         query = query.filter(HistoricoVenda.tipo_operacao == tipo_operacao)
62     if busca:
63         query = query.filter(
64             or_(
65                 HistoricoVenda.numero_pedido.ilike(f"%{busca}%"),
66                 HistoricoVenda.cliente_nome.ilike(f"%{busca}%"),
67                 HistoricoVenda.usuario_nome.ilike(f"%{busca}%"),
68             )
69         )
70
71     total = query.count()
72     registros = query.order_by(HistoricoVenda.data_criacao.desc()).offset(offset).limit(limite).all()
73
74     return {
75         "total": total,
76         "offset": offset,
77         "limite": limite,
78         "registros": [
79             {
80                 "id": r.id,
81                 "venda_id": r.venda_id,
82                 "numero_pedido": r.numero_pedido,
83                 "usuario_nome": r.usuario_nome,
84                 "usuario_papel": r.usuario_papel,
85                 "cliente_nome": r.cliente_nome,
86                 "valor_total": float(r.valor_total) if r.valor_total else 0,
87                 "valor_desconto": float(r.valor_desconto) if r.valor_desconto else 0,
88                 "forma_pagamento": r.forma_pagamento,
89                 "status": r.status,
90                 "tipo_operacao": r.tipo_operacao,
91                 "origem": r.origem,
92                 "data_criacao": str(r.data_criacao),
93                 "data_cancelamento": str(r.data_cancelamento) if r.data_cancelamento else None,
94                 "motivo_cancelamento": r.motivo_cancelamento,
95                 "observacao": r.observacao,
96             }
97             for r in registros
98         ]
99     }
100
101
102 @router.get("/vendas/{historico_id}")
103 def detalhe_historico(
104     historico_id: int,
105     empresa_documento: str = Depends(get_empresa_from_user),
106     current_user: Usuario = Depends(get_current_user),
107     db: Session = Depends(get_db_salto),
108 ):
109     """Retorna os detalhes completos de um registro de historico."""
110     registro = db.query(HistoricoVenda).filter(
111         HistoricoVenda.id == historico_id,
112         HistoricoVenda.empresa_documento == empresa_documento
113     ).first()
114
115     if not registro:
116         raise HTTPException(status_code=404, detail="Registro de historico nao encontrado.")
117
118     import json
119     return {
120         "id": registro.id,
121         "venda_id": registro.venda_id,
122         "numero_pedido": registro.numero_pedido,
123         "empresa_documento": registro.empresa_documento,
124         "usuario": {
125             "id": registro.usuario_id,
126             "nome": registro.usuario_nome,
127             "papel": registro.usuario_papel,
128         },
129         "cliente": {
130             "id": registro.cliente_id,
131             "nome": registro.cliente_nome,
132         },
133         "financeiro": {
134             "valor_total": float(registro.valor_total) if registro.valor_total else 0,

```



```

135         "valor_desconto": float(registro.valor_desconto) if registro.valor_desconto else 0,
136         "valor_recebido": float(registro.valor_recebido) if registro.valor_recebido else 0,
137         "troco": float(registro.troco) if registro.troco else 0,
138         "forma_pagamento": registro.forma_pagamento,
139     },
140     "itens": json.loads(registro.itens_json) if registro.itens_json else [],
141     "status": registro.status,
142     "tipo_operacao": registro.tipo_operacao,
143     "origem": registro.origem,
144     "observacao": registro.observacao,
145     "data_criacao": str(registro.data_criacao),
146     "data_alteracao": str(registro.data_alteracao) if registro.data_alteracao else None,
147     "data_cancelamento": str(registro.data_cancelamento) if registro.data_cancelamento else None,
148     "motivo_cancelamento": registro.motivo_cancelamento,
149     "erro_detalhes": registro.erro_detalhes,
150     "versao": registro.versao,
151 }
152
153
154 @router.get("/estatisticas")
155 def estatisticas_historico(
156     empresa_documento: str = Depends(get_empresa_from_user),
157     data_inicio: Optional[str] = None,
158     data_fim: Optional[str] = None,
159     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
160     db: Session = Depends(get_db_salto),
161 ):
162     """Retorna estatísticas consolidadas do histórico de vendas."""
163     query = db.query(HistoricoVenda).filter(
164         HistoricoVenda.empresa_documento == empresa_documento,
165         HistoricoVenda.tipo_operacao == "VENDA"
166     )
167
168     if data_inicio:
169         try:
170             di = datetime.strptime(data_inicio, "%Y-%m-%d")
171             query = query.filter(HistoricoVenda.data_criacao >= di)
172         except ValueError:
173             pass
174
175     if data_fim:
176         try:
177             df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
178             query = query.filter(HistoricoVenda.data_criacao < df)
179         except ValueError:
180             pass
181
182     total_vendas = query.filter(HistoricoVenda.status == "CONCLUIDA").count()
183     total_canceladas = query.filter(HistoricoVenda.status == "CANCELADA").count()
184
185     faturamento = db.query(func.sum(HistoricoVenda.valor_total)).filter(
186         HistoricoVenda.empresa_documento == empresa_documento,
187         HistoricoVenda.status == "CONCLUIDA"
188     ).scalar() or 0
189
190     ticket_medio = faturamento / total_vendas if total_vendas > 0 else 0
191
192     return {
193         "total_vendas": total_vendas,
194         "total_canceladas": total_canceladas,
195         "faturamento_total": float(faturamento),
196         "ticket_medio": round(float(ticket_medio), 2),
197         "taxa_cancelamento": round(total_canceladas / (total_vendas + total_canceladas) * 100, 2) if (total_vendas + total_canceladas) > 0 else 0,
198     }
199

```

```

1 """
2 routers/login_operacional.py - Login de colaborador (vendedor/operador) do SaltoGestão
3 Endpoint: POST /api/auth/login
4 """
5 from datetime import datetime
6 from fastapi import APIRouter, Depends, HTTPException
7 from sqlalchemy.orm import Session
8 from pydantic import BaseModel
9
10 from database import get_db, SessionLocal
11 from models_core import Usuario, Empresa, AdminCorumba
12 from security import hash_senha, verificar_senha, criar_token, get_current_user
13
14 router = APIRouter(tags=["Autenticação Operacional"])
15
16
17 class LoginOperacionalRequest(BaseModel):
18     """Login com username + senha OU email + senha."""
19     username: str | None = None
20     email: str | None = None
21     senha: str
22
23
24 class LoginResponse(BaseModel):
25     access_token: str = ""
26     token_type: str = "bearer"
27     usuario: dict
28
29
30 @router.post("/login")
31 def login_operacional(req: LoginOperacionalRequest, db: Session = Depends(get_db)):
32     """
33     Login de colaborador do SaltoGestão.
34     Aceita username ou email.
35     Também aceita login de admin Corumbá como fallback.
36     """
37     user = None
38     user_type = None
39
40     # 1) Tenta buscar por username no banco de usuários
41     if req.username:
42         user = db.query(Usuario).filter(Usuario.username == req.username).first()
43         if user:
44             user_type = "colaborador"
45
46     # 2) Tenta buscar por email no banco de usuários
47     if not user and req.email:
48         user = db.query(Usuario).filter(Usuario.email == req.email).first()
49         if user:
50             user_type = "colaborador"
51
52     # 3) Fallback: tenta login de admin Corumbá
53     if not user and req.email:
54         user = db.query(AdminCorumba).filter(AdminCorumba.email == req.email).first()
55         if user:
56             user_type = "admin"
57
58     if not user:
59         raise HTTPException(status_code=401, detail="Usuário não encontrado.")
60
61     # Verifica senha
62     senha_ok = verificar_senha(req.senha, user.senha_hash)
63     if not senha_ok:
64         raise HTTPException(status_code=401, detail="Senha incorreta.")
65
66     if not user.ativo:
67         raise HTTPException(status_code=403, detail="Conta desativada.")
68
69     # Gera token
70     if user_type == "colaborador":
71         token = criar_token({
72             "sub": user.username,
73             "username": user.username,
74             "nome": user.nome,
75             "papal": user.papel,
76             "empresa_documento": user.empresa_documento,
77             "tipo": "colaborador",
78             "usuario_id": user.id,
79         })
80         return {
81             "access_token": token,
82             "token_type": "bearer",
83             "usuario": {
84                 "id": user.id,
85                 "nome": user.nome,
86                 "username": user.username,
87                 "email": getattr(user, 'email', ''),
88                 "papal": user.papel,
89                 "empresa_documento": user.empresa_documento,
90                 "tipo": "colaborador",
91             }
92         }
93     else: # admin
94         token = criar_token({
95             "sub": user.email,
96             "username": user.email,
97             "nome": user.nome,
98             "papal": "ADMIN",
99             "empresa_documento": "00.000.000/0001-00",
100             "tipo": "admin",
101             "admin_id": user.id,
102         })
103         return {
104             "access_token": token,
105             "token_type": "bearer",
106             "usuario": {
107                 "id": user.id,
108                 "nome": user.nome,
109                 "email": user.email,
110                 "papal": "ADMIN",
111                 "tipo": "admin",
112             }
113         }
114

```

```

1 """
2 routers/painel.py - Painel do cliente: produtos, assinaturas, status
3 """
4 from fastapi import APIRouter, Depends, HTTPException
5 from sqlalchemy.orm import Session
6
7 from database import get_db
8 from models_core import Cliente, Produto, Assinatura
9 from security import get_current_cliente
10
11 router = APIRouter(prefix="/painel", tags=["Painel do Cliente"])
12
13
14 @router.get("/produtos")
15 def listar_produtos(
16     cliente: Cliente = Depends(get_current_cliente),
17     db: Session = Depends(get_db)
18 ):
19     """
20     Lista todos os produtos do catálogo com o status do cliente:
21     - adquirido: produto que o cliente já tem ativo
22     - bloqueado: produto que o cliente não adquiriu ou está inadimplente
23     """
24     produtos = db.query(Produto).filter(Produto.ativo == True).order_by(Produto.ordem).all()
25
26     # Buscar assinaturas do cliente
27     assinaturas = db.query(Assinatura).filter(
28         Assinatura.cliente_id == cliente.id
29     ).all()
30
31     adquiridos = {a.produto_id: a for a in assinaturas if a.status == "ATIVA"}
32     inadimplentes = {a.produto_id: a for a in assinaturas if a.status == "INADIMIENTE"}
33
34     resultado = []
35     for p in produtos:
36         if p.id in adquiridos:
37             status = "adquirido"
38             assinatura = adquiridos[p.id]
39         elif p.id in inadimplentes:
40             status = "inadimplente"
41             assinatura = inadimplentes[p.id]
42         else:
43             status = "bloqueado"
44             assinatura = None
45
46         resultado.append({
47             "id": p.id,
48             "nome": p.nome,
49             "descricao": p.descricao,
50             "preco_mensal": float(p.preco_mensal) if p.preco_mensal else 0,
51             "preco_anual": float(p.preco_anual) if p.preco_anual else None,
52             "icone": p.icone,
53             "status": status,
54             "data_aquisicao": str(assinatura.data_aquisicao) if assinatura else None,
55             "data_expiracao": str(assinatura.data_expiracao) if assinatura and assinatura.data_expiracao else None,
56         })
57
58     return resultado
59
60
61 @router.get("/produto/{produto_id}")
62 def detalhe_produto(
63     produto_id: int,
64     cliente: Cliente = Depends(get_current_cliente),
65     db: Session = Depends(get_db)
66 ):
67     """
68     Detalhes de um produto específico.
69     Se o cliente não adquiriu, mostra info + botão de compra.
70     """
71     produto = db.query(Produto).filter(Produto.id == produto_id, Produto.ativo == True).first()
72     if not produto:
73         raise HTTPException(status_code=404, detail="Produto não encontrado.")
74
75     # Verificar se cliente tem assinatura
76     assinatura = db.query(Assinatura).filter(
77         Assinatura.cliente_id == cliente.id,
78         Assinatura.produto_id == produto_id,
79     ).first()
80
81     if assinatura and assinatura.status == "ATIVA":
82         status = "adquirido"
83     elif assinatura and assinatura.status == "INADIMIENTE":
84         status = "inadimplente"
85     else:
86         status = "bloqueado"
87
88     return {
89         "id": produto.id,
90         "nome": produto.nome,
91         "descricao": produto.descricao,
92         "preco_mensal": float(produto.preco_mensal) if produto.preco_mensal else 0,
93         "preco_anual": float(produto.preco_anual) if produto.preco_anual else None,
94         "icone": produto.icone,
95         "status": status,
96         "data_aquisicao": str(assinatura.data_aquisicao) if assinatura else None,
97         "data_expiracao": str(assinatura.data_expiracao) if assinatura and assinatura.data_expiracao else None,
98     }
99
100
101 @router.get("/minhas-assinaturas")
102 def minhas_assinaturas(
103     cliente: Cliente = Depends(get_current_cliente),
104     db: Session = Depends(get_db)
105 ):
106     """Lista todas as assinaturas do cliente."""
107     assinaturas = db.query(Assinatura).filter(
108         Assinatura.cliente_id == cliente.id
109     ).all()
110
111     resultado = []
112     for a in assinaturas:
113         produto = db.query(Produto).filter(Produto.id == a.produto_id).first()
114         resultado.append({
115             "id": a.id,
116             "produto_id": a.produto_id,
117             "produto_nome": produto.nome if produto else "Desconhecido",
118             "status": a.status,
119             "valor_mensal": float(a.valor_mensal) if a.valor_mensal else 0,
120             "data_aquisicao": str(a.data_aquisicao),
121             "data_expiracao": str(a.data_expiracao) if a.data_expiracao else None,
122         })
123
124     return resultado
125

```

```

1 """
2 routers/produtos.py - CRUD de produtos + gestao de estoque
3 Multi-tenant: todas as operacoes filtram por empresa_documento (via JWT)
4 """
5 from fastapi import APIRouter, Depends, HTTPException, Query
6 from sqlalchemy.orm import Session
7 from sqlalchemy import func
8 from typing import Optional
9
10 from database import get_db_salto, get_db_core
11 from models import Produto, EstoqueMovimentacao
12 from schemas import ProdutoCreate, ProdutoUpdate, ProdutoResponse, EstoqueMovimentacaoCreate
13 from models_core import Usuario
14 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
15
16 router = APIRouter(prefix="/produtos", tags=["Produtos"])
17
18
19 def get_empresa(current_user):
20     return current_user.empresa_documento
21
22
23 @router.post("/", response_model=ProdutoResponse)
24 def criar_produto(
25     prod: ProdutoCreate,
26     empresa_documento: str = Depends(get_empresa_from_user),
27     current_user: Usuario = Depends(get_current_user),
28     db: Session = Depends(get_db_salto)
29 ):
30     produto = Produto(empresa_documento=empresa_documento, **prod.model_dump())
31     db.add(produto)
32     db.commit()
33     db.refresh(produto)
34
35     # Registrar movimentacao de estoque inicial se > 0
36     if produto.estoque_atual > 0:
37         mov = EstoqueMovimentacao(
38             empresa_documento=empresa_documento,
39             produto_id=produto.id,
40             tipo="ENTRADA",
41             quantidade=produto.estoque_atual,
42             motivo="Cadastro inicial",
43             usuario=current_user.nome
44         )
45         db.add(mov)
46         db.commit()
47     return produto
48
49
50 @router.get("/", response_model=list[ProdutoResponse])
51 def listar_produtos(
52     empresa_documento: str = Depends(get_empresa_from_user),
53     busca: Optional[str] = None,
54     ativo: Optional[bool] = True,
55     estoque_baixo: Optional[bool] = None,
56     current_user: Usuario = Depends(get_current_user),
57     db: Session = Depends(get_db_salto)
58 ):
59     query = db.query(Produto).filter(Produto.empresa_documento == empresa_documento)
60     if ativo is not None:
61         query = query.filter(Produto.ativo == ativo)
62     if busca:
63         query = query.filter(
64             Produto.nome.ilike(f"%{busca}%") |
65             Produto.sku.ilike(f"%{busca}%") |
66             Produto.codigo_barras.ilike(f"%{busca}%")
67         )
68     if estoque_baixo:
69         query = query.filter(Produto.estoque_atual <= Produto.estoque_minimo)
70     return query.order_by(Produto.nome).all()
71
72
73 @router.get("/{produto_id}", response_model=ProdutoResponse)
74 def get_produto(
75     produto_id: int,
76     empresa_documento: str = Depends(get_empresa_from_user),
77     current_user: Usuario = Depends(get_current_user),
78     db: Session = Depends(get_db_salto)
79 ):
80     produto = db.query(Produto).filter(
81         Produto.id == produto_id,
82         Produto.empresa_documento == empresa_documento
83     ).first()
84     if not produto:
85         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
86     return produto
87
88
89 @router.put("/{produto_id}", response_model=ProdutoResponse)
90 def atualizar_produto(
91     produto_id: int,
92     dados: ProdutoUpdate,
93     empresa_documento: str = Depends(get_empresa_from_user),
94     current_user: Usuario = Depends(get_current_user),
95     db: Session = Depends(get_db_salto)
96 ):
97     produto = db.query(Produto).filter(
98         Produto.id == produto_id,
99         Produto.empresa_documento == empresa_documento
100     ).first()
101     if not produto:
102         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
103
104     update_data = dados.model_dump(exclude_unset=True)
105     for key, value in update_data.items():
106         setattr(produto, key, value)
107     db.commit()
108     db.refresh(produto)
109     return produto
110
111
112 @router.delete("/{produto_id}")
113 def desativar_produto(
114     produto_id: int,
115     empresa_documento: str = Depends(get_empresa_from_user),
116     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
117     db: Session = Depends(get_db_salto)
118 ):
119     produto = db.query(Produto).filter(
120         Produto.id == produto_id,
121         Produto.empresa_documento == empresa_documento
122     ).first()
123     if not produto:
124         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
125     produto.ativo = False
126     db.commit()
127     return {"status": "Produto desativado"}
128
129
130 # =====
131 # MOVIMENTACAO DE ESTOQUE
132 # =====
133
134 @router.post("/movimentacao")

```

```

135 def movimentar_estoque(
136     mov: EstoqueMovimentacaoCreate,
137     empresa_documento: str = Depends(get_empresa_from_user),
138     current_user: Usuario = Depends(get_current_user),
139     db: Session = Depends(get_db_salto)
140 ):
141     produto = db.query(Produto).filter(
142         Produto.id == mov.produto_id,
143         Produto.empresa_documento == empresa_documento
144     ).first()
145     if not produto:
146         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
147
148     if mov.tipo == "SAIDA" and produto.estoque_atual < mov.quantidade:
149         raise HTTPException(status_code=400, detail="Estoque insuficiente.")
150
151     # Atualizar estoque
152     if mov.tipo == "ENTRADA":
153         produto.estoque_atual += mov.quantidade
154     elif mov.tipo == "SAIDA":
155         produto.estoque_atual -= mov.quantidade
156     elif mov.tipo == "AJUSTE":
157         produto.estoque_atual = mov.quantidade # Ajuste define o valor absoluto
158
159     # Registrar movimentacao
160     reg = EstoqueMovimentacao(
161         empresa_documento=empresa_documento,
162         produto_id=mov.produto_id,
163         tipo=mov.tipo,
164         quantidade=mov.quantidade,
165         motivo=mov.motivo,
166         usuario=current_user.nome
167     )
168     db.add(reg)
169     db.commit()
170     return {"status": "Movimentacao registrada", "estoque_atual": float(produto.estoque_atual)}
171
172
173 @router.get("/{produto_id}/movimentacoes")
174 def historico_movimentacoes(
175     produto_id: int,
176     empresa_documento: str = Depends(get_empresa_from_user),
177     current_user: Usuario = Depends(get_current_user),
178     db: Session = Depends(get_db_salto)
179 ):
180     return db.query(EstoqueMovimentacao).filter(
181         EstoqueMovimentacao.produto_id == produto_id,
182         EstoqueMovimentacao.empresa_documento == empresa_documento
183     ).order_by(EstoqueMovimentacao.data_movimentacao.desc()).limit(50).all()
184
185
186 @router.get("/estoque/alertas")
187 def alertas_estoque(
188     empresa_documento: str = Depends(get_empresa_from_user),
189     current_user: Usuario = Depends(get_current_user),
190     db: Session = Depends(get_db_salto)
191 ):
192     """Retorna produtos com estoque abaixo do minimo."""
193     return db.query(Produto).filter(
194         Produto.empresa_documento == empresa_documento,
195         Produto.ativo == True,
196         Produto.estoque_atual <= Produto.estoque_minimo
197     ).all()
198

```

```

1 """
2 routers/usuarios.py - CRUD de colaboradores/equipe
3 Permissões: ADMIN pode criar/editar/remover qualquer usuario.
4             GERENTE pode criar apenas VENDEDOR e CAIXA.
5 """
6 from fastapi import APIRouter, Depends, HTTPException
7 from sqlalchemy.orm import Session
8 from typing import Optional
9
10 from database import get_db_core
11 from models_core import Usuario
12 from schemas import UsuarioCreate, UsuarioUpdate, UsuarioResponse
13 from security import hash_senha, require_role
14
15 router = APIRouter(prefix="/usuarios", tags=["Colaboradores"])
16
17
18 @router.post("/", response_model=UsuarioResponse)
19 def criar_usuario(
20     usuario: UsuarioCreate,
21     empresa_documento: str,
22     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
23     db: Session = Depends(get_db_core)
24 ):
25     if current_user.empresa_documento != empresa_documento:
26         raise HTTPException(status_code=403, detail="So pode criar usuarios da sua empresa.")
27
28     # Verificar se username ja existe na empresa
29     existing = db.query(Usuario).filter(
30         Usuario.username == usuario.username,
31         Usuario.empresa_documento == empresa_documento
32     ).first()
33     if existing:
34         raise HTTPException(status_code=400, detail="Username ja existe nesta empresa.")
35
36     # GERENTE so pode criar VENDEDOR e CAIXA
37     if current_user.papel == "GERENTE" and usuario.papel in ("ADMIN", "GERENTE"):
38         raise HTTPException(status_code=403, detail="Gerente so pode criar vendedores e operadores de caixa.")
39
40     user = Usuario(
41         empresa_documento=empresa_documento,
42         nome=usuario.nome,
43         username=usuario.username,
44         senha_hash=hash_senha(usuario.senha),
45         papel=usuario.papel,
46         codigo=usuario.codigo,
47         ativo=True
48     )
49     db.add(user)
50     db.commit()
51     db.refresh(user)
52     return user
53
54
55 @router.get("/", response_model=list[UsuarioResponse])
56 def listar_usuarios(
57     empresa_documento: str,
58     ativo: Optional[bool] = None,
59     papel: Optional[str] = None,
60     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
61     db: Session = Depends(get_db_core)
62 ):
63     query = db.query(Usuario).filter(Usuario.empresa_documento == empresa_documento)
64     if ativo is not None:
65         query = query.filter(Usuario.ativo == ativo)
66     if papel:
67         query = query.filter(Usuario.papel == papel)
68     return query.order_by(Usuario.nome).all()
69
70
71 @router.get("/{usuario_id}", response_model=UsuarioResponse)
72 def get_usuario(
73     usuario_id: int,
74     empresa_documento: str,
75     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
76     db: Session = Depends(get_db_core)
77 ):
78     user = db.query(Usuario).filter(
79         Usuario.id == usuario_id,
80         Usuario.empresa_documento == empresa_documento
81     ).first()
82     if not user:
83         raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
84     return user
85
86
87 @router.put("/{usuario_id}", response_model=UsuarioResponse)
88 def atualizar_usuario(
89     usuario_id: int,
90     dados: UsuarioUpdate,
91     empresa_documento: str,
92     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
93     db: Session = Depends(get_db_core)
94 ):
95     user = db.query(Usuario).filter(
96         Usuario.id == usuario_id,
97         Usuario.empresa_documento == empresa_documento
98     ).first()
99     if not user:
100         raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
101
102     update_data = dados.model_dump(exclude_unset=True)
103     if "senha" in update_data:
104         update_data["senha_hash"] = hash_senha(update_data.pop("senha"))
105
106     for key, value in update_data.items():
107         setattr(user, key, value)
108
109     db.commit()
110     db.refresh(user)
111     return user
112
113
114 @router.delete("/{usuario_id}")
115 def desativar_usuario(
116     usuario_id: int,
117     empresa_documento: str,
118     current_user: Usuario = Depends(require_role("ADMIN")),
119     db: Session = Depends(get_db_core)
120 ):
121     user = db.query(Usuario).filter(
122         Usuario.id == usuario_id,
123         Usuario.empresa_documento == empresa_documento
124     ).first()
125     if not user:
126         raise HTTPException(status_code=404, detail="Usuario nao encontrado.")
127     if user.id == current_user.id:
128         raise HTTPException(status_code=400, detail="Voce nao pode desativar a si mesmo.")
129
130     user.ativo = False
131     db.commit()
132     return {"status": "Usuario desativado"}
133

```

```

1 """
2 routers/vendas.py - Registro de vendas (PDV) + cancelamento + descontos
3 Multi-tenant com filtros em todas as queries.
4 Fluxo PDV:
5 1. Cliente busca produtos por nome/sku
6 2. Adiciona ao carrinho
7 3. Escolhe vendedor (por código)
8 4. Aplica desconto (se houver, exige senha admin)
9 5. Registra pagamento
10 6. Finaliza -> gera Venda + VendaItens + atualiza estoque + pontos cliente + transação caixa
11 """
12 from fastapi import APIRouter, Depends, HTTPException, Query
13 from sqlalchemy.orm import Session
14 from sqlalchemy import func
15 from datetime import datetime, timedelta
16 from typing import Optional
17
18 from database import get_db_salto, get_db_core
19 from models import Venda, VendaItem, Produto, Cliente, CaixaSessao, CaixaTransacao
20 from schemas import VendaCreate, VendaResponse, DescontoVendaRequest, VendaCancelRequest
21 from models_core import Usuario
22 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
23 from historico import registrar_venda, registrar_cancelamento, registrar_erro_venda
24
25 router = APIRouter(prefix="/vendas", tags=["Vendas"])
26
27
28 @router.post("/", response_model=dict)
29 def registrar_venda(
30     venda: VendaCreate,
31     empresa_documento: str = Depends(get_empresa_from_user),
32     current_user: Usuario = Depends(get_current_user),
33     db: Session = Depends(get_db_salto)
34 ):
35     """
36     Registra uma venda completa com itens, atualiza estoque e registra na caixa.
37     """
38     # 1. Verificar se caixa está aberto
39     caixa_aberto = db.query(CaixaSessao).filter(
40         CaixaSessao.empresa_documento == empresa_documento,
41         CaixaSessao.status == "ABERTO"
42     ).first()
43     if not caixa_aberto:
44         raise HTTPException(status_code=400, detail="Caixa fechado. Abra o caixa antes de vender.")
45
46     # 2. Calcular total e validar itens
47     total = 0
48     itens_venda = []
49     for item in venda.itens:
50         produto = db.query(Produto).filter(
51             Produto.id == item.produto_id,
52             Produto.empresa_documento == empresa_documento,
53             Produto.ativo == True
54         ).first()
55         if not produto:
56             raise HTTPException(status_code=404, detail=f"Produto ID {item.produto_id} não encontrado ou inativo.")
57         if produto.estoque_atual < item.quantidade:
58             raise HTTPException(status_code=400, detail=f"Estoque insuficiente para {produto.nome}. Disponível: {float(produto.estoque_atual)}")
59
60         subtotal = float(produto.preco_venda) * float(item.quantidade)
61         itens_venda.append({
62             "produto": produto,
63             "quantidade": item.quantidade,
64             "preco_unitario": float(produto.preco_venda),
65             "subtotal": subtotal
66         })
67         total += subtotal
68
69     # 3. Determinar forma de pagamento principal (primeira forma listada)
70     forma_pagamento = venda.pagamentos[0].forma if venda.pagamentos else "DESCONHECIDO"
71     valor_recebido = sum(p.valor for p in venda.pagamentos)
72     troco = max(0, valor_recebido - total)
73
74     # 4. Buscar dados do vendedor
75     vendedor_nome = None
76     if venda.vendedor_id:
77         # Buscar nome do vendedor pelo id (tabela usuarios no core)
78         from database import get_db_core
79         # Nota: usamos o id como referência
80         vendedor_nome = venda.vendedor_nome # O frontend já envia o nome
81
82     # 5. Criar a venda
83     nova_venda = Venda(
84         empresa_documento=empresa_documento,
85         cliente_id=venda.cliente_id,
86         vendedor_id=venda.vendedor_id,
87         vendedor_nome=vendedor_nome,
88         operador=current_user.nome,
89         total=total,
90         desconto=0,
91         valor_recebido=valor_recebido,
92         troco=troco,
93         forma_pagamento=forma_pagamento,
94         status="CONCLUIDA",
95         observacao=venda.observacao,
96         data_venda=datetime.utcnow()
97     )
98     db.add(nova_venda)
99     db.flush() # Para obter o ID
100
101     # 6. Criar itens da venda e atualizar estoque
102     for iv in itens_venda:
103         item_db = VendaItem(
104             empresa_documento=empresa_documento,
105             venda_id=nova_venda.id,
106             produto_id=iv["produto"].id,
107             produto_nome=iv["produto"].nome,
108             quantidade=iv["quantidade"],
109             preco_unitario=iv["preco_unitario"],
110             subtotal=iv["subtotal"]
111         )
112         db.add(item_db)
113         # Baixar estoque
114         iv["produto"].estoque_atual -= iv["quantidade"]
115
116     # 7. Registrar transação no caixa
117     transacao = CaixaTransacao(
118         empresa_documento=empresa_documento,
119         sessao_id=caixa_aberto.id,
120         tipo="VENDA",
121         valor=total,
122         descricao=f"Venda #{nova_venda.id}",
123         venda_id=nova_venda.id,
124         operador=current_user.nome
125     )
126     db.add(transacao)
127
128     # 8. Atualizar totais da sessão do caixa
129     caixa_aberto.total_vendas += total
130
131     # 9. Atualizar pontos do cliente (fidelidade: 1 ponto por R$ 1 gasto)
132     if venda.cliente_id:
133         cliente = db.query(Cliente).filter(
134             Cliente.id == venda.cliente_id,

```

```

135         Cliente.empresa_documento == empresa_documento
136     ).first()
137     if cliente:
138         pontos_ganhos = int(total) # 1 ponto por real
139         cliente.pontos_fidelidade += pontos_ganhos
140
141     db.commit()
142
143     # 10. Registrar no historico de vendas
144     try:
145         itens_para_historico = [
146             {
147                 "produto_id": iv["produto"].id,
148                 "produto_nome": iv["produto"].nome,
149                 "quantidade": float(iv["quantidade"]),
150                 "preco_unitario": float(iv["preco_unitario"]),
151                 "subtotal": float(iv["subtotal"])
152             }
153             for iv in itens_venda
154         ]
155
156         registrar_venda(
157             db=db,
158             empresa_documento=empresa_documento,
159             usuario_id=current_user.id,
160             usuario_nome=current_user.nome,
161             usuario_papel=current_user.papel,
162             venda_id=nova_venda.id,
163             itens=itens_para_historico,
164             pagamentos=[{"forma": p.forma, "valor": float(p.valor)} for p in venda.pagamentos],
165             cliente_id=venda.cliente_id,
166             cliente_nome=cliente.nome if venda.cliente_id and cliente else None,
167             valor_total=total,
168             valor_desconto=0,
169             valor_recebido=valor_recebido,
170             troco=troco,
171             forma_pagamento=forma_pagamento,
172             origem="PDV",
173             observacao=venda.observacao,
174         )
175         db.commit()
176     except Exception as e:
177         # Se falhar o historico, nao bloqueia a venda
178         import logging
179         logging.error(f"Erro ao registrar historico de venda: {e}")
180
181     return {
182         "status": "Venda registrada com sucesso",
183         "venda_id": nova_venda.id,
184         "total": float(total),
185         "troco": float(troco),
186         "forma_pagamento": forma_pagamento
187     }
188
189 @router.get("/", response_model=list[dict])
190 def listar_vendas():
191     empresa_documento: str = Depends(get_empresa_from_user),
192     data_inicio: Optional[str] = None,
193     data_fim: Optional[str] = None,
194     vendedor_id: Optional[int] = None,
195     cliente_id: Optional[int] = None,
196     status: Optional[str] = None,
197     limite: int = Query(50, ge=1, le=200),
198     offset: int = Query(0, ge=0),
199     current_user: Usuario = Depends(get_current_user),
200     db: Session = Depends(get_db_salto)
201 ):
202     query = db.query(Venda).filter(Venda.empresa_documento == empresa_documento)
203
204     if data_inicio:
205         try:
206             di = datetime.strptime(data_inicio, "%Y-%m-%d")
207             query = query.filter(Venda.data_venda >= di)
208         except ValueError:
209             pass
210
211     if data_fim:
212         try:
213             df = datetime.strptime(data_fim, "%Y-%m-%d") + timedelta(days=1)
214             query = query.filter(Venda.data_venda < df)
215         except ValueError:
216             pass
217
218     if vendedor_id:
219         query = query.filter(Venda.vendedor_id == vendedor_id)
220     if cliente_id:
221         query = query.filter(Venda.cliente_id == cliente_id)
222     if status:
223         query = query.filter(Venda.status == status)
224
225     vendas = query.order_by(Venda.data_venda.desc()).offset(offset).limit(limite).all()
226     return [
227         {
228             "id": v.id,
229             "cliente_id": v.cliente_id,
230             "vendedor_nome": v.vendedor_nome,
231             "operador": v.operador,
232             "total": float(v.total),
233             "desconto": float(v.desconto),
234             "forma_pagamento": v.forma_pagamento,
235             "status": v.status,
236             "data_venda": str(v.data_venda),
237             "quantidade_itens": len(v.itens)
238         }
239         for v in vendas
240     ]
241
242
243
244 @router.get("/{venda_id}")
245 def get_venda(
246     venda_id: int,
247     empresa_documento: str = Depends(get_empresa_from_user),
248     current_user: Usuario = Depends(get_current_user),
249     db: Session = Depends(get_db_salto)
250 ):
251     venda = db.query(Venda).filter(
252         Venda.id == venda_id,
253         Venda.empresa_documento == empresa_documento
254     ).first()
255     if not venda:
256         raise HTTPException(status_code=404, detail="Venda nao encontrada.")
257
258     return {
259         "id": venda.id,
260         "cliente_id": venda.cliente_id,
261         "vendedor_id": venda.vendedor_id,
262         "vendedor_nome": venda.vendedor_nome,
263         "operador": venda.operador,
264         "total": float(venda.total),
265         "desconto": float(venda.desconto),
266         "valor_recebido": float(venda.valor_recebido),
267         "troco": float(venda.troco),
268         "forma_pagamento": venda.forma_pagamento,
269         "status": venda.status,
270         "observacao": venda.observacao,
271         "data_venda": str(venda.data_venda),
272         "itens": [

```



```

273         {
274             "produto_id": item.produto_id,
275             "produto_nome": item.produto_nome,
276             "quantidade": float(item.quantidade),
277             "preco_unitario": float(item.preco_unitario),
278             "subtotal": float(item.subtotal)
279         }
280     for item in venda.itens
281 }
282 }
283
284
285 @router.post("/{venda_id}/cancelar")
286 def cancelar_venda(
287     venda_id: int,
288     req: VendaCancelRequest,
289     empresa_documento: str = Depends(get_empresa_from_user),
290     current_user: Usuario = Depends(get_current_user),
291     db_core: Session = Depends(get_db_core),
292     db: Session = Depends(get_db_salto)
293 ):
294     """
295     Cancela uma venda. Exige senha de admin para autorizar.
296     Devolve estoque, remove pontos do cliente e registra transacao de cancelamento no caixa.
297     """
298     # 1. Verificar senha admin
299     admin_user = db_core.query(Usuario).filter(
300         Usuario.empresa_documento == empresa_documento,
301         Usuario.papel.in_({"ADMIN", "GERENTE"}),
302         Usuario.ativo == True
303     ).all()
304
305     senha_valida = False
306     for u in admin_user:
307         if verificar_senha(req.senha_admin, u.senha_hash):
308             senha_valida = True
309             break
310
311     if not senha_valida:
312         raise HTTPException(status_code=403, detail="Senha de administrador invalida.")
313
314     # 2. Buscar venda
315     venda = db.query(Venda).filter(
316         Venda.id == venda_id,
317         Venda.empresa_documento == empresa_documento,
318         Venda.status == "CONCLUIDA"
319     ).first()
320     if not venda:
321         raise HTTPException(status_code=404, detail="Venda nao encontrada ou ja cancelada.")
322
323     # 3. Devolver estoque
324     for item in venda.itens:
325         produto = db.query(Produto).filter(
326             Produto.id == item.produto_id,
327             Produto.empresa_documento == empresa_documento
328         ).first()
329         if produto:
330             produto.estoque_antigo = produto.estoque_atual
331             produto.estoque_atual += item.quantidade
332
333     # 4. Remover pontos do cliente
334     if venda.cliente_id:
335         cliente = db.query(Cliente).filter(
336             Cliente.id == venda.cliente_id,
337             Cliente.empresa_documento == empresa_documento
338         ).first()
339         if cliente:
340             pontos_perdidos = int(venda.total)
341             cliente.pontos_fidelidade = max(0, cliente.pontos_fidelidade - pontos_perdidos)
342
343     # 5. Registrar transacao de cancelamento no caixa
344     caixa_aberto = db.query(CaixaSessao).filter(
345         CaixaSessao.empresa_documento == empresa_documento,
346         CaixaSessao.status == "ABERTO"
347     ).first()
348     if caixa_aberto:
349         transacao = CaixaTransacao(
350             empresa_documento=empresa_documento,
351             sessao_id=caixa_aberto.id,
352             tipo="CANCELAMENTO",
353             valor=venda.total,
354             descricao=f"Cancelamento Venda #{venda_id}",
355             venda_id=venda_id,
356             operador=current_user.nome,
357             autorizado_por="VERIFICADO_SENHA_ADMIN"
358         )
359         db.add(transacao)
360         caixa_aberto.total_vendas -= venda.total
361
362     # 6. Atualizar status
363     venda.status = "CANCELADA"
364     venda.observacao = f"{venda.observacao or ''} [CANCELADA: {req.motivo or 'Sem motivo informado'}]"
365
366     # 7. Registrar cancelamento no historico
367     try:
368         registrar_cancelamento(
369             db=db,
370             empresa_documento=empresa_documento,
371             venda_id=venda_id,
372             usuario_id=current_user.id,
373             usuario_nome=current_user.nome,
374             motivo=req.motivo,
375         )
376     except Exception as e:
377         import logging
378         logging.error(f"Erro ao registrar cancelamento no historico: {e}")
379
380     db.commit()
381
382     return {"status": "Venda cancelada", "venda_id": venda_id, "valor_devolvido": float(venda.total)}
383
384
385 @router.post("/{venda_id}/desconto")
386 def aplicar_desconto(
387     venda_id: int,
388     req: DescontoVendaRequest,
389     empresa_documento: str = Depends(get_empresa_from_user),
390     current_user: Usuario = Depends(get_current_user),
391     db_core: Session = Depends(get_db_core),
392     db: Session = Depends(get_db_salto)
393 ):
394     """
395     Aplica desconto em uma venda ja concluida. Exige senha admin.
396     Registra transacao de desconto no caixa.
397     """
398     # 1. Verificar senha admin
399     admin_user = db_core.query(Usuario).filter(
400         Usuario.empresa_documento == empresa_documento,
401         Usuario.papel.in_({"ADMIN", "GERENTE"}),
402         Usuario.ativo == True
403     ).all()
404
405     senha_valida = False
406     for u in admin_user:
407         if verificar_senha(req.senha_admin, u.senha_hash):
408             senha_valida = True
409             break
410

```

```
411 if not senha_valida:
412     raise HTTPException(status_code=403, detail="Senha de administrador invalida.")
413
414 # 2. Buscar venda
415 venda = db.query(Venda).filter(
416     Venda.id == venda_id,
417     Venda.empresa_documento == empresa_documento,
418     Venda.status == "CONCLUIDA"
419 ).first()
420 if not venda:
421     raise HTTPException(status_code=404, detail="Venda nao encontrada.")
422
423 if req.valor_desconto >= float(venda.total):
424     raise HTTPException(status_code=400, detail="Desconto nao pode ser maior ou igual ao total da venda.")
425
426 venda.desconto = req.valor_desconto
427 venda.total = float(venda.total) - req.valor_desconto
428
429 # Registrar no caixa
430 caixa_aberto = db.query(CaixaSessao).filter(
431     CaixaSessao.empresa_documento == empresa_documento,
432     CaixaSessao.status == "ABERTO"
433 ).first()
434 if caixa_aberto:
435     transacao = CaixaTransacao(
436         empresa_documento=empresa_documento,
437         sessao_id=caixa_aberto.id,
438         tipo="DESCONTO",
439         valor=req.valor_desconto,
440         descricao=f"Desconto Venda #{venda_id}: {req.motivo or 'Sem motivo'}",
441         venda_id=venda_id,
442         operador=current_user.nome,
443         autorizado_por="VERIFICADO_SENHA_ADMIN"
444     )
445     db.add(transacao)
446
447 db.commit()
448 return {"status": "Desconto aplicado", "novo_total": float(venda.total), "desconto": float(venda.desconto)}
449
```

```

1 """
2 schemas.py - Schemas Pydantic para validacao de entrada/saida da API
3 """
4 from pydantic import BaseModel, Field
5 from typing import Optional, List
6 from datetime import datetime
7
8
9 # =====
10 # AUTENTICACAO
11 # =====
12 class LoginRequest(BaseModel):
13     username: str # Pode ser username ou codigo do colaborador
14     senha: str
15
16
17 class LoginResponse(BaseModel):
18     access_token: str
19     token_type: str = "bearer"
20     nome: str
21     username: str
22     papel: str
23     empresa_documento: str
24
25
26 class TokenData(BaseModel):
27     username: str
28     empresa_documento: str
29     papel: str
30     usuario_id: int
31
32
33 # =====
34 # USUARIOS (Colaboradores)
35 # =====
36 class UsuarioCreate(BaseModel):
37     nome: str
38     username: str
39     senha: str
40     papel: str = "OPERADOR"
41     codigo: Optional[str] = None
42
43
44 class UsuarioUpdate(BaseModel):
45     nome: Optional[str] = None
46     senha: Optional[str] = None
47     papel: Optional[str] = None
48     codigo: Optional[str] = None
49     ativo: Optional[bool] = None
50
51
52 class UsuarioResponse(BaseModel):
53     id: int
54     nome: str
55     username: str
56     papel: str
57     codigo: Optional[str]
58     ativo: bool
59     empresa_documento: str
60
61     class Config:
62         from_attributes = True
63
64
65 # =====
66 # PRODUTOS
67 # =====
68 class ProdutoCreate(BaseModel):
69     nome: str
70     descricao: Optional[str] = None
71     sku: Optional[str] = None
72     codigo_barras: Optional[str] = None
73     preco_venda: float = Field(ge=0)
74     preco_custo: float = Field(ge=0, default=0)
75     estoque_atual: float = Field(ge=0, default=0)
76     estoque_minimo: float = Field(ge=0, default=0)
77     unidade: str = "UN"
78     ncm: Optional[str] = None
79     cfop: Optional[str] = None
80     cest: Optional[str] = None
81
82
83 class ProdutoUpdate(BaseModel):
84     nome: Optional[str] = None
85     descricao: Optional[str] = None
86     sku: Optional[str] = None
87     codigo_barras: Optional[str] = None
88     preco_venda: Optional[float] = None
89     preco_custo: Optional[float] = None
90     estoque_minimo: Optional[float] = None
91     unidade: Optional[str] = None
92     ativo: Optional[bool] = None
93     ncm: Optional[str] = None
94     cfop: Optional[str] = None
95     cest: Optional[str] = None
96
97
98 class ProdutoResponse(BaseModel):
99     id: int
100     nome: str
101     descricao: Optional[str]
102     sku: Optional[str]
103     codigo_barras: Optional[str]
104     preco_venda: float
105     preco_custo: float
106     estoque_atual: float
107     estoque_minimo: float
108     unidade: str
109     ativo: bool
110     ncm: Optional[str]
111     cfop: Optional[str]
112     cest: Optional[str]
113
114     class Config:
115         from_attributes = True
116
117
118 # =====
119 # ESTOQUE MOVIMENTACAO
120 # =====
121 class EstoqueMovimentacaoCreate(BaseModel):
122     produto_id: int
123     tipo: str # ENTRADA, SAIDA, AJUSTE
124     quantidade: float
125     motivo: Optional[str] = None
126
127
128 class EstoqueMovimentacaoResponse(BaseModel):
129     id: int
130     produto_id: int
131     tipo: str
132     quantidade: float
133     motivo: Optional[str]
134     usuario: Optional[str]

```

```

135     data_transacao: Optional[datetime]
136
137     class Config:
138         from_attributes = True
139
140
141 # =====
142 # CLIENTES
143 # =====
144 class ClienteCreate(BaseModel):
145     nome: str
146     telefone: Optional[str] = None
147     email: Optional[str] = None
148     cpf_cnpj: Optional[str] = None
149     endereco: Optional[str] = None
150     cidade: Optional[str] = None
151     estado: Optional[str] = None
152     cep: Optional[str] = None
153     data_nascimento: Optional[str] = None
154
155
156 class ClienteUpdate(BaseModel):
157     nome: Optional[str] = None
158     telefone: Optional[str] = None
159     email: Optional[str] = None
160     cpf_cnpj: Optional[str] = None
161     endereco: Optional[str] = None
162     cidade: Optional[str] = None
163     estado: Optional[str] = None
164     cep: Optional[str] = None
165     data_nascimento: Optional[str] = None
166     ativo: Optional[bool] = None
167
168
169 class ClienteResponse(BaseModel):
170     id: int
171     nome: str
172     telefone: Optional[str]
173     email: Optional[str]
174     cpf_cnpj: Optional[str]
175     endereco: Optional[str]
176     cidade: Optional[str]
177     estado: Optional[str]
178     cep: Optional[str]
179     data_nascimento: Optional[str]
180     pontos_fidelidade: int
181     ativo: bool
182
183     class Config:
184         from_attributes = True
185
186
187 class ClienteHistoricoResponse(BaseModel):
188     compras: List[dict]
189     total_gasto: float
190     pontos: int
191
192
193 # =====
194 # FORNECEDORES
195 # =====
196 class FornecedorCreate(BaseModel):
197     nome_fantasia: str
198     razao_social: Optional[str] = None
199     cnpj_cpf: Optional[str] = None
200     telefone: Optional[str] = None
201     email: Optional[str] = None
202     endereco: Optional[str] = None
203
204
205 class FornecedorResponse(BaseModel):
206     id: int
207     nome_fantasia: str
208     razao_social: Optional[str]
209     cnpj_cpf: Optional[str]
210     telefone: Optional[str]
211     email: Optional[str]
212     endereco: Optional[str]
213     ativo: bool
214
215     class Config:
216         from_attributes = True
217
218
219 # =====
220 # VENDAS
221 # =====
222 class ItemVendaCreate(BaseModel):
223     produto_id: int
224     quantidade: float
225
226
227 class PagamentoVenda(BaseModel):
228     forma: str
229     valor: float
230
231
232 class DescontoVendaRequest(BaseModel):
233     valor_desconto: float = Field(ge=0)
234     motivo: Optional[str] = None
235     senha_admin: str # Exige senha admin para desconto
236
237
238 class VendaCreate(BaseModel):
239     cliente_id: Optional[int] = None
240     vendedor_id: Optional[int] = None
241     vendedor_nome: Optional[str] = None
242     itens: List[ItemVendaCreate]
243     pagamentos: List[PagamentoVenda]
244     observacao: Optional[str] = None
245
246
247 class VendaCancelRequest(BaseModel):
248     senha_admin: str
249     motivo: Optional[str] = None
250
251
252 class VendaItemResponse(BaseModel):
253     id: int
254     produto_id: int
255     produto_nome: Optional[str]
256     quantidade: float
257     preco_unitario: float
258     subtotal: float
259
260     class Config:
261         from_attributes = True
262
263
264 class VendaResponse(BaseModel):
265     id: int
266     cliente_id: Optional[int]
267     vendedor_id: Optional[int]
268     vendedor_nome: Optional[str]
269     operador: Optional[str]
270     total: float
271     desconto: float
272     valor_recebido: float

```

```

273     troco: float
274     forma_pagamento: Optional[str]
275     status: str
276     observacao: Optional[str]
277     data_venda: Optional[datetime]
278     itens: List[VendaItemResponse]
279
280     class Config:
281         from_attributes = True
282
283
284 # =====
285 # CAIXA
286 # =====
287 class CaixaAbrirRequest(BaseModel):
288     operador: str
289     saldo_inicial: float = 0
290
291
292 class CaixaFecharRequest(BaseModel):
293     operador: str
294     saldo_final: float
295
296
297 class CaixaTransacaoRequest(BaseModel):
298     tipo: str # ENTRADA, SAIDA, SANGRIA
299     valor: float = Field(gt=0)
300     descricao: Optional[str] = None
301     senha_admin: Optional[str] = None # Exigido para SAIDA e SANGRIA
302
303
304 class CaixaStatusResponse(BaseModel):
305     status: str
306     sessao_id: Optional[int]
307     operador: Optional[str]
308     saldo_inicial: Optional[float]
309     total_vendas: Optional[float]
310     total_entradas: Optional[float]
311     total_saidas: Optional[float]
312     saldo_esperado: Optional[float]
313     data_abertura: Optional[datetime]
314     transacoes: List[dict] = []
315
316
317 # =====
318 # DASHBOARD
319 # =====
320 class DashboardResponse(BaseModel):
321     vendas_hoje: float
322     vendas_ontem: float
323     vendas_mes: float
324     quantidade_vendas_hoje: int
325     ticket_medio_hoje: float
326     produtos_estoque_baixo: int
327     clientes_ativos: int
328     vendedores_do_dia: List[dict]
329     ultimas_vendas: List[dict]
330
331
332 # =====
333 # EMPRESA (validacao de CNPJ)
334 # =====
335 class EmpresaValidaRequest(BaseModel):
336     documento: str # CNPJ
337
338
339 class EmpresaValidaResponse(BaseModel):
340     valido: bool
341     nome_fantasia: Optional[str] = None
342     plano: Optional[str] = None
343     ativo: bool
344

```

```

1 """
2 security.py - Utilitários de segurança: hash de senha (PBKDF2-SHA256), JWT, dependências FastAPI
3 Usa hash nativo do Python (hashlib) – compatível com Python 3.13 sem dependências externas
4 """
5 import os
6 import hashlib
7 import hmac
8 import secrets
9 from datetime import datetime, timedelta
10 from typing import Optional
11 from fastapi import Depends, HTTPException, status
12 from fastapi.security import OAuth2PasswordBearer
13 from jose import JWTError, jwt
14 from sqlalchemy.orm import Session
15
16 from database import get_db, SessionLocal
17 from models_core import Cliente, Usuario
18
19 # =====
20 # CONFIGURAÇÃO
21 # =====
22 SECRET_KEY = os.getenv("SECRET_KEY", "dev-local-key-change-in-production")
23 ALGORITHM = "HS256"
24 ACCESS_TOKEN_EXPIRE_DAYS = 30
25 PBKDF2_ITERATIONS = 600_000 # OWASP 2024 recommendation
26
27 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/cliente/login")
28
29 # =====
30 # HASH DE SENHA (PBKDF2-SHA256 + salt)
31 # =====
32 def hash_senha(senha: str) -> str:
33     """Gera hash PBKDF2-SHA256 com salt aleatório. Formato: salt_hex$hash_hex"""
34     salt = secrets.token_bytes(32)
35     dk = hashlib.pbkdf2_hmac("sha256", senha.encode("utf-8"), salt, PBKDF2_ITERATIONS)
36     return salt.hex() + "$" + dk.hex()
37
38
39 def verificar_senha(senha_plana: str, senha_hash: str) -> bool:
40     """Verifica senha contra hash PBKDF2. Formato esperado: salt_hex$hash_hex"""
41     try:
42         salt_hex, hash_hex = senha_hash.split("$", 1)
43         salt = bytes.fromhex(salt_hex)
44         expected = bytes.fromhex(hash_hex)
45         dk = hashlib.pbkdf2_hmac("sha256", senha_plana.encode("utf-8"), salt, PBKDF2_ITERATIONS)
46         return hmac.compare_digest(dk, expected)
47     except (ValueError, AttributeError):
48         return False
49
50
51 # =====
52 # JWT
53 # =====
54 def criar_token(data: dict, expires_delta: Optional[timedelta] = None) -> str:
55     to_encode = data.copy()
56     expire = datetime.utcnow() + (expires_delta or timedelta(days=ACCESS_TOKEN_EXPIRE_DAYS))
57     to_encode.update({"exp": expire})
58     return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
59
60
61 def decodificar_token(token: str) -> dict:
62     try:
63         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
64         return payload
65     except JWTError:
66         raise HTTPException(
67             status_code=status.HTTP_401_UNAUTHORIZED,
68             detail="Token inválido ou expirado",
69             headers={"WWW-Authenticate": "Bearer"},
70         )
71
72
73 # =====
74 # DEPENDENCY: obter cliente atual
75 # =====
76 async def get_current_cliente(
77     token: str = Depends(oauth2_scheme),
78     db: Session = Depends(get_db),
79 ) -> Cliente:
80     payload = decodificar_token(token)
81     email: str = payload.get("sub")
82     if email is None:
83         raise HTTPException(status_code=401, detail="Token inválido")
84
85     cliente = db.query(Cliente).filter(Cliente.email == email).first()
86     if cliente is None or not cliente.ativo:
87         raise HTTPException(status_code=401, detail="Cliente inativo ou não encontrado")
88     return cliente
89
90
91 # =====
92 # DEPENDENCY: obter usuário colaborador atual (SaltoGestão desktop)
93 # =====
94 async def get_current_user(
95     token: str = Depends(oauth2_scheme),
96     db: Session = Depends(get_db),
97 ) -> Usuario:
98     """Obtém o usuário colaborador (vendedor/operador) a partir do token JWT."""
99     core_db = SessionLocal()
100     try:
101         payload = decodificar_token(token)
102         username: str = payload.get("username") or payload.get("sub")
103         empresa_documento: str = payload.get("empresa_documento")
104         if username is None:
105             raise HTTPException(status_code=401, detail="Token inválido")
106         # Busca no banco core via models_core
107         user = core_db.query(Usuario).filter(Usuario.username == username).first()
108         if user is None or not user.ativo:
109             raise HTTPException(status_code=401, detail="Usuário inativo ou não encontrado")
110         return user
111     finally:
112         core_db.close()
113
114
115 async def get_empresa_from_user(
116     current_user: Usuario = Depends(get_current_user),
117 ) -> str:
118     """Extrai o CNPJ da empresa do usuário autenticado."""
119     return current_user.empresa_documento
120
121
122 def require_role(*roles):
123     """Dependência que exige que o usuário tenha um dos papéis listados."""
124     async def _check(
125         current_user: Usuario = Depends(get_current_user),
126     ) -> Usuario:
127         if current_user.papel not in roles:
128             raise HTTPException(
129                 status_code=403,
130                 detail=f"Papel '{current_user.papel}' não autorizado. Necessário: {', '.join(roles)}",
131             )
132         return current_user
133     return _check
134

```



```

1  #!/usr/bin/env python3
2  """
3  seed_admin.py - Cria o primeiro administrador/vendedor da Corumbá Sistemas.
4  Uso: python seed_admin.py
5  """
6  import sys
7  import os
8
9  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
10
11 from database import SessionLocal, engine
12 from models_core import BaseCore, AdminCorumba
13 from security import hash_senha
14
15 print("=" * 50)
16 print("\n Seed Admin - Corumbá Sistemas")
17 print("=" * 50)
18
19 BaseCore.metadata.create_all(bind=engine)
20 print("✓ Tabelas atualizadas")
21
22 db = SessionLocal()
23
24 # Verificar se já existe admin
25 existing = db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumbasistemas.com.br").first()
26 if existing:
27     print("⚠ Admin já existe: admin@corumbasistemas.com.br")
28     db.close()
29     sys.exit(0)
30
31 # Criar admin master
32 admin = AdminCorumba(
33     nome="Administrador Corumbá",
34     email="admin@corumbasistemas.com.br",
35     senha_hash=hash_senha("corumba2026"),
36     papel="ADMIN",
37     ativo=True,
38 )
39 db.add(admin)
40
41 # Criar vendedor de teste
42 vendedor = AdminCorumba(
43     nome="Vendedor Teste",
44     email="vendedor@corumbasistemas.com.br",
45     senha_hash=hash_senha("vendedor123"),
46     papel="VENDEDOR",
47     ativo=True,
48 )
49 db.add(vendedor)
50
51 db.commit()
52 db.close()
53
54 print()
55 print("=" * 50)
56 print("✓ Admins criados com sucesso!")
57 print("=" * 50)
58 print()
59 print("👤 ADMIN MASTER:")
60 print("  Email: admin@corumbasistemas.com.br")
61 print("  Senha: corumba2026")
62 print("  Papel: ADMIN")
63 print()
64 print("👤 VENDEDOR:")
65 print("  Email: vendedor@corumbasistemas.com.br")
66 print("  Senha: vendedor123")
67 print("  Papel: VENDEDOR")
68 print()
69 print("📄 URL Login Admin: http://191.252.210.235:3000/admin/login")
70

```



```

1 #!/usr/bin/env python3
2 """
3 seed_clientes.py - Cria clientes de teste com assinaturas reais.
4 Uso: python seed_clientes.py
5
6 Clientes:
7 - Maria Silva (CPF) - SaltoGestão ATIVA, MesaPronta INADIMPLENTE
8 - João Oliveira (CNPJ) - SaltoGestão ATIVA, MesaPronta ATIVA
9 """
10 import sys
11 import os
12 import datetime
13
14 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
15
16 from database import SessionLocal, engine
17 from models_core import BaseCore, Cliente, Produto, Assinatura, ChaveAtivacao
18 from security import hash_senha
19
20 print("=" * 50)
21 print("\n Seed Clientes - Corumbá Sistemas")
22 print("=" * 50)
23
24 BaseCore.metadata.create_all(bind=engine)
25
26 db = SessionLocal()
27
28 # Buscar produtos
29 salto = db.query(Produto).filter(Produto.nome == "SaltoGestão").first()
30 mesa = db.query(Produto).filter(Produto.nome == "MesaPronta").first()
31
32 if not salto or not mesa:
33     print("✖ Produtos não encontrados! Rode seed_produtos.py primeiro.")
34     db.close()
35     sys.exit(1)
36
37 print(f"✓ Produtos encontrados: SaltoGestão (id={salto.id}), MesaPronta (id={mesa.id})")
38
39 # Limpar clientes de teste existentes
40 for email in ["maria.silva@teste.com", "joao.oliveira@teste.com"]:
41     existing = db.query(Cliente).filter(Cliente.email == email).first()
42     if existing:
43         db.query(Assinatura).filter(Assinatura.cliente_id == existing.id).delete()
44         db.query(ChaveAtivacao).filter(ChaveAtivacao.cliente_id == existing.id).delete()
45         db.delete(existing)
46         db.commit()
47         print(f"    Removido cliente existente: {email}")
48
49 now = datetime.datetime.utcnow()
50
51 # =====
52 # MARIA SILVA (CPF)
53 # =====
54 maria = Cliente(
55     nome="Maria Silva",
56     email="maria.silva@teste.com",
57     telefone="(11) 98765-4321",
58     cpf_cnpj="52998224725",
59     senha_hash=hash_senha("maria123"),
60     ativo=True,
61     data_cadastro=now,
62 )
63 db.add(maria)
64 db.flush()
65
66 # Maria tem SaltoGestão ATIVA
67 maria_salto = Assinatura(
68     cliente_id=maria.id,
69     produto_id=salto.id,
70     status="ATIVA",
71     data_aquisicao=now,
72     valor_mensal=salto.preco_mensal,
73 )
74 db.add(maria_salto)
75
76 # Maria tem MesaPronta INADIMPLENTE
77 maria_mesa = Assinatura(
78     cliente_id=maria.id,
79     produto_id=mesa.id,
80     status="INADIMPLENTE",
81     data_aquisicao=now,
82     valor_mensal=mesa.preco_mensal,
83 )
84 db.add(maria_mesa)
85
86 # Chave de ativação para MesaPronta (inadimplente)
87 maria_chave = ChaveAtivacao(
88     cliente_id=maria.id,
89     chave="CORUMBA-MARI-0001",
90     produto_id=mesa.id,
91     usada=False,
92     data_criacao=now,
93 )
94 db.add(maria_chave)
95
96 print(f"✓ Maria Silva criada (id={maria.id})")
97 print(f"    SaltoGestão: ATIVA")
98 print(f"    MesaPronta: INADIMPLENTE")
99
100 # =====
101 # JOÃO OLIVEIRA (CNPJ)
102 # =====
103 joao = Cliente(
104     nome="João Oliveira",
105     email="joao.oliveira@teste.com",
106     telefone="(62) 99876-5432",
107     cpf_cnpj="11223333000181",
108     senha_hash=hash_senha("joao123"),
109     ativo=True,
110     data_cadastro=now,
111 )
112 db.add(joao)
113 db.flush()
114
115 # João tem SaltoGestão ATIVA
116 joao_salto = Assinatura(
117     cliente_id=joao.id,
118     produto_id=salto.id,
119     status="ATIVA",
120     data_aquisicao=now,
121     valor_mensal=salto.preco_mensal,
122 )
123 db.add(joao_salto)
124
125 # João tem MesaPronta ATIVA
126 joao_mesa = Assinatura(
127     cliente_id=joao.id,
128     produto_id=mesa.id,
129     status="ATIVA",
130     data_aquisicao=now,
131     valor_mensal=mesa.preco_mensal,
132 )
133 db.add(joao_mesa)
134

```

```

135 # Chave de ativação para MesaPronta
136 joao_chave = ChaveAtivacao(
137     cliente_id=joao.id,
138     chave="CORUMBA-JOAO-0001",
139     produto_id=mesa.id,
140     usada=False,
141     data_criacao=now,
142 )
143 db.add(joao_chave)
144
145 print(f"✓ João Oliveira criado (id={joao.id})")
146 print(f"    SaltoGestão: ATIVA")
147 print(f"    MesaPronta: ATIVA")
148
149 db.commit()
150 db.close()
151
152 print()
153 print(f"=" * 50)
154 print(f"✓ Seed de clientes concluído!")
155 print(f"=" * 50)
156 print()
157 print(f"🔑 CREDENCIAIS DE TESTE:")
158 print()
159 print(f"    Maria Silva (CPF):")
160 print(f"        Email: maria.silva@teste.com")
161 print(f"        Senha: maria123")
162 print(f"        SaltoGestão: ✓ Ativo")
163 print(f"        MesaPronta: ⚠ Inadimplente")
164 print()
165 print(f"    João Oliveira (CNPJ):")
166 print(f"        Email: joao.oliveira@teste.com")
167 print(f"        Senha: joao123")
168 print(f"        SaltoGestão: ✓ Ativo")
169 print(f"        MesaPronta: ✓ Ativo")
170 print()
171

```

```

1  #!/usr/bin/env python3
2  """
3  seed_inicial.py - Popula o banco com produtos do catálogo e cliente de teste.
4  Uso: python seed_inicial.py
5  """
6  import sys
7  import os
8
9  sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
10
11 os.environ.setdefault("DATABASE_URL", "sqlite:///./salto_principal.db")
12 os.environ.setdefault("SECRET_KEY", "dev-seed-key-2026")
13
14 from database import SessionLocal, engine
15 from models_core import BaseCore, Cliente, Produto, Assinatura, ChaveAtivacao
16 from security import hash_senha
17
18 print("==" * 50)
19 print("🐍 Seed Inicial - Corumbá Sistemas v3.0")
20 print("==" * 50)
21
22 # Criar todas as tabelas
23 print("\n📦 Criando tabelas...")
24 BaseCore.metadata.create_all(bind=engine)
25 print("    ✓ Tabelas criadas")
26
27 db = SessionLocal()
28
29 # =====
30 # 1. PRODUTOS DO CATÁLOGO
31 # =====
32 print("\n📦 Cadastrando produtos do catálogo...")
33 produtos_data = [
34     {
35         "nome": "Salto PDV",
36         "descricao": "Frente de caixa ágil e sem travamentos. Controle suas vendas, emita cupons fiscais e atenda seus clientes com máxima velocidade. Inclui PDV completo, controle de
estoque e relatórios gerenciais.",
37         "preco_mensal": 149.90,
38         "preco_anual": 1499.90,
39         "icone": "👉",
40         "ordem": 1,
41     },
42     {
43         "nome": "MesaPronta",
44         "descricao": "Acompanhe o fluxo do pedido desde a mesa ou delivery até a entrega. Organização completa para não perder nenhuma venda. Ideal para restaurantes, bares e
lanchonetes.",
45         "preco_mensal": 99.90,
46         "preco_anual": 999.90,
47         "icone": "👈",
48         "ordem": 2,
49     },
50     {
51         "nome": "Corumbá Financeiro",
52         "descricao": "Controle financeiro completo para sua empresa. Contas a pagar, contas a receber, fluxo de caixa e DRE. Tudo integrado com o PDV.",
53         "preco_mensal": 79.90,
54         "preco_anual": 799.90,
55         "icone": "💰",
56         "ordem": 3,
57     },
58     {
59         "nome": "Corumbá Delivery",
60         "descricao": "Módulo de delivery integrado. Receba pedidos pelo WhatsApp, cardápio online e gestão de entregas. Compatível com iFood e Rappi.",
61         "preco_mensal": 129.90,
62         "preco_anual": 1299.90,
63         "icone": "🍷",
64         "ordem": 4,
65     },
66     {
67         "nome": "Corumbá NFC-e",
68         "descricao": "Emissão de Nota Fiscal de Consumidor Eletrônica. Integração com SEFAZ, envio automático por email e geração de DANFE.",
69         "preco_mensal": 59.90,
70         "preco_anual": 599.90,
71         "icone": "🧾",
72         "ordem": 5,
73     },
74     {
75         "nome": "Corumbá CRM",
76         "descricao": "Gestão de relacionamento com clientes. Cadastro completo, histórico de compras, programa de fidelidade e campanhas de marketing.",
77         "preco_mensal": 69.90,
78         "preco_anual": 699.90,
79         "icone": "👤",
80         "ordem": 6,
81     },
82 ]
83
84 produtos_criados = []
85 for p in produtos_data:
86     existing = db.query(Produto).filter(Produto.nome == p["nome"]).first()
87     if not existing:
88         produto = Produto(**p, ativo=True)
89         db.add(produto)
90         db.flush()
91         produtos_criados.append(produto)
92         print(f"    ✓ {p['icone']} {p['nome']} – R$ {p['preco_mensal']}/mês")
93     else:
94         produtos_criados.append(existing)
95         print(f"    △ {p['nome']} já existe")
96
97 db.commit()
98
99 # =====
100 # 2. CLIENTE DE TESTE
101 # =====
102 print("\n👤 Criando cliente de teste...")
103 cliente_teste = db.query(Cliente).filter(Cliente.email == "cliente@exemplo.com").first()
104 if not cliente_teste:
105     cliente_teste = Cliente(
106         nome="João Silva",
107         email="cliente@exemplo.com",
108         telefone="(62) 99999-9999",
109         cpf_cnpj="12345678901",
110         senha_hash=hash_senha("cliente123"),
111         ativo=True,
112     )
113     db.add(cliente_teste)
114     db.commit()
115     db.refresh(cliente_teste)
116     print(f"    ✓ Cliente criado: {cliente_teste.nome} ({cliente_teste.email})")
117 else:
118     print(f"    △ Cliente já existe: {cliente_teste.email}")
119
120 # =====
121 # 3. ASSINATURA ATIVA (Salto PDV)
122 # =====
123 print("\n📝 Criando assinatura de teste...")
124 assinatura_existente = db.query(Assinatura).filter(
125     Assinatura.cliente_id == cliente_teste.id,
126     Assinatura.produto_id == produtos_criados[0].id,
127 ).first()
128
129 if not assinatura_existente:
130     assinatura = Assinatura(
131         cliente_id=cliente_teste.id,
132         produto_id=produtos_criados[0].id, # Salto PDV

```

```

133         status="ATIVA",
134         valor_mensal=149.90,
135     )
136     db.add(assinatura)
137     db.commit()
138     print(f" ✓ Assinatura ativa: Salto PDV")
139 else:
140     print(f" △ Assinatura já existe")
141
142 # =====
143 # 4. CHAVE DE ATIVAÇÃO
144 # =====
145 print("\n➡ Gerando chave de ativação de teste...")
146 chave_existente = db.query(ChaveAtivacao).filter(
147     ChaveAtivacao.cliente_id == cliente_teste.id,
148     ChaveAtivacao.usada == False,
149 ).first()
150
151 if not chave_existente:
152     chave = ChaveAtivacao(
153         cliente_id=cliente_teste.id,
154         chave="CORUMBA-DEST-0001",
155         produto_id=produtos_criados[1].id, # MesaPronta
156         usada=False,
157     )
158     db.add(chave)
159     db.commit()
160     print(f" ✓ Chave gerada: CORUMBA-DEST-0001 (para MesaPronta)")
161 else:
162     print(f" △ Chave já existe: {chave_existente.chave}")
163
164 db.close()
165
166 print("\n" + "=" * 50)
167 print("✓ Seed concluído!")
168 print("=" * 50)
169 print("")
170 ☐ Dados de acesso do cliente de teste:
171
172 ✉ E-mail: cliente@exemplo.com
173 📌 Senha: cliente123
174
175 📦 Produtos cadastrados: 6
176 ✓ Salto PDV – Adquirido (ativo)
177   └─ MesaPronta – Bloqueado (chave: CORUMBA-DEST-0001)
178   └─ Corumbá Financeiro – Bloqueado
179   └─ Corumbá Delivery – Bloqueado
180   └─ Corumbá NFC-e – Bloqueado
181   └─ Corumbá CRM – Bloqueado
182 """)
183

```

```

1  #!/usr/bin/env python3
2  """
3  seed_produtos.py - Popula o catálogo de produtos da Corumbá Sistemas.
4  Uso: python seed_produtos.py
5
6  Produtos:
7  1. SaltoGestão - Sistema de gestão empresarial completo
8  2. MesaPronta - Sistema de gestão para restaurantes
9  """
10 import sys
11 import os
12
13 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
14
15 from database import SessionLocal, engine
16 from models_core import BaseCore, Produto
17
18 print("=" * 50)
19 print("🌱 Seed Produtos - Corumbá Sistemas")
20 print("=" * 50)
21
22 BaseCore.metadata.create_all(bind=engine)
23 print("✓ Tabelas atualizadas")
24
25 db = SessionLocal()
26
27 # Verificar se já existem produtos
28 existing = db.query(Produto).count()
29 if existing > 0:
30     print(f"⚠ Já existem {existing} produtos no catálogo. Limpando...")
31     db.query(Produto).delete()
32     db.commit()
33
34 # =====
35 # PRODUTOS DA CORUMBÁ SISTEMAS (apenas 2)
36 # =====
37
38 produtos = [
39     Produto(
40         nome="SaltoGestão",
41         descricao="Sistema de gestão empresarial completo. Controle de vendas, estoque, financeiro, clientes e fornecedores. Ideal para comércio varejista e atacadista.",
42         preco_mensal=149.90,
43         preco_anual=1499.90,
44         icone="📊",
45         ordem=1,
46         ativo=True,
47     ),
48     Produto(
49         nome="MesaPronta",
50         descricao="Sistema de gestão para restaurantes, bares e lanchonetes. Controle de mesas, pedidos, delivery e comandas. Organização completa para seu estabelecimento.",
51         preco_mensal=99.90,
52         preco_anual=999.90,
53         icone="🍽",
54         ordem=2,
55         ativo=True,
56     ),
57     Produto(
58         nome="ConnectFood",
59         descricao="Integração direta com iFood e 99Food. Receba pedidos das plataformas de delivery direto no seu MesaPronta. Gerencie pedidos, status de preparo e entrega em tempo
60 real. Ideal para restaurantes e lanchonetes que vendem por delivery.",
61         preco_mensal=49.90,
62         preco_anual=499.90,
63         icone="🚚",
64         ordem=3,
65         ativo=True,
66     ),
67 ]
68
69 for p in produtos:
70     db.add(p)
71
72 db.commit()
73
74 print()
75 print("=" * 50)
76 print("✓ Produtos criados com sucesso!")
77 print("=" * 50)
78 print()
79 print("📋 CATÁLOGO:")
80 for p in produtos:
81     print(f"    {p.icone} {p.nome}")
82     print(f"        Mensal: R$ {p.preco_mensal:.2f}")
83     print(f"        Anual: R$ {p.preco_anual:.2f}")
84     print()
85
86 db.close()

```

```

1 #!/usr/bin/env python3
2 """
3 seed_test.py - Script de Seed e Teste Completo do Sistema Corumbá Sistemas
4
5 Fase 2: Cria 100 empresas com dados isolados
6 - 100 empresas (CNPJs fictícios válidos)
7 - 10 usuários por empresa (1000 total)
8 - 100 produtos por empresa (10000 total)
9 - 100 vendas por empresa (10000 total)
10
11 Fase 3-7: Testes automatizados
12 - Isolamento multi-tenant
13 - Segurança
14 - Performance
15 - Integridade
16 - Correções automáticas
17
18 Fase 8: Relatório final
19 """
20 import sys
21 import os
22 import time
23 import json
24 import random
25 import string
26 import hashlib
27 import statistics
28 from datetime import datetime, timedelta
29 from concurrent.futures import ThreadPoolExecutor, as_completed
30
31 # Adicionar o path do backend
32 sys.path.insert(0, '/home/hermesos/sistema-loja/backendPy')
33
34 from sqlalchemy import create_engine, text, func
35 from sqlalchemy.orm import sessionmaker
36
37 # Importar configurações
38 from database import engine_core, engine_salto, SessionCore, SessionSalto
39 from models_core import Empresa, Usuario, Licenca
40 from models import Produto, Cliente, Venda, VendaItem, CaixaSessao, CaixaTransacao, HistoricoVenda
41 from licenca import gerar_chave_licenca
42 from security import hash_senha
43
44 # =====
45 # CONFIGURAÇÃO
46 # =====
47 NUM_EMPRESAS = 100
48 USUARIOS_POR_EMPRESA = 10
49 PRODUTOS_POR_EMPRESA = 100
50 VENDAS_POR_EMPRESA = 100
51
52 PAPEIS = ['ADMIN', 'GERENTE', 'CAIXA', 'VENDEDOR', 'OPERADOR', 'FINANCEIRO', 'ESTOQUE', 'SUPPORT']
53 FORMAS_PAGAMENTO = ['Dinheiro', 'Cartao Credito', 'Cartao Debito', 'PIX']
54 CATEGORIAS_PRODUTO = ['Alimentos', 'Bebidas', 'Limpeza', 'Higiene', 'Eletronicos', 'Vestuario', 'Ferramentas', 'Escritorio', 'Pet', 'Automotivo']
55
56 # Cores para output
57 class COR:
58     VERDE = '\033[92m'
59     AMARELO = '\033[93m'
60     VERMELHO = '\033[91m'
61     AZUL = '\033[94m'
62     ROXO = '\033[95m'
63     CIANO = '\033[96m'
64     NEGRO = '\033[1m'
65     FIM = '\033[0m'
66
67 def log(msg, cor=COR.CIANO):
68     print(f"{cor}{msg}{COR.FIM}")
69
70 def gerar_cnpj():
71     """Gera CNPJ fictício válido."""
72     def digito_verificador(base, pesos):
73         soma = sum(int(b) * p for b, p in zip(base, pesos))
74         resto = soma % 11
75         return '0' if resto < 2 else str(11 - resto)
76
77     # Gerar base aleatória
78     base = ''.join([str(random.randint(0, 9)) for _ in range(12)])
79
80     # Calcular dígitos verificadores
81     pesos1 = [5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2]
82     d1 = digito_verificador(base, pesos1)
83
84     pesos2 = [6, 5, 4, 3, 2, 9, 8, 7, 6, 5, 4, 3, 2]
85     d2 = digito_verificador(base + d1, pesos2)
86
87     cnpj = base + d1 + d2
88     return f"{cnpj[:2]}.{cnpj[2:5]}.{cnpj[5:8]}-{cnpj[8:12]}-{cnpj[12:]}"
89
90 def gerar_nome_empresa(i):
91     """Gera nome de empresa único."""
92     prefixos = ['Super', 'Mega', 'Ultra', 'Top', 'Prime', 'Max', 'Pro', 'Elite', 'Gold', 'Silver']
93     nomes = ['Comercio', 'Distribuidora', 'Atacado', 'Varejo', 'Mercado', 'Loja', 'Shop', 'Center', 'Store', 'Mart']
94     sufixos = ['LTDA', 'ME', 'EPP', 'SA', 'MEI']
95     return f"{random.choice(prefixos)} {random.choice(nomes)} {i+1} {random.choice(sufixos)}"
96
97 def gerar_nome_usuario(papel, i):
98     """Gera nome de usuário baseado no papel."""
99     nomes = ['Joao', 'Maria', 'Pedro', 'Ana', 'Carlos', 'Lucia', 'Fernando', 'Patricia', 'Roberto', 'Juliana',
100             'Marcos', 'Camila', 'Rafael', 'Beatriz', 'Thiago', 'Larissa', 'Gustavo', 'Amanda', 'Diego', 'Isabela']
101     sobrenomes = ['Silva', 'Santos', 'Oliveira', 'Souza', 'Rodrigues', 'Ferreira', 'Almeida', 'Pereira', 'Lima', 'Gomes']
102     return f"{random.choice(nomes)} {random.choice(sobrenomes)}"
103
104 def gerar_nome_produto(categoria, i):
105     """Gera nome de produto baseado na categoria."""
106     adjetivos = ['Premium', 'Basico', 'Especial', 'Classico', 'Novo', 'Original', 'Natural', 'Importado', 'Nacional', 'Economico']
107     nomes = {
108         'Alimentos': ['Arroz', 'Feijao', 'Macarrao', 'Farinha', 'Acucar', 'Sal', 'Oleo', 'Leite', 'Cafe', 'Biscoito'],
109         'Bebidas': ['Agua', 'Refrigerante', 'Suco', 'Cerveja', 'Vinho', 'Cha', 'Energetico', 'Leite', 'Iogurte', 'Vitamina'],
110         'Limpeza': ['Detergente', 'Sabao', 'Desinfetante', 'Amaciante', 'Alcool', 'Agua Sanitaria', 'Esponja', 'Vassoura', 'Rodo', 'Pano'],
111         'Higiene': ['Shampoo', 'Condicionador', 'Sabonete', 'Pasta Dental', 'Escova', 'Desodorante', 'Papel Higienico', 'Absorvente', 'Fralda', 'Creme'],
112         'Eletronicos': ['Cabo USB', 'Carregador', 'Fone', 'Pilha', 'Lampada', 'Extensao', 'Tomada', 'Interruptor', 'Bateria', 'Adaptador'],
113         'Vestuario': ['Camiseta', 'Calca', 'Meia', 'Sapato', 'Chinelo', 'Bone', 'Cinto', 'Luva', 'Chapeu', 'Mochila'],
114         'Ferramentas': ['Martelo', 'Chave', 'Alicate', 'Parafuso', 'Prego', 'Fita', 'Cola', 'Lixa', 'Serra', 'Trena'],
115         'Escritorio': ['Caneta', 'Lapis', 'Caderno', 'Papel', 'Grampeador', 'Tesoura', 'Cola', 'Fita', 'Envelope', 'Clips'],
116         'Pet': ['Racao', 'Petisco', 'Brinquedo', 'Coleira', 'Shampoo Pet', 'Antipulga', 'Vermifugo', 'Cama Pet', 'Bebedouro', 'Comedouro'],
117         'Automotivo': ['Oleo Motor', 'Filtro', 'Palheta', 'Limpador', 'Cera', 'Desodorizador', 'Suporte', 'Capa', 'Tapete', 'Adesivo'],
118     }
119     nome_base = random.choice(nomes.get(categoria, ['Produto']))
120     return f"{nome_base} {random.choice(adjetivos)} {i+1}"
121
122 # =====
123 # FASE 2: SEED DE DADOS
124 # =====
125
126 class SeedData:
127     """Classe para gerar e inserir dados de teste."""
128
129     def __init__(self):
130         self.db_core = SessionCore()
131         self.db_salto = SessionSalto()
132         self.empresas = []
133         self.usuarios = []
134         self.produtos = []

```

```

135     self.vendas = []
136     self.erros = []
137     self.métricas = {
138         'tempo_empresas': 0,
139         'tempo_usuarios': 0,
140         'tempo_produtos': 0,
141         'tempo_vendas': 0,
142         'total_empresas': 0,
143         'total_usuarios': 0,
144         'total_produtos': 0,
145         'total_vendas': 0,
146         'total_historico': 0,
147         'total_cancelamentos': 0,
148         'erros': [],
149     }
150
151     def limpar_dados_anteriores(self):
152         """Limpa todos os dados de teste anteriores."""
153         log(f"\n[LIMPEZA] Removendo dados de teste anteriores...", COR.AMARELO)
154
155         try:
156             # Limpar banco Salto (ordem: FK primeiro)
157             self.db_salto.query(HistoricoVenda).delete()
158             self.db_salto.query(VendaItem).delete()
159             self.db_salto.query(Venda).delete()
160             self.db_salto.query(CaixaTransacao).delete()
161             self.db_salto.query(CaixaSessao).delete()
162             self.db_salto.query(Produto).delete()
163             self.db_salto.query(Cliente).delete()
164             self.db_salto.commit()
165
166             # Limpar banco Core
167             self.db_core.query(Licenca).delete()
168             self.db_core.query(Usuario).delete()
169             self.db_core.query(Empresa).delete()
170             self.db_core.commit()
171
172             log(f"[LIMPEZA] Dados removidos com sucesso", COR.VERDE)
173         except Exception as e:
174             log(f"[LIMPEZA] Erro: {e}", COR.VERMELHO)
175             self.db_salto.rollback()
176             self.db_core.rollback()
177
178     def criar_empresas(self):
179         """Cria N empresas com licenças."""
180         log(f"\n[EMPRESAS] Criando {NUM_EMPRESAS} empresas...", COR.AZUL)
181         inicio = time.time()
182
183         for i in range(NUM_EMPRESAS):
184             try:
185                 cnpj = gerar_cnpj()
186                 nome = gerar_nome_empresa(i)
187                 chave = gerar_chave_licenca()
188
189                 # Criar empresa no banco Core
190                 empresa = Empresa(
191                     documento=cnpj,
192                     razao_social=nome,
193                     nome_fantasia=nome.split(' LTDA')[0].split(' ME')[0].split(' SA')[0],
194                     email=f"contato@{nome.lower().split()[0].replace(' ', '')}.com.br",
195                     telefone=f"629{random.randint(10000000, 99999999)}",
196                     plano=random.choice(['BASIC0', 'PRO', 'PREMIUM']),
197                     ativo=True,
198                 )
199                 self.db_core.add(empresa)
200                 self.db_core.flush()
201
202                 # Criar licença
203                 licenca = Licenca(
204                     empresa_documento=cnpj,
205                     chave=chave,
206                     status="ATIVA",
207                     plano=empresa.plano,
208                     data_expiracao=datetime.utcnow() + timedelta(days=random.choice([30, 90, 180, 365])),
209                     max_usuarios=50,
210                     max_produtos=5000,
211                     ativo=True,
212                 )
213                 self.db_core.add(licenca)
214                 self.db_core.flush()
215
216                 self.empresas.append({
217                     'cnpj': cnpj,
218                     'nome': nome,
219                     'chave': chave,
220                     'plano': empresa.plano,
221                 })
222
223                 if (i + 1) % 10 == 0:
224                     self.db_core.commit()
225                     log(f" {EMPRESAS} {i+1}/{NUM_EMPRESAS} criadas", COR.VERDE)
226
227             except Exception as e:
228                 self.db_core.rollback()
229                 self.métricas['erros'].append(f"Empresa {i}: {str(e)}")
230                 log(f" [ERRO] Empresa {i}: {e}", COR.VERMELHO)
231
232         self.db_core.commit()
233         self.métricas['tempo_empresas'] = time.time() - inicio
234         self.métricas['total_empresas'] = len(self.empresas)
235         log(f"[EMPRESAS] {len(self.empresas)} empresas criadas em {self.métricas['tempo_empresas']:.2f}s", COR.VERDE)
236
237     def criar_usuarios(self):
238         """Cria usuários para cada empresa."""
239         log(f"\n[USUARIOS] Criando {USUARIOS_POR_EMPRESA} usuarios por empresa...", COR.AZUL)
240         inicio = time.time()
241
242         for emp_idx, empresa in enumerate(self.empresas):
243             try:
244                 for u_idx in range(USUARIOS_POR_EMPRESA):
245                     papel = PAPEIS[u_idx % len(PAPEIS)]
246                     nome = gerar_nome_usuario(papel, u_idx)
247                     username = f"{nome.lower().split()[0]}{emp_idx:03d}{u_idx:02d}"
248                     codigo = f"{u_idx+1:04d}"
249
250                     usuario = Usuario(
251                         empresa_documento=empresa['cnpj'],
252                         nome=nome,
253                         username=username,
254                         senha_hash=hash_senha(f"senha{codigo}"),
255                         papel=papel,
256                         codigo=codigo,
257                         ativo=True,
258                     )
259                     self.db_core.add(usuario)
260                     self.usuarios.append({
261                         'empresa_cnpj': empresa['cnpj'],
262                         'username': username,
263                         'codigo': codigo,
264                         'papel': papel,
265                         'senha': f"senha{codigo}",
266                     })
267
268                 if (emp_idx + 1) % 10 == 0:
269                     self.db_core.commit()
270                     log(f" {USUARIOS} Empresa {emp_idx+1}/{NUM_EMPRESAS}: {USUARIOS_POR_EMPRESA} usuarios", COR.VERDE)
271
272             except Exception as e:

```

```

273         self.db_core.rollback()
274         self.métricas['erros'].append(f"Usuarios empresa {emp_idx}: {str(e)}")
275
276     self.db_core.commit()
277     self.métricas['tempo_usuarios'] = time.time() - inicio
278     self.métricas['total_usuarios'] = len(self.usuarios)
279     log(f"[USUARIOS] {len(self.usuarios)} usuarios criados em {self.métricas['tempo_usuarios']:.2f}s", COR.VERDE)
280
281 def criar_produtos(self):
282     """Cria produtos para cada empresa."""
283     log(f"\n[PRODUTOS] Criando {PRODUTOS_POR_EMPRESA} produtos por empresa...", COR.AZUL)
284     inicio = time.time()
285
286     for emp_idx, empresa in enumerate(self.empresas):
287         try:
288             for p_idx in range(PRODUTOS_POR_EMPRESA):
289                 categoria = random.choice(CATEGORIAS_PRODUTO)
290                 nome = gerar_nome_produto(categoria, p_idx)
291                 preco = round(random.uniform(1.99, 999.99), 2)
292                 estoque = random.randint(0, 500)
293
294                 produto = Produto(
295                     empresa_documento=empresa['cnpj'],
296                     nome=nome,
297                     sku=f"SKU-{emp_idx:03d}-{p_idx:04d}",
298                     codigo_barras=f"{random.randint(1000000000000, 999999999999)}",
299                     preco_venda=preco,
300                     preco_custo=round(preco * random.uniform(0.4, 0.7), 2),
301                     estoque_atual=estoque,
302                     estoque_minimo=random.randint(5, 50),
303                     unidade=random.choice(['UN', 'KG', 'LT', 'CX', 'PC']),
304                     ncm=f"{random.randint(1000, 9999)}.{random.randint(10, 99)}.{random.randint(10, 99)}",
305                     ativo=True,
306                 )
307                 self.db_salto.add(produto)
308                 self.produtos.append({
309                     'empresa_cnpj': empresa['cnpj'],
310                     'sku': produto.sku,
311                     'nome': nome,
312                     'preco': preco,
313                 })
314
315             if (emp_idx + 1) % 10 == 0:
316                 self.db_salto.commit()
317                 log(f" [PRODUTOS] Empresa {emp_idx+1}/{NUM_EMPRESAS}: {PRODUTOS_POR_EMPRESA} produtos", COR.VERDE)
318
319         except Exception as e:
320             self.db_salto.rollback()
321             self.métricas['erros'].append(f"Produtos empresa {emp_idx}: {str(e)}")
322
323     self.db_salto.commit()
324     self.métricas['tempo_produtos'] = time.time() - inicio
325     self.métricas['total_produtos'] = len(self.produtos)
326     log(f"[PRODUTOS] {len(self.produtos)} produtos criados em {self.métricas['tempo_produtos']:.2f}s", COR.VERDE)
327
328 def criar_clientes(self):
329     """Cria clientes para cada empresa."""
330     log(f"\n[CLIENTES] Criando 20 clientes por empresa...", COR.AZUL)
331
332     from models import Cliente
333
334     for emp_idx, empresa in enumerate(self.empresas):
335         try:
336             for c_idx in range(20):
337                 nome = gerar_nome_usuario("CLIENTE", c_idx)
338                 cliente = Cliente(
339                     empresa_documento=empresa['cnpj'],
340                     nome=nome,
341                     telefone=f"629{random.randint(10000000, 99999999)}",
342                     email=f"{nome.lower().split()[0]}@email.com",
343                     cpf_cnpj=f"{random.randint(10000000000, 99999999999)}",
344                     pontos_fidelidade=random.randint(0, 1000),
345                     ativo=True,
346                 )
347                 self.db_salto.add(cliente)
348
349             if (emp_idx + 1) % 20 == 0:
350                 self.db_salto.commit()
351
352         except Exception as e:
353             self.db_salto.rollback()
354
355     self.db_salto.commit()
356     log(f"[CLIENTES] Clientes criados", COR.VERDE)
357
358 def criar_vendas(self):
359     """Cria vendas para cada empresa (com histórico automático)."""
360     log(f"\n[VENDAS] Criando {VENDAS_POR_EMPRESA} vendas por empresa...", COR.AZUL)
361     inicio = time.time()
362
363     from models import CaixaSessao, CaixaTransacao
364     from historico import registrar_venda, registrar_cancelamento
365
366     for emp_idx, empresa in enumerate(self.empresas):
367         try:
368             # Abrir caixa para a empresa
369             admin_user = [u for u in self.usuarios if u['empresa_cnpj'] == empresa['cnpj'] and u['papel'] == 'ADMIN'][0]
370
371             caixa = CaixaSessao(
372                 empresa_documento=empresa['cnpj'],
373                 operador=admin_user['username'],
374                 usuario_id=1,
375                 saldo_inicial=100.00,
376                 status="ABERTO",
377             )
378             self.db_salto.add(caixa)
379             self.db_salto.flush()
380
381             # Buscar produtos da empresa
382             produtos_empresa = [p for p in self.produtos if p['empresa_cnpj'] == empresa['cnpj']]
383
384             for v_idx in range(VENDAS_POR_EMPRESA):
385                 try:
386                     # Selecionar itens aleatórios
387                     num_itens = random.randint(1, 10)
388                     itens_selecionados = random.sample(produtos_empresa, min(num_itens, len(produtos_empresa)))
389
390                     itens_venda = []
391                     total = 0
392                     for prod in itens_selecionados:
393                         qtd = random.randint(1, 5)
394                         subtotal = prod['preco'] * qtd
395                         itens_venda.append({
396                             'produto_id': random.randint(1, 10000),
397                             'produto_nome': prod['nome'],
398                             'quantidade': qtd,
399                             'preco_unitario': prod['preco'],
400                             'subtotal': subtotal,
401                         })
402                     total += subtotal
403
404                     # Forma de pagamento
405                     forma = random.choice(FORMAS_PAGAMENTO)
406                     valor_recebido = total + random.uniform(0, 50)
407                     troco = max(0, valor_recebido - total)
408
409                     # Criar venda
410                     venda = Venda(

```



```

411         empresa_documento=empresa['cnpj'],
412         vendedor_id=random.randint(1, 100),
413         vendedor_nome=f"Vendedor {random.randint(1, 5)}",
414         operador=admin_user['username'],
415         total=total,
416         desconto=random.choice([0, 0, 0, total * 0.05, total * 0.1]),
417         valor_recebido=valor_recebido,
418         troco=troco,
419         forma_pagamento=forma,
420         status="CONCLUIDA",
421         data_venda=datetime.utcnow() - timedelta(days=random.randint(0, 30)),
422     )
423     self.db_salto.add(venda)
424     self.db_salto.flush()
425
426     # Criar itens da venda
427     for iv in itens_venda:
428         item_db = VendaItem(
429             empresa_documento=empresa['cnpj'],
430             venda_id=venda.id,
431             produto_id=iv['produto_id'],
432             produto_nome=iv['produto_nome'],
433             quantidade=iv['quantidade'],
434             preco_unitario=iv['preco_unitario'],
435             subtotal=iv['subtotal'],
436         )
437         self.db_salto.add(item_db)
438
439     # Registrar transação no caixa
440     transacao = CaixaTransacao(
441         empresa_documento=empresa['cnpj'],
442         sessao_id=caixa.id,
443         tipo="VENDA",
444         valor=total,
445         descricao=f"Venda #{venda.id}",
446         venda_id=venda.id,
447         operador=admin_user['username'],
448     )
449     self.db_salto.add(transacao)
450
451     # Registrar no histórico
452     registrar_venda(
453         db=self.db_salto,
454         empresa_documento=empresa['cnpj'],
455         usuario_id=random.randint(1, 100),
456         usuario_nome=admin_user['username'],
457         usuario_papel='ADMIN',
458         venda_id=venda.id,
459         itens=itens_venda,
460         pagamentos=[{'forma': forma, 'valor': float(valor_recebido)}],
461         valor_total=total,
462         valor_recebido=float(valor_recebido),
463         troco=float(troco),
464         forma_pagamento=forma,
465         origem='PDV',
466     )
467
468     # Algumas vendas canceladas (10%)
469     if random.random() < 0.1:
470         venda.status = "CANCELADA"
471         registrar_cancelamento(
472             db=self.db_salto,
473             empresa_documento=empresa['cnpj'],
474             venda_id=venda.id,
475             usuario_id=random.randint(1, 100),
476             usuario_nome=admin_user['username'],
477             motivo=random.choice(['Cliente desistiu', 'Produto indisponível', 'Erro de pagamento', 'Troca']),
478         )
479         self.métricas['total_cancelamentos'] += 1
480
481     self.vendas.append({
482         'empresa_cnpj': empresa['cnpj'],
483         'venda_id': venda.id,
484         'total': total,
485     })
486
487     except Exception as e:
488         self.db_salto.rollback()
489         self.métricas['erros'].append(f"Venda {v_idx} empresa {emp_idx}: {str(e)}")
490         continue
491
492     self.db_salto.commit()
493
494     if (emp_idx + 1) % 10 == 0:
495         log(f" [VENDAS] Empresa {emp_idx+1}/{NUM_EMPRESAS}: {VENDAS_POR_EMPRESA} vendas", COR.VERDE)
496
497     except Exception as e:
498         self.db_salto.rollback()
499         self.métricas['erros'].append(f"Vendas empresa {emp_idx}: {str(e)}")
500
501     self.métricas['tempo_vendas'] = time.time() - inicio
502     self.métricas['total_vendas'] = len(self.vendas)
503     log(f"[VENDAS] {len(self.vendas)} vendas criadas em {self.métricas['tempo_vendas']:.2f}s", COR.VERDE)
504
505     def fechar(self):
506         """Fecha as conexões com o banco."""
507         self.db_core.close()
508         self.db_salto.close()
509
510     # =====
511     # FASE 3: TESTES DE ISOLAMENTO MULTI-TENANT
512     # =====
513
514     class TestesIsolamento:
515         """Testa o isolamento de dados entre empresas."""
516
517         def __init__(self):
518             self.db_core = SessionCore()
519             self.db_salto = SessionSalto()
520             self.resultados = []
521             self.erros = []
522
523         def testar_isolamento_vendas(self):
524             """Verifica que cada empresa vê apenas suas próprias vendas."""
525             log("\n[TESTE] Isolamento de Vendas...", COR.ROXO)
526
527             # Pegar 10 empresas aleatórias
528             empresas = self.db_core.query(Empresa).limit(10).all()
529
530             for empresa in empresas:
531                 # Contar vendas da empresa
532                 vendas_empresa = self.db_salto.query(Venda).filter(
533                     Venda.empresa_documento == empresa.documento
534                 ).count()
535
536                 # Verificar que NÃO há vendas de outras empresas
537                 vendas_outras = self.db_salto.query(Venda).filter(
538                     Venda.empresa_documento != empresa.documento
539                 ).count()
540
541                 # Verificar que o histórico também está isolado
542                 historico_empresa = self.db_salto.query(HistoricoVenda).filter(
543                     HistoricoVenda.empresa_documento == empresa.documento
544                 ).count()
545
546                 self.resultados.append({
547                     'empresa': empresa.documento,

```

```

549         'vendas': vendas_empresa,
550         'historico': historico_empresa,
551         'vendas_outras_visiveis': vendas_outras,
552     })
553
554     # Verificar que não há vendas cruzadas
555     vendas_cruzadas = self.db_salto.query(Venda).filter(
556         Venda.empresa_documento.in_([e.documento for e in empresas[:5]])
557     ).filter(
558         ~Venda.empresa_documento.in_([e.documento for e in empresas[:5]])
559     ).count()
560
561     if vendas_cruzadas == 0:
562         log(" [OK] Nenhuma venda cruzada detectada", COR.VERDE)
563     else:
564         log(f" [FALHA] {vendas_cruzadas} vendas cruzadas!", COR.VERMELHO)
565         self.erros.append(f"Vendas cruzadas: {vendas_cruzadas}")
566
567 def testar_isolamento_usuarios(self):
568     """Verifica que usuários de uma empresa não aparecem em outra."""
569     log("\n[TESTE] Isolamento de Usuários...", COR.ROXO)
570
571     # Verificar que existem usuários com mesmo código em empresas diferentes
572     resultado = self.db_core.execute(text("""
573         SELECT codigo, COUNT(DISTINCT empresa_documento) as num_empresas
574         FROM usuarios
575         WHERE codigo IS NOT NULL
576         GROUP BY codigo
577         HAVING COUNT(DISTINCT empresa_documento) > 1
578         LIMIT 5
579     """)).fetchall()
580
581     if resultado:
582         log(f" [OK] {len(resultado)} códigos de usuário existem em múltiplas empresas (isolamento correto)", COR.VERDE)
583     else:
584         log(" [INFO] Nenhum código duplicado entre empresas", COR.AMARELO)
585
586 def testar_isolamento_produtos(self):
587     """Verifica que produtos de uma empresa não aparecem em outra."""
588     log("\n[TESTE] Isolamento de Produtos...", COR.ROXO)
589
590     # Verificar SKUs únicos por empresa
591     resultado = self.db_salto.execute(text("""
592         SELECT sku, COUNT(DISTINCT empresa_documento) as num_empresas
593         FROM produtos
594         GROUP BY sku
595         HAVING COUNT(DISTINCT empresa_documento) > 1
596     """)).fetchall()
597
598     if not resultado:
599         log(" [OK] SKUs são únicos por empresa", COR.VERDE)
600     else:
601         log(f" [FALHA] {len(resultado)} SKUs duplicados entre empresas!", COR.VERMELHO)
602         self.erros.append(f"SKUs duplicados: {len(resultado)}")
603
604 def testar_isolamento_historico(self):
605     """Verifica que o histórico de vendas está isolado."""
606     log("\n[TESTE] Isolamento do Histórico...", COR.ROXO)
607
608     # Contar registros de histórico por empresa
609     resultado = self.db_salto.execute(text("""
610         SELECT empresa_documento, COUNT(*) as total
611         FROM historico_vendas
612         GROUP BY empresa_documento
613         ORDER BY total DESC
614         LIMIT 5
615     """)).fetchall()
616
617     if resultado:
618         log(f" [OK] Histórico isolado: {len(resultado)} empresas com registros", COR.VERDE)
619         for r in resultado[:3]:
620             log(f" - {r[0]}: {r[1]} registros", COR.CIANO)
621     else:
622         log(" [FALHA] Nenhum registro de histórico encontrado!", COR.VERMELHO)
623         self.erros.append("Histórico vazio")
624
625 def fechar(self):
626     self.db_core.close()
627     self.db_salto.close()
628
629
630 # =====
631 # FASE 8: RELATÓRIO FINAL
632 # =====
633
634 def gerar_relatorio(metricas, testes_isolamento):
635     """Gera o relatório final completo."""
636     log("\n" + "="*70, COR.NEGRITO)
637     log("RELATÓRIO FINAL - TESTE COMPLETO DO SISTEMA CORUMBÁ SISTEMAS", COR.NEGRITO)
638     log("="*70, COR.NEGRITO)
639
640     log("\n■ ESTATÍSTICAS:", COR.AZUL)
641     log(f" Total de empresas: {metricas['total_empresas']}")
642     log(f" Total de usuários: {metricas['total_usuarios']}")
643     log(f" Total de produtos: {metricas['total_produtos']}")
644     log(f" Total de vendas: {metricas['total_vendas']}")
645     log(f" Total de cancelamentos: {metricas['total_cancelamentos']}")
646
647     log("\n⌚ TEMPOS:", COR.AZUL)
648     log(f" Criação de empresas: {metricas['tempo_empresas']:.2f}s")
649     log(f" Criação de usuários: {metricas['tempo_usuarios']:.2f}s")
650     log(f" Criação de produtos: {metricas['tempo_produtos']:.2f}s")
651     log(f" Criação de vendas: {metricas['tempo_vendas']:.2f}s")
652     tempo_total = metricas['tempo_empresas'] + metricas['tempo_usuarios'] + metricas['tempo_produtos'] + metricas['tempo_vendas']
653     log(f" Tempo total: {tempo_total:.2f}s")
654
655     log("\n⚠️ BUGS ENCONTRADOS:", COR.AMARELO)
656     if metricas['erros']:
657         log(f" Total: {len(metricas['erros'])} erros", COR.VERMELHO)
658         for erro in metricas['erros'][:10]:
659             log(f" - {erro}", COR.VERMELHO)
660         if len(metricas['erros']) > 10:
661             log(f" ... e {len(metricas['erros']) - 10} mais", COR.VERMELHO)
662     else:
663         log(" Nenhum erro encontrado!", COR.VERDE)
664
665     log("\n■ ISOLAMENTO MULTI-TENANT:", COR.ROXO)
666     if testes_isolamento.erros:
667         for erro in testes_isolamento.erros:
668             log(f" [FALHA] {erro}", COR.VERMELHO)
669     else:
670         log(" [APROVADO] Isolamento completo entre empresas", COR.VERDE)
671
672     log("\n⬇️ RESULTADO FINAL:", COR.NEGRITO)
673     total_erros = len(metricas['erros']) + len(testes_isolamento.erros)
674     if total_erros == 0:
675         log(" ✓ SISTEMA 100% FUNCIONAL", COR.VERDE)
676         log(" ✓ ISOLAMENTO MULTI-TENANT VALIDADO", COR.VERDE)
677         log(" ✓ HISTÓRICO DE VENDAS FUNCIONANDO", COR.VERDE)
678         log(" ✓ PRONTO PARA PRODUÇÃO", COR.VERDE)
679     else:
680         log(f" ⚠️ {total_erros} PROBLEMAS ENCONTRADOS", COR.VERMELHO)
681         log(" ⚠️ NECESSÁRIO CORRIGIR ANTES DE PRODUÇÃO", COR.VERMELHO)
682
683     log("\n" + "="*70, COR.NEGRITO)
684
685
686 # =====

```

```
687 # EXECUÇÃO PRINCIPAL
688 # =====
689
690 if __name__ == "__main__":
691     log("=*70, COR.NEGRITO)
692     log("CORUMBA SISTEMAS - TESTE COMPLETO DO SISTEMA ERP/PDV SAAS", COR.NEGRITO)
693     log("=*70, COR.NEGRITO)
694
695     inicio_total = time.time()
696
697     # Fase 2: Seed
698     seed = SeedData()
699     seed.limpar_dados_anteriores()
700     seed.criar_empresas()
701     seed.criar_usuarios()
702     seed.criar_produtos()
703     seed.criar_clientes()
704     seed.criar_vendas()
705     seed.fechar()
706
707     # Fase 3: Testes de isolamento
708     testes = TestesIsolamento()
709     testes.testar_isolamento_vendas()
710     testes.testar_isolamento_usuarios()
711     testes.testar_isolamento_produtos()
712     testes.testar_isolamento_historico()
713     testes.fechar()
714
715     # Fase 8: Relatório
716     gerar_relatorio(seed.mtricas, testes)
717
718     tempo_total = time.time() - inicio_total
719     log(f"\n⦿ Tempo total de execução: {tempo_total:.2f}s", COR.NEGRITO)
720
```

```

1 #!/usr/bin/env python3
2 import sys, os, time, json, random, string, hashlib
3 from datetime import datetime, timedelta
4 from sqlalchemy import create_engine, func, text
5 from sqlalchemy.orm import sessionmaker, declarative_base
6 from sqlalchemy import Column, Integer, String, Boolean, DateTime, Text, Numeric
7
8 sys.path.insert(0, '/home/hermesos/sistema-loja/backendPy')
9
10 engine_core = create_engine('sqlite:///memory:', echo=False)
11 engine_salto = create_engine('sqlite:///memory:', echo=False)
12 BaseCore = declarative_base()
13 BaseSalto = declarative_base()
14
15 class Empresa(BaseCore):
16     __tablename__ = "empresas"
17     id = Column(Integer, primary_key=True, autoincrement=True)
18     documento = Column(String(20), unique=True, index=True, nullable=False)
19     razao_social = Column(String(255), nullable=False)
20     nome_fantasia = Column(String(255))
21     plano = Column(String(50), default="BASICO")
22     ativo = Column(Boolean, default=True)
23
24 class Licenca(BaseCore):
25     __tablename__ = "licencas"
26     id = Column(Integer, primary_key=True, autoincrement=True)
27     empresa_documento = Column(String(20), index=True, nullable=False)
28     chave = Column(String(50), unique=True, index=True, nullable=False)
29     status = Column(String(20), default="ATIVA")
30     plano = Column(String(50), default="BASICO")
31     max_usuarios = Column(Integer, default=10)
32     ativo = Column(Boolean, default=True)
33
34 class Usuario(BaseCore):
35     __tablename__ = "usuarios"
36     id = Column(Integer, primary_key=True, autoincrement=True)
37     empresa_documento = Column(String(20), index=True, nullable=False)
38     nome = Column(String(255), nullable=False)
39     username = Column(String(100), index=True, nullable=False)
40     senha_hash = Column(String(255), nullable=False)
41     papel = Column(String(30), default="OPERADOR")
42     codigo = Column(String(20), nullable=True)
43     ativo = Column(Boolean, default=True)
44
45 class Produto(BaseSalto):
46     __tablename__ = "produtos"
47     id = Column(Integer, primary_key=True, autoincrement=True)
48     empresa_documento = Column(String(20), index=True, nullable=False)
49     nome = Column(String(255), nullable=False)
50     preco_venda = Column(Numeric(10, 2), default=0)
51     estoque_atual = Column(Numeric(10, 3), default=0)
52     ativo = Column(Boolean, default=True)
53
54 class Venda(BaseSalto):
55     __tablename__ = "vendas"
56     id = Column(Integer, primary_key=True, autoincrement=True)
57     empresa_documento = Column(String(20), index=True, nullable=False)
58     total = Column(Numeric(10, 2), default=0)
59     valor_recebido = Column(Numeric(10, 2), default=0)
60     troco = Column(Numeric(10, 2), default=0)
61     forma_pagamento = Column(String(50))
62     status = Column(String(20), default="CONCLUIDA")
63     data_venda = Column(DateTime, default=datetime.utcnow)
64
65 class VendaItem(BaseSalto):
66     __tablename__ = "venda itens"
67     id = Column(Integer, primary_key=True, autoincrement=True)
68     empresa_documento = Column(String(20), index=True, nullable=False)
69     venda_id = Column(Integer, nullable=False)
70     produto_id = Column(Integer, nullable=False)
71     produto_nome = Column(String(255))
72     quantidade = Column(Numeric(10, 3), nullable=False)
73     preco_unitario = Column(Numeric(10, 2), nullable=False)
74     subtotal = Column(Numeric(10, 2), nullable=False)
75
76 class HistoricoVenda(BaseSalto):
77     __tablename__ = "historico_vendas"
78     id = Column(Integer, primary_key=True, autoincrement=True)
79     empresa_documento = Column(String(20), index=True, nullable=False)
80     venda_id = Column(Integer, nullable=True)
81     numero_pedido = Column(String(50), index=True)
82     usuario_nome = Column(String(255), nullable=True)
83     valor_total = Column(Numeric(10, 2), default=0)
84     forma_pagamento = Column(String(50), nullable=True)
85     itens_json = Column(Text, nullable=True)
86     status = Column(String(20), default="CONCLUIDA")
87     tipo_operacao = Column(String(30), default="VENDA")
88     data_criacao = Column(DateTime, default=datetime.utcnow)
89     motivo_cancelamento = Column(Text, nullable=True)
90
91 class CaixaSessao(BaseSalto):
92     __tablename__ = "caixa_sesseoes"
93     id = Column(Integer, primary_key=True, autoincrement=True)
94     empresa_documento = Column(String(20), index=True, nullable=False)
95     operador = Column(String(255), nullable=False)
96     status = Column(String(20), default="ABERTO")
97     saldo_inicial = Column(Numeric(10, 2), default=0)
98
99 class CaixaTransacao(BaseSalto):
100     __tablename__ = "caixa_transacoes"
101     id = Column(Integer, primary_key=True, autoincrement=True)
102     empresa_documento = Column(String(20), index=True, nullable=False)
103     sessao_id = Column(Integer, nullable=False)
104     tipo = Column(String(20), nullable=False)
105     valor = Column(Numeric(10, 2), nullable=False)
106     descricao = Column(String(255), nullable=True)
107     operador = Column(String(255), nullable=True)
108
109 BaseCore.metadata.create_all(bind=engine_core)
110 BaseSalto.metadata.create_all(bind=engine_salto)
111 SessionCore = sessionmaker(bind=engine_core)
112 SessionSalto = sessionmaker(bind=engine_salto)
113
114 from historico import registrar_venda, registrar_cancelamento
115
116 class C:
117     V='\\033[92m';A='\\033[93m';R='\\033[91m';AZ='\\033[94m'
118     R0='\\033[95m';CI='\\033[96m';N='\\033[1m';F='\\033[0m'
119
120 def log(m,p=C.CI): print(f"{p}{m}{C.F}"); sys.stdout.flush()
121
122 def cnpj():
123     b=''
124     def d(b,p):
125         s=sum(int(x)*y for x,y in zip(b,p));r=s%11;return '0' if r<2 else str(11-r)
126     return f"{b[2:]}.{b[2:5]}.{b[5:8]}.{b[8:12]}.{d(b,[5,4,3,2,9,8,7,6,5,4,3,2])}{d(b+d(b,[5,4,3,2,9,8,7,6,5,4,3,2]))}"
127
128 def hsh(s): return hashlib.sha256(s.encode()).hexdigest()
129
130 PAPEIS=['ADMIN','GERENTE','CAIXA','VENDEDOR','OPERADOR','FINANCEIRO','ESTOQUE','SUPORTE']
131 PAG=['Dinheiro','Cartao Credito','Cartao Debito','PIX']
132 R={'e':0,'u':0,'p':0,'v':0,'h':0,'ca':0,'er':[],'ta':0,'tf':0}
133
134 log(f"{N{'='*60}\\nCORUMBA SISTEMAS - TESTE COMPLETO (SQLite)\\n{'='*60}")

```

```

135 t0=time.time()
136 db_c=SessionCore(); db_s=SessionSalto()
137
138 log("\n[1/4] 100 empresas...",C.AZ)
139 t=time.time()
140 for i in range(100):
141     try:
142         cn=cnpj()
143         db_c.add(Empresa(documento=cn,razao_social=f"Empresa {i+1} LTDA",nome_fantasia=f"Emp {i+1}",plano=random.choice(['BASICO','PRO','PREMIUM'])))
144         db_c.add(Licenca(empresa_documento=cn,chave=f"CORUMBA-{''.join(random.choices(string.ascii_uppercase+string.digits,k=4))}").
{''.join(random.choices(string.ascii_uppercase+string.digits,k=4))}-{''.join(random.choices(string.ascii_uppercase+string.digits,k=4))},status="ATIVA"))
145         if (i+1)%25==0: db_c.commit()
146         except Exception as e: db_c.rollback(); R['er'].append(f"E{i}:{e}")
147     db_c.commit(); R['e']=db_c.query(Empresa).count()
148     log(f" OK {R['e']} empresas em {time.time()-t:.1f}s",C.V)
149
150 log("\n[2/4] 1000 usuarios...",C.AZ)
151 t=time.time()
152 for em in db_c.query(Empresa).all():
153     try:
154         for u in range(10):
155             db_c.add(Usuario(empresa_documento=em.documento,nome=f"User{u+1}",username=f"u{em.id:03d}{u:02d}",senha_hash=hsh(f"{u+1:04d}"),papel=PAPEIS[u%8],codigo=f"{u+1:04d}"))
156             db_c.commit()
157         except Exception as e: db_c.rollback(); R['er'].append(f"U{e}:{e}")
158     R['u']=db_c.query(Usuario).count()
159     log(f" OK {R['u']} usuarios em {time.time()-t:.1f}s",C.V)
160
161 log("\n[3/4] 10000 produtos...",C.AZ)
162 t=time.time()
163 for em in db_c.query(Empresa).all():
164     try:
165         for p in range(100):
166             db_s.add(Produto(empresa_documento=em.documento,nome=f"Prod {p+1}",preco_venda=round(random.uniform(1.99,999.99),2),estoque_atual=random.randint(0,500)))
167             db_s.commit()
168         except Exception as e: db_s.rollback(); R['er'].append(f"P{e}:{e}")
169     R['p']=db_s.query(Produto).count()
170     log(f" OK {R['p']} produtos em {time.time()-t:.1f}s",C.V)
171
172 log("\n[4/4] 10000 vendas + historico...",C.AZ)
173 t=time.time()
174 for em in db_c.query(Empresa).all():
175     try:
176         cx=CaixaSessao(empresa_documento=em.documento,operador="admin",saldo_inicial=100,status="ABERTO")
177         db_s.add(cx); db_s.flush()
178         for v in range(100):
179             ni=random.randint(1,5);itens=[];tot=0
180             for _ in range(ni):
181                 pr=round(random.uniform(5,200),2);q=random.randint(1,3);st=pr*q;tot+=st
182                 itens.append({'produto_id':random.randint(1,10000),'produto_nome':f"P{random.randint(1,100)}",'quantidade':q,'preco_unitario':pr,'subtotal':st})
183             vr=Venda(empresa_documento=em.documento,total=tot,valor_recebido=tot+random.uniform(0,20),forma_pagamento=random.choice(PAG),status="CONCLUIDA")
184             db_s.add(vr); db_s.flush()
185             for it in itens:
186                 db_s.add(VendaItem(empresa_documento=em.documento,venda_id=vr.id,produto_id=it['produto_id'],produto_nome=it['produto_nome'],quantidade=it['quantidade'],preco_unitario=it['preco_unitario'],su
btotal=it['subtotal']))
187                 db_s.add(CaixaTransacao(empresa_documento=em.documento,sessao_id=cx.id,tipo="VENDA",valor=tot,descricao=f"V#{vr.id}",operador="admin"))
188                 registrar_venda(db=db_s,empresa_documento=em.documento,usuario_id=1,usuario_nome="Admin",usuario_papel="ADMIN",venda_id=vr.id,itens=itens,pagamentos=
[{'forma':vr.forma_pagamento,'valor':float(vr.valor_recebido)]),valor_total=tot,valor_recebido=float(vr.valor_recebido),forma_pagamento=vr.forma_pagamento,origem="PDV")
189                 if random.random()<0.1:
190                     vr.status="CANCELADA"
191                     registrar_cancelamento(db=db_s,empresa_documento=em.documento,venda_id=vr.id,usuario_id=1,usuario_nome="Admin",motivo=random.choice(["Cliente desistiu","Produto
indisponivel","Erro pagamento"]))
192                     R['ca']+=1
193             db_s.commit()
194         except Exception as e: db_s.rollback(); R['er'].append(f"V{e}:{e}");continue
195     R['v']=db_s.query(Venda).count();R['h']=db_s.query(HistoricoVenda).count()
196     log(f" OK {R['v']} vendas em {time.time()-t:.1f}s",C.V)
197     log(f" OK {R['h']} registros de historico",C.V)
198     log(f" OK {R['ca']} cancelamentos",C.V)
199     log(f"\n{' '*60}\nTESTES DE ISOLAMENTO MULTI-TENANT\n{' '*60}")
200     e1,e2=db_c.query(Empresa).limit(2).all()
201
202     v1=db_s.query(Venda).filter(Venda.empresa_documento==e1.documento).count()
203     v2=db_s.query(Venda).filter(Venda.empresa_documento==e2.documento).count()
204     if v1>0 and v2>0: log(f" OK Vendas isoladas: E1={v1}, E2={v2}",C.V);R['ta']+=1
205     else: log(f" FAIL Vendas: E1={v1}, E2={v2}",C.R);R['tf']+=1
206
207     h1=db_s.query(HistoricoVenda).filter(HistoricoVenda.empresa_documento==e1.documento).count()
208     h2=db_s.query(HistoricoVenda).filter(HistoricoVenda.empresa_documento==e2.documento).count()
209     if h1>0 and h2>0: log(f" OK Historico isolado: E1={h1}, E2={h2}",C.V);R['ta']+=1
210     else: log(f" FAIL Historico: E1={h1}, E2={h2}",C.R);R['tf']+=1
211
212     u1=db_c.query(Usuario).filter(Usuario.empresa_documento==e1.documento,Usuario.codigo=="0001").count()
213     u2=db_c.query(Usuario).filter(Usuario.empresa_documento==e2.documento,Usuario.codigo=="0001").count()
214     if u1>0 and u2>0: log(f" OK Codigo 0001 existe em ambas empresas",C.V);R['ta']+=1
215     else: log(f" INFO Codigo duplicado nao encontrado",C.A)
216
217     p1=db_s.query(Produto).filter(Produto.empresa_documento==e1.documento).count()
218     p2=db_s.query(Produto).filter(Produto.empresa_documento==e2.documento).count()
219     if p1==p2==100: log(f" OK Produtos isolados: E1={p1}, E2={p2}",C.V);R['ta']+=1
220     else: log(f" FAIL Produtos: E1={p1}, E2={p2}",C.R);R['tf']+=1
221
222     sd=db_s.execute(text("SELECT empresa_documento, COUNT(*) as c FROM historico_vendas GROUP BY empresa_documento ORDER BY c DESC LIMIT 5")).fetchall()
223     if sd: log(f" OK Historico por empresa: {len(sd)} empresas",C.V);R['ta']+=1
224     else: log(f" FAIL Sem registros de historico",C.R);R['tf']+=1
225
226     hc=db_s.query(HistoricoVenda).filter(HistoricoVenda.status=="CONCLUIDA").count()
227     hca=db_s.query(HistoricoVenda).filter(HistoricoVenda.status=="CANCELADA").count()
228     if hc>0 and hca>0: log(f" OK Consistencia: {hc} concluidos, {hca} cancelados",C.V);R['ta']+=1
229     else: log(f" FAIL Consistencia",C.R);R['tf']+=1
230
231     tt=time.time()-t0
232     log(f"\n{' '*60}\nRELATORIO FINAL\n{' '*60}")
233     log(f"Empresas: {R['e']} | Usuarios: {R['u']} | Produtos: {R['p']} | Vendas: {R['v']}")
234     log(f"Historicos: {R['h']} | Cancelamentos: {R['ca']}")
235     log(f"Testes aprovados: {R['ta']} | Falhas: {R['tf']}")
236     log(f"Erros: {len(R['er'])} | Tempo: {tt:.1f}s")
237     tp=len(R['er'])+R['tf']
238     if tp==0: log("\nSISTEMA 100% FUNCIONAL - PRONTO PARA PRODUCAO",C.V)
239     else: log(f"\n{tp} PROBLEMAS ENCONTRADOS",C.R)
240     for e in R['er'][:5]: log(f" - {e}",C.R)
241     log(f"\n{' '*60}")
242     db_c.close(); db_s.close()
243

```

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Painel Admin - Corumbá Sistemas</title>
7   <script src="https://cdn.tailwindcss.com"></script>
8   <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css" rel="stylesheet">
9 </head>
10 <body class="bg-slate-50 font-sans antialiased text-slate-800">
11
12   <div class="flex h-screen overflow-hidden">
13
14     <aside class="w-64 bg-slate-900 text-white flex flex-col shadow-xl">
15       <div class="p-6 flex items-center justify-center border-b border-slate-700">
16         <span class="text-2xl font-bold text-emerald-400"><i class="fa-solid fa-bolt mr-2"></i>Corumbá</span>
17       </div>
18
19       <nav class="flex-1 px-4 py-6 space-y-2">
20         <a href="#" class="flex items-center px-4 py-3 bg-blue-600 text-white rounded-lg transition-colors">
21           <i class="fa-solid fa-chart-line w-6"></i>
22           <span class="font-medium">Dashboard</span>
23         </a>
24         <a href="#" class="flex items-center px-4 py-3 text-slate-300 hover:bg-slate-800 hover:text-white rounded-lg transition-colors">
25           <i class="fa-solid fa-users-gear w-6"></i>
26           <span class="font-medium">Usuários e Permissões</span>
27         </a>
28         <a href="#" class="flex items-center px-4 py-3 text-slate-300 hover:bg-slate-800 hover:text-white rounded-lg transition-colors">
29           <i class="fa-solid fa-key w-6"></i>
30           <span class="font-medium">Licenças (Máquinas)</span>
31         </a>
32         <a href="#" class="flex items-center px-4 py-3 text-slate-300 hover:bg-slate-800 hover:text-white rounded-lg transition-colors">
33           <i class="fa-solid fa-store w-6"></i>
34           <span class="font-medium">Lojas Clientes</span>
35         </a>
36       </nav>
37
38       <div class="p-4 border-t border-slate-700">
39         <div class="flex items-center gap-3">
40           <div class="w-10 h-10 rounded-full bg-slate-700 flex items-center justify-center text-emerald-400 font-bold">
41             AD
42           </div>
43           <div>
44             <p class="text-sm font-medium">Administrador</p>
45             <p class="text-xs text-slate-400">Master</p>
46           </div>
47         </div>
48       </div>
49     </aside>
50
51     <main class="flex-1 relative overflow-y-auto focus:outline-none">
52       <div class="p-8">
53         <h1 class="text-3xl font-bold text-slate-800 mb-6">Visão Geral do Sistema</h1>
54
55         <div class="grid grid-cols-1 md:grid-cols-3 gap-6 mb-8">
56           <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
57             <div class="flex items-center justify-between mb-4">
58               <h3 class="text-slate-500 font-medium">Lojas Ativas</h3>
59               <span class="flex items-center justify-center w-8 h-8 rounded-full bg-blue-100 text-blue-600">
60                 <i class="fa-solid fa-building"></i>
61               </span>
62             </div>
63             <p class="text-3xl font-bold text-slate-800">1</p>
64           </div>
65
66           <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
67             <div class="flex items-center justify-between mb-4">
68               <h3 class="text-slate-500 font-medium">Licenças Emitidas</h3>
69               <span class="flex items-center justify-center w-8 h-8 rounded-full bg-emerald-100 text-emerald-600">
70                 <i class="fa-solid fa-key"></i>
71               </span>
72             </div>
73             <p class="text-3xl font-bold text-slate-800">1</p>
74           </div>
75
76           <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
77             <div class="flex items-center justify-between mb-4">
78               <h3 class="text-slate-500 font-medium">Usuários Totais</h3>
79               <span class="flex items-center justify-center w-8 h-8 rounded-full bg-purple-100 text-purple-600">
80                 <i class="fa-solid fa-users"></i>
81               </span>
82             </div>
83             <p class="text-3xl font-bold text-slate-800">1</p>
84           </div>
85         </div>
86
87         <div class="bg-white rounded-xl shadow-sm border border-slate-200 p-6">
88           <h2 class="text-xl font-bold text-slate-800 mb-4">Gestão Rápida</h2>
89           <p class="text-slate-600 mb-6">Selecione uma opção no menu lateral para gerenciar as permissões de acesso ao PDV Desktop, ou criar novas chaves de licença para máquinas.</p>
90           <button class="bg-slate-900 text-white px-6 py-2 rounded-lg font-medium hover:bg-slate-800 transition-colors">
91             <i class="fa-solid fa-plus mr-2"></i> Nova Loja Cliente
92           </button>
93         </div>
94
95       </div>
96     </main>
97   </div>
98
99 </body>
100 </html>
```

```

1 """
2 routers/clientes.py - CRUD de clientes + historico de compras + fidelidade
3 Multi-tenant: todas as operacoes filtram por empresa_documento.
4 """
5 from fastapi import APIRouter, Depends, HTTPException, Query
6 from sqlalchemy.orm import Session
7 from sqlalchemy import func
8 from typing import Optional
9 from datetime import datetime, timedelta
10
11 from database import get_db_salto, get_db_core
12 from models import Cliente, Venda, VendaItem
13 from schemas import ClienteCreate, ClienteUpdate, ClienteResponse, ClienteHistoricoResponse
14 from models_core import Usuario
15 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
16
17 router = APIRouter(prefix="/clientes", tags=["Clientes"])
18
19
20 @router.post("/", response_model=ClienteResponse)
21 def criar_cliente(
22     cliente: ClienteCreate,
23     empresa_documento: str = Depends(get_empresa_from_user),
24     current_user: Usuario = Depends(get_current_user),
25     db: Session = Depends(get_db_salto)):
26     obj = Cliente(empresa_documento=empresa_documento, **cliente.model_dump())
27     db.add(obj)
28     db.commit()
29     db.refresh(obj)
30     return obj
31
32
33 @router.get("/", response_model=list[ClienteResponse])
34 def listar_clientes(
35     empresa_documento: str = Depends(get_empresa_from_user),
36     busca: Optional[str] = None,
37     ativo: Optional[bool] = True,
38     ordem: str = "nome", # nome, pontos, recente
39     current_user: Usuario = Depends(get_current_user),
40     db: Session = Depends(get_db_salto)
41 ):
42     query = db.query(Cliente).filter(Cliente.empresa_documento == empresa_documento)
43     if ativo is not None:
44         query = query.filter(Cliente.ativo == ativo)
45     if busca:
46         query = query.filter(
47             Cliente.nome.ilike(f"%{busca}%") |
48             Cliente.telefone.ilike(f"%{busca}%") |
49             Cliente.cpf_cnpj.ilike(f"%{busca}%") |
50             Cliente.email.ilike(f"%{busca}%")
51         )
52     if ordem == "pontos":
53         query = query.order_by(Cliente.pontos_fidelidade.desc())
54     elif ordem == "recente":
55         query = query.order_by(Cliente.id.desc())
56     else:
57         query = query.order_by(Cliente.nome)
58     return query.all()
59
60
61 @router.get("/{cliente_id}", response_model=ClienteResponse)
62 def get_cliente(
63     cliente_id: int,
64     empresa_documento: str = Depends(get_empresa_from_user),
65     current_user: Usuario = Depends(get_current_user),
66     db: Session = Depends(get_db_salto)
67 ):
68     obj = db.query(Cliente).filter(
69         Cliente.id == cliente_id,
70         Cliente.empresa_documento == empresa_documento
71     ).first()
72     if not obj:
73         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
74     return obj
75
76
77 @router.put("/{cliente_id}", response_model=ClienteResponse)
78 def atualizar_cliente(
79     cliente_id: int,
80     dados: ClienteUpdate,
81     empresa_documento: str = Depends(get_empresa_from_user),
82     current_user: Usuario = Depends(get_current_user),
83     db: Session = Depends(get_db_salto)
84 ):
85     obj = db.query(Cliente).filter(
86         Cliente.id == cliente_id,
87         Cliente.empresa_documento == empresa_documento
88     ).first()
89     if not obj:
90         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
91     for key, value in dados.model_dump(exclude_unset=True).items():
92         setattr(obj, key, value)
93     db.commit()
94     db.refresh(obj)
95     return obj
96
97
98 @router.delete("/{cliente_id}")
99 def desativar_cliente(
100     cliente_id: int,
101     empresa_documento: str = Depends(get_empresa_from_user),
102     current_user: Usuario = Depends(get_current_user),
103     db: Session = Depends(get_db_salto)
104 ):
105     obj = db.query(Cliente).filter(
106         Cliente.id == cliente_id,
107         Cliente.empresa_documento == empresa_documento
108     ).first()
109     if not obj:
110         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
111     obj.ativo = False
112     db.commit()
113     return {"status": "Cliente desativado"}
114
115
116 # =====
117 # HISTORICO DE COMPRAS + FIDELIDADE
118 # =====
119
120 @router.get("/{cliente_id}/historico")
121 def historico_compras(
122     cliente_id: int,
123     empresa_documento: str = Depends(get_empresa_from_user),
124     limite: int = 30,
125     current_user: Usuario = Depends(get_current_user),
126     db: Session = Depends(get_db_salto)
127 ):
128     """Retorna historico de compras do cliente com detalhes."""
129     cliente = db.query(Cliente).filter(
130         Cliente.id == cliente_id,
131         Cliente.empresa_documento == empresa_documento
132     ).first()
133     if not cliente:
134         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")

```

```

135
136 vendas = db.query(Venda).filter(
137     Venda.cliente_id == cliente_id,
138     Venda.empresa_documento == empresa_documento,
139     Venda.status == "CONCLUIDA"
140 ).order_by(Venda.data_venda.desc()).limit(limite).all()
141
142 total_gasto = db.query(func.sum(Venda.total)).filter(
143     Venda.cliente_id == cliente_id,
144     Venda.empresa_documento == empresa_documento,
145     Venda.status == "CONCLUIDA"
146 ).scalar() or 0
147
148 return {
149     "cliente": cliente.nome,
150     "total_gasto": float(total_gasto),
151     "pontos_fidelidade": cliente.pontos_fidelidade,
152     "compras": [
153         {
154             "venda_id": v.id,
155             "data": str(v.data_venda),
156             "total": float(v.total),
157             "forma_pagamento": v.forma_pagamento,
158             "itens": [
159                 {
160                     "produto": item.produto_nome,
161                     "quantidade": float(item.quantidade),
162                     "preco": float(item.preco_unitario),
163                     "subtotal": float(item.subtotal)
164                 }
165                 for item in v.itens
166             ]
167         }
168         for v in vendas
169     ]
170 }
171
172
173 @router.post("/{cliente_id}/pontos")
174 def adicionar_pontos(
175     cliente_id: int,
176     pontos: int,
177     empresa_documento: str = Depends(get_empresa_from_user),
178     current_user: Usuario = Depends(get_current_user),
179     db: Session = Depends(get_db_salto)
180 ):
181     """Adiciona pontos de fidelidade ao cliente."""
182     cliente = db.query(Cliente).filter(
183         Cliente.id == cliente_id,
184         Cliente.empresa_documento == empresa_documento
185     ).first()
186     if not cliente:
187         raise HTTPException(status_code=404, detail="Cliente nao encontrado.")
188     cliente.pontos_fidelidade += pontos
189     db.commit()
190     return {"pontos_totais": cliente.pontos_fidelidade}
191
192
193 @router.get("/fidelidade/ranking")
194 def ranking_fidelidade(
195     empresa_documento: str = Depends(get_empresa_from_user),
196     limite: int = 20,
197     current_user: Usuario = Depends(get_current_user),
198     db: Session = Depends(get_db_salto)
199 ):
200     """Top clientes por pontuacao de fidelidade."""
201     return db.query(Cliente).filter(
202         Cliente.empresa_documento == empresa_documento,
203         Cliente.ativo == True,
204         Cliente.pontos_fidelidade > 0
205     ).order_by(Cliente.pontos_fidelidade.desc()).limit(limite).all()
206

```



```

1 """
2 routers/dashboard.py - Metricas e informacoes consolidadas
3 """
4 from fastapi import APIRouter, Depends, HTTPException, Query
5 from sqlalchemy.orm import Session
6 from sqlalchemy import func
7 from datetime import datetime, timedelta
8
9 from database import get_db_salto, get_db_core
10 from models import Venda, Produto, Cliente, EstoqueMovimentacao
11 from models_core import Usuario
12 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
13
14 router = APIRouter(prefix="/dashboard", tags=["Dashboard"])
15
16
17 @router.get("/")
18 def get_dashboard(
19     empresa_documento: str = Depends(get_empresa_from_user),
20     current_user: Usuario = Depends(get_current_user),
21     db: Session = Depends(get_db_salto)
22 ):
23     hoje = datetime.utcnow().date()
24     ontem = hoje - timedelta(days=1)
25     inicio_mes = hoje.replace(day=1)
26
27     def _date_str(d):
28         return d.isoformat()
29
30     # ---- Vendas hoje ----
31     vendas_hoje_q = db.query(func.sum(Venda.total), func.count(Venda.id), func.avg(Venda.total)).filter(
32         Venda.empresa_documento == empresa_documento,
33         Venda.status == "CONCLUIDA",
34         func.date(Venda.data_venda) == hoje
35     ).first()
36     vendas_hoje = float(vendas_hoje_q[0] or 0)
37     qtd_vendas_hoje = int(vendas_hoje_q[1] or 0)
38     ticket_medio_hoje = float(vendas_hoje_q[2] or 0)
39
40     # ---- Vendas ontem ----
41     vendas_ontem_q = db.query(func.sum(Venda.total)).filter(
42         Venda.empresa_documento == empresa_documento,
43         Venda.status == "CONCLUIDA",
44         func.date(Venda.data_venda) == ontem
45     ).first()
46     vendas_ontem = float(vendas_ontem_q[0] or 0)
47
48     # ---- Vendas mes ----
49     vendas_mes_q = db.query(func.sum(Venda.total)).filter(
50         Venda.empresa_documento == empresa_documento,
51         Venda.status == "CONCLUIDA",
52         func.date(Venda.data_venda) >= inicio_mes
53     ).first()
54     vendas_mes = float(vendas_mes_q[0] or 0)
55
56     # ---- Estoque baixo ----
57     produtos_estoque_baixo = db.query(func.count(Produto.id)).filter(
58         Produto.empresa_documento == empresa_documento,
59         Produto.ativo == True,
60         Produto.estoque_atual <= Produto.estoque_minimo
61     ).scalar() or 0
62
63     # ---- Clientes ativos ----
64     clientes_ativos = db.query(func.count(Cliente.id)).filter(
65         Cliente.empresa_documento == empresa_documento,
66         Cliente.ativo == True
67     ).scalar() or 0
68
69     # ---- Vendedores do dia (who made sales today) ----
70     vendedores_dia = db.query(
71         Venda.vendedor_nome,
72         func.count(Venda.id).label("qtd_vendas"),
73         func.sum(Venda.total).label("total_vendido")
74     ).filter(
75         Venda.empresa_documento == empresa_documento,
76         Venda.status == "CONCLUIDA",
77         func.date(Venda.data_venda) == hoje
78     ).group_by(Venda.vendedor_nome).order_by(func.sum(Venda.total).desc()).all()
79
80     # ---- Ultimas vendas ----
81     ultimas = db.query(Venda).filter(
82         Venda.empresa_documento == empresa_documento,
83         Venda.status == "CONCLUIDA"
84     ).order_by(Venda.data_venda.desc()).limit(10).all()
85
86     return {
87         "vendas_hoje": vendas_hoje,
88         "vendas_ontem": vendas_ontem,
89         "vendas_mes": vendas_mes,
90         "quantidade_vendas_hoje": qtd_vendas_hoje,
91         "ticket_medio_hoje": round(ticket_medio_hoje, 2),
92         "produtos_estoque_baixo": produtos_estoque_baixo,
93         "clientes_ativos": clientes_ativos,
94         "vendedores_do_dia": [
95             {
96                 "nome": v[0] or "Sem vendedor",
97                 "qtd_vendas": int(v[1]),
98                 "total_vendido": float(v[2])
99             }
100             for v in vendedores_dia
101         ],
102         "ultimas_vendas": [
103             {
104                 "id": v.id,
105                 "total": float(v.total),
106                 "vendedor": v.vendedor_nome,
107                 "cliente_id": v.cliente_id,
108                 "data": str(v.data_venda)
109             }
110             for v in ultimas
111         ]
112     }
113

```

```
1 # SaltoGestão - Sistema Desktop
2
3 Sistema de gestão comercial desktop desenvolvido em Python com CustomTkinter.
4
5 ## Como Executar
6
7 ```bash
8 cd SaltoGestao/desktop
9 chmod +x run.sh
10 ./run.sh
11 ```
12
13 Ou manualmente:
14
15 ```bash
16 cd SaltoGestao/desktop
17 python3 -m venv venv
18 source venv/bin/activate
19 pip install -r requirements.txt
20 python3 main.py
21 ```
22
23 ## Requisitos
24
25 - Python 3.10+
26 - tkinter (geralmente já vem com o Python)
27 - customtkinter (instalado automaticamente pelo run.sh)
28
29 ## Telas
30
31 - 🏠 **Login** - Admin, Vendedor ou Cliente
32 - 📊 **Dashboard** - Estatísticas e atividades recentes
33 - 👤 **Clientes** - CRUD completo com busca
34 - 📦 **Produtos** - Catálogo de produtos (grid visual)
35 - 💰 **Vendas** - Histórico e nova venda
36 - ✍️ **Assinaturas** - Gestão de assinaturas
37 - 📄 **Relatórios** - Exportação de relatórios
38
39 ## Configuração
40
41 O sistema conecta ao backend FastAPI. A URL do servidor pode ser configurada na tela de login.
42
43 URL padrão: `http://localhost:8080`
44
```

```

1  """
2  =====
3  SALTO GESTAO - SISTEMA DESKTOP v2.0
4  Design Moderno e Clean - Paleta Corumba Sistemas
5  =====
6
7  Paleta de cores:
8  --cor-primaria:      #0b1a30 (azul escuro - marca Corumba)
9  --cor-primaria-80:   #1a2d4a (variante 80%)
10 --cor-primaria-60:   #2d4a6e (variante 60%)
11 --cor-primaria-40:   #5a7ea8 (variante 40%)
12 --cor-primaria-20:   #9bb5d0 (variante 20%)
13 --cor-primaria-10:   #c5d5e8 (variante 10%)
14
15 --cor-fundo:         #f5f7fa (cinza muito claro - fundo app)
16 --cor-fundo-card:    #ffffff (branco puro - cards)
17 --cor-fundo-hover:   #ebeeef5 (hover sutil)
18
19 --cor-texto:         #0b1a30 (texto principal)
20 --cor-texto-80:      #3c4a5e (texto secundario)
21 --cor-texto-60:      #6d7a8e (texto terciario)
22 --cor-texto-40:      #9eaabe (texto desabilitado)
23
24 --cor-sucesso:       #22c55e (verde)
25 --cor-alerta:        #f59e0b (amarelo/laranja)
26 --cor-erro:          #ef4444 (vermelho)
27 --cor-info:          #3b82f6 (azul claro)
28
29 --cor-borda:         #e2e8f0 (bordas sutis)
30 --cor-borda-focus:   #0b1a30 (borda em foco)
31
32 Tipografia:
33 --font-family: 'Segoe UI', 'Inter', system-ui, sans-serif
34 --font-mono: 'Cascadia Code', 'Consolas', monospace
35
36 Espacamento:
37 --space-xs: 4px
38 --space-sm: 8px
39 --space-md: 16px
40 --space-lg: 24px
41 --space-xl: 32px
42 --space-2xl: 48px
43
44 Borda:
45 --radius-sm: 8px
46 --radius-md: 12px
47 --radius-lg: 16px
48 --radius-xl: 20px
49 --radius-full: 9999px
50
51 Sombras:
52 --shadow-sm: 0 1px 3px rgba(11,26,48,0.08)
53 --shadow-md: 0 4px 12px rgba(11,26,48,0.10)
54 --shadow-lg: 0 8px 24px rgba(11,26,48,0.12)
55 --shadow-xl: 0 16px 48px rgba(11,26,48,0.16)
56
57 Transicoes:
58 --transition-fast: 150ms ease
59 --transition-normal: 250ms ease
60 --transition-slow: 350ms ease
61 """
62
63 import flet as ft
64 import requests
65 import json
66 import threading
67 from datetime import datetime
68
69 # =====
70 # DESIGN SYSTEM - CONSTANTES
71 # =====
72
73 # Paleta Corumba Sistemas
74 class Cores:
75     """Paleta de cores Corumba Sistemas"""
76     # Primaria
77     P = "#0b1a30" # Primaria (marca)
78     P80 = "#1a2d4a"
79     P60 = "#2d4a6e"
80     P40 = "#5a7ea8"
81     P20 = "#9bb5d0"
82     P10 = "#c5d5e8"
83     P5 = "#e2e8f0"
84
85     # Fundos
86     FUNDO = "#f5f7fa"
87     FUNDO_CARD = "#ffffff"
88     FUNDO_HOVER = "#ebeeef5"
89
90     # Texto
91     TEXTO = "#0b1a30"
92     TEXTO_80 = "#3c4a5e"
93     TEXTO_60 = "#6d7a8e"
94     TEXTO_40 = "#9eaabe"
95
96     # Status
97     SUCESSO = "#22c55e"
98     ALERTA = "#f59e0b"
99     ERRO = "#ef4444"
100    INFO = "#3b82f6"
101
102    # Bordas
103    BORDA = "#e2e8f0"
104    BORDA_FOCUS = "#0b1a30"
105    ESCURO = "#0b1a30"
106
107    # Transparentes
108    OVERLAY = "rgba(11,26,48,0.5)"
109
110
111 class Espacamento:
112     """Escala de espacamento"""
113     XS = 4
114     SM = 8
115     MD = 16
116     LG = 24
117     XL = 32
118     XXL = 48
119
120
121 class Bordas:
122     """Raios de borda"""
123     SM = 8
124     MD = 12
125     LG = 16
126     XL = 20
127     FULL = 9999
128
129
130 # =====
131 # COMPONENTES BASE
132 # =====
133
134 class DS:

```

```

135 """Design System - Componentes reutilizaveis"""
136
137 @staticmethod
138 def container(
139     content=None,
140     bgcolor=Cores.FUNDO_CARD,
141     border_radius=Bordas.MD,
142     padding=Espacamento.MD,
143     margin=None,
144     width=None,
145     height=None,
146     border=None,
147     shadow=True,
148     expand=False,
149     col=None,
150 ):
151     """Card/container base com estilo padrao"""
152     return ft.Container(
153         content=content,
154         bgcolor=bgcolor,
155         border_radius=border_radius,
156         padding=padding,
157         margin=margin,
158         width=width,
159         height=height,
160         border=border,
161         expand=expand,
162         col=col,
163         shadow=ft.BoxShadow(
164             spread_radius=0,
165             blur_radius=8,
166             color=ft.Colors.with_opacity(0.06, Cores.ESCURO),
167             offset=ft.Offset(0, 2),
168         ) if shadow else None,
169     )
170
171 @staticmethod
172 def titulo(texto, size=16, weight=ft.FontWeight.W_600, color=Cores.TEXT0):
173     """Titulo padronizado"""
174     return ft.Text(
175         texto,
176         size=size,
177         weight=weight,
178         color=color,
179         font_family="Segoe UI",
180         selectable=True,
181     )
182
183 @staticmethod
184 def subtitulo(texto, size=12, color=Cores.TEXT0_60):
185     """Subtitulo padronizado"""
186     return ft.Text(
187         texto,
188         size=size,
189         color=color,
190         font_family="Segoe UI",
191         selectable=True,
192     )
193
194 @staticmethod
195 def paragrafo(texto, size=13, color=Cores.TEXT0_80, selectable_text=True):
196     """Paragrafo padronizado"""
197     return ft.Text(
198         texto,
199         size=size,
200         color=color,
201         font_family="Segoe UI",
202         selectable=selectable_text,
203         weight=ft.FontWeight.W_400,
204     )
205
206 @staticmethod
207 def campo(
208     label,
209     prefix_icon=None,
210     password=False,
211     can_reveal_password=False,
212     width=None,
213     height=48,
214     text_size=14,
215     hint_text=None,
216     on_change=None,
217     on_focus=None,
218     suffix=None,
219 ):
220     """Campo de entrada padronizado estilo moderno"""
221     return ft.TextField(
222         label=label,
223         prefix_icon=prefix_icon,
224         password=password,
225         can_reveal_password=can_reveal_password,
226         width=width,
227         height=height,
228         text_size=text_size,
229         hint_text=hint_text,
230         hint_style=ft.TextStyle(
231             size=13,
232             color=Cores.TEXT0_40,
233             font_family="Segoe UI",
234         ),
235         label_style=ft.TextStyle(
236             size=12,
237             color=Cores.TEXT0_60,
238             font_family="Segoe UI",
239         ),
240         text_style=ft.TextStyle(
241             size=text_size,
242             color=Cores.TEXT0,
243             font_family="Segoe UI",
244         ),
245         border_color=Cores.BORDA,
246         focused_border_color=Cores.P,
247         focused_border_width=2,
248         border_radius=Bordas.MD,
249         bgcolor=Cores.FUNDO_CARD,
250         content_padding=ft.Padding.only(left=16, right=16, top=12, bottom=12),
251         on_change=on_change,
252         on_focus=on_focus,
253         suffix=suffix,
254     )
255
256 @staticmethod
257 def botao(
258     texto,
259     on_click=None,
260     bgcolor=Cores.P,
261     color=Cores.FUNDO_CARD,
262     width=None,
263     height=44,
264     icon=None,
265     disabled=False,
266     style=None,
267     expand=False,
268 ):
269     """Botao padrao com estilo moderno"""
270     btn_style = ft.ButtonStyle(
271         bgcolor={
272             ft.ControlState.DEFAULT: bgcolor,

```

```

273         ft.ControlState.HOVERED: Cores.P80,
274         ft.ControlState.DISABLED: Cores.TEXT0_40,
275     },
276     color=color,
277     shape=ft.RoundedRectangleBorder(radius=Bordas.MD),
278     padding=ft.padding.symmetric(horizontal=24, vertical=12),
279     elevation={
280         ft.ControlState.DEFAULT: 0,
281         ft.ControlState.HOVERED: 2,
282     },
283     animation_duration=150,
284     text_style=ft.TextStyle(
285         size=14,
286         weight=ft.FontWeight.W_600,
287         font_family="Segoe UI",
288     ),
289 )
290
291 content = texto
292 if icon:
293     content = ft.Row(
294         [
295             ft.Icon(icon, size=18, color=color),
296             ft.Text(texto, color=color, size=14, weight=ft.FontWeight.W_600),
297         ],
298         alignment=ft.MainAxisAlignment.CENTER,
299         spacing=8,
300     )
301
302 return ft.ElevatedButton(
303     content=content,
304     on_click=on_click,
305     width=width,
306     height=height,
307     disabled=disabled,
308     style=btn_style if style is None else style,
309     expand=expand,
310 )
311
312 @staticmethod
313 def botao_outline(
314     texto,
315     on_click=None,
316     color=Cores.P,
317     width=None,
318     height=44,
319     icon=None,
320     expand=False,
321 ):
322     """Botao secundario outline"""
323     style = ft.ButtonStyle(
324         bgcolor={
325             ft.ControlState.DEFAULT: Cores.FUNDO_CARD,
326             ft.ControlState.HOVERED: Cores.P5,
327         },
328         color={
329             ft.ControlState.DEFAULT: color,
330             ft.ControlState.HOVERED: Cores.P,
331         },
332         shape=ft.RoundedRectangleBorder(radius=Bordas.MD),
333         side=ft.BorderSide(width=1.5, color=Cores.BORDA),
334         padding=ft.padding.symmetric(horizontal=24, vertical=12),
335         animation_duration=150,
336         text_style=ft.TextStyle(
337             size=14,
338             weight=ft.FontWeight.W_500,
339             font_family="Segoe UI",
340         ),
341     )
342
343 content = texto
344 if icon:
345     content = ft.Row(
346         [
347             ft.Icon(icon, size=18, color=color),
348             ft.Text(texto, color=color, size=14, weight=ft.FontWeight.W_500),
349         ],
350         alignment=ft.MainAxisAlignment.CENTER,
351         spacing=8,
352     )
353
354 return ft.OutlinedButton(
355     content=content,
356     on_click=on_click,
357     width=width,
358     height=height,
359     style=style,
360     expand=expand,
361 )
362
363 @staticmethod
364 def badge(texto, color=Cores.P20, text_color=Cores.P, border_radius=Bordas.FULL):
365     """Badge/tag padrao"""
366     return ft.Container(
367         content=ft.Text(
368             texto,
369             size=11,
370             weight=ft.FontWeight.W_500,
371             color=text_color,
372             font_family="Segoe UI",
373         ),
374         bgcolor=color,
375         border_radius=border_radius,
376         padding=ft.padding.symmetric(horizontal=10, vertical=4),
377     )
378
379 @staticmethod
380 def icone_botao(icon, tooltip=None, on_click=None, icon_color=Cores.TEXT0_60, size=20):
381     """Botao de icone circular"""
382     return ft.IconButton(
383         icon=icon,
384         tooltip=tooltip,
385         icon_color=icon_color,
386         icon_size=size,
387         on_click=on_click,
388         style=ft.ButtonStyle(
389             shape=ft.CircleBorder(),
390             padding=8,
391             bgcolor={
392                 ft.ControlState.DEFAULT: Cores.P5,
393                 ft.ControlState.HOVERED: Cores.P10,
394             },
395         ),
396     )
397
398 @staticmethod
399 def card(
400     titulo=None,
401     subtitulo=None,
402     content=None,
403     padding=Espacamento.MD,
404     margin=None,
405     width=None,
406     height=None,
407     on_click=None,
408     header_icon=None,
409     header_color=Cores.P,
410     header_actions=None,

```

```

411     expand=False,
412     col=None,
413 ):
414     """Card com estilo moderno"""
415     # Header do card
416     header = None
417     if titulo:
418         header_content = []
419         if header_icon:
420             header_content.append(ft.Icon(header_icon, size=18, color=header_color))
421             header_content.append(DS.titulo(titulo, size=14, color=Cores.TEXT0))
422         if subtítulo:
423             header_content.append(ft.Container(width=8))
424             header_content.append(DS.subtitulo(subtitulo))
425         if header_actions:
426             header_content.append(ft.Row(expand=True, controls=[], alignment=ft.MainAxisAlignment.END))
427             for action in header_actions:
428                 header_content.append(action)
429
430         header = ft.Container(
431             content=ft.Row(header_content, spacing=8, alignment=ft.MainAxisAlignment.START),
432             padding=ft.padding.only(left=padding, right=padding, top=padding, bottom=Espacamento.SM) if content else ft.padding.all(padding),
433             border=ft.border.only(bottom=ft.BorderSide(1, Cores.BORDA)) if content else None,
434         )
435
436     # Footer com conteudo
437     body = None
438     if content:
439         body = ft.Container(
440             content=content,
441             padding=ft.padding.all(padding),
442             expand=True,
443         )
444
445     children = [c for c in [header, body] if c]
446
447     card = ft.Container(
448         content=ft.Column(children, spacing=0, expand=True) if len(children) > 1 else children[0],
449         bgcolor=Cores.FUNDO_CARD,
450         border_radius=Bordas.LG,
451         padding=0,
452         margin=margin,
453         width=width,
454         height=height,
455         expand=expand,
456         col=col,
457         shadow=ft.BoxShadow(
458             spread_radius=0,
459             blur_radius=8,
460             color=ft.Colors.with_opacity(0.06, Cores.ESCURO),
461             offset=ft.Offset(0, 2),
462         ),
463         animate=ft.Animation(150, ft.AnimationCurve.EASE_OUT),
464         on_hover=Lambda e: None, # Placeholder para hover animation
465         on_click=on_click,
466     )
467
468     return card
469
470 @staticmethod
471 def divider():
472     """Divisor padrao"""
473     return ft.Container(
474         height=1,
475         bgcolor=Cores.BORDA,
476         margin=ft.padding.symmetric(vertical=Espacamento.SM),
477     )
478
479 @staticmethod
480 def loading_spinner(size=24, color=Cores.P):
481     """Spinner de carregamento"""
482     return ft.ProgressRing(
483         width=size,
484         height=size,
485         stroke_width=3,
486         color=color,
487         bgcolor=ft.Colors.with_opacity(0.1, color),
488     )
489
490 @staticmethod
491 def stat_card(titulo, valor, icone=None, cor=Cores.P, subtitulo=None, tendencia=None):
492     """Card de estatistica/dashboard"""
493     children = []
494
495     # Icon + Titulo row
496     titulo_row = []
497     if icone:
498         titulo_row.append(ft.Container(
499             content=ft.Icon(icone, size=20, color=cor),
500             bgcolor=ft.Colors.with_opacity(0.1, cor),
501             border_radius=Bordas.SM,
502             padding=8,
503         ))
504     titulo_row.append(ft.Container(width=4))
505     titulo_row.append(DS.subtitulo(titulo, size=11))
506     children.append(ft.Row(titulo_row, spacing=8))
507
508     children.append(ft.Container(height=4))
509
510     # Valor grande
511     children.append(ft.Text(
512         str(valor),
513         size=20,
514         weight=ft.FontWeight.W_700,
515         color=Cores.TEXT0,
516         font_family="Segoe UI",
517     ))
518
519     # Subtitulo/Tendencia
520     if subtitulo or tendencia:
521         extras = []
522         if subtitulo:
523             extras.append(DS.subtitulo(subtitulo, size=10))
524         if tendencia:
525             cor_trend = Cores.SUCESSO if tendencia > 0 else Cores.ERRO
526             extras.append(ft.Text(
527                 ("+" if tendencia > 0 else "") + str(tendencia) + "%",
528                 size=10,
529                 weight=ft.FontWeight.W_600,
530                 color=cor_trend,
531             ))
532         children.append(ft.Row(extras, spacing=4))
533
534     return DS.card(
535         content=ft.Column(children, spacing=2, alignment=ft.MainAxisAlignment.START),
536         padding=Espacamento.MD,
537         width=200,
538         height=90,
539     )
540
541 @staticmethod
542 def tabela_header(*headers):
543     """Header de tabela"""
544     cells = [ft.Text(h, size=11, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60, font_family="Segoe UI") for h in headers]
545     return ft.Container(
546         content=ft.Row(cells, spacing=Espacamento.MD, alignment=ft.MainAxisAlignment.START),
547         padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=Espacamento.SM),
548         bgcolor=Cores.FUNDO,

```

```

549         border_radius=ft.border_radius.only(top_left=Bordas.MD, top_right=Bordas.MD),
550     )
551
552     @staticmethod
553     def tabela_row(*cells, selected=False, hover=True):
554         """Linha de tabela"""
555         controls = []
556         for cell in cells:
557             if isinstance(cell, str):
558                 controls.append(ft.Text(cell, size=12, color=Cores.TEXT0_80, font_family="Segoe UI"))
559             else:
560                 controls.append(cell)
561
562         return ft.Container(
563             content=ft.Row(controls, spacing=Espacamento.MD, alignment=ft.MainAxisAlignment.START),
564             padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
565             bgcolor=Cores.P5 if selected else Cores.FUNDO_CARD,
566             border=ft.border.only(bottom=ft.BorderSide(0.5, Cores.BORDA)),
567             animate=ft.Animation(100, ft.AnimationCurve.EASE_OUT),
568         )
569
570     @staticmethod
571     def sidebar_item(label, icon, selected=False, on_click=None, badge_count=None):
572         """Item da sidebar de navegacao"""
573         bg = ft.Colors.with_opacity(0.08, Cores.P) if selected else ft.Colors.TRANSPARENT
574         text_weight = ft.FontWeight.W_600 if selected else ft.FontWeight.W_400
575         icon_color = Cores.P if selected else Cores.TEXT0_60
576         text_color = Cores.P if selected else Cores.TEXT0_80
577
578         row_controls = [
579             ft.Icon(icon, size=20, color=icon_color),
580             ft.Container(width=4,
581                 ft.Text(label, size=13, weight=text_weight, color=text_color, font_family="Segoe UI"),
582             ]
583
584         if badge_count:
585             row_controls.append(
586                 ft.Container(expand=True, content=None, alignment=ft.alignment.center_right)
587             )
588             row_controls.append(DS.badge(str(badge_count), color=Cores.ERRO, text_color=Cores.FUNDO_CARD))
589
590         return ft.Container(
591             content=ft.Row(row_controls, spacing=12, alignment=ft.MainAxisAlignment.START),
592             padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
593             bgcolor=bg,
594             border_radius=Bordas.MD,
595             on_click=on_click,
596             animate=ft.Animation(150, ft.AnimationCurve.EASE_OUT),
597             on_hover=lambda e: (
598                 e.control.__setattr__('bgcolor', ft.Colors.with_opacity(0.08, Cores.P) if e.data == "true" and not selected else None),
599                 e.control.update(),
600             ) if True else None,
601             margin=ft.padding.symmetric(horizontal=Espacamento.SM, vertical=2),
602         )
603
604     @staticmethod
605     def nav_bar(title, page, user_name=None, avatar_url=None, actions=None):
606         """Barra de navegacao superior"""
607         left = [
608             DS.titulo(title, size=18, color=Cores.P),
609         ]
610
611         right = []
612         if actions:
613             right.extend(actions)
614
615         if user_name:
616             if avatar_url is None:
617                 right.append(
618                     ft.Row([
619                         ft.CircleAvatar(
620                             content=ft.Text(user_name[0].upper(), size=14, weight=ft.FontWeight.W_600, color=Cores.FUNDO_CARD),
621                             bgcolor=Cores.P,
622                             radius=16,
623                         ),
624                         ft.Container(width=4,
625                             ft.Text(user_name.split()[0], size=12, weight=ft.FontWeight.W_500, color=Cores.TEXT0_80, font_family="Segoe UI"),
626                             ], spacing=8, alignment=ft.MainAxisAlignment.CENTER)
627                     )
628                 )
629
630         return ft.Container(
631             content=ft.Row(
632                 [
633                     ft.Row(left, spacing=Espacamento.MD),
634                     ft.Row(right, spacing=Espacamento.SM, expand=True, alignment=ft.MainAxisAlignment.END),
635                 ],
636                 alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
637             ),
638             padding=ft.padding.symmetric(horizontal=Espacamento.LG, vertical=Espacamento.SM),
639             bgcolor=Cores.FUNDO_CARD,
640             border=ft.border.only(bottom=ft.BorderSide(1, Cores.BORDA)),
641             shadow=ft.BoxShadow(
642                 spread_radius=0,
643                 blur_radius=4,
644                 color=ft.Colors.with_opacity(0.04, Cores.ESCURO),
645                 offset=ft.Offset(0, 1),
646             ),
647         )
648
649     @staticmethod
650     def page_header(title, subtitle=None, actions=None):
651         """Header de pagina"""
652         left = ft.Column(
653             [ft.Text(title, size=22, weight=ft.FontWeight.W_700, color=Cores.TEXT0, font_family="Segoe UI")]
654             + ([ft.Container(height=2), ft.Text(subtitle, size=13, color=Cores.TEXT0_60, font_family="Segoe UI")] if subtitle else []),
655             spacing=4,
656         )
657
658         return ft.Container(
659             content=ft.Row(
660                 [
661                     left,
662                     ft.Row(actions or [], spacing=Espacamento.SM, expand=True, alignment=ft.MainAxisAlignment.END),
663                 ],
664                 alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
665             ),
666             padding=ft.padding.only(left=Espacamento.LG, right=Espacamento.LG, top=Espacamento.LG, bottom=Espacamento.SM),
667             bgcolor=Cores.FUNDO,
668         )
669
670     @staticmethod
671     def empty_state(icon=ft.Icons.INBOX_OUTLINED, title="Nenhum dado encontrado", subtitle=None, action=None):
672         """Estado vazio elegante"""
673         return ft.Container(
674             content=ft.Column(
675                 [ft.Icon(icon, size=48, color=Cores.TEXT0_40)]
676                 + [ft.Container(height=Espacamento.MD)]
677                 + [ft.Text(title, size=16, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60, text_align=ft.TextAlign.CENTER)]
678                 + [ft.Container(height=Espacamento.XS)]
679                 + [ft.Text(subtitle or "", size=12, color=Cores.TEXT0_40, text_align=ft.TextAlign.CENTER)]
680                 + ([ft.Container(height=Espacamento.MD), action] if action else []),
681                 alignment=ft.MainAxisAlignment.CENTER,
682                 horizontal_alignment=ft.CrossAxisAlignment.CENTER,
683             ),
684             alignment=ft.alignment.center,
685             padding=Espacamento.XL,
686         )

```

```

687 @staticmethod
688 def snackbar(message, type="info", duration=3000):
689     """Notificacao toast"""
690     type_config = {
691         "success": {"icon": ft.Icons.CHECK_CIRCLE, "color": Cores.SUCESSO, "bg": ft.Colors.with_opacity(0.1, Cores.SUCESSO)},
692         "error": {"icon": ft.Icons.ERROR_OUTLINED, "color": Cores.ERRO, "bg": ft.Colors.with_opacity(0.1, Cores.ERRO)},
693         "warning": {"icon": ft.Icons.WARNING_OUTLINED, "color": Cores.ALERTA, "bg": ft.Colors.with_opacity(0.1, Cores.ALERTA)},
694         "info": {"icon": ft.Icons.INFO_OUTLINED, "color": Cores.INFO, "bg": ft.Colors.with_opacity(0.1, Cores.INFO)},
695     }
696     cfg = type_config.get(type, type_config["info"])
697
698     return ft.Container(
699         content=ft.Row([
700             ft.Icon(cfg["icon"], size=18, color=cfg["color"]),
701             ft.Container(width=8,
702                 ft.Text(message, size=13, color=Cores.TEXT0_80, font_family="Segoe UI", expand=True),
703             ], spacing=8),
704         bgcolor=cfg["bg"],
705         border_radius=Bordas.MD,
706         padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=Espacamento.SM),
707         border=ft.border.all(1, ft.Colors.with_opacity(0.2, cfg["color"])),
708         shadow=ft.BoxShadow(
709             spread_radius=0,
710             blur_radius=12,
711             color=ft.Colors.with_opacity(0.1, Cores.ESCUR0),
712             offset=ft.Offset(0, 4),
713         ),
714         animate=ft.Animation(200, ft.AnimationCurve.EASE_OUT),
715     )
716

```



```

1 """
2 SALTO GESTAO Desktop v2.0 - ARQUIVO UNICO
3 Design Moderno & Clean - Paleta Corumba Sistemas
4 NAO precisa de tkinter! Usa Flet (Flutter-based).
5
6 Instalacao: pip install flet requests
7 Execuciao: python main.py
8 """
9
10 import flet as ft
11 import requests
12 import threading
13 from datetime import datetime
14
15 # =====
16 # PALETA DE CORES CORUMBA SISTEMAS
17 # =====
18 P = "#0b1a30"
19 P80 = "#1a2d4a"
20 P50 = "#5a7ea8"
21 P20 = "#9bb5d0"
22 P10 = "#c5d5e8"
23 P5 = "#e2e8f0"
24 FUNDO = "#f5f7fa"
25 BRANCO = "#ffffff"
26 TEXTO = "#0b1a30"
27 TEXTO80 = "#3c4a5e"
28 TEXTO60 = "#6d7a8e"
29 TEXTO40 = "#9eaabe"
30 OK = "#22c55e"
31 ERRO = "#ef4444"
32 ALERTA = "#f59e0b"
33 INFO = "#3b82f6"
34 FUNDO_CARD = "#ffffff"
35 BORDA = "#e2e8f0"
36
37 API_URL = "http://191.252.210.235:8080"
38
39
40 # =====
41 # CLIENTE API
42 # =====
43 class API:
44     def __init__(self):
45         self.base = API_URL
46         self.token = None
47         self.user = None
48         self.h = {"Content-Type": "application/json"}
49
50     def set_token(self, t):
51         self.token = t
52         self.h["Authorization"] = "Bearer " + t
53
54     def login(self, u, p):
55         try:
56             r = requests.post(self.base + "/api/login", json={"username": u, "password": p}, timeout=10)
57             if r.status_code == 200:
58                 d = r.json()
59                 self.set_token(d.get("access_token", ""))
60                 self.user = d.get("user", {})
61                 return {"ok": True}
62             return {"ok": False, "err": r.json().get("detail", "Erro")}
63         except requests.exceptions.ConnectionError:
64             return {"ok": False, "err": "Sem conexao com o servidor"}
65         except Exception as e:
66             return {"ok": False, "err": str(e)}
67
68     def get(self, ep):
69         try:
70             r = requests.get(self.base + ep, headers=self.h, timeout=15)
71             return {"ok": True, "data": r.json()} if r.status_code == 200 else {"ok": False}
72         except:
73             return {"ok": False}
74
75     def post(self, ep, data):
76         try:
77             r = requests.post(self.base + ep, json=data, headers=self.h, timeout=15)
78             return {"ok": True} if r.status_code in (200, 201) else {"ok": False}
79         except:
80             return {"ok": False}
81
82     def put(self, ep, data):
83         try:
84             r = requests.put(self.base + ep, json=data, headers=self.h, timeout=15)
85             return {"ok": True} if r.status_code == 200 else {"ok": False}
86         except:
87             return {"ok": False}
88
89     def delete(self, ep):
90         try:
91             r = requests.delete(self.base + ep, headers=self.h, timeout=15)
92             return {"ok": True}
93         except:
94             return {"ok": False}
95
96 api = API()
97
98
99 # =====
100 # HELPER: Campo de entrada moderno
101 # =====
102 def campo(label, icon=None, pwd=False, hint=None, w=None, **kw):
103     return ft.TextField(
104         label=label, prefix_icon=icon, password=pwd,
105         hint_text=hint, hint_style=ft.TextStyle(size=12, color=TEXTO40),
106         label_style=ft.TextStyle(size=11, color=TEXTO60),
107         text_style=ft.TextStyle(size=13, color=TEXTO),
108         border_color=BORDA, focused_border_color=P, focused_border_width=2,
109         border_radius=12, bgcolor=BRANCO,
110         content_padding=ft.padding.only(left=16, right=16, top=12, bottom=12),
111         width=w, **kw
112     )
113
114
115 # =====
116 # HELPER: Botao primario
117 # =====
118 def btn(texto, icon=None, w=None, h=44, on_click=None, **kw):
119     content = ft.Row([
120         ft.Icon(icon, size=16, color=BRANCO) if icon else ft.Container(),
121         ft.Text(texto, color=BRANCO, size=14, weight=ft.FontWeight.W_600),
122     ], alignment=ft.MainAxisAlignment.CENTER, spacing=8) if icon else texto
123
124     return ft.ElevatedButton(
125         content=content, width=w, height=h, on_click=on_click,
126         style=ft.ButtonStyle(
127             bgcolor={ft.ControlState.DEFAULT: P, ft.ControlState.HOVERED: P80},
128             color=BRANCO, shape=ft.RoundedRectangleBorder(radius=12),
129             padding=ft.padding.symmetric(horizontal=24, vertical=12),
130             elevation={ft.ControlState.DEFAULT: 0, ft.ControlState.HOVERED: 2},
131             animation_duration=150,
132         ), **kw
133     )
134

```

```

135
136
137 # =====
138 # HELPER: Card
139 # =====
140 def card(content=None, padding=16, shadow=True, border=False, **kw):
141     return ft.Container(
142         content=content, bgcolor=BRANCO, border_radius=16, padding=padding,
143         shadow=ft.BoxShadow(spread_radius=0, blur_radius=8,
144             color=ft.Colors.with_opacity(0.06, TEXT0), offset=ft.Offset(0, 2)) if shadow else None,
145         border=ft.border.all(1, BORDA) if border else None, **kw
146     )
147
148
149 # =====
150 # COMPONENTE: Sidebar Item
151 # =====
152 def sidebar_item(label, icon, selected=False, on_click=None):
153     return ft.Container(
154         content=ft.Row([
155             ft.Icon(icon, size=20, color=P if selected else TEXT060),
156             ft.Container(width=4,
157                 ft.Text(label, size=13, weight=ft.FontWeight.W_600 if selected else ft.FontWeight.W_400,
158                     color=P if selected else TEXT080),
159             ], spacing=12, alignment=ft.MainAxisAlignment.START),
160         padding=ft.padding.symmetric(horizontal=16, vertical=10),
161         bgcolor=ft.Colors.with_opacity(0.08, P) if selected else ft.Colors.TRANSPARENT,
162         border_radius=12, on_click=on_click, animate=ft.Animation(150),
163         margin=ft.padding.symmetric(horizontal=8, vertical=2),
164     )
165
166
167 # =====
168 # COMPONENTE: Badge
169 # =====
170 def badge(texto, cor=P):
171     return ft.Container(
172         content=ft.Text(texto, size=10, weight=ft.FontWeight.W_600, color=cor),
173         bgcolor=ft.Colors.with_opacity(0.1, cor), border_radius=9999,
174         padding=ft.padding.symmetric(horizontal=8, vertical=3),
175     )
176
177
178 # =====
179 # COMPONENTE: Stat Card
180 # =====
181 def stat_card(titulo, valor, icone, cor=P):
182     return card(content=ft.Column([
183         ft.Row([
184             ft.Container(content=ft.Icon(icone, size=20, color=cor),
185                 bgcolor=ft.Colors.with_opacity(0.1, cor), border_radius=8, padding=8),
186             ],),
187         ft.Container(height=4),
188         ft.Text(titulo, size=11, color=TEXT060),
189         ft.Text(str(valor), size=26, weight=ft.FontWeight.W_700, color=TEXT0),
190     ], spacing=2), padding=16, width=200, height=90)
191
192
193 # =====
194 # TELA DE LOGIN
195 # =====
196 def tela_login(page, on_ok):
197     user = campo("Usuario", icon=ft.Icons.PERSON_OUTLINED, hint="Digite seu usuario", w=360)
198     pwd = campo("Senha", icon=ft.Icons.LOCK_OUTLINED, pwd=True, can_reveal_password=True, hint="Digite sua senha", w=360)
199     msg = ft.Container(visible=False)
200
201     def show(text, tipo="err"):
202         cols = {"err": ERRO, "ok": OK}
203         bgs = {"err": ft.Colors.with_opacity(0.08, ERRO), "ok": ft.Colors.with_opacity(0.08, OK)}
204         icons = {"err": ft.Icons.ERROR_OUTLINED, "ok": ft.Icons.CHECK_CIRCLE_OUTLINED}
205         msg.content = ft.Container(
206             content=ft.Row([ft.Icon(icons.get(tipo), size=16, color=TEXT0), ft.Container(width=8,
207                 ft.Text(text, size=12, color=TEXT080)], spacing=8),
208             bgcolor=bgs.get(tipo), border_radius=8, padding=ft.padding.symmetric(horizontal=12, vertical=8),
209             border=ft.border.all(1, ft.Colors.with_opacity(0.15, cols.get(tipo))), w=360)
210         msg.visible = True
211         page.update()
212
213     def do_login(e):
214         if not user.value or not pwd.value:
215             show("Preencha todos os campos"); return
216         result = api.login(user.value, pwd.value)
217         if result["ok"]:
218             show("Login realizado!", "ok")
219             page.update()
220             import time; time.sleep(0.4)
221             on_ok()
222         else:
223             show(result["err"])
224         page.update()
225
226     return ft.Container(
227         alignment=ft.alignment.center, expand=True,
228         gradient=ft.LinearGradient(
229             begin=ft.alignment.top_center, end=ft.alignment.bottom_center,
230             colors=[FUNDO, BRANCO]),
231         content=card(content=ft.Column([
232             # Logo
233             ft.Container(
234                 content=ft.Column([
235                     ft.Container(content=ft.Icon(ft.Icons.STORE_ROUNDED, size=40, color=P),
236                         bgcolor=ft.Colors.with_opacity(0.1, P), border_radius=9999, padding=20,
237                         width=80, height=80, alignment=ft.alignment.center),
238                     ft.Container(height=8,
239                         ft.Text("Salto Gestao", size=24, weight=ft.FontWeight.W_700, color=P),
240                         ft.Text("Sistema de Gestao Empresarial", size=12, color=TEXT060),
241                     ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
242                     alignment=ft.alignment.center, padding=ft.padding.only(top=16)),
243                     ft.Container(height=32),
244                     user, ft.Container(height=8), pwd, ft.Container(height=8),
245                     msg, ft.Container(height=8),
246                     btn("Entrar", icon=ft.Icons.ARROW_FORWARD, w=360, h=48, on_click=do_login),
247                     ft.Container(height=16),
248                 ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=4),
249                 padding=40, w=420,
250                 shadow=ft.BoxShadow(spread_radius=0, blur_radius=24,
251                     color=ft.Colors.with_opacity(0.12, TEXT0), offset=ft.Offset(0, 8)))
252         ])
253
254
255 # =====
256 # TELA: DASHBOARD
257 # =====
258 def tela_dashboard(page):
259     stats = ft.Row(spacing=12, wrap=True, scroll=ft.ScrollMode.AUTO)
260
261     def load():
262         stats.controls.clear()
263         cr = api.get("/api/clientes/")
264         pr = api.get("/api/produtos/")
265         vr = api.get("/api/vendas/")
266         nc = len(cr["data"]) if cr["ok"] else 0
267         np = len(pr["data"]) if pr["ok"] else 0
268         nv = len(vr["data"]) if vr["ok"] else 0
269         total = sum(v.get("total", 0) for v in vr["data"]) if vr["ok"] and vr["data"] else 0
270
271         for t, v, ic, c in [("Clientes", str(nc), ft.Icons.PEOPLE_OUTLINED, P),
272             ("Produtos", str(np), ft.Icons.INVENTORY_2_OUTLINED, INFO),

```

```

273         ("Vendas", str(nv), ft.Icons.POINT_OF_SALE, OK),
274         ("Faturamento", "R$ %.2f" % total, ft.Icons.ATTACH_MONEY, ALERTA)]:
275     stats.controls.append(stat_card(t, v, ic, c))
276     page.update()
277
278     return ft.Container(
279         content=ft.Column([
280             ft.Container(
281                 content=ft.Row([
282                     ft.Column([
283                         ft.Text("Dashboard", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
284                         ft.Text("Bem-vindo, " + str(api.user.get("nome", "Usuario")) + "!", size=13, color=TEXT060),
285                     ], spacing=4),
286                     ft.Row([ft.IconButton(icon=ft.Icons.REFRESH, icon_color=TEXT060,
287                                     bgcolor=P5, on_click=lambda e: load(),
288                                     style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8))],
289                             expand=True, alignment=ft.MainAxisAlignment.END),
290                     ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
291                     padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUND0),
292                     stats,
293                     ft.Container(content=ft.Divider(height=1, color=BORDA), padding=ft.padding.symmetric(horizontal=24)),
294                     ft.Text("Resumo do Negocio", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
295                             padding=ft.padding.only(left=28, top=8)),
296                     ], spacing=16, expand=True),
297                     bgcolor=FUND0, expand=True, on_mount=lambda e: load()
298                 )
299             )
300
301 # =====
302 # TELA: CLIENTES
303 # =====
304 def tela_clientes(page):
305     lista = ft.ListView(spacing=8, padding=0, expand=True)
306     nome = campo("Nome completo", hint="Ex: Joao Silva")
307     cpf = campo("CPF/CNPJ", hint="000.000.000-00")
308     email = campo("E-mail", hint="email@exemplo.com")
309     tel = campo("Telefone", hint="(62) 99999-9999")
310     msg = ft.Container(visible=False)
311     edit_id = {"v": None}
312
313     def show(text, tipo="ok"):
314         icons = {"ok": ft.Icons.CHECK_CIRCLE_OUTLINED, "err": ft.Icons.ERROR_OUTLINED,
315                 "info": ft.Icons.INFO_OUTLINED}
316         bgs = {"ok": ft.Colors.with_opacity(0.08, OK), "err": ft.Colors.with_opacity(0.08, ERRO),
317               "info": ft.Colors.with_opacity(0.08, INFO)}
318         msg.content = ft.Container(
319             content=ft.Row([ft.Text(text, size=12, color=TEXT080)], spacing=8),
320             bgcolor=bgs.get(tipo), border_radius=8, padding=ft.padding.symmetric(horizontal=12, vertical=8),
321             margin=ft.padding.only(bottom=8))
322         msg.visible = True
323         page.update()
324
325     def load():
326         lista.controls.clear()
327         r = api.get("/api/clientes/")
328         if r["ok"] and r["data"]:
329             for c in r["data"]:
330                 lista.controls.append(card(
331                     padding=0, shadow=False, border=True,
332                     content=ft.Container(
333                         content=ft.Row([
334                             ft.CircleAvatar(content=ft.Text(c.get("nome", "?")[0].upper(), size=16,
335                                                         weight=ft.FontWeight.W_600, color=BRANCO), bgcolor=P20, radius=20),
336                             ft.Container(width=8),
337                             ft.Column([
338                                 ft.Text(c.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=TEXT0),
339                                 ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=11, color=TEXT060),
340                                 ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=11, color=TEXT060),
341                             ], spacing=2, expand=True),
342                             ft.IconButton(icon=ft.Icons.EDIT_OUTLINED, icon_color=INFO,
343                                     bgcolor=ft.Colors.with_opacity(0.08, INFO), tooltip="Editar",
344                                     on_click=lambda e, cli=c: edit(cli),
345                                     style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
346                             ft.IconButton(icon=ft.Icons.DELETE_OUTLINED, icon_color=ERRO,
347                                     bgcolor=ft.Colors.with_opacity(0.08, ERRO), tooltip="Excluir",
348                                     on_click=lambda e, cli=c: delete(cli),
349                                     style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
350                             ], spacing=8, alignment=ft.MainAxisAlignment.START),
351                             padding=ft.padding.symmetric(horizontal=16, vertical=10)))
352             else:
353                 lista.controls.append(ft.Container(
354                     content=ft.Column([
355                         ft.Icon(ft.Icons.PEOPLE_OUTLINED, size=40, color=TEXT040),
356                         ft.Text("Nenhum cliente cadastrado", size=14, color=TEXT060),
357                     ], horizontal_alignment=ft.CrossAxisAlignment.CENTER),
358                     alignment=ft.Alignment.CENTER, padding=40))
359         page.update()
360
361     def save(e):
362         data = {"nome": nome.value or "", "cpf_cnpj": cpf.value or "", "email": email.value or "", "telefone": tel.value or ""}
363         if not data["nome"]:
364             show("Nome e obrigatorio", "err"); return
365         if edit_id["v"]:
366             r = api.put("/api/clientes/" + str(edit_id["v"]), data)
367         else:
368             r = api.post("/api/clientes/", data)
369         if r["ok"]:
370             show("Cliente salvo!", "ok")
371             for f in [nome, cpf, email, tel]: f.value = ""
372             edit_id["v"] = None
373             load()
374         else:
375             show("Erro ao salvar", "err")
376
377     def edit(c):
378         edit_id["v"] = c.get("id")
379         nome.value = c.get("nome", "")
380         cpf.value = c.get("cpf_cnpj", "")
381         email.value = c.get("email", "")
382         tel.value = c.get("telefone", "")
383         show("Editando: " + str(c.get("nome", "")), "info")
384         page.update()
385
386     def delete(c):
387         def ok(e):
388             if e.control.text == "Sim":
389                 r = api.delete("/api/clientes/" + str(c.get("id")))
390                 if r["ok"]: show("Excluido!", "ok"); load()
391                 else: show("Erro ao excluir", "err")
392             dlg.open = False; page.update()
393         dlg = ft.AlertDialog(
394             title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
395             content=ft.Text("Excluir " + str(c.get("nome")) + "?" ),
396             actions=[ft.TextButton("Nao", on_click=ok),
397                     ft.TextButton("Sim", on_click=ok, style=ft.ButtonStyle(color=ERRO))]
398         )
399         page.overlay.append(dlg); dlg.open = True; page.update()
400
401     def novo(e):
402         edit_id["v"] = None
403         for f in [nome, cpf, email, tel]: f.value = ""
404         show("Novo cliente", "info"); page.update()
405
406     return ft.Container(
407         content=ft.Column([
408             ft.Container(
409                 content=ft.Row([
410                     ft.Column([ft.Text("Clientes", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
411                                 ft.Text("Gerencie seus clientes", size=13, color=TEXT060)], spacing=4),

```

```

411         ft.Row([btn("Novo", icon=ft.Icons.ADD, w=100, on_click=novo)], spacing=8,
412               expand=True, alignment=ft.MainAxisAlignment.END),
413         ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
414         padding=ft.Padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUND0),
415         # Form
416         card(
417             titulo="Formulário",
418             content=ft.Column([
419                 ft.Row([nome, cpf], spacing=12),
420                 ft.Row([email, tel], spacing=12),
421                 ft.Row([btn("Salvar", icon=ft.Icons.SAVE_OUTLINED, w=140, on_click=save),
422                        ft.OutlinedButton(text="Limpar", on_click=lambda e: novo(e),
423                                         style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=12),
424                                                         side=ft.BorderSide(1.5, BORDA),
425                                                         padding=ft.Padding.symmetric(horizontal=20, vertical=12)))]),
426                 spacing=8),
427             ], spacing=12), padding=16),
428         ft.Text(" Clientes cadastrados", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
429               padding=ft.Padding.only(left=4, bottom=8)),
430         ft.Container(content=lista, expand=True),
431         ], spacing=16, expand=True),
432         padding=ft.Padding.all(24), bgcolor=FUND0, expand=True, on_mount=lambda e: load())
433     )
434
435
436
437 # =====
438 # TELA: PRODUTOS
439 # =====
440 def tela_produtos(page):
441     lista = ft.ListView(spacing=8, padding=0, expand=True)
442     nome = campo("Nome do produto", hint="Ex: Arroz 5kg")
443     preco = campo("Preço (R$)", hint="0.00")
444     cat = campo("Categoria", hint="Alimentos")
445     estoque = campo("Estoque", hint="0")
446     msg = ft.Container(visible=False)
447     edit_id = {"v": None}
448
449     def show(text, tipo="ok"):
450         icons = {"ok": ft.Icons.CHECK_CIRCLE_OUTLINED, "err": ft.Icons.ERROR_OUTLINED}
451         bgs = {"ok": ft.Colors.with_opacity(0.08, OK), "err": ft.Colors.with_opacity(0.08, ERRO)}
452         msg.content = ft.Container(
453             content=ft.Row([ft.Text(text, size=12, color=TEXT080)], spacing=8),
454             bgcolor=bgs.get(tipo), border_radius=8,
455             padding=ft.Padding.symmetric(horizontal=12, vertical=8), margin=ft.Padding.only(bottom=8))
456         msg.visible = True
457         page.update()
458
459     def load():
460         lista.controls.clear()
461         r = api.get("/api/produtos/")
462         if r["ok"] and r["data"]:
463             for p in r["data"]:
464                 lista.controls.append(card(
465                     padding=0, shadow=False, border=True,
466                     content=ft.Container(
467                         content=ft.Row([
468                             ft.Container(content=ft.Icon(ft.Icons.INVENTORY_2, size=16, color=INFO),
469                                     bgcolor=ft.Colors.with_opacity(0.1, INFO), border_radius=8,
470                                     padding=8, w=40, h=40, alignment=ft.Alignment.center),
471                             ft.Container(width=8),
472                             ft.Column([
473                                 ft.Text(p.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=TEXT0),
474                                 ft.Text("R$ %.2f" % p.get("preco", 0), size=12, weight=ft.FontWeight.W_700, color=OK),
475                                 ft.Text(str(p.get("estoque", 0)) + " em estoque | " + str(p.get("categoria", "")), size=10, color=TEXT060),
476                             ], spacing=2, expand=True),
477                             ft.IconButton(icon=ft.Icons.EDIT_OUTLINED, icon_color=INFO,
478                                         on_click=lambda e, pr=p: edit(pr),
479                                         style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
480                             ft.IconButton(icon=ft.Icons.DELETE_OUTLINED, icon_color=ERRO,
481                                         on_click=lambda e, pr=p: delete(pr),
482                                         style=ft.ButtonStyle(shape=ft.CircleBorder(), padding=8)),
483                             ], spacing=8, alignment=ft.MainAxisAlignment.START),
484                             padding=ft.Padding.symmetric(horizontal=16, vertical=10)))
485             else:
486                 lista.controls.append(ft.Container(
487                     content=ft.Column([ft.Icon(ft.Icons.INVENTORY_2_OUTLINED, size=40, color=TEXT040),
488                                         ft.Text("Nenhum produto", size=14, color=TEXT060)],
489                                         horizontal_alignment=ft.CrossAxisAlignment.CENTER),
490                     alignment=ft.Alignment.center, padding=40))
491         page.update()
492
493     def save(e):
494         try:
495             pr = float(preco.value or 0)
496             es = int(estoque.value or 0)
497         except ValueError:
498             show("Preço e estoque devem ser numeros", "err"); return
499         data = {"nome": nome.value or "", "preco": pr, "categoria": cat.value or "", "estoque": es}
500         if not data["nome"]: show("Nome é obrigatorio", "err"); return
501         if edit_id["v"]:
502             r = api.put("/api/produtos/" + str(edit_id["v"]), data)
503         else:
504             r = api.post("/api/produtos/", data)
505         if r["ok"]:
506             show("Produto salvo!", "ok")
507             for f in [nome, preco, cat, estoque]: f.value = ""
508             edit_id["v"] = None; load()
509         else:
510             show("Erro ao salvar", "err")
511
512     def edit(p):
513         edit_id["v"] = p.get("id"); nome.value = p.get("nome", "")
514         preco.value = str(p.get("preco", "")); cat.value = p.get("categoria", "")
515         estoque.value = str(p.get("estoque", "")); page.update()
516
517     def delete(p):
518         def ok(e):
519             if e.control.text == "Sim":
520                 r = api.delete("/api/produtos/" + str(p.get("id")))
521                 if r["ok"]: show("Excluido!", "ok"); load()
522                 dlg.open = False; page.update()
523             dlg = ft.AlertDialog(title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),
524                                content=ft.Text("Excluir '" + str(p.get("nome")) + "'?",
525                                                  actions=[ft.TextButton("Nao", on_click=ok), ft.TextButton("Sim", on_click=ok)])
526                                page.overlay.append(dlg); dlg.open = True; page.update())
527
528     return ft.Container(
529         content=ft.Column([
530             ft.Container(
531                 content=ft.Row([
532                     ft.Column([ft.Text("Produtos", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
533                                 ft.Text("Gerencie seu estoque", size=13, color=TEXT060)], spacing=4),
534                     ft.Row([btn("Novo", icon=ft.Icons.ADD, w=100, on_click=lambda e: (edit_id.__setitem__("v", None), [f.__setattr__("value", "") for f in [nome, preco, cat, estoque]]),
535                             spacing=8, expand=True, alignment=ft.MainAxisAlignment.END),
536                     ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
537                     padding=ft.Padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUND0),
538                     card(content=ft.Column([
539                         ft.Row([nome, preco], spacing=12), ft.Row([cat, estoque], spacing=12),
540                         ft.Row([btn("Salvar", icon=ft.Icons.SAVE_OUTLINED, w=140, on_click=save)], spacing=8),
541                         msg,
542                     ], spacing=12), padding=16),
543                     ft.Text("Produtos cadastrados", size=13, weight=ft.FontWeight.W_600, color=TEXT060,
544                           padding=ft.Padding.only(left=4, bottom=8)),
545                     ft.Container(content=lista, expand=True),
546                     ], spacing=16, expand=True),
547                     padding=ft.Padding.all(24), bgcolor=FUND0, expand=True, on_mount=lambda e: load())

```

```

548 )
549
550
551 # =====
552 # TELA: VENDAS
553 # =====
554 def tela_vendas(page):
555     lista = ft.ListView(spacing=8, padding=0, expand=True)
556
557     def load():
558         lista.controls.clear()
559         r = api.get("/api/vendas/")
560         if r["ok"] and r["data"]:
561             total = 0
562             for v in r["data"]:
563                 t = v.get("total", 0); total += t
564                 lista.controls.append(card(
565                     padding=0, shadow=False, border=True,
566                     content=ft.Container(
567                         content=ft.Row([
568                             ft.Container(content=ft.Icon(ft.Icons.POINT_OF_SALE, size=16, color=OK),
569                                         bgcolor=ft.Colors.with_opacity(0.1, OK), border_radius=8,
570                                         padding=6, w=36, h=36, alignment=ft.alignment.center),
571                             ft.Container(width=8),
572                             ft.Column([
573                                 ft.Text("Venda #" + str(v.get("id",0)), size=13, weight=ft.FontWeight.W_600),
574                                 ft.Text(str(v.get("cliente_nome","N/A")), size=11, color=TEXT060),
575                                 ], spacing=2, expand=True),
576                                 ft.Text("R$ %.2f" % t, size=14, weight=ft.FontWeight.W_700, color=OK),
577                                 ], spacing=8, alignment=ft.MainAxisAlignment.START),
578                                 padding=ft.padding.symmetric(horizontal=16, vertical=10)))
579                 lista.controls.insert(0, card(
580                     content=ft.Row([
581                         ft.Text(str(len(r["data"])) + " vendas", size=12, color=TEXT060),
582                         ft.Text("Total: R$ %.2f" % total, size=14, weight=ft.FontWeight.W_700, color=P),
583                         ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
584                         padding=16, shadow=False, bgcolor=ft.Colors.with_opacity(0.03, P),
585                         border=ft.border.all(1, P20)))
586             page.update()
587
588     return ft.Container(
589         content=ft.Column([
590             ft.Container(
591                 content=ft.Column([ft.Text("Vendas", size=22, weight=ft.FontWeight.W_700, color=TEXT0),
592                                     ft.Text("Historico de vendas", size=13, color=TEXT060)], spacing=4),
593                 padding=ft.padding.only(left=24, right=24, top=24, bottom=8), bgcolor=FUND0),
594             ft.Container(content=lista, expand=True),
595             ], spacing=16),
596             padding=ft.padding.all(24), bgcolor=FUND0, expand=True, on_mount=lambda e: load())
597 )
598
599 # =====
600 # LAYOUT PRINCIPAL
601 # =====
602 def layout_principal():
603     main = ft.Container(expand=True, bgcolor=FUND0)
604     sel = {"i": 0}
605
606     def nav(i):
607         sel["i"] = i
608         telas = {0: tela_dashboard, 1: tela_clientes, 2: tela_produtos, 3: tela_vendas}
609         if i in telas:
610             main.content = telas[i](main.page)
611             main.page.update()
612
613     for j, item in enumerate(nav.items):
614         item.bgcolor = ft.Colors.with_opacity(0.08, P) if j == i else ft.Colors.TRANSPARENT
615     page.update()
616
617     nav_items = [
618         sidebar_item("Dashboard", ft.Icons.DASHBOARD, True, lambda e: nav(0)),
619         sidebar_item("Clientes", ft.Icons.PEOPLE_OUTLINED, False, lambda e: nav(1)),
620         sidebar_item("Produtos", ft.Icons.INVENTORY_2_OUTLINED, False, lambda e: nav(2)),
621         sidebar_item("Vendas", ft.Icons.POINT_OF_SALE_OUTLINED, False, lambda e: nav(3)),
622     ]
623
624     sidebar = ft.Container(
625         content=ft.Column([
626             ft.Container(content=ft.Row([
627                 ft.Container(content=ft.Icon(ft.Icons.STORE, size=20, color=BRANCO),
628                             bgcolor=P, border_radius=8, padding=8),
629                 ft.Container(width=6),
630                 ft.Column([ft.Text("SaltoGestao", size=13, weight=ft.FontWeight.W_700, color=BRANCO),
631                             ft.Text("v2.0", size=8, color=P20)], spacing=0),
632             ]), padding=ft.padding.symmetric(horizontal=12, vertical=16),
633             margin=ft.padding.only(top=8, bottom=16)),
634             ft.Column(nav_items, spacing=2),
635             ft.Container(expand=True),
636             ft.Container(content=ft.Text("Corumba Sistemas", size=8, color=P20),
637                     padding=ft.padding.all(12)),
638             ], spacing=0),
639             width=220, bgcolor=P, padding=ft.padding.symmetric(vertical=8))
640
641     return ft.Container(
642         content=ft.Row([
643             sidebar,
644             ft.VerticalDivider(width=1, color=BORDA),
645             main,
646             ], expand=True, spacing=0),
647             expand=True)
648
649 # =====
650 # APP PRINCIPAL
651 # =====
652 def main(page: ft.Page):
653     page.title = "SaltoGestao v2.0"
654     page.padding = 0
655     page.window.width = 1280
656     page.window.height = 800
657     page.window.min_width = 1024
658     page.window.min_height = 700
659     page.bgcolor = FUND0
660     page.theme = ft.Theme(color_scheme_seed=P, font_family="Segoe UI", use_material3=True)
661
662     mc = ft.Container(expand=True, bgcolor=FUND0)
663
664     def on_login():
665         mc.content = layout_principal()
666         page.update()
667
668     mc.content = tela_login(page, on_login)
669     page.add(mc)
670
671 if __name__ == "__main__":
672     ft.app(target=main)
673
674
675

```

```

1 """
2 SaltoGestao - Sistema Desktop com Flet
3 Desenvolvido por Corumba Sistemas
4 Interface moderna para gestao de empresas
5
6 Instalacao: pip install flet requests
7 Execucacao: flet run main_flet.py
8 ou: python main_flet.py
9 """
10
11 import flet as ft
12 import requests
13 import json
14 import threading
15 from datetime import datetime
16
17 # =====
18 # CONFIGURACAO
19 # =====
20 API_URL = "http://191.252.210.235:8080"
21
22
23 # =====
24 # CLIENTE DA API
25 # =====
26 class APIClient:
27     """Cliente para comunicacao com a API FastAPI do SaltoGestao"""
28
29     def __init__(self):
30         self.base_url = API_URL
31         self.token = None
32         self.user = None
33         self.headers = {"Content-Type": "application/json"}
34
35     def set_token(self, token):
36         self.token = token
37         self.headers["Authorization"] = "Bearer " + token
38
39     def login(self, username, password):
40         try:
41             resp = requests.post(
42                 self.base_url + "/api/login",
43                 json={"username": username, "password": password},
44                 timeout=10,
45             )
46             if resp.status_code == 200:
47                 data = resp.json()
48                 self.set_token(data.get("access_token", ""))
49                 self.user = data.get("user", {})
50                 return {"success": True, "data": data}
51             return {"success": False, "error": resp.json().get("detail", "Erro no login")}
52         except requests.exceptions.ConnectionError:
53             return {"success": False, "error": "Nao foi possivel conectar ao servidor"}
54         except Exception as e:
55             return {"success": False, "error": str(e)}
56
57     def get(self, endpoint):
58         try:
59             resp = requests.get(self.base_url + endpoint, headers=self.headers, timeout=15)
60             if resp.status_code == 200:
61                 return {"success": True, "data": resp.json()}
62             return {"success": False, "error": resp.text}
63         except Exception as e:
64             return {"success": False, "error": str(e)}
65
66     def post(self, endpoint, data):
67         try:
68             resp = requests.post(
69                 self.base_url + endpoint, json=data, headers=self.headers, timeout=15
70             )
71             if resp.status_code in (200, 201):
72                 return {"success": True, "data": resp.json()}
73             return {"success": False, "error": resp.text}
74         except Exception as e:
75             return {"success": False, "error": str(e)}
76
77     def put(self, endpoint, data):
78         try:
79             resp = requests.put(
80                 self.base_url + endpoint, json=data, headers=self.headers, timeout=15
81             )
82             if resp.status_code == 200:
83                 return {"success": True, "data": resp.json()}
84             return {"success": False, "error": resp.text}
85         except Exception as e:
86             return {"success": False, "error": str(e)}
87
88     def delete(self, endpoint):
89         try:
90             resp = requests.delete(self.base_url + endpoint, headers=self.headers, timeout=15)
91             if resp.status_code in (200, 204):
92                 return {"success": True}
93             return {"success": False, "error": resp.text}
94         except Exception as e:
95             return {"success": False, "error": str(e)}
96
97
98 api = APIClient()
99
100
101 # =====
102 # TELA DE LOGIN
103 # =====
104 def criar_tela_login(page, on_login_success):
105     username_field = ft.TextField(
106         label="Usuario",
107         prefix_icon=ft.Icons.PERSON,
108         border_radius=10,
109         width=350,
110         text_size=15,
111     )
112     password_field = ft.TextField(
113         label="Senha",
114         prefix_icon=ft.Icons.LOCK,
115         password=True,
116         can_reveal_password=True,
117         border_radius=10,
118         width=350,
119         text_size=15,
120     )
121     msg_text = ft.Text("", size=14, weight=ft.FontWeight.BOLD)
122     loading = ft.ProgressRing(width=20, height=20, visible=False)
123
124     def do_login(e):
125         if not username_field.value or not password_field.value:
126             msg_text.value = "Preencha todos os campos!"
127             msg_text.color = ft.Colors.ORANGE
128             page.update()
129             return
130         loading.visible = True
131         msg_text.value = "Conectando..."
132         msg_text.color = ft.Colors.BLUE
133         page.update()
134         result = api.login(username_field.value, password_field.value)

```

```

135     loading.visible = False
136     if result["success"]:
137         msg_text.value = "Login realizado com sucesso!"
138         msg_text.color = ft.Colors.GREEN
139         page.update()
140         on_login_success()
141     else:
142         msg_text.value = result["error"]
143         msg_text.color = ft.Colors.RED
144         page.update()
145
146     return ft.Container(
147         content=ft.Column(
148             [
149                 ft.Icon(ft.Icons.STORE, size=80, color=ft.Colors.BLUE_700),
150                 ft.Text("SaltoGestao", size=36, weight=ft.FontWeight.BOLD, color=ft.Colors.BLUE_900),
151                 ft.Text("Sistema de Gestao Empresarial", size=14, color=ft.Colors.GREY_600),
152                 ft.Divider(height=20, color=ft.Colors.TRANSPARENT),
153                 username_field,
154                 ft.Container(height=5),
155                 password_field,
156                 ft.Container(height=10),
157                 ft.ElevatedButton(
158                     content=ft.Row(
159                         [ft.Icon(ft.Icons.LOGIN), ft.Text("Entrar", size=16, weight=ft.FontWeight.BOLD)],
160                         alignment=ft.MainAxisAlignment.CENTER,
161                     ),
162                     width=350,
163                     height=50,
164                     bgcolor=ft.Colors.BLUE_700,
165                     color=ft.Colors.WHITE,
166                     style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=10)),
167                     on_click=do_login,
168                 ),
169                 ft.Container(height=10),
170                 ft.Row([loading, msg_text], alignment=ft.MainAxisAlignment.CENTER),
171                 ft.Divider(height=20),
172                 ft.Text("Corumba Sistemas (c) 2026", size=11, color=ft.Colors.GREY_500),
173             ],
174             horizontal_alignment=ft.CrossAxisAlignment.CENTER,
175             alignment=ft.MainAxisAlignment.CENTER,
176         ),
177         alignment=ft.alignment.center,
178         expand=True,
179     )
180
181
182 # =====
183 # TELA DE CLIENTES
184 # =====
185 def criar_tela_clientes(page):
186     clientes_list = ft.ListView(spacing=5, padding=10, expand=True)
187     search_field = ft.TextField(
188         label="Buscar cliente...",
189         prefix_icon=ft.Icons.SEARCH,
190         border_radius=10,
191         expand=True,
192     )
193     nome_field = ft.TextField(label="Nome completo", border_radius=8, expand=True)
194     cpf_field = ft.TextField(label="CPF/CNPJ", border_radius=8, expand=True)
195     email_field = ft.TextField(label="E-mail", border_radius=8, expand=True)
196     telefone_field = ft.TextField(label="Telefone", border_radius=8, expand=True)
197     endereco_field = ft.TextField(label="Endereco", border_radius=8, expand=True)
198     msg_text = ft.Text("", size=13, weight=ft.FontWeight.BOLD)
199     edit_id = {"value": None}
200
201     def carregar_clientes():
202         clientes_list.controls.clear()
203         result = api.get("/api/clientes/")
204         if result["success"]:
205             for c in result["data"]:
206                 clientes_list.controls.append(
207                     ft.Card(
208                         content=ft.Container(
209                             content=ft.Row(
210                                 [
211                                     ft.Icon(ft.Icons.PERSON, color=ft.Colors.BLUE_600),
212                                     ft.Column(
213                                         [
214                                             ft.Text(c.get("nome", ""), weight=ft.FontWeight.BOLD, size=14),
215                                             ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=12, color=ft.Colors.GREY_600),
216                                             ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=12, color=ft.Colors.GREY_600),
217                                         ],
218                                         expand=True,
219                                         spacing=2,
220                                     ),
221                                     ft.IconButton(
222                                         icon=ft.Icons.EDIT,
223                                         icon_color=ft.Colors.BLUE_600,
224                                         tooltip="Editar",
225                                         on_click=lambda e, cli=c: editar_cliente(cli),
226                                     ),
227                                     ft.IconButton(
228                                         icon=ft.Icons.DELETE,
229                                         icon_color=ft.Colors.RED_600,
230                                         tooltip="Excluir",
231                                         on_click=lambda e, cli=c: excluir_cliente(cli),
232                                     ),
233                                 ],
234                                 alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
235                             ),
236                             padding=12,
237                         ),
238                         elevation=2,
239                     )
240                 )
241         else:
242             clientes_list.controls.append(
243                 ft.Text("Erro ao carregar: " + str(result["error"]), color=ft.Colors.RED)
244             )
245         page.update()
246
247     def salvar_cliente(e):
248         data = {
249             "nome": nome_field.value or "",
250             "cpf_cnpj": cpf_field.value or "",
251             "email": email_field.value or "",
252             "telefone": telefone_field.value or "",
253             "endereco": endereco_field.value or "",
254         }
255         if not data["nome"]:
256             msg_text.value = "Nome e obrigatorio!"
257             msg_text.color = ft.Colors.ORANGE
258             page.update()
259             return
260         if edit_id["value"]:
261             result = api.put("/api/clientes/" + str(edit_id["value"]), data)
262         else:
263             result = api.post("/api/clientes/", data)
264         if result["success"]:
265             msg_text.value = "Cliente salvo com sucesso!"
266             msg_text.color = ft.Colors.GREEN
267             limpar_form()
268             carregar_clientes()
269         else:
270             msg_text.value = "Erro: " + str(result["error"])[0:80]
271             msg_text.color = ft.Colors.RED
272         page.update()

```

```

273
274 def editar_cliente(cliente):
275     edit_id["value"] = cliente.get("id")
276     nome_field.value = cliente.get("nome", "")
277     cpf_field.value = cliente.get("cpf_cnpj", "")
278     email_field.value = cliente.get("email", "")
279     telefone_field.value = cliente.get("telefone", "")
280     endereco_field.value = cliente.get("endereco", "")
281     msg_text.value = "Editando cliente..."
282     msg_text.color = ft.Colors.BLUE
283     page.update()
284
285 def excluir_cliente(cliente):
286     def confirmar(e):
287         if e.control.text == "Sim":
288             result = api.delete("/api/clientes/" + str(cliente.get("id")))
289             if result["success"]:
290                 msg_text.value = "Cliente excluído!"
291                 msg_text.color = ft.Colors.GREEN
292                 carregar_clientes()
293             else:
294                 msg_text.value = "Erro: " + str(result["error"])[0:80]
295                 msg_text.color = ft.Colors.RED
296             dlg.open = False
297             page.update()
298         dlg = ft.AlertDialog(
299             title=ft.Text("Confirmar exclusão"),
300             content=ft.Text("Excluir cliente '" + str(cliente.get("nome")) + "'?"),
301             actions=[
302                 ft.TextButton("Não", on_click=confirmar),
303                 ft.TextButton("Sim", on_click=confirmar),
304             ],
305         )
306     page.overlay.append(dlg)
307     dlg.open = True
308     page.update()
309
310 def limpar_form():
311     edit_id["value"] = None
312     nome_field.value = ""
313     cpf_field.value = ""
314     email_field.value = ""
315     telefone_field.value = ""
316     endereco_field.value = ""
317
318 def novo_cliente(e):
319     limpar_form()
320     msg_text.value = "Novo cliente"
321     msg_text.color = ft.Colors.BLUE
322     page.update()
323
324 return ft.Container(
325     content=ft.Column(
326         [
327             ft.Row(
328                 [
329                     ft.Icon(ft.Icons.PEOPLE, size=32, color=ft.Colors.BLUE_700),
330                     ft.Text("Gestão de Clientes", size=24, weight=ft.FontWeight.BOLD),
331                     ft.ElevatedButton(
332                         "Novo",
333                         icon=ft.Icons.ADD,
334                         on_click=novo_cliente,
335                         bgcolor=ft.Colors.GREEN_600,
336                         color=ft.Colors.WHITE,
337                     ),
338                 ],
339                 alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
340             ),
341             ft.Divider(),
342             ft.Row([search_field]),
343             ft.Card(
344                 content=ft.Container(
345                     content=ft.Column(
346                         [
347                             ft.Text("Formulário", weight=ft.FontWeight.BOLD, size=16),
348                             ft.Row([nome_field, cpf_field]),
349                             ft.Row([email_field, telefone_field]),
350                             endereco_field,
351                             ft.Row(
352                                 [
353                                     ft.ElevatedButton(
354                                         "Salvar",
355                                         icon=ft.Icons.SAVE,
356                                         on_click=salvar_cliente,
357                                         bgcolor=ft.Colors.BLUE_700,
358                                         color=ft.Colors.WHITE,
359                                     ),
360                                     ft.OutlinedButton(
361                                         "Limpar",
362                                         on_click=lambda e: (limpar_form(), page.update()),
363                                     ),
364                                 ],
365                                 msg_text,
366                             ],
367                             alignment=ft.MainAxisAlignment.START,
368                             spacing=10,
369                         ),
370                     ],
371                     spacing=8,
372                 ),
373                 padding=15,
374             ),
375             ft.Text("Clientes cadastrados:", weight=ft.FontWeight.BOLD, size=14),
376             clientes_list,
377         ],
378         spacing=10,
379         expand=True,
380     ),
381     padding=15,
382     expand=True,
383     on_mount=lambda e: carregar_clientes(),
384 )
385
386
387
388 # =====
389 # TELA DE PRODUTOS
390 # =====
391
392 def criar_tela_produtos(page):
393     produtos_list = ft.ListView(spacing=5, padding=10, expand=True)
394     nome_field = ft.TextField(label="Nome do produto", border_radius=8, expand=True)
395     preco_field = ft.TextField(label="Preço (R$)", border_radius=8, expand=True, keyboard_type=ft.KeyboardType.NUMBER)
396     categoria_field = ft.TextField(label="Categoria", border_radius=8, expand=True)
397     estoque_field = ft.TextField(label="Estoque", border_radius=8, expand=True, keyboard_type=ft.KeyboardType.NUMBER)
398     descricao_field = ft.TextField(label="Descrição", border_radius=8, multiline=True, min_lines=2)
399     msg_text = ft.Text("", size=13, weight=ft.FontWeight.BOLD)
400     edit_id = {"value": None}
401
402     def carregar_produtos():
403         produtos_list.controls.clear()
404         result = api.get("/api/produtos/")
405         if result["success"]:
406             for p in result["data"]:
407                 produtos_list.controls.append(
408                     ft.Card(
409                         content=ft.Container(
410                             content=ft.Row(
411

```



```

411         ft.Icon(ft.Icons.INVENTORY_2, color=ft.Colors.PURPLE_600),
412         ft.Column(
413             [
414                 ft.Text(p.get("nome", ""), weight=ft.FontWeight.BOLD, size=14),
415                 ft.Text("R$ %.2f" % p.get("preco", 0), size=13, color=ft.Colors.GREEN_700, weight=ft.FontWeight.BOLD),
416                 ft.Text("Estoque: %d | %s" % (p.get("estoque", 0), p.get("categoria", "")), size=11, color=ft.Colors.GREY_600),
417             ],
418             expand=True,
419             spacing=2,
420         ),
421         ft.IconButton(
422             icon=ft.Icons.EDIT,
423             icon_color=ft.Colors.BLUE_600,
424             on_click=lambda e, prod=p: editar_produto(prod),
425         ),
426         ft.IconButton(
427             icon=ft.Icons.DELETE,
428             icon_color=ft.Colors.RED_600,
429             on_click=lambda e, prod=p: excluir_produto(prod),
430         ),
431     ],
432 ),
433 padding=12,
434 ),
435 elevation=2,
436 )
437 )
438 else:
439     produtos_list.controls.append(ft.Text("Erro: " + str(result["error"]), color=ft.Colors.RED))
440     page.update()
441
442 def salvar_produto(e):
443     try:
444         preco = float(preco_field.value or 0)
445         estoque = int(estoque_field.value or 0)
446     except ValueError:
447         msg_text.value = "Preco e estoque devem ser numeros!"
448         msg_text.color = ft.Colors.ORANGE
449         page.update()
450         return
451     data = {
452         "nome": nome_field.value or "",
453         "preco": preco,
454         "categoria": categoria_field.value or "",
455         "estoque": estoque,
456         "descricao": descricao_field.value or "",
457     }
458     if not data["nome"]:
459         msg_text.value = "Nome e obrigatorio!"
460         msg_text.color = ft.Colors.ORANGE
461         page.update()
462         return
463     if edit_id["value"]:
464         result = api.put("/api/produtos/" + str(edit_id["value"]), data)
465     else:
466         result = api.post("/api/produtos/", data)
467     if result["success"]:
468         msg_text.value = "Produto salvo!"
469         msg_text.color = ft.Colors.GREEN
470         limpar_form()
471         carregar_produtos()
472     else:
473         msg_text.value = str(result["error"][:80])
474         msg_text.color = ft.Colors.RED
475     page.update()
476
477 def editar_produto(produto):
478     edit_id["value"] = produto.get("id")
479     nome_field.value = produto.get("nome", "")
480     preco_field.value = str(produto.get("preco", ""))
481     categoria_field.value = produto.get("categoria", "")
482     estoque_field.value = str(produto.get("estoque", ""))
483     descricao_field.value = produto.get("descricao", "")
484     msg_text.value = "Editando produto..."
485     msg_text.color = ft.Colors.BLUE
486     page.update()
487
488 def excluir_produto(produto):
489     def confirmar():
490         if e.control.text == "Sim":
491             result = api.delete("/api/produtos/" + str(produto.get("id")))
492             if result["success"]:
493                 msg_text.value = "Produto excluido!"
494                 msg_text.color = ft.Colors.GREEN
495                 carregar_produtos()
496             dlg.open = False
497             page.update()
498     dlg = ft.AlertDialog(
499         title=ft.Text("Confirmar"),
500         content=ft.Text("Excluir '" + str(produto.get("nome")) + "'?"),
501         actions=[ft.TextButton("Nao", on_click=confirmar), ft.TextButton("Sim", on_click=confirmar)],
502     )
503     page.overlay.append(dlg)
504     dlg.open = True
505     page.update()
506
507 def limpar_form():
508     edit_id["value"] = None
509     nome_field.value = ""
510     preco_field.value = ""
511     categoria_field.value = ""
512     estoque_field.value = ""
513     descricao_field.value = ""
514
515 return ft.Container(
516     content=ft.Column(
517         [
518             ft.Row(
519                 [
520                     ft.Icon(ft.Icons.INVENTORY, size=32, color=ft.Colors.PURPLE_700),
521                     ft.Text("Gestao de Produtos", size=24, weight=ft.FontWeight.BOLD),
522                     ft.ElevatedButton(
523                         "Novo",
524                         icon=ft.Icons.ADD,
525                         on_click=lambda e: (limpar_form(), page.update()),
526                         bgcolor=ft.Colors.GREEN_600,
527                         color=ft.Colors.WHITE,
528                     ),
529                 ],
530                 alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
531             ),
532             ft.Divider(),
533             ft.Card(
534                 content=ft.Container(
535                     content=ft.Column(
536                         [
537                             ft.Text("Formulario", weight=ft.FontWeight.BOLD, size=16),
538                             ft.Row([nome_field, preco_field]),
539                             ft.Row([categoria_field, estoque_field]),
540                             descricao_field,
541                             ft.Row(
542                                 [
543                                     ft.ElevatedButton("Salvar", icon=ft.Icons.SAVE, on_click=salvar_produto, bgcolor=ft.Colors.PURPLE_700, color=ft.Colors.WHITE),
544                                     ft.OutlinedButton("Limpar", on_click=lambda e: (limpar_form(), page.update())),
545                                 ],
546                                 msg_text,
547                                 spacing=10,
548                             ),
549                         ],
550                     ),
551                 ),
552             ),
553         ],
554     ),
555 )

```

```

549         ],
550         spacing=8,
551     ),
552     padding=15,
553 ),
554     elevation=3,
555 ),
556     ft.Text("Produtos cadastrados:", weight=ft.FontWeight.BOLD, size=14),
557     produtos_list,
558 ],
559     spacing=10,
560     expand=True,
561 ),
562     padding=15,
563     expand=True,
564     on_mount=lambda e: carregar_produtos(),
565 )
566
567 # =====
568 # TELA DE VENDAS
569 # =====
570
571 def criar_tela_vendas(page):
572     vendas_list = ft.ListView(spacing=5, padding=10, expand=True)
573
574     def carregar_vendas():
575         vendas_list.controls.clear()
576         result = api.get("/api/vendas/")
577         if result["success"]:
578             total_geral = 0
579             for v in result["data"]:
580                 total = v.get("total", 0)
581                 total_geral += total
582                 vendas_list.controls.append(
583                     ft.Card(
584                         content=ft.Container(
585                             content=ft.Row(
586                                 [
587                                     ft.Icon(ft.Icons.SHOPPING_CART, color=ft.Colors.GREEN_600),
588                                     ft.Column(
589                                         [
590                                             ft.Text("Venda #" + str(v.get("id", 0)), weight=ft.FontWeight.BOLD, size=14),
591                                             ft.Text("Cliente: " + str(v.get("cliente nome", "N/A")), size=12),
592                                             ft.Text("Data: " + str(v.get("data", "N/A")), size=11, color=ft.Colors.GREY_600),
593                                         ],
594                                         expand=True,
595                                         spacing=2,
596                                     ),
597                                     ft.Text("R$ %.2f" % total, size=16, weight=ft.FontWeight.BOLD, color=ft.Colors.GREEN_700),
598                                 ],
599                             ),
600                             padding=12,
601                         ),
602                         elevation=2,
603                     )
604                 )
605             vendas_list.controls.insert(
606                 0,
607                 ft.Container(
608                     content=ft.Text(
609                         "Total: R$ %.2f | %d vendas" % (total_geral, len(result["data"])),
610                         weight=ft.FontWeight.BOLD, size=16, color=ft.Colors.BLUE_900,
611                     ),
612                     padding=10,
613                     bgcolor=ft.Colors.BLUE_50,
614                     border_radius=8,
615                 ),
616             )
617         else:
618             vendas_list.controls.append(ft.Text("Erro: " + str(result["error"]), color=ft.Colors.RED))
619     page.update()
620
621     return ft.Container(
622         content=ft.Column(
623             [
624                 ft.Row(
625                     [
626                         ft.Icon(ft.Icons.POINT_OF_SALE, size=32, color=ft.Colors.GREEN_700),
627                         ft.Text("Vendas Realizadas", size=24, weight=ft.FontWeight.BOLD),
628                     ],
629                     alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
630                 ),
631                 ft.Divider(),
632                 vendas_list,
633             ],
634             spacing=10,
635             expand=True,
636         ),
637         padding=15,
638         expand=True,
639         on_mount=lambda e: carregar_vendas(),
640     )
641
642 # =====
643 # TELA DE DASHBOARD
644 # =====
645
646 def criar_tela_dashboard(page):
647     stats_column = ft.Column(spacing=10)
648
649     def carregar_stats():
650         stats_column.controls.clear()
651         clientes_r = api.get("/api/clientes/")
652         produtos_r = api.get("/api/produtos/")
653         vendas_r = api.get("/api/vendas/")
654
655         n_clientes = len(clientes_r["data"]) if clientes_r["success"] else 0
656         n_produtos = len(produtos_r["data"]) if produtos_r["success"] else 0
657         n_vendas = len(vendas_r["data"]) if vendas_r["success"] else 0
658         total_vendas = 0
659         if vendas_r["success"]:
660             for v in vendas_r["data"]:
661                 total_vendas += v.get("total", 0)
662
663         cards = [
664             ("Clientes", str(n_clientes), ft.Colors.BLUE_700, ft.Colors.BLUE_50),
665             ("Produtos", str(n_produtos), ft.Colors.PURPLE_700, ft.Colors.PURPLE_50),
666             ("Vendas", str(n_vendas), ft.Colors.GREEN_700, ft.Colors.GREEN_50),
667             ("Faturamento", "R$ %.2f" % total_vendas, ft.Colors.ORANGE_700, ft.Colors.ORANGE_50),
668         ]
669
670         for titulo, valor, cor, bg in cards:
671             stats_column.controls.append(
672                 ft.Card(
673                     content=ft.Container(
674                         content=ft.Column(
675                             [
676                                 ft.Text(titulo, size=14, color=ft.Colors.GREY_700),
677                                 ft.Text(valor, size=32, weight=ft.FontWeight.BOLD, color=cor),
678                             ],
679                             horizontal_alignment=ft.CrossAxisAlignment.CENTER,
680                             spacing=5,
681                         ),
682                         padding=20,
683                         bgcolor=bg,
684                         border_radius=12,
685                         width=200,
686                     ),

```

```

687         elevation=4,
688     )
689 )
690 page.update()
691
692 return ft.Container(
693     content=ft.Column(
694         [
695             ft.Row(
696                 [
697                     ft.Icon(ft.Icons.DASHBOARD, size=32, color=ft.Colors.BLUE_700),
698                     ft.Text("Dashboard", size=24, weight=ft.FontWeight.BOLD),
699                     ft.IconButton(
700                         icon=ft.Icons.REFRESH,
701                         icon_color=ft.Colors.BLUE_700,
702                         tooltip="Atualizar",
703                         on_click=lambda e: carregar_stats(),
704                     ),
705                 ],
706                 alignment=ft.MainAxisAlignment.SPACE_BETWEEN,
707             ),
708             ft.Divider(),
709             ft.Text("Bem-vindo, " + str(api.user.get("nome", "Usuario"))) + "!", size=16, color=ft.Colors.GREY_700),
710             ft.Text("Empresa: " + str(api.user.get("empresa_nome", "N/A")), size=13, color=ft.Colors.GREY_600),
711             ft.Divider(height=10),
712             ft.Text("Resumo do Negocio", size=18, weight=ft.FontWeight.BOLD),
713             ft.Row([stats_column], wrap=True, spacing=15, run_spacing=15),
714         ],
715         spacing=10,
716         expand=True,
717     ),
718     padding=15,
719     expand=True,
720     on_mount=lambda e: carregar_stats(),
721 )
722
723
724 # =====
725 # TELA DE TESTE 100 CLIENTES
726 # =====
727 def criar_tela_teste(page):
728     result_text = ft.Text("", size=13, selectable=True)
729     progress_bar = ft.ProgressBar(width=400, visible=False)
730     progress_text = ft.Text("", size=12)
731
732     def executar_teste(e):
733         def run_test():
734             progress_bar.visible = True
735             result_text.value = "Iniciando teste de 100 clientes..."
736             page.update()
737             sucessos = 0
738             falhas = 0
739             for i in range(1, 101):
740                 data = {
741                     "nome": "Cliente Teste %03d" % i,
742                     "cpf_cnpj": "%011d" % i,
743                     "email": "cliente%03d@teste.com" % i,
744                     "telefone": "(62) 9%08d" % i,
745                     "endereco": "Rua Teste, %d - Centro" % i,
746                 }
747                 result = api.post("/api/clientes/", data)
748                 if result["success"]:
749                     sucessos += 1
750                 else:
751                     falhas += 1
752                 if i % 10 == 0:
753                     progress_bar.value = i / 100
754                     progress_text.value = "Progresso: %d/100" % i
755                     page.update()
756             result_text.value = (
757                 "Teste concluido!\n\n"
758                 "Resultados:\n"
759                 "  Sucessos: %d/100\n"
760                 "  Falhas: %d/100\n"
761                 "  Taxa de sucesso: %d%%" % (sucessos, falhas, sucessos)
762             )
763             progress_bar.visible = False
764             page.update()
765             threading.Thread(target=run_test, daemon=True).start()
766
767     return ft.Container(
768         content=ft.Column(
769             [
770                 ft.Row(
771                     [
772                         ft.Icon(ft.Icons.SCIENCE, size=32, color=ft.Colors.AMBER_700),
773                         ft.Text("Teste de Carga - 100 Clientes", size=24, weight=ft.FontWeight.BOLD),
774                     ],
775                 ),
776                 ft.Divider(),
777                 ft.Text(
778                     "Este teste insere 100 clientes de exemplo no sistema para validar a performance da API.",
779                     size=14, color=ft.Colors.GREY_700,
780                 ),
781                 ft.Container(height=10),
782                 ft.ElevatedButton(
783                     "Executar Teste de 100 Clientes",
784                     icon=ft.Icons.PLAY_ARROW,
785                     on_click=executar_teste,
786                     bgcolor=ft.Colors.AMBER_700,
787                     color=ft.Colors.WHITE,
788                     style=ft.ButtonStyle(shape=ft.RoundedRectangleBorder(radius=10)),
789                     height=50,
790                 ),
791                 ft.Container(height=15),
792                 progress_bar,
793                 progress_text,
794                 ft.Divider(),
795                 ft.Container(
796                     content=ft.Column([result_text]),
797                     padding=15,
798                     bgcolor=ft.Colors.GREY_100,
799                     border_radius=10,
800                     width=500,
801                 ),
802             ],
803             spacing=10,
804             expand=True,
805         ),
806         padding=15,
807         expand=True,
808     )
809
810
811 # =====
812 # APLICACAO PRINCIPAL
813 # =====
814 def main(page: ft.Page):
815     page.title = "SaltoGestao - Sistema de Gestao Empresarial"
816     page.theme.mode = ft.ThemeMode.LIGHT
817     page.padding = 0
818     page.window.width = 1200
819     page.window.height = 750
820     page.window.min_width = 900
821     page.window.min_height = 600
822     page.theme = ft.Theme(color_scheme_seed=ft.Colors.BLUE, font_family="Segoe UI")
823
824     main_content = ft.Container(expand=True)

```

```

825
826 def mostrar_tela(tela_nome):
827     telas = {
828         "dashboard": criar_tela_dashboard,
829         "clientes": criar_tela_clientes,
830         "produtos": criar_tela_produtos,
831         "vendas": criar_tela_vendas,
832         "teste": criar_tela_teste,
833     }
834     if tela_nome in telas:
835         main_content.content = telas[tela_nome](page)
836         page.update()
837
838 def on_login_success():
839     main_content.content = criar_layout_principal()
840     page.update()
841
842 def criar_layout_principal():
843     def navegar(e):
844         idx = e.control.selected_index
845         telas_map = {0: "dashboard", 1: "clientes", 2: "produtos", 3: "vendas", 4: "teste"}
846         if idx in telas_map:
847             mostrar_tela(telas_map[idx])
848
849     sidebar = ft.NavigationRail(
850         selected_index=0,
851         label_type=ft.NavigationRailLabelType.ALL,
852         min_width=100,
853         min_extended_width=200,
854         group_alignment=-0.9,
855         destinations=[
856             ft.NavigationRailDestination(
857                 icon=ft.Icons.DASHBOARD_OUTLINED,
858                 selected_icon=ft.Icons.DASHBOARD,
859                 label="Dashboard",
860             ),
861             ft.NavigationRailDestination(
862                 icon=ft.Icons.PEOPLE_OUTLINED,
863                 selected_icon=ft.Icons.PEOPLE,
864                 label="Clientes",
865             ),
866             ft.NavigationRailDestination(
867                 icon=ft.Icons.INVENTORY_2_OUTLINED,
868                 selected_icon=ft.Icons.INVENTORY_2,
869                 label="Produtos",
870             ),
871             ft.NavigationRailDestination(
872                 icon=ft.Icons.POINT_OF_SALE_OUTLINED,
873                 selected_icon=ft.Icons.POINT_OF_SALE,
874                 label="Vendas",
875             ),
876             ft.NavigationRailDestination(
877                 icon=ft.Icons.SCIENCE_OUTLINED,
878                 selected_icon=ft.Icons.SCIENCE,
879                 label="Teste 100",
880             ),
881         ],
882         on_change=navegar,
883         bgcolor=ft.Colors.BLUE_50,
884     )
885
886     content_area = ft.Container(
887         content=criar_tela_dashboard(page),
888         expand=True,
889         padding=0,
890     )
891
892     nonlocal main_content
893     main_content = content_area
894
895     return ft.Container(
896         content=ft.Row(
897             [sidebar, ft.VerticalDivider(width=1), content_area],
898             expand=True,
899             spacing=0,
900         ),
901         expand=True,
902     )
903
904 main_content.content = criar_tela_login(page, on_login_success)
905 page.add(main_content)
906
907
908 if __name__ == "__main__":
909     ft.app(target=main)
910

```

```

1 """
2 SALTO GESTAO Desktop v2.0 - Design Moderno & Clean
3 Paleta: Corumbá Sistemas (#0b1a30)
4 """
5
6 import flet as ft
7 import requests
8 import json
9 import threading
10 from datetime import datetime
11
12 # Importar design system
13 from design_system import Cores, Espacamento, Bordas, DS
14
15 # Configuracao
16 API_URL = "http://191.252.210.235:8080"
17
18
19 # =====
20 # CLIENTE API
21 # =====
22
23 class APIClient:
24     def __init__(self):
25         self.base_url = API_URL
26         self.token = None
27         self.user = None
28         self.headers = {"Content-Type": "application/json"}
29
30     def set_token(self, token):
31         self.token = token
32         self.headers["Authorization"] = "Bearer " + token
33
34     def login(self, username, password):
35         try:
36             resp = requests.post(
37                 self.base_url + "/api/login",
38                 json={"username": username, "password": password},
39                 timeout=10,
40             )
41             if resp.status_code == 200:
42                 data = resp.json()
43                 self.set_token(data.get("access_token", ""))
44                 self.user = data.get("user", {})
45                 return {"success": True, "data": data}
46             return {"success": False, "error": resp.json().get("detail", "Erro no login")}
47         except requests.exceptions.ConnectionError:
48             return {"success": False, "error": "Nao foi possivel conectar ao servidor"}
49         except Exception as e:
50             return {"success": False, "error": str(e)}
51
52     def get(self, endpoint):
53         try:
54             resp = requests.get(self.base_url + endpoint, headers=self.headers, timeout=15)
55             if resp.status_code == 200:
56                 return {"success": True, "data": resp.json()}
57             return {"success": False, "error": resp.text}
58         except Exception as e:
59             return {"success": False, "error": str(e)}
60
61     def post(self, endpoint, data):
62         try:
63             resp = requests.post(self.base_url + endpoint, json=data, headers=self.headers, timeout=15)
64             if resp.status_code in (200, 201):
65                 return {"success": True, "data": resp.json()}
66             return {"success": False, "error": resp.text}
67         except Exception as e:
68             return {"success": False, "error": str(e)}
69
70     def put(self, endpoint, data):
71         try:
72             resp = requests.put(self.base_url + endpoint, json=data, headers=self.headers, timeout=15)
73             if resp.status_code == 200:
74                 return {"success": True, "data": resp.json()}
75             return {"success": False, "error": resp.text}
76         except Exception as e:
77             return {"success": False, "error": str(e)}
78
79     def delete(self, endpoint):
80         try:
81             resp = requests.delete(self.base_url + endpoint, headers=self.headers, timeout=15)
82             if resp.status_code in (200, 204):
83                 return {"success": True}
84             return {"success": False, "error": resp.text}
85         except Exception as e:
86             return {"success": False, "error": str(e)}
87
88
89 api = APIClient()
90
91
92 # =====
93 # TELA DE LOGIN v2.0
94 # =====
95
96 def criar_tela_login(page, on_login_success):
97     """Tela de login com design moderno e clean"""
98
99     username_field = DS.campo(
100         label="Usuário",
101         prefix_icon=ft.Icons.PERSON_OUTLINED,
102         hint_text="Digite seu usuário",
103         width=360,
104     )
105     password_field = DS.campo(
106         label="Senha",
107         prefix_icon=ft.Icons.LOCK_OUTLINED,
108         password=True,
109         can_reveal_password=True,
110         hint_text="Digite sua senha",
111         width=360,
112     )
113
114     msg_banner = ft.Container(visible=False, animate=ft.Animation(200, ft.AnimationCurve.EASE_OUT))
115     loading = DS.loading_spinner(size=20)
116     loading.visible = False
117
118     def show_message(text, msg_type="error"):
119         icons = {"error": ft.Icons.ERROR_OUTLINED, "success": ft.Icons.CHECK_CIRCLE_OUTLINED}
120         colors = {"error": Cores.ERRO, "success": Cores.SUCESSO}
121         bg_colors = {"error": ft.Colors.with_opacity(0.08, Cores.ERRO), "success": ft.Colors.with_opacity(0.08, Cores.SUCESSO)}
122
123         msg_banner.content = ft.Container(
124             content=ft.Row(
125                 ft.Icon(icons.get(msg_type), size=16, color=colors.get(msg_type)),
126                 ft.Container(width=8),
127                 ft.Text(text, size=12, color=Cores.TEXT0_80),
128             ), spacing=8),
129             bgcolor=bg_colors.get(msg_type),
130             border_radius=Bordas.SM,
131             padding=ft.Padding.symmetric(horizontal=12, vertical=8),
132             border=ft.Border.all(1, ft.Colors.with_opacity(0.15, colors.get(msg_type))),
133             width=360,
134         )

```

```

135     msg_banner.visible = True
136     page.update()
137
138 def do_login(e):
139     if not username_field.value or not password_field.value:
140         show_message("Preencha todos os campos", "error")
141         return
142
143     loading.visible = True
144     msg_banner.visible = False
145     page.update()
146
147     result = api.login(username_field.value, password_field.value)
148     loading.visible = False
149
150     if result["success"]:
151         show_message("Login realizado com sucesso!", "success")
152         page.update()
153         # Pequena pausa para mostrar mensagem de sucesso
154         import time
155         time.sleep(0.5)
156         on_login_success()
157     else:
158         show_message(result["error"], "error")
159         page.update()
160
161 # Layout do login
162 login_card = DS.card(
163     content=ft.Column([
164         # Logo area
165         ft.Container(
166             content=ft.Column([
167                 # Icon com fundo circular
168                 ft.Container(
169                     content=ft.Icon(ft.Icons.STORE_ROUNDED, size=40, color=Cores.P),
170                     bgcolor=ft.Colors.with_opacity(0.1, Cores.P),
171                     border_radius=Bordas.FULL,
172                     padding=20,
173                     width=80,
174                     height=80,
175                     alignment=ft.alignment.center,
176                 ),
177                 ft.Container(height=Espacamento.SM),
178                 DS.titulo("SaltoGestao", size=24, weight=ft.FontWeight.W_700, color=Cores.P),
179                 ft.Container(height=2),
180                 DS.subtitulo("Sistema de Gestao Empresarial", size=12),
181             ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=2),
182             alignment=ft.alignment.center,
183             padding=ft.padding.only(top=Espacamento.MD),
184         ),
185         ft.Container(height=Espacamento.LG),
186         # Campos
187         username_field,
188         ft.Container(height=Espacamento.SM),
189         password_field,
190         ft.Container(height=Espacamento.SM),
191         # Mensagem
192         msg_banner,
193         ft.Container(height=Espacamento.SM),
194         # Botao
195         DS.botao(
196             texto="Entrar",
197             icon=ft.Icons.ARROW_FORWARD,
198             width=360,
199             height=48,
200             on_click=do_login,
201         ),
202         ft.Container(height=Espacamento.SM),
203         # Loading row
204         ft.Row([loading], alignment=ft.MainAxisAlignment.CENTER),
205         ft.Container(height=Espacamento.MD),
206     ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=4),
207     padding=Espacamento.XL,
208     width=420,
209     shadow=ft.BoxShadow(
210         spread_radius=0,
211         blur_radius=24,
212         color=ft.Colors.with_opacity(0.12, Cores.ESCURO),
213         offset=ft.Offset(0, 8),
214     ),
215 )
216
217 return ft.Container(
218     content=login_card,
219     alignment=ft.alignment.center,
220     expand=True,
221     gradient=ft.LinearGradient(
222         begin=ft.alignment.top_center,
223         end=ft.alignment.bottom_center,
224         colors=[Cores.FUNDO, Cores.FUNDO_CARD],
225     ),
226 )
227
228 # =====
229 # TELA DE CLIENTES v2.0
230 # =====
231
232 def criar_tela_clientes(page):
233     """Tela de clientes com design moderno"""
234
235     clientes_list = ft.ListView(spacing=Espacamento.SM, padding=0, expand=True)
236     search_field = DS.campo(
237         label="Buscar...",
238         prefix_icon=ft.Icons.SEARCH,
239         hint_text="Nome, CPF, telefone...",
240     )
241
242     # Formulário
243     nome_field = DS.campo(label="Nome completo", hint_text="Ex: Joao Silva")
244     cpf_field = DS.campo(label="CPF/CNPJ", hint_text="000.000.000-00")
245     email_field = DS.campo(label="E-mail", hint_text="email@exemplo.com")
246     telefone_field = DS.campo(label="Telefone", hint_text="(62) 99999-9999")
247
248     msg_banner = ft.Container(visible=False, animate=ft.Animation(200, ft.AnimationCurve.EASE_OUT))
249     edit_id = {"value": None}
250
251     def show_msg(text, msg_type="success"):
252         icons = {"success": ft.Icons.CHECK_CIRCLE_OUTLINED, "error": ft.Icons.ERROR_OUTLINED, "info": ft.Icons.INFO_OUTLINED}
253         colors = {"success": Cores.SUCESSO, "error": Cores.ERRO, "info": Cores.INFO}
254         bg = {"success": ft.Colors.with_opacity(0.08, Cores.SUCESSO), "error": ft.Colors.with_opacity(0.08, Cores.ERRO), "info": ft.Colors.with_opacity(0.08, Cores.INFO)}
255         msg_banner.content = ft.Container(
256             content=ft.Row([
257                 ft.Icon(icons.get(msg_type), size=16, color=Cores.TEXT0),
258                 ft.Container(width=8),
259                 ft.Text(text, size=12, color=Cores.TEXT0_80),
260             ], spacing=8),
261             bgcolor=bg.get(msg_type),
262             border_radius=Bordas.SM,

```

```

273         padding=ft.padding.symmetric(horizontal=12, vertical=8),
274         margin=ft.padding.only(bottom=Espacamento.SM),
275     )
276     msg_banner.visible = True
277     page.update()
278
279 def carregar_clientes():
280     clientes_list.controls.clear()
281     result = api.get("/api/clientes/")
282
283     if result["success"]:
284         clientes = result["data"]
285         if not clientes:
286             clientes_list.controls.append(DS.empty_state(
287                 icon=ft.Icons.PEOPLE_OUTLINED,
288                 title="Nenhum cliente cadastrado",
289                 subtitle="Clique em 'Novo' para adicionar",
290             ))
291         for c in clientes:
292             clientes_list.controls.append(DS.card(
293                 padding=0,
294                 shadow=False,
295                 border=ft.border.all(1, Cores.BORDA),
296                 content=ft.Container(
297                     content=ft.Row(
298                         ft.CircleAvatar(
299                             content=ft.Text(
300                                 c.get("nome", "?").split()[0][0].upper() if c.get("nome") else "?",
301                                 size=16, weight=ft.FontWeight.W_600, color=Cores.FUNDO_CARD,
302                             ),
303                             bgcolor=Cores.P20,
304                             radius=20,
305                         ),
306                         ft.Container(width=Espacamento.SM),
307                         ft.Column(
308                             ft.Text(c.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0),
309                             ft.Text("CPF: " + str(c.get("cpf_cnpj", "N/A")), size=11, color=Cores.TEXT0_60),
310                             ft.Text("Tel: " + str(c.get("telefone", "N/A")), size=11, color=Cores.TEXT0_60),
311                             ], spacing=2, expand=True),
312                             DS.icone_botao(ft.Icons.EDIT_OUTLINED, "Editar", on_click=lambda e, cli=c: editar_cliente(cli)),
313                             DS.icone_botao(ft.Icons.DELETE_OUTLINED, "Excluir", icon_color=Cores.ERRO, on_click=lambda e, cli=c: excluir_cliente(cli)),
314                             ], spacing=Espacamento.SM, alignment=ft.MainAxisAlignment.START),
315                             padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
316                         ),
317                     ))
318     else:
319         clientes_list.controls.append(
320             ft.Text("Erro: " + str(result["error"])[0:80], color=Cores.ERRO, size=12)
321         )
322     page.update()
323
324 def salvar_cliente(e):
325     data = {
326         "nome": nome_field.value or "",
327         "cpf_cnpj": cpf_field.value or "",
328         "email": email_field.value or "",
329         "telefone": telefone_field.value or "",
330     }
331     if not data["nome"]:
332         show_msg("Nome é obrigatorio", "error")
333         return
334
335     if edit_id["value"]:
336         result = api.put("/api/clientes/" + str(edit_id["value"]), data)
337     else:
338         result = api.post("/api/clientes/", data)
339
340     if result["success"]:
341         show_msg("Cliente salvo com sucesso!", "success")
342         limpar_form()
343         carregar_clientes()
344     else:
345         show_msg("Erro: " + str(result["error"])[0:80], "error")
346
347 def editar_cliente(cliente):
348     edit_id["value"] = cliente.get("id")
349     nome_field.value = cliente.get("nome", "")
350     cpf_field.value = cliente.get("cpf_cnpj", "")
351     email_field.value = cliente.get("email", "")
352     telefone_field.value = cliente.get("telefone", "")
353     show_msg("Editando: " + str(cliente.get("nome", "")), "info")
354     page.update()
355
356 def excluir_cliente(cliente):
357     def confirmar(e):
358         if e.control.text == "Sim":
359             result = api.delete("/api/clientes/" + str(cliente.get("id")))
360             if result["success"]:
361                 show_msg("Cliente excluido!", "success")
362                 carregar_clientes()
363             else:
364                 show_msg("Erro ao excluir", "error")
365         dlg.open = False
366         page.update()
367
368     dlg = ft.AlertDialog(
369         title=ft.Text("Confirmar exclusao", weight=ft.FontWeight.W_600),
370         content=ft.Text("Excluir " + str(cliente.get("nome")) + "?" ),
371         actions=[
372             ft.TextButton("Nao", on_click=confirmar),
373             ft.TextButton("Sim", on_click=confirmar, style=ft.ButtonStyle(color=Cores.ERRO)),
374         ],
375     )
376     page.overlay.append(dlg)
377     dlg.open = True
378     page.update()
379
380 def limpar_form():
381     edit_id["value"] = None
382     nome_field.value = cpf_field.value = email_field.value = telefone_field.value = ""
383
384 def Novo_cliente(e):
385     limpar_form()
386     show_msg("Novo cliente - preencha os campos", "info")
387     page.update()
388
389 # Layout
390 return ft.Container(
391     content=ft.Column([
392         DS.page_header("Clientes", subtitle="Gerencie seus clientes", actions=[
393             DS.botao_outline("Limpar", icon=ft.Icons.REFRESH, on_click=lambda e: (limpar_form(), page.update())),
394             DS.botao("Novo Cliente", icon=ft.Icons.ADD, on_click=Novo_cliente),
395         ]),
396
397         # Busca
398         DS.card(content=search_field, padding=Espacamento.SM, shadow=False),
399
400         # Formulario
401         DS.card(
402             titulo="Formulario",
403             content=ft.Column([
404                 ft.Row([nome_field, cpf_field], spacing=Espacamento.SM),
405                 ft.Row([email_field, telefone_field], spacing=Espacamento.SM),
406                 ft.Row([
407                     DS.botao("Salvar", icon=ft.Icons.SAVE_OUTLINED, on_click=salvar_cliente),
408                     DS.botao_outline("Limpar", on_click=lambda e: (limpar_form(), page.update())),
409                 ], spacing=Espacamento.SM),
410             ]),
411             msg_banner,

```

```

411         ], spacing=Espacamento.SM),
412         padding=Espacamento.MD,
413     ),
414     # Lista
415     ft.Container(
416         content=ft.Text(str("Cadastrados"), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60),
417         padding=ft.padding.only(left=4, bottom=Espacamento.SM),
418     ),
419     ft.Container(
420         content=clientes_list,
421         expand=True,
422     ),
423 ], spacing=Espacamento.MD, expand=True),
424 padding=Espacamento.LG,
425 bgcolor=Cores.FUNDO,
426 expand=True,
427 on_mount=lambda e: carregar_clientes(),
428 )
429
430
431
432 # =====
433 # TELA DE PRODUTOS v2.0
434 # =====
435
436 def criar_tela_produtos(page):
437     produtos_list = ft.ListView(spacing=Espacamento.SM, padding=0, expand=True)
438
439     nome_field = DS.campo(label="Nome do produto", hint_text="Ex: Arroz 5kg")
440     preco_field = DS.campo(label="Preço (R$)", hint_text="0.00")
441     categoria_field = DS.campo(label="Categoria", hint_text="Alimentos")
442     estoque_field = DS.campo(label="Estoque", hint_text="0")
443     descricao_field = DS.campo(label="Descricao", multiline=True, height=72)
444
445     msg_banner = ft.Container(visible=False)
446     edit_id = {"value": None}
447
448     def show_msg(text, msg_type="success"):
449         colors = {"success": Cores.SUCESSO, "error": Cores.ERRO, "info": Cores.INFO}
450         bg = {"success": ft.Colors.with_opacity(0.08, Cores.SUCESSO), "error": ft.Colors.with_opacity(0.08, Cores.ERRO), "info": ft.Colors.with_opacity(0.08, Cores.INFO)}
451         msg_banner.content = ft.Container(
452             content=ft.Row([ft.Text(text, size=12, color=Cores.TEXT0_80)], spacing=8),
453             bgcolor=bg.get(msg_type),
454             border_radius=Bordas.SM,
455             padding=ft.padding.symmetric(horizontal=12, vertical=8),
456             margin=ft.padding.only(bottom=Espacamento.SM),
457         )
458         msg_banner.visible = True
459         page.update()
460
461     def carregar_produtos():
462         produtos_list.controls.clear()
463         result = api.get("/api/produtos/")
464         if result["success"]:
465             produtos = result["data"]
466             if not produtos:
467                 produtos_list.controls.append(DS.empty_state(
468                     icon=ft.Icons.INVENTORY_2_OUTLINED,
469                     title="Nenhum produto cadastrado",
470                 ))
471             for p in produtos:
472                 preco = p.get("preco", 0)
473                 estoque = p.get("estoque", 0)
474                 produtos_list.controls.append(DS.card(
475                     padding=0,
476                     shadow=False,
477                     border=ft.border.all(1, Cores.BORDA),
478                     content=ft.Container(
479                         content=ft.Row([
480                             ft.Container(
481                                 content=ft.Icon(ft.Icons.INVENTORY_2_OUTLINED, size=18, color=Cores.INFO),
482                                 bgcolor=ft.Colors.with_opacity(0.1, Cores.INFO),
483                                 border_radius=Bordas.SM,
484                                 padding=8,
485                                 width=40, height=40,
486                                 alignment=ft.alignment.center,
487                             ),
488                             ft.Container(width=Espacamento.SM),
489                             ft.Column([
490                                 ft.Text(p.get("nome", ""), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0),
491                                 ft.Text("R$ %.2f" % preco, size=12, weight=ft.FontWeight.W_700, color=Cores.SUCESSO, font family="Courier New"),
492                                 ft.Text(str(estoque) + " em estoque | " + str(p.get("categoria", ""))), size=10, color=Cores.TEXT0_60,
493                             ], spacing=2, expand=True),
494                             DS.icone_botao(ft.Icons.EDIT_OUTLINED, "Editar", on_click=lambda e, prod=p: editar_produto(prod)),
495                             DS.icone_botao(ft.Icons.DELETE_OUTLINED, "Excluir", icon_color=Cores.ERRO, on_click=lambda e, prod=p: excluir_produto(prod)),
496                             ], spacing=Espacamento.SM, alignment=ft.MainAxisAlignment.START),
497                             padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
498                         ),
499                     ),
500                 else:
501                     produtos_list.controls.append(ft.Text("Erro: " + str(result["error"])[0:80], color=Cores.ERRO, size=12))
502                 page.update()
503
504     def salvar_produto(e):
505         try:
506             preco = float(preco_field.value or 0)
507             estoque = int(estoque_field.value or 0)
508         except ValueError:
509             show_msg("Preço e estoque devem ser numeros", "error")
510             return
511         data = {"nome": nome_field.value or "", "preco": preco, "categoria": categoria_field.value or "", "estoque": estoque, "descricao": descricao_field.value or ""}
512         if not data["nome"]:
513             show_msg("Nome é obrigatorio", "error")
514             return
515         if edit_id["value"]:
516             result = api.put("/api/produtos/" + str(edit_id["value"]), data)
517         else:
518             result = api.post("/api/produtos/", data)
519         if result["success"]:
520             show_msg("Produto salvo!", "success")
521             for f in [nome_field, preco_field, categoria_field, estoque_field, descricao_field]:
522                 f.value = ""
523             edit_id["value"] = None
524             carregar_produtos()
525         else:
526             show_msg("Erro: " + str(result["error"])[0:80], "error")
527
528     def editar_produto(produto):
529         edit_id["value"] = produto.get("id")
530         nome_field.value = produto.get("nome", "")
531         preco_field.value = str(produto.get("preco", ""))
532         categoria_field.value = produto.get("categoria", "")
533         estoque_field.value = str(produto.get("estoque", ""))
534         descricao_field.value = produto.get("descricao", "")
535         show_msg("Editando: " + str(produto.get("nome", "")), "info")
536         page.update()
537
538     def excluir_produto(produto):
539         def confirmar(e):
540             if e.control.text == "Sim":
541                 result = api.delete("/api/produtos/" + str(produto.get("id")))
542                 if result["success"]:
543                     show_msg("Produto excluido!", "success")
544                     carregar_produtos()
545             dlg.open = False
546             page.update()
547         dlg = ft.AlertDialog(
548             title=ft.Text("Confirmar", weight=ft.FontWeight.W_600),

```



```

549         content=ft.Text("Excluir " + str(produto.get("nome")) + "?"?),
550         actions=[ft.TextButton("Nao", on_click=confirmar), ft.TextButton("Sim", on_click=confirmar)],
551     )
552     page.overlay.append(dlg)
553     dlg.open = True
554     page.update()
555
556     return ft.Container(
557         content=ft.Column([
558             DS.page_header("Produtos", subtitle="Gerencie seu estoque", actions=[
559                 DS.botoao_outline("Limpar", icon=ft.Icons.REFRESH, on_click=lambda e: page.update()),
560                 DS.botoao("Novo Produto", icon=ft.Icons.ADD, on_click=lambda e: None),
561             ]),
562             DS.card(
563                 titulo="Formulario",
564                 content=ft.Column([
565                     ft.Row([nome_field, preco_field], spacing=Espacamento.SM),
566                     ft.Row([categoria_field, estoque_field], spacing=Espacamento.SM),
567                     descricao_field,
568                     ft.Row([
569                         DS.botoao("Salvar", icon=ft.Icons.SAVE_OUTLINED, on_click=salvar_produto),
570                         DS.botoao_outline("Limpar", on_click=lambda e: page.update()),
571                     ], spacing=Espacamento.SM),
572                     msg_banner,
573                 ], spacing=Espacamento.SM),
574                 padding=Espacamento.MD,
575             ),
576             ft.Container(
577                 content=ft.Text("Produtos cadastrados", size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0_60),
578                 padding=ft.padding.only(left=4, bottom=Espacamento.SM),
579             ),
580             ft.Container(content=produtos_list, expand=True),
581         ], spacing=Espacamento.MD, expand=True),
582         padding=Espacamento.LG,
583         bgcolor=Cores.FUNDO,
584         expand=True,
585         on_mount=lambda e: carregar_produtos(),
586     )
587
588 # =====
589 # TELA DE VENDAS v2.0
590 # =====
591
592 def criar_tela_vendas(page):
593     vendas_list = ft.ListView(spacing=Espacamento.SM, padding=0, expand=True)
594
595     def carregar_vendas():
596         vendas_list.controls.clear()
597         result = api.get("/api/vendas/")
598         if result["success"]:
599             vendas = result["data"]
600             total_geral = 0
601             for v in vendas:
602                 total = v.get("total", 0)
603                 total_geral += total
604             vendas_list.controls.append(DS.card(
605                 padding=0,
606                 shadow=False,
607                 border=ft.border.all(1, Cores.BORDA),
608                 content=ft.Container(
609                     content=ft.Row([
610                         ft.Container(
611                             content=ft.Icon(ft.Icons.POINT_OF_SALE, size=16, color=Cores.SUCESSO),
612                             bgcolor=ft.Colors.with_opacity(0.1, Cores.SUCESSO),
613                             border_radius=Bordas.SM,
614                             padding=6, width=36, height=36,
615                             alignment=ft.alignment.center,
616                         ),
617                         ft.Container(width=Espacamento.SM),
618                         ft.Column([
619                             ft.Text("Venda #" + str(v.get("id", 0)), size=13, weight=ft.FontWeight.W_600, color=Cores.TEXT0),
620                             ft.Text(str(v.get("cliente_nome", "N/A")), size=11, color=Cores.TEXT0_60),
621                             ft.Text(str(v.get("data", "N/A")), size=10, color=Cores.TEXT0_40),
622                         ], spacing=2, expand=True),
623                         ft.Text("R$ %.2f" % total, size=14, weight=ft.FontWeight.W_700, color=Cores.SUCESSO, font_family="Courier New"),
624                     ], spacing=Espacamento.SM, alignment=ft.MainAxisAlignment.START),
625                 padding=ft.padding.symmetric(horizontal=Espacamento.MD, vertical=10),
626             ),
627         ))
628
629     vendas_list.controls.insert(0, DS.card(
630         content=ft.Row([
631             ft.Text(str(len(vendas)) + " vendas", size=12, color=Cores.TEXT0_60),
632             ft.Text("Total: R$ %.2f" % total_geral, size=14, weight=ft.FontWeight.W_700, color=Cores.P, font_family="Courier New"),
633         ], alignment=ft.MainAxisAlignment.SPACE_BETWEEN),
634         padding=Espacamento.MD,
635         shadow=False,
636         border=ft.border.all(1, Cores.P20),
637         bgcolor=ft.Colors.with_opacity(0.03, Cores.P),
638     ))
639     page.update()
640
641     return ft.Container(
642         content=ft.Column([
643             DS.page_header("Vendas", subtitle="Historico de vendas"),
644             ft.Container(content=vendas_list, expand=True),
645         ], spacing=Espacamento.MD),
646         padding=Espacamento.LG,
647         bgcolor=Cores.FUNDO,
648         expand=True,
649         on_mount=lambda e: carregar_vendas(),
650     )
651
652 # =====
653 # TELA DE DASHBOARD v2.0
654 # =====
655
656 def criar_tela_dashboard(page):
657     stats_column = ft.Row(spacing=Espacamento.SM, wrap=True, scroll=ft.ScrollMode.AUTO)
658
659     def carregar_stats():
660         stats_column.controls.clear()
661
662         clientes_r = api.get("/api/clientes/")
663         produtos_r = api.get("/api/produtos/")
664         vendas_r = api.get("/api/vendas/")
665
666         n_clientes = len(clientes_r["data"]) if clientes_r["success"] else 0
667         n_produtos = len(produtos_r["data"]) if produtos_r["success"] else 0
668         n_vendas = len(vendas_r["data"]) if vendas_r["success"] else 0
669
670         total_vendas = 0
671         if vendas_r["success"]:
672             for v in vendas_r["data"]:
673                 total_vendas += v.get("total", 0)
674
675         stats = [
676             ("Clientes", str(n_clientes), ft.Icons.PEOPLE_OUTLINED, Cores.P),
677             ("Produtos", str(n_produtos), ft.Icons.INVENTORY_2_OUTLINED, Cores.INFO),
678             ("Vendas", str(n_vendas), ft.Icons.POINT_OF_SALE, Cores.SUCESSO),
679             ("Faturamento", "R$ %.2f" % total_vendas, ft.Icons.ATTACH_MONEY, Cores.ALERTA),
680         ]
681
682         for titulo, valor, icone, cor in stats:
683             stats_column.controls.append(DS.stat_card(titulo, valor, icone, cor))
684
685     page.update()

```

```

687
688 return ft.Container(
689     content=ft.Column([
690         DS.page_header("Dashboard",
691             subtitle="Bem-vindo, " + str(api.user.get("nome", "Usuario")) + "!",
692             actions=[DS.icone_botao(ft.Icons.REFRESH, "Atualizar", on_click=lambda e: carregar_stats())]
693         ),
694         stats_column,
695         DS.divider(),
696         DS.subtitulo("Resumo do Negocio", size=13),
697     ], spacing=Espacamento.MD),
698     padding=Espacamento.LG,
699     bgcolor=Cores.FUNDO,
700     expand=True,
701     on_mount=lambda e: carregar_stats(),
702 )
703
704
705 # =====
706 # TELA DE TESTE v2.0
707 # =====
708
709 def criar_tela_teste(page):
710     result_text = ft.Text("", size=12, selectable=True, color=Cores.TEXT0_80)
711     progress_bar = ft.ProgressBar(width=300, visible=False, color=Cores.P, bgcolor=Cores.P10)
712     progress_text = ft.Text("", size=11, color=Cores.TEXT0_60)
713
714     def executar_teste(e):
715         def run_test():
716             progress_bar.visible = True
717             result_text.value = "Iniciando teste de 100 clientes..."
718             page.update()
719             sucessos = 0
720             falhas = 0
721             for i in range(1, 101):
722                 data = {
723                     "nome": "Cliente Teste %03d" % i,
724                     "cpf_cnpj": "%011d" % i,
725                     "email": "cliente%03d@teste.com" % i,
726                     "telefone": "(62) 9%08d" % i,
727                     "endereco": "Rua Teste, %d" % i,
728                 }
729                 result = api.post("/api/clientes/", data)
730                 if result["success"]:
731                     sucessos += 1
732                 else:
733                     falhas += 1
734                 if i % 10 == 0:
735                     progress_bar.value = i / 100
736                     progress_text.value = "Progresso: %d/100" % i
737                     page.update()
738                 result_text.value = "Teste concluido!\n Sucessos: %d/100\n Falhas: %d" % (sucessos, falhas)
739                 progress_bar.visible = False
740                 page.update()
741             threading.Thread(target=run_test, daemon=True).start()
742
743     return ft.Container(
744         content=ft.Column([
745             DS.page_header("Teste de Carga", subtitle="Insira 100 clientes de teste"),
746             DS.card(
747                 content=ft.Column([
748                     DS.paragrafo("Este teste insere 100 clientes de exemplo para validar a performance da API."),
749                     ft.Container(height=Espacamento.SM),
750                     DS.botao("Executar Teste - 100 Clientes", icon=ft.Icons.PLAY_ARROW_ROUNDED, on_click=executar_teste),
751                     ft.Container(height=Espacamento.SM),
752                     progress_bar,
753                     progress_text,
754                 ], spacing=Espacamento.SM),
755                 padding=Espacamento.MD,
756             ),
757             ft.Container(height=Espacamento.SM),
758             DS.titulo("Resultado", size=14),
759             DS.card(
760                 content=result_text,
761                 padding=Espacamento.MD,
762                 shadow=False,
763                 border=ft.border.all(1, Cores.BORDA),
764             ),
765         ], spacing=Espacamento.MD),
766         padding=Espacamento.LG,
767         bgcolor=Cores.FUNDO,
768         expand=True,
769     )
770
771
772 # =====
773 # LAYOUT PRINCIPAL v2.0
774 # =====
775
776 def criar_layout_principal():
777     """Layout principal com sidebar moderna"""
778
779     main_content = ft.Container(
780         content=criar_tela_dashboard(None),
781         expand=True,
782         bgcolor=Cores.FUNDO,
783     )
784
785     # Indice selecionado (usando closure)
786     state = {"selected": 0}
787
788     def navegar(index):
789         state["selected"] = index
790         telas = {
791             0: criar_tela_dashboard,
792             1: criar_tela_clientes,
793             2: criar_tela_produtos,
794             3: criar_tela_vendas,
795             4: criar_tela_teste,
796         }
797         if index in telas:
798             main_content.content = telas[index](main_content.page)
799             main_content.page.update()
800
801     # Sidebar items
802     sidebar_items = [
803         ("Dashboard", ft.Icons.DASHBOARD_OUTLINED, ft.Icons.DASHBOARD),
804         ("Clientes", ft.Icons.PEOPLE_OUTLINED, ft.Icons.PEOPLE),
805         ("Produtos", ft.Icons.INVENTORY_2_OUTLINED, ft.Icons.INVENTORY_2),
806         ("Vendas", ft.Icons.POINT_OF_SALE_OUTLINED, ft.Icons.POINT_OF_SALE),
807         ("Teste 100", ft.Icons.SCIENCE_OUTLINED, ft.Icons.SCIENCE),
808     ]
809
810     sidebar_controls = []
811     for i, (label, icon_out, icon_fill) in enumerate(sidebar_items):
812         idx = i
813         is_selected = i == 0
814
815         sidebar_controls.append(DS.sidebar_item(
816             label=label,
817             icon=icon_fill if is_selected else icon_out,
818             selected=is_selected,
819             on_click=lambda e, i=idx: navegar(i),
820         ))
821
822     # Sidebar
823     sidebar = ft.Container(
824         content=ft.Column([

```

```

825         # Logo
826         ft.Container(
827             content=ft.Row([
828                 ft.Container(
829                     content=ft.Icon(ft.Icons.STORE, size=22, color=Cores.FUNDO_CARD),
830                     bgcolor=Cores.P,
831                     border_radius=Bordas.SM,
832                     padding=8,
833                 ),
834                 ft.Container(width=8),
835                 ft.Column([
836                     ft.Text("SaltoGestao", size=14, weight=ft.FontWeight.W 700, color=Cores.FUNDO_CARD, font_family="Segoe UI"),
837                     ft.Text("v2.0", size=9, color=Cores.P20, font_family="Segoe UI"),
838                 ], spacing=0),
839             ], spacing=4, alignment=ft.MainAxisAlignment.START),
840             padding=ft.padding.symmetric(horizontal=Espacamento.SM, vertical=Espacamento.MD),
841             margin=ft.padding.only(top=Espacamento.SM, bottom=Espacamento.MD),
842         ),
843
844         ft.Container(height=Espacamento.SM),
845
846         # Nav items
847         ft.Column(sidebar_controls, spacing=2),
848
849         ft.Container(expand=True),
850
851         # Footer sidebar
852         ft.Container(
853             content=DS.subtitulo("Corumba Sistemas (c) 2026", size=9),
854             padding=ft.padding.all(Espacamento.SM),
855         ),
856         ], spacing=0),
857         width=220,
858         bgcolor=Cores.P,
859         padding=ft.padding.symmetric(vertical=Espacamento.SM),
860     )
861
862     return ft.Container(
863         content=ft.Row([
864             sidebar,
865             ft.VerticalDivider(width=1, color=Cores.BORDA),
866             main_content,
867         ], expand=True, spacing=0),
868         expand=True,
869     )
870
871
872 # =====
873 # APP PRINCIPAL
874 # =====
875
876 def main(page: ft.Page):
877     page.title = "SaltoGestao v2.0"
878     page.padding = 0
879     page.window.width = 1280
880     page.window.height = 800
881     page.window.min_width = 1024
882     page.window.min_height = 700
883     page.bgcolor = Cores.FUNDO
884     page.theme = ft.Theme(
885         color_scheme_seed=Cores.P,
886         font_family="Segoe UI",
887         use_material3=True,
888     )
889
890     main_content = ft.Container(expand=True, bgcolor=Cores.FUNDO)
891
892     def on_login_success():
893         main_content.content = criar_layout_principal()
894         page.update()
895
896     main_content.content = criar_tela_login(page, on_login_success)
897     page.add(main_content)
898
899
900 if __name__ == "__main__":
901     ft.app(target=main)
902

```

```
1 flet==0.24.1
2 requests>=2.31.0
3
```

```
1 #!/bin/bash
2 # =====
3 # SaltoGestao Desktop - Script de Execucao
4 # Usa Flet (baseado em Flutter) - NAO precisa de tkinter!
5 # =====
6
7 echo "=====
8 echo " SaltoGestao Desktop"
9 echo " Corumba Sistemas"
10 echo "=====
11 echo ""
12
13 # Verifica se o Python existe
14 if ! command -v python3 &> /dev/null; then
15     echo "ERRO: python3 nao encontrado!"
16     echo "Instale o Python 3.10+: https://www.python.org/downloads/"
17     exit 1
18 fi
19
20 echo "Python: $(python3 --version)"
21 echo ""
22
23 # Instala dependencias se necessario
24 echo "Verificando dependencias..."
25 pip install --user flet requests 2>/dev/null
26
27 echo ""
28 echo "Iniciando SaltoGestao Desktop..."
29 echo ""
30
31 # Executa o sistema
32 cd "$(dirname "$0")"
33 python3 main_flet.py
34
```

```

1 """
2 =====
3 Teste de 100 Clientes - SaltoGestão Desktop
4 =====
5 Insere 100 clientes via API e valida cada operação.
6 Uso: python3 testar_100_clientes.py
7 """
8 import requests
9 import json
10 import time
11 import sys
12
13 # =====
14 # CONFIGURAÇÃO
15 # =====
16 # Tenta local primeiro, depois VPS
17 URLS = [
18     "http://localhost:8080",
19     "http://191.252.210.235:8080",
20     "http://192.168.100.68:8080",
21 ]
22
23 API_BASE = None
24 for url in URLS:
25     try:
26         r = requests.get(f"{url}/health", timeout=3)
27         if r.status_code == 200:
28             API_BASE = url
29             print(f"✓ API conectada em: {url}")
30             break
31     except Exception:
32         continue
33
34 if not API_BASE:
35     print("× Nenhuma API encontrada. Inicie o backend primeiro.")
36     sys.exit(1)
37
38 # =====
39 # LOGIN PARA OBTER TOKEN
40 # =====
41 def fazer_login():
42     """Faz login e retorna o token JWT."""
43     credenciais = [
44         {"email": "admin@corumba.com", "senha": "admin123"},
45         {"email": "admin@salgestao.com", "senha": "admin123"},
46         {"email": "admin@saltogestao.com", "senha": "admin123"},
47     ]
48     for cred in credenciais:
49         try:
50             r = requests.post(f"{API_BASE}/api/auth/login", json=cred, timeout=5)
51             if r.status_code == 200:
52                 data = r.json()
53                 token = data.get("access_token") or data.get("token")
54                 if token:
55                     print(f"✓ Login OK com: {cred['email']}")
56                     return token
57             except Exception:
58                 continue
59     # Tenta registrar novo admin
60     try:
61         dados = {
62             "nome": "Admin Teste",
63             "email": "admin@teste.com",
64             "senha": "testel23",
65             "tipo": "admin",
66             "cnpj_empresa": "00.000.000/0001-00"
67         }
68         r = requests.post(f"{API_BASE}/api/auth/register", json=dados, timeout=5)
69         if r.status_code in [200, 201]:
70             token = r.json().get("access_token") or r.json().get("token")
71             if token:
72                 print("✓ Admin registrado e logado")
73                 return token
74     except Exception:
75         pass
76     print("× Falha no login. Verifique as credenciais.")
77     return None
78
79 # =====
80 # GERA DADOS DE 100 CLIENTES
81 # =====
82 def gerar_clientes_100():
83     """Gera 100 clientes com dados variados e realistas."""
84     nomes_base = [
85         "João", "Maria", "José", "Ana", "Pedro", "Carla", "Lucas", "Fernanda",
86         "Marcos", "Juliana", "Rafael", "Patricia", "Bruno", "Camila", "Diego",
87         "Amanda", "Gustavo", "Larissa", "Thiago", "Beatriz", "Felipe", "Mariana",
88         "Rodrigo", "Letícia", "André", "Vanessa", "Marcelo", "Tatiana", "Leonardo",
89         "Priscila", "Eduardo", "Sabrina", "Ricardo", "Isabela", "Fábio", "Natália",
90         "Alexandre", "Daniela", "Guilherme", "Bianca", "Vinicius", "Lorena",
91         "Matheus", "Gabriela", "Samuel", "Helena", "Isaac", "Valentina", "Daniel",
92         "Laura", "Henrique", "Manuela", "Caio", "Sophia", "Enzo", "Heloisa",
93         "Miguel", "Alice", "Arthur", "Júlia", "Bernardo", "Luísa", "Heitor",
94         "Isabella", "Davi", "Lara", "Lorenzo", "Cecília", "Theo", "Eloá",
95         "Gabriel", "Yasmin", "Noah", "Rafaela", "Benjamin", "Giovanna", "Samuel",
96         "Clara", "Anthony", "Livia", "Joaquim", "Melissa", "Nicolas", "Rebeca",
97         "Lucca", "Agatha", "Ryan", "Esther", "Ian", "Nicole", "David", "Sarah",
98         "Pedro Henrique", "Vitória", "Bruno Henrique", "Luna", "Cauã", "Marina",
99         "Kaique", "Valentim", "Yuri", "Antonella"
100     ]
101
102     sobrenomes = [
103         "Silva", "Santos", "Oliveira", "Souza", "Rodrigues", "Ferreira", "Alves",
104         "Pereira", "Lima", "Gomes", "Costa", "Ribeiro", "Martins", "Carvalho",
105         "Araújo", "Melo", "Barbosa", "Rocha", "Dias", "Nascimento", "Andrade",
106         "Moreira", "Nunes", "Marques", "Machado", "Mendes", "Freitas", "Cardoso",
107         "Ramos", "Vieira", "Monteiro", "Cunha", "Guimarães", "Figueiredo",
108         "Pinto", "Correia", "Teixeira", "Caldeira", "Borges", "Miranda"
109     ]
110
111     cidades = [
112         ("Aparecida de Goiânia", "GO"), ("Goiânia", "GO"), ("Anápolis", "GO"),
113         ("Rio Verde", "GO"), ("Luziânia", "GO"), ("Águas Lindas", "GO"),
114         ("Valparaíso", "GO"), ("Trindade", "GO"), ("Novo Gama", "GO"),
115         ("Senador Canedo", "GO"), ("Itumbiara", "GO"), ("Jataí", "GO"),
116         ("Formosa", "GO"), ("Catalão", "GO"), ("Caldas Novas", "GO")
117     ]
118
119     bairros = [
120         "Centro", "Jardim América", "Setor Oeste", "Setor Bueno",
121         "Jardim Goiás", "Vila Brasília", "Setor Central", "Campinas",
122         "Vila Aurora", "Jardim Europa", "Setor Marista", "PQ Oeste",
123         "Porto Grande", "Jardim Novo Mundo", "Vila Aurora"
124     ]
125
126     ruas = [
127         "Rua das Flores", "Av. Brasil", "Rua São João", "Av. Goiás",
128         "Rua 15 de Novembro", "Rua da Independência", "Av. Anhanguera",
129         "Rua São Paulo", "Rua Minas Gerais", "Av. Tóquio",
130         "Rua Vitória", "Av. Independência", "Rua Boa Vista",
131         "Av. Presidente Vargas", "Rua das Palmeiras"
132     ]
133
134

```

```

135 clientes = []
136 usados = set()
137
138 for i in range(100):
139     import random
140     random.seed(1 + 42) # Reprodutível
141
142     nome = nomes_base[i % len(nomes_base)]
143     sobrenome1 = sobrenomes[i % len(sobrenomes)]
144     sobrenome2 = sobrenomes[(i * 3 + 7) % len(sobrenomes)]
145     nome_completo = f"{nome} {sobrenome1} {sobrenome2}"
146
147     cidade, estado = cidades[i % len(cidades)]
148     bairro = bairros[i % len(bairros)]
149     rua = ruas[i % len(ruas)]
150     numero = random.randint(1, 999)
151     cep = f"{random.randint(10000, 99999)}-{random.randint(100, 999)}"
152
153     # Gera documento único
154     doc_num = f"{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(0, 99):02d}"
155     doc_key = doc_num[:9]
156     if doc_key in usados:
157         doc_num = f"{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(100, 999)}{random.randint(0, 99):02d}"
158     usados.add(doc_num[:9])
159
160     email = f"{nome.lower()}_{i+1}@email.com"
161
162     # Telefone variado
163     ddd = random.choice(["62", "11", "21", "31", "41", "51", "61", "71", "81", "91"])
164     telefone = f"{ddd}9{random.randint(8000, 9999)}-{random.randint(1000, 9999)}"
165
166     cliente = {
167         "nome": nome_completo.strip(),
168         "documento": doc_num,
169         "email": email,
170         "telefone": telefone,
171         "endereco": {
172             "rua": f"{rua}, {numero}",
173             "bairro": bairro,
174             "cidade": cidade,
175             "estado": estado,
176             "cep": cep,
177             "complemento": f"Apto {random.randint(1, 20)}" if random.random() > 0.5 else ""
178         },
179         "observacoes": f"Cliente teste #{i+1} - gerado automaticamente"
180     }
181     clientes.append(cliente)
182
183 return clientes
184
185
186 # =====
187 # TESTE PRINCIPAL
188 # =====
189 def main():
190     print("=" * 50)
191     print("🔍 TESTE: 100 CLIENTES - SALTOSTÃO DESKTOP")
192     print("=" * 50)
193
194     # 1. Login
195     print("\n➤ ETAPA 1: Login")
196     token = fazer_login()
197     if not token:
198         return
199
200     headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
201
202     # 2. Limpa clientes de teste anteriores
203     print("\n➤ ETAPA 2: Limpando clientes de teste anteriores")
204     try:
205         r = requests.get(f"{API_BASE}/api/admin/clientes", headers=headers, timeout=10)
206         if r.status_code == 200:
207             existentes = r.json()
208             for c in existentes:
209                 if "teste" in c.get("email", "") or "Cliente teste" in str(c.get("observacoes", "")):
210                     cid = c.get("id")
211                     requests.delete(f"{API_BASE}/api/admin/clientes/{cid}", headers=headers, timeout=5)
212             print(f"🗑️ Clientes existentes verificados: {len(existentes)}")
213     except Exception as e:
214         print(f"⚠️ Aviso: {e}")
215
216     # 3. Gera 100 clientes
217     print("\n➤ ETAPA 3: Gerando 100 clientes")
218     clientes = gerar_clientes_100()
219     print(f"✅ {len(clientes)} clientes gerados")
220
221     # 4. Insere todos
222     print("\n➤ ETAPA 4: Inserindo clientes via API")
223     inseridos = 0
224     falhas = 0
225     erros_detalhados = []
226     ids_inseridos = []
227
228     inicio = time.time()
229     for i, cliente in enumerate(clientes):
230         try:
231             r = requests.post(
232                 f"{API_BASE}/api/admin/clientes",
233                 json=cliente,
234                 headers=headers,
235                 timeout=10
236             )
237             if r.status_code in [200, 201]:
238                 inseridos += 1
239                 resp_data = r.json()
240                 cid = resp_data.get("id") or resp_data.get("cliente", {}).get("id")
241                 if cid:
242                     ids_inseridos.append(cid)
243             else:
244                 falhas += 1
245                 erros_detalhados.append(f"#{i+1} {cliente['nome']}: HTTP {r.status_code} - {r.text[:100]}")
246         except Exception as e:
247             falhas += 1
248             erros_detalhados.append(f"#{i+1} {cliente['nome']}: {str(e)[:100]}")
249
250     # Progress bar
251     if (i + 1) % 10 == 0:
252         pct = (i + 1) % 10
253         print(f"📊 {i+1}/100 inseridos... ({inseridos} OK, {falhas} falhas)")
254
255     tempo = time.time() - inicio
256     print(f"\n🏁 Inseridos: {inseridos}/100")
257     if falhas > 0:
258         print(f"❌ Falhas: {falhas}")
259         for err in erros_detalhados[:10]:
260             print(f"📄 {err}")
261     print(f"🕒 Tempo: {tempo:.1f}s ({tempo/max(inseridos,1)*1000:.0f}ms/cliente)")
262
263     # 5. Validação: Busca todos
264     print("\n➤ ETAPA 5: Validação - Buscando todos os clientes")
265     try:
266         r = requests.get(f"{API_BASE}/api/admin/clientes", headers=headers, timeout=10)
267         if r.status_code == 200:
268             todos = r.json()
269             total = len(todos) if isinstance(todos, list) else todos.get("total", 0)
270             print(f"✅ Total na API: {total} clientes")
271
272     # Verifica dados de 5 clientes aleatórios

```

```

273         if isinstance(todos, list) and len(todos) > 0:
274             print("\n    Q Verificação de 5 clientes aleatórios:")
275             import random
276             amostra = random.sample(todos, min(5, len(todos)))
277             for c in amostra:
278                 nome = c.get("nome", "?")[:30]
279                 email = c.get("email", "?")[:25]
280                 doc = c.get("documento", "?")[:14]
281                 cli_id = c.get("id", "?")
282                 print(f"        ID:{cli_id} | {nome} | {email} | {doc}")
283     else:
284         print(f"    x Erro na busca: HTTP {r.status_code}")
285 except Exception as e:
286     print(f"    x Erro: {e}")
287
288 # 6. Teste de busca individual
289 if ids_inseridos:
290     print("\n* ETAPA 6: Teste de busca por ID")
291     test_id = ids_inseridos[0]
292     try:
293         r = requests.get(f"{API_BASE}/api/admin/clientes/{test_id}", headers=headers, timeout=5)
294         if r.status_code == 200:
295             c = r.json()
296             print(f"    ✓ Busca por ID {test_id}: {c.get('nome', '?')[:40]}")
297         else:
298             print(f"    x Falha: HTTP {r.status_code}")
299     except Exception as e:
300         print(f"    x Erro: {e}")
301
302 # 7. Teste de atualização
303 if ids_inseridos:
304     print("\n* ETAPA 7: Teste de atualização")
305     test_id = ids_inseridos[1] if len(ids_inseridos) > 1 else ids_inseridos[0]
306     try:
307         r = requests.put(
308             f"{API_BASE}/api/admin/clientes/{test_id}",
309             json={"telefone": "6299999-8888", "observacoes": "Atualizado via teste"},
310             headers=headers,
311             timeout=5
312         )
313         if r.status_code in [200, 201]:
314             print(f"    ✓ Atualização ID {test_id}: OK")
315         else:
316             print(f"    x Falha: HTTP {r.status_code}")
317     except Exception as e:
318         print(f"    x Erro: {e}")
319
320 # 8. RESULTADO FINAL
321 print("\n" + "=" * 50)
322 print("❏ RESULTADO FINAL")
323 print("=" * 50)
324 print(f"    API:                {API_BASE}")
325 print(f"    Inseridos:          {inseridos}/100")
326 print(f"    Falhas:             {falhas}")
327 print(f"    Tempo total:        {tempo:.1f}s")
328 print(f"    Performance:        {tempo/max(inseridos,1)*1000:.0f}ms/cliente")
329
330 if inseridos == 100:
331     print("\n    ✎ PASSOU! Todos os 100 clientes inseridos com sucesso!")
332 else:
333     print(f"\n    ⚠ ATENÇÃO: {falhas} clientes falharam")
334
335 # Teste de paginação (se suportado)
336 print("\n* ETAPA 8: Teste de paginação")
337 for page in [1, 2, 5]:
338     try:
339         r = requests.get(
340             f"{API_BASE}/api/admin/clientes?page={page}&limit=20",
341             headers=headers, timeout=5
342         )
343         if r.status_code == 200:
344             dados = r.json()
345             if isinstance(dados, list):
346                 print(f"    ✓ Página {page}: {len(dados)} registros")
347             else:
348                 print(f"    ✓ Página {page}: {dados}")
349     except Exception as e:
350         print(f"    ⚠ Página {page}: {e}")
351
352 print("\n✓ Teste concluído!")
353
354
355 if __name__ == "__main__":
356     main()
357

```



```
1 # SaltoGestão Desktop - Utilitários
2
```

```

1 """
2 =====
3 SaltoGestão - Sistema Desktop
4 Módulo de conexão com API (FastAPI Backend)
5 =====
6 """
7 import requests
8 import json
9 from datetime import datetime
10 from typing import Optional
11
12
13 class APIClient:
14     """Cliente para comunicação com o backend FastAPI."""
15
16     def __init__(self, base_url: str = "http://localhost:8080"):
17         self.base_url = base_url.rstrip("/")
18         self.token: Optional[str] = None
19         self.usuario = None
20         self.session = requests.Session()
21         self.timeout = 10
22
23     # =====
24     # AUTENTICAÇÃO
25     # =====
26
27     def login_admin(self, email: str, senha: str) -> dict:
28         """Login como administrador (Corumbá Sistemas)."""
29         resp = self._post("/api/admin/auth/login", {
30             "email": email,
31             "senha": senha
32         })
33         if resp.get("access_token"):
34             self.token = resp["access_token"]
35             self.usuario = resp.get("usuario", {"tipo": "admin"})
36             self.session.headers.update({"Authorization": f"Bearer {self.token}"})
37             return resp
38
39     def login_cliente(self, email: str, senha: str) -> dict:
40         """Login como cliente (empresa)."""
41         resp = self._post("/api/auth/cliente/login", {
42             "email": email,
43             "senha": senha
44         })
45         if resp.get("access_token"):
46             self.token = resp["access_token"]
47             self.usuario = resp.get("usuario", {"tipo": "cliente"})
48             self.session.headers.update({"Authorization": f"Bearer {self.token}"})
49             return resp
50
51     def login_vendedor(self, email: str, senha: str) -> dict:
52         """Login como vendedor."""
53         resp = self._post("/api/vendedor/auth/login", {
54             "email": email,
55             "senha": senha
56         })
57         if resp.get("access_token"):
58             self.token = resp["access_token"]
59             self.usuario = resp.get("usuario", {"tipo": "vendedor"})
60             self.session.headers.update({"Authorization": f"Bearer {self.token}"})
61             return resp
62
63     def logout(self):
64         """Limpa sessão."""
65         self.token = None
66         self.usuario = None
67         self.session.headers.pop("Authorization", None)
68
69     @property
70     def autenticado(self) -> bool:
71         return self.token is not None
72
73     @property
74     def tipo_usuario(self) -> str:
75         if self.usuario:
76             return self.usuario.get("tipo", "desconhecido")
77         return "desconhecido"
78
79     # =====
80     # ADMIN - DASHBOARD
81     # =====
82
83     def admin_dashboard(self) -> dict:
84         """Estatísticas do painel admin."""
85         return self._get("/api/admin/dashboard")
86
87     # =====
88     # ADMIN - PRODUTOS (Catálogo)
89     # =====
90
91     def admin_listar_produtos(self) -> list:
92         """Lista todos os produtos do catálogo."""
93         return self._get("/api/admin/produtos") or []
94
95     # =====
96     # ADMIN - CLIENTES
97     # =====
98
99     def admin_listar_clientes(self, pagina: int = 1, por_pagina: int = 50) -> dict:
100         """Lista clientes com paginação."""
101         return self._get(f"/api/admin/clientes?pagina={pagina}&por_pagina={por_pagina}")
102
103     def admin_criar_cliente(self, dados: dict) -> dict:
104         """Cria um novo cliente (empresa)."""
105         return self._post("/api/admin/clientes/criar", dados)
106
107     def admin_buscar_cliente(self, cliente_id: int) -> dict:
108         """Busca dados de um cliente específico."""
109         return self._get(f"/api/admin/clientes/{cliente_id}")
110
111     def admin_editar_cliente(self, cliente_id: int, dados: dict) -> dict:
112         """Edita dados de um cliente."""
113         return self._put(f"/api/admin/clientes/{cliente_id}", dados)
114
115     def admin_excluir_cliente(self, cliente_id: int) -> bool:
116         """Exclui um cliente."""
117         resp = self._delete(f"/api/admin/clientes/{cliente_id}")
118         return resp is not None
119
120     # =====
121     # ADMIN - ASSINATURAS
122     # =====
123
124     def admin_listar_assinaturas(self) -> list:
125         """Lista todas as assinaturas."""
126         return self._get("/api/admin/assinaturas") or []
127
128     # =====
129     # ADMIN - VENDAS
130     # =====
131
132     def admin_listar_vendas(self, pagina: int = 1) -> dict:
133         """Lista vendas com paginação."""
134         return self._get(f"/api/admin/vendas?pagina={pagina}")

```

```

135
136 # =====
137 # VENDEDOR
138 # =====
139
140 def vendedor_listar_clientes(self) -> list:
141     """Lista clientes do vendedor."""
142     return self._get("/api/vendedor/clientes") or []
143
144 def vendedor_cadastrar_cliente(self, dados: dict) -> dict:
145     """Vendedor cadastra novo cliente."""
146     return self._post("/api/vendedor/clientes/criar", dados)
147
148 def vendedor gerar_chave(self, cliente_id: int, produto_id: int) -> dict:
149     """Gera chave de ativação para cliente."""
150     return self._post("/api/vendedor/gerar-chave", {
151         "cliente_id": cliente_id,
152         "produto_id": produto_id
153     })
154
155 # =====
156 # CLIENTE
157 # =====
158
159 def cliente_dashboard(self) -> dict:
160     """Dashboard do cliente."""
161     return self._get("/api/cliente/dashboard")
162
163 def cliente_meus_dados(self) -> dict:
164     """Dados da empresa cliente."""
165     return self._get("/api/cliente/meus-dados")
166
167 def cliente_produtos_ativos(self) -> list:
168     """Produtos ativos do cliente."""
169     return self._get("/api/cliente/produtos-ativos") or []
170
171 def cliente_gerar_chave(self, produto_id: int) -> dict:
172     """Cliente gera chave de ativação."""
173     return self._post("/api/cliente/gerar-chave", {"produto_id": produto_id})
174
175 # =====
176 # GERAIS
177 # =====
178
179 def health_check(self) -> bool:
180     """Verifica se a API está online."""
181     try:
182         resp = self._get("/health")
183         return resp is not None
184     except Exception:
185         return False
186
187 # =====
188 # MÉTODOS HTTP INTERNOS
189 # =====
190
191 def _get(self, endpoint: str) -> any:
192     try:
193         resp = self.session.get(f"{self.base_url}{endpoint}", timeout=self.timeout)
194         if resp.status_code == 401:
195             self.logout()
196             raise Exception("Sessão expirada. Faça login novamente.")
197         if resp.status_code >= 400:
198             return {"erro": resp.text, "status": resp.status_code}
199         return resp.json() if resp.text else None
200     except requests.ConnectionError:
201         raise Exception(f"Não foi possível conectar ao servidor ({self.base_url})")
202     except requests.Timeout:
203         raise Exception("Tempo de conexão esgotado. Verifique sua internet.")
204     except Exception as e:
205         raise Exception(f"Erro na requisição: {str(e)}")
206
207 def _post(self, endpoint: str, dados: dict) -> dict:
208     try:
209         resp = self.session.post(f"{self.base_url}{endpoint}", json=dados, timeout=self.timeout)
210         if resp.status_code == 401:
211             self.logout()
212             raise Exception("Sessão expirada. Faça login novamente.")
213         if resp.status_code >= 400:
214             return {"erro": resp.text, "status": resp.status_code}
215         return resp.json() if resp.text else {}
216     except requests.ConnectionError:
217         raise Exception(f"Não foi possível conectar ao servidor ({self.base_url})")
218     except requests.Timeout:
219         raise Exception("Tempo de conexão esgotado.")
220     except Exception as e:
221         raise Exception(f"Erro na requisição: {str(e)}")
222
223 def _put(self, endpoint: str, dados: dict) -> dict:
224     try:
225         resp = self.session.put(f"{self.base_url}{endpoint}", json=dados, timeout=self.timeout)
226         if resp.status_code == 401:
227             self.logout()
228             raise Exception("Sessão expirada.")
229         if resp.status_code >= 400:
230             return {"erro": resp.text, "status": resp.status_code}
231         return resp.json() if resp.text else {}
232     except requests.ConnectionError:
233         raise Exception(f"Não foi possível conectar ao servidor.")
234     except Exception as e:
235         raise Exception(f"Erro na requisição: {str(e)}")
236
237 def _delete(self, endpoint: str) -> any:
238     try:
239         resp = self.session.delete(f"{self.base_url}{endpoint}", timeout=self.timeout)
240         if resp.status_code == 401:
241             self.logout()
242             raise Exception("Sessão expirada.")
243         if resp.status_code >= 400:
244             return None
245         return resp.json() if resp.text else {"sucesso": True}
246     except requests.ConnectionError:
247         raise Exception(f"Não foi possível conectar ao servidor.")
248     except Exception as e:
249         raise Exception(f"Erro na requisição: {str(e)}")
250

```

```

1 """
2 =====
3 SaltoGestão - Configurações e Tema
4 =====
5 """
6 import json
7 import os
8
9 # Caminho do arquivo de configuração
10 CONFIG_DIR = os.path.join(os.path.expanduser("~"), ".saltogestao")
11 CONFIG_FILE = os.path.join(CONFIG_DIR, "config.json")
12
13 # =====
14 # CORES DO TEMA (Dark Mode Profissional)
15 # =====
16 CORES = {
17     # Primárias
18     "primaria": "#1E88E5",          # Azul principal
19     "primaria_hover": "#1565C0",    # Azul hover
20     "primaria_clara": "#42A5F5",    # Azul claro
21
22     # Backgrounds
23     "bg_principal": "#0B1A30",      # Azul escuro (fundo principal)
24     "bg_card": "#0F2138",           # Azul mais claro (cards)
25     "bg_input": "#162D4A",          # Fundo dos inputs
26     "bg_header": "#0A1628",         # Fundo do header
27
28     # Textos
29     "texto_primario": "#FFFFFF",     # Texto principal (branco)
30     "texto_secundario": "#90A4AE",   # Texto secundário (cinza azulado)
31     "texto_terciario": "#546E7A",   # Texto terciário (cinza escuro)
32
33     # Status
34     "sucesso": "#43A047",            # Verde sucesso
35     "sucesso_bg": "#1B5E20",         # Fundo verde escuro
36     "erro": "#E53935",               # Vermelho erro
37     "erro_bg": "#B71C1C",            # Fundo vermelho escuro
38     "aviso": "#FB8C00",              # Laranja aviso
39     "aviso_bg": "#E65100",           # Fundo laranja escuro
40     "info": "#039BE5",               # Azul info
41
42     # Bordas
43     "borda": "#1E3A5F",              # Borda padrão
44     "borda_foco": "#1E88E5",         # Borda com foco
45
46     # Botões
47     "btn_primario": "#1E88E5",
48     "btn_primario_hover": "#1565C0",
49     "btn_sucesso": "#43A047",
50     "btn_sucesso_hover": "#2E7D32",
51     "btn_erro": "#E53935",
52     "btn_erro_hover": "#C62828",
53     "btn_secundario": "#37474F",
54     "btn_secundario_hover": "#455A64",
55
56     # Tabela
57     "tabela_header": "#0F2138",
58     "tabela_linha_par": "#0D1E36",
59     "tabela_linha_impar": "#112540",
60     "tabela_linha_hover": "#162D4A",
61     "tabela_selecionada": "#1A3A5C",
62 }
63
64 # =====
65 # FONTE
66 # =====
67 FONTE = "Segoe UI"
68 FONTE_MONO = "Consolas"
69
70 # =====
71 # DIMENSÕES
72 # =====
73 JANELA_LARGURA = 1280
74 JANELA_ALTURA = 780
75 SIDEBAR_LARGURA = 240
76 HEADER_ALTURA = 60
77
78 # =====
79 # CONFIGURAÇÃO DO SERVIDOR
80 # =====
81 DEFAULT_API_URL = "http://localhost:8080"
82
83
84 def carregar_config() -> dict:
85     """Carrega configurações salvas."""
86     if os.path.exists(CONFIG_FILE):
87         try:
88             with open(CONFIG_FILE, "r", encoding="utf-8") as f:
89                 return json.load(f)
90         except Exception:
91             pass
92     return {"api_url": DEFAULT_API_URL, "ultimo_email": ""}
93
94
95 def salvar_config(config: dict):
96     """Salva configurações."""
97     os.makedirs(CONFIG_DIR, exist_ok=True)
98     with open(CONFIG_FILE, "w", encoding="utf-8") as f:
99         json.dump(config, f, ensure_ascii=False, indent=2)
100
101
102 def obter_api_url() -> str:
103     """Retorna a URL da API configurada."""
104     config = carregar_config()
105     return config.get("api_url", DEFAULT_API_URL)
106
107
108 def salvar_api_url(url: str):
109     """Salva a URL da API."""
110     config = carregar_config()
111     config["api_url"] = url
112     salvar_config(config)
113
114
115 def salvar_ultimo_email(email: str):
116     """Salva o último email usado no login."""
117     config = carregar_config()
118     config["ultimo_email"] = email
119     salvar_config(config)
120

```

```
1 # 📁 Documentos
2
3 Esta pasta armazena documentos que não são código-fonte do projeto.
4
5 ## 0 que vai aqui
6
7 - PDFs – manuais, guias, propostas comerciais, contratos
8 - Planilhas – Excel (.xlsx), CSV, Google Sheets exportadas
9 - Apresentações – PowerPoint (.pptx), PDFs de slides
10 - Imagens de referência – wireframes, mockups, prints de tela
11 - Outros documentos – documentos fiscais, certificados, etc.
12
13 ## Organização
14
15 Crie subpastas por tipo ou projeto se necessário:
16
17 ```
18 documentos/
19 └─ guias/
20 └─ propostas/
21 └─ planilhas/
22 └─ ...
23 ```
24
25 > ⚠ Nota: Não coloque código-fonte aqui. Use as pastas `backendPy/`, `frontend-web/`, `desktop-app/` para isso.
26
```

```
1 <!-- BEGIN:nextjs-agent-rules -->
2 # This is NOT the Next.js you know
3
4 This version has breaking changes – APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in `node_modules/next/dist/docs/` before
writing any code. Heed deprecation notices.
5 <!-- END:nextjs-agent-rules -->
6
```

```

1 'use client';
2
3 import { useState } from 'react';
4 import { useRouter } from 'next/navigation';
5 import Link from 'next/link';
6
7 interface Produto {
8   id: number;
9   nome: string;
10  descricao: string;
11  preco_mensal: number;
12  icone: string;
13 }
14
15 export default function AdminCriarClientePage() {
16   const router = useRouter();
17   const [produtos, setProdutos] = useState<Produto[]>([]);
18   const [produtosSelecionados, setProdutosSelecionados] = useState<number[]>([]);
19   const [carregandoProdutos, setCarregandoProdutos] = useState(true);
20
21   const [form, setForm] = useState({
22     nome: '',
23     email: '',
24     telefone: '',
25     cpf_cnpj: '',
26     senha: '',
27   });
28   const [isSaving, setIsSaving] = useState(false);
29   const [erro, setErro] = useState('');
30   const [sucesso, setSucesso] = useState('');
31
32   // Carregar produtos do catálogo
33   useState(() => {
34     fetch('/api/admin/produtos')
35       .then((res) => res.json())
36       .then((data) => setProdutos(data))
37       .catch(() => {})
38       .finally(() => setCarregandoProdutos(false));
39   });
40
41   const toggleProduto = (id: number) => {
42     setProdutosSelecionados((prev) =>
43       prev.includes(id) ? prev.filter((p) => p !== id) : [...prev, id]
44     );
45   };
46
47   const handleSubmit = async (e: React.FormEvent) => {
48     e.preventDefault();
49     setIsSaving(true);
50     setErro('');
51     setSucesso('');
52
53     try {
54       const res = await fetch('/api/admin/clientes/criar', {
55         method: 'POST',
56         headers: { 'Content-Type': 'application/json' },
57         body: JSON.stringify({
58           nome: form.nome,
59           email: form.email,
60           telefone: form.telefone,
61           cpf_cnpj: form.cpf_cnpj,
62           senha: form.senha,
63           produto_ids: produtosSelecionados,
64         }),
65       });
66
67       if (res.ok) {
68         const data = await res.json();
69         setSucesso(`Cliente ${data.nome} cadastrado com sucesso! ID: ${data.cliente_id}`);
70         setForm({ nome: '', email: '', telefone: '', cpf_cnpj: '', senha: '' });
71         setProdutosSelecionados([]);
72       } else {
73         const err = await res.json();
74         setErro(err.detail || 'Erro ao cadastrar cliente.');
```

```

135         value={form.email}
136         onChange={(e) => setForm({ ...form, email: e.target.value })}
137         className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
138         placeholder="cliente@email.com"
139     />
140 </div>
141 <div>
142     <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Telefone</label>
143     <input
144         type="text"
145         value={form.telefone}
146         onChange={(e) => setForm({ ...form, telefone: e.target.value })}
147         className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
148         placeholder="(62) 99999-9999"
149     />
150 </div>
151 <div>
152     <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">CPF ou CNPJ</label>
153     <input
154         type="text"
155         required
156         value={form.cpf_cnpj}
157         onChange={(e) => setForm({ ...form, cpf_cnpj: e.target.value })}
158         className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
159         placeholder="000.000.000-00 ou 00.000.000/0000-00"
160     />
161 </div>
162 <div className="md:col-span-2">
163     <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Senha de Acesso</label>
164     <input
165         type="password"
166         required
167         value={form.senha}
168         onChange={(e) => setForm({ ...form, senha: e.target.value })}
169         className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none"
170         placeholder="Senha para acesso ao painel"
171     />
172 </div>
173 </div>
174 </div>
175
176 {/* Produtos */}
177 <div className="bg-white rounded-xl border border-gray-200 p-6">
178     <h3 className="text-lg font-bold text-[#0b1a30] mb-4">Produtos / Serviços</h3>
179     <p className="text-sm text-gray-500 mb-4">Selecione os produtos que o cliente está adquirindo:</p>
180
181     {carregandoProdutos ? (
182         <p className="text-gray-400">Carregando produtos...</p>
183     ) : (
184         <div className="grid grid-cols-1 md:grid-cols-2 gap-3">
185             {produtos.map((p) => (
186                 <div
187                     key={p.id}
188                     onClick={() => toggleProduto(p.id)}
189                     className={`p-4 rounded-lg border-2 cursor-pointer transition-all ${
190                         produtosSelecionados.includes(p.id)
191                             ? 'border-[#84cc16] bg-green-50'
192                             : 'border-gray-200 hover:border-gray-300'
193                     }`}
194                 >
195                     <div className="flex items-center gap-3">
196                         <span className="text-2xl">{p.icone}</span>
197                         <div className="flex-1">
198                             <p className="font-bold text-[#0b1a30]">{p.nome}</p>
199                             <p className="text-xs text-gray-500">
200                                 R$ {p.preco_mensal.toFixed(2).replace('.', ',')} / mês
201                             </p>
202                         </div>
203                     <div className={`w-6 h-6 rounded-full border-2 flex items-center justify-center ${
204                         produtosSelecionados.includes(p.id)
205                             ? 'border-[#84cc16] bg-[#84cc16]'
206                             : 'border-gray-300'
207                     }`}>
208                         {produtosSelecionados.includes(p.id) && (
209                             <span className="text-white text-xs"></span>
210                         )}
211                     </div>
212                 </div>
213             )}
214         </div>
215     )}
216 </div>
217
218     {produtosSelecionados.length > 0 && (
219         <div className="mt-4 p-3 bg-green-50 rounded-lg">
220             <p className="text-sm text-green-700 font-medium">
221                 {produtosSelecionados.length} produto(s) selecionado(s)
222                 { ' - ' }
223                 Total: R$ { ' ' }
224                 {produtos
225                     .filter((p) => produtosSelecionados.includes(p.id))
226                     .reduce((sum, p) => sum + p.preco_mensal, 0)
227                     .toFixed(2)
228                     .replace('.', ',')}
229                 / mês
230             </p>
231         </div>
232     )}
233 </div>
234
235 {/* Botões */}
236 <div className="flex gap-4">
237     <Link
238         href="/admin/clientes/listar"
239         className="flex-1 py-3 border border-gray-300 text-gray-700 font-bold rounded-md text-center hover:bg-gray-50 transition-colors"
240     >
241         Cancelar
242     </Link>
243     <button
244         type="submit"
245         disabled={isSaving}
246         className="flex-1 py-3 bg-[#84cc16] text-[#0b1a30] font-bold rounded-md hover:bg-[#65a30d] hover:text-white transition-colors disabled:opacity-50"
247     >
248         {isSaving ? 'Cadastrando...' : 'Cadastrar Cliente'}
249     </button>
250 </div>
251 </form>
252 </main>
253 </div>
254 );
255 }
256

```



```

1 'use client';
2
3 import { useEffect, useState } from 'react';
4 import { useRouter } from 'next/navigation';
5 import Link from 'next/link';
6
7 interface Cliente {
8   id: number;
9   nome: string;
10  email: string;
11  telefone: string;
12  cpf_cnpj: string;
13  ativo: boolean;
14  tem_inadimplencia: boolean;
15  total_produtos: number;
16 }
17
18 export default function AdminClientesPage() {
19   const router = useRouter();
20   const [clientes, setClientes] = useState<Cliente[]>([]);
21   const [carregando, setCarregando] = useState(true);
22   const [busca, setBusca] = useState('');
23   const [filtro, setFiltro] = useState<string>('TODO');
24   const [erro, setErro] = useState('');
25
26   const carregarClientes = async () => {
27     try {
28       let url = '/api/admin/clientes';
29       if (filtro === 'INADIMIENTE') url += '?status=INADIMIENTE';
30       if (busca) url += (url.includes('?') ? '&' : '?') + `busca=${encodeURIComponent(busca)}`;
31
32       const res = await fetch(url);
33       if (res.ok) {
34         const data = await res.json();
35         setClientes(data);
36       } else {
37         setErro('Erro ao carregar clientes');
38       }
39     } catch {
40       setErro('Erro de conexão com o servidor');
41     } finally {
42       setCarregando(false);
43     }
44   };
45
46   useEffect(() => {
47     carregarClientes();
48   }, [filtro, busca]);
49
50   return (
51     <div className="min-h-screen bg-[#f8fafc]">
52       {/* Header */}
53       <header className="bg-[#0b1a30] text-white px-8 py-5">
54         <div className="max-w-6xl mx-auto flex justify-between items-center">
55           <div>
56             <h1 className="text-2xl font-bold">Corumbá Sistemas</h1>
57             <p className="text-sm text-slate-400">Área do Vendedor – Gestão de Clientes</p>
58           </div>
59           <Link
60             href="/admin/clientes/criar"
61             className="bg-[#84cc16] text-[#0b1a30] font-bold py-2 px-6 rounded-md hover:bg-[#65a30d] hover:text-white transition-colors"
62             >
63             + Novo Cliente
64           </Link>
65         </div>
66       </header>
67
68       <main className="max-w-6xl mx-auto px-8 py-8">
69         {/* Filtros */}
70         <div className="bg-white rounded-xl border border-gray-200 p-6 mb-6">
71           <div className="flex flex-col md:flex-row gap-4">
72             <div className="flex-1">
73               <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Buscar Cliente</label>
74               <input
75                 type="text"
76                 placeholder="Nome, email ou CPF/CNPJ..."
77                 value={busca}
78                 onChange={(e) => setBusca(e.target.value)}
79                 className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] focus:border-transparent outline-none"
80               />
81             </div>
82             <div className="md:w-48">
83               <label className="block text-xs font-bold text-[#0b1a30] uppercase mb-1.5">Status</label>
84               <select
85                 value={filtro}
86                 onChange={(e) => setFiltro(e.target.value)}
87                 className="w-full border border-gray-300 rounded-md px-4 py-2.5 text-sm focus:ring-2 focus:ring-[#84cc16] outline-none bg-white"
88               >
89                 <option value="TODO">Todos</option>
90                 <option value="ATIVO">Ativos</option>
91                 <option value="INADIMIENTE">Inadimplentes</option>
92               </select>
93             </div>
94           </div>
95         </div>
96
97         {/* Stats rápidos */}
98         <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
99           <div className="bg-white rounded-xl border border-gray-200 p-5">
100             <p className="text-xs font-bold text-gray-400 uppercase">Total Clientes</p>
101             <p className="text-3xl font-extrabold text-[#0b1a30]">{clientes.length}</p>
102           </div>
103           <div className="bg-white rounded-xl border border-gray-200 p-5">
104             <p className="text-xs font-bold text-gray-400 uppercase">Ativos</p>
105             <p className="text-3xl font-extrabold text-green-600">{clientes.filter(c => !c.tem_inadimplencia).length}</p>
106           </div>
107           <div className="bg-white rounded-xl border border-gray-200 p-5">
108             <p className="text-xs font-bold text-gray-400 uppercase">Inadimplentes</p>
109             <p className="text-3xl font-extrabold text-red-600">{clientes.filter(c => c.tem_inadimplencia).length}</p>
110           </div>
111         </div>
112
113         {/* Tabela */}
114         <div className="bg-white rounded-xl border border-gray-200 overflow-hidden">
115           <div className="px-6 py-4 border-b border-gray-200 bg-gray-50">
116             <h3 className="text-sm font-bold text-[#0b1a30] uppercase tracking-wider">Clientes Cadastrados</h3>
117           </div>
118
119           {erro && (
120             <div className="p-4 bg-red-50 border-b border-red-200 text-red-700 text-sm">{erro}</div>
121           )}
122
123           {carregando ? (
124             <div className="p-12 text-center text-gray-400">Carregando clientes...</div>
125           ) : clientes.length === 0 ? (
126             <div className="p-12 text-center text-gray-400">
127               Nenhum cliente encontrado.
128               <Link href="/admin/clientes/criar" className="text-[#84cc16] font-bold hover:underline ml-1">
129                 Cadastre o primeiro cliente.
130               </Link>
131             </div>
132           ) : (
133             <table>
134               <thead>
135                 <tr>
136                   <th>ID</th>
137                   <th>Nome</th>
138                   <th>Email</th>
139                   <th>Telefone</th>
140                   <th>CPF/CNPJ</th>
141                   <th>Status</th>
142                   <th>Produtos</th>
143                 </tr>
144               </thead>
145               <tbody>
146                 <tr>
147                   <td>1</td>
148                   <td>João da Silva</td>
149                   <td>josilva@email.com</td>
150                   <td>(11) 9999-9999</td>
151                   <td>123.456.789-00</td>
152                   <td>ATIVO</td>
153                   <td>3</td>
154                 </tr>
155               </tbody>
156             </table>
157           )}
158         </div>
159       </main>
160     </div>
161   );
162 }

```

```

135     </div>
136   ) : (
137     <div className="overflow-x-auto">
138       <table className="w-full text-left">
139         <thead>
140           <tr className="border-b border-gray-200 text-xs text-gray-500 uppercase bg-gray-50">
141             <th className="p-4 font-bold">Cliente</th>
142             <th className="p-4 font-bold">Contato</th>
143             <th className="p-4 font-bold">CPF/CNPJ</th>
144             <th className="p-4 font-bold">Produtos</th>
145             <th className="p-4 font-bold">Status</th>
146             <th className="p-4 font-bold text-center">Ações</th>
147           </tr>
148         </thead>
149         <tbody className="divide-y divide-gray-100">
150           {clientes.map((c) => (
151             <tr key={c.id} className="hover:bg-gray-50 transition-colors">
152               <td className="p-4">
153                 <p className="text-sm font-bold text-[#0b1a30]">{c.nome}</p>
154                 <p className="text-xs text-gray-500">{c.email}</p>
155               </td>
156               <td className="p-4 text-sm text-gray-600">{c.telefone}</td>
157               <td className="p-4 text-sm text-gray-600 font-mono">{c.cpf_cnpj}</td>
158               <td className="p-4 text-sm text-center">
159                 <span className="bg-blue-50 text-blue-700 font-bold px-2 py-1 rounded">
160                   {c.total_produtos}
161                 </span>
162               </td>
163               <td className="p-4">
164                 {c.tem_inadimplencia ? (
165                   <span className="bg-red-50 text-red-700 text-xs font-bold px-3 py-1 rounded-full border border-red-200">
166                     ⚠ Inadimplente
167                   </span>
168                 ) : (
169                   <span className="bg-green-50 text-green-700 text-xs font-bold px-3 py-1 rounded-full border border-green-200">
170                     ✓ Ativo
171                   </span>
172                 )}
173               </td>
174               <td className="p-4 text-center">
175                 <button
176                   onClick={() => router.push(`/admin/clientes/detalhe?id=${c.id}`)}
177                   className="text-[#0b1a30] hover:text-[#84cc16] font-bold text-sm transition-colors"
178                 >
179                   Ver Detalhes
180                 </button>
181               </td>
182             </tr>
183           ))}
184         </tbody>
185       </table>
186     </div>
187   )}
188 </div>
189 </main>
190 </div>
191 );
192 }
193

```

```

1 'use client';
2
3 import { useEffect, useState } from 'react';
4 import { useRouter } from 'next/navigation';
5 import Link from 'next/link';
6
7 interface DashboardData {
8   clientes: { total: number; ativos: number };
9   assinaturas: { total: number; ativas: number; inadimplentes: number; canceladas: number };
10  chaves_ativacao: { total: number; usadas: number; pendentes: number };
11  receita_mensal_estimada: number;
12 }
13
14 export default function AdminDashboardPage() {
15   const router = useRouter();
16   const [dash, setDash] = useState<DashboardData | null>(null);
17   const [admin, setAdmin] = useState<{ nome: string; papel: string } | null>(null);
18   const [carregando, setCarregando] = useState(true);
19
20   useEffect(() => {
21     const token = localStorage.getItem('admin_token');
22     if (!token) {
23       router.push('/admin/login');
24       return;
25     }
26
27     const adminData = localStorage.getItem('admin_data');
28     if (adminData) {
29       setAdmin(JSON.parse(adminData));
30     }
31
32     fetch('/api/admin/dashboard', {
33       headers: { Authorization: `Bearer ${token}` },
34     })
35       .then((res) => (res.ok ? res.json() : null))
36       .then((data) => setDash(data))
37       .catch(() => {})
38       .finally(() => setCarregando(false));
39   }, [router]);
40
41   const handleLogout = () => {
42     localStorage.removeItem('admin_token');
43     localStorage.removeItem('admin_data');
44     router.push('/admin/login');
45   };
46
47   if (carregando) {
48     return (
49       <div className="min-h-screen flex items-center justify-center bg-[#f8fafc]">
50         <p className="text-gray-500">Carregando...</p>
51       </div>
52     );
53   }
54
55   return (
56     <div className="min-h-screen bg-[#f8fafc]">
57       </* Header */>
58       <header className="bg-[#0b1a30] text-white px-8 py-5">
59         <div className="max-w-6xl mx-auto flex justify-between items-center">
60           <div>
61             <h1 className="text-2xl font-bold">Corumbá Sistemas</h1>
62             <p className="text-sm text-slate-400">Painel Administrativo – {admin?.nome}</p>
63           </div>
64           <div className="flex items-center gap-4">
65             <span className="bg-[#84cc16] text-[#0b1a30] text-xs font-bold px-3 py-1 rounded-full uppercase">
66               {admin?.papel}
67             </span>
68             <button
69               onClick={handleLogout}
70               className="text-sm text-slate-400 hover:text-white transition-colors"
71             >
72               Sair
73             </button>
74           </div>
75         </div>
76       </header>
77
78       <main className="max-w-6xl mx-auto px-8 py-8">
79         </* Stats */>
80         <div className="grid grid-cols-1 md:grid-cols-4 gap-4 mb-8">
81           <div className="bg-white rounded-xl border border-gray-200 p-6">
82             <p className="text-xs font-bold text-gray-400 uppercase">Clientes</p>
83             <p className="text-3xl font-extrabold text-[#0b1a30]">{dash?.clientes.total || 0}</p>
84             <p className="text-xs text-green-600 font-semibold mt-1">{dash?.clientes.ativos || 0} ativos</p>
85           </div>
86           <div className="bg-white rounded-xl border border-gray-200 p-6">
87             <p className="text-xs font-bold text-gray-400 uppercase">Assinaturas Ativas</p>
88             <p className="text-3xl font-extrabold text-green-600">{dash?.assinaturas.ativas || 0}</p>
89             <p className="text-xs text-gray-500 font-medium mt-1">de {dash?.assinaturas.total || 0} total</p>
90           </div>
91           <div className="bg-white rounded-xl border border-gray-200 p-6">
92             <p className="text-xs font-bold text-gray-400 uppercase">Inadimplentes</p>
93             <p className="text-3xl font-extrabold text-red-600">{dash?.assinaturas.inadimplentes || 0}</p>
94             <p className="text-xs text-red-500 font-medium mt-1">requerem atenção</p>
95           </div>
96           <div className="bg-white rounded-xl border border-gray-200 p-6">
97             <p className="text-xs font-bold text-gray-400 uppercase">Receita Mensal</p>
98             <p className="text-3xl font-extrabold text-[#0b1a30]">
99               R$ {(dash?.receita_mensal_estimada || 0).toFixed(2).replace('.', ',')}
100             </p>
101             <p className="text-xs text-blue-600 font-semibold mt-1">estimada</p>
102           </div>
103         </div>
104
105         </* Ações Rápidas */>
106         <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mb-8">
107           <Link
108             href="/admin/clientes/listar"
109             className="bg-white p-6 rounded-xl border border-gray-200 hover:border-[#84cc16] hover:shadow-md transition-all group"
110           >
111             <div className="text-3xl mb-3">+</div>
112             <h3 className="text-lg font-bold text-[#0b1a30] group-hover:text-[#65a30d]">Clientes</h3>
113             <p className="text-sm text-gray-500 mt-1">Gerenciar clientes cadastrados</p>
114           </Link>
115
116           <Link
117             href="/admin/clientes/criar"
118             className="bg-white p-6 rounded-xl border border-gray-200 hover:border-[#84cc16] hover:shadow-md transition-all group"
119           >
120             <div className="text-3xl mb-3">+</div>
121             <h3 className="text-lg font-bold text-[#0b1a30] group-hover:text-[#65a30d]">Novo Cliente</h3>
122             <p className="text-sm text-gray-500 mt-1">Cadastrar novo cliente e vincular produtos</p>
123           </Link>
124
125           <div className="bg-white p-6 rounded-xl border border-gray-200">
126             <div className="text-3xl mb-3">+</div>
127             <h3 className="text-lg font-bold text-[#0b1a30]">Chaves de Ativação</h3>
128             <p className="text-sm text-gray-500 mt-1">
129               {dash?.chaves_ativacao.pendentes || 0} chaves pendentes
130             </p>
131           </div>
132         </div>
133
134         </* Últimos clientes */>

```

```
135     <div className="bg-white rounded-xl border border-gray-200 overflow-hidden">
136       <div className="px-6 py-4 border-b border-gray-200 flex justify-between items-center">
137         <h3 className="text-sm font-bold text-[#0b1a30] uppercase tracking-wider">Últimos Clientes</h3>
138         <Link href="/admin/clientes/listar" className="text-sm text-[#65a30d] font-bold hover:underline">
139           Ver todos →
140         </Link>
141       </div>
142       <div className="p-6 text-center text-gray-400 text-sm">
143         <Link href="/admin/clientes/criar" className="text-[#84cc16] font-bold hover:underline">
144           Cadastre o primeiro cliente →
145         </Link>
146       </div>
147     </div>
148   </main>
149 </div>
150 );
151 }
152
```

```

1 'use client';
2
3 import { useState } from 'react';
4 import { useRouter } from 'next/navigation';
5
6 export default function AdminLoginPage() {
7   const router = useRouter();
8   const [email, setEmail] = useState('');
9   const [senha, setSenha] = useState('');
10  const [isLoading, setIsLoading] = useState(false);
11  const [error, setError] = useState('');
12
13  const handleLogin = async (e: React.FormEvent<HTMLFormElement>) => {
14    e.preventDefault();
15    setIsLoading(true);
16    setError('');
17
18    try {
19      const response = await fetch('/api/admin/auth/login', {
20        method: 'POST',
21        headers: { 'Content-Type': 'application/json' },
22        body: JSON.stringify({ email, senha }),
23      });
24
25      if (response.ok) {
26        const data = await response.json();
27        localStorage.setItem('admin_token', data.access_token);
28        localStorage.setItem('admin_data', JSON.stringify(data.admin));
29        window.location.href = '/admin/dashboard';
30      } else {
31        const errorData = await response.json();
32        setError(errorData.detail || 'Erro no login.');

```

```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function POST(request: Request) {
6   try {
7     const { email, senha } = await request.json();
8
9     const res = await fetch(`${API_URL}/api/admin/auth/login`, {
10       method: 'POST',
11       headers: { 'Content-Type': 'application/json' },
12       body: JSON.stringify({ email, senha }),
13     });
14
15     const data = await res.json();
16
17     if (res.ok) {
18       return NextResponse.json(data);
19     }
20     return NextResponse.json(data, { status: res.status });
21   } catch (error) {
22     console.error('Erro no login admin:', error);
23     return NextResponse.json({ error: 'Erro de conexão' }, { status: 500 });
24   }
25 }
26
```

```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function POST(request: Request) {
6   try {
7     const body = await request.json();
8
9     const res = await fetch(`${API_URL}/api/admin/clientes/criar`, {
10       method: 'POST',
11       headers: { 'Content-Type': 'application/json' },
12       body: JSON.stringify(body),
13     });
14
15     const data = await res.json();
16
17     if (res.ok) {
18       return NextResponse.json(data);
19     }
20     return NextResponse.json(data, { status: res.status });
21   } catch (error) {
22     console.error('Erro ao criar cliente:', error);
23     return NextResponse.json({ error: 'Erro de conexão com o servidor.' }, { status: 500 });
24   }
25 }
26
```

```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function GET(request: Request) {
6   try {
7     const { searchParams } = new URL(request.url);
8     const status = searchParams.get('status');
9     const busca = searchParams.get('busca');
10
11     let url = `${API_URL}/api/admin/clientes`;
12     const params = new URLSearchParams();
13     if (status) params.set('status', status);
14     if (busca) params.set('busca', busca);
15     if (params.toString()) url += `?${params.toString()}`;
16
17     const res = await fetch(url);
18     if (res.ok) {
19       const data = await res.json();
20       return NextResponse.json(data);
21     }
22     return NextResponse.json([], { status: res.status });
23   } catch (error) {
24     console.error('Erro ao buscar clientes:', error);
25     return NextResponse.json([], { status: 500 });
26   }
27 }
28
29 export async function POST(request: Request) {
30   try {
31     const body = await request.json();
32
33     const res = await fetch(`${API_URL}/api/admin/clientes/criar`, {
34       method: 'POST',
35       headers: { 'Content-Type': 'application/json' },
36       body: JSON.stringify(body),
37     });
38
39     const data = await res.json();
40
41     if (res.ok) {
42       return NextResponse.json(data);
43     }
44     return NextResponse.json(data, { status: res.status });
45   } catch (error) {
46     console.error('Erro ao criar cliente:', error);
47     return NextResponse.json({ error: 'Erro de conexão' }, { status: 500 });
48   }
49 }
50
```



```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function GET(request: Request) {
6   try {
7     const authHeader = request.headers.get('authorization') || '';
8     const token = authHeader.replace('Bearer ', '');
9
10    if (!token) {
11      return NextResponse.json({ error: 'Não autorizado' }, { status: 401 });
12    }
13
14    const res = await fetch(`${API_URL}/api/admin/dashboard`, {
15      headers: { Authorization: `Bearer ${token}` },
16    });
17
18    if (res.ok) {
19      const data = await res.json();
20      return NextResponse.json(data);
21    }
22    return NextResponse.json({ error: 'Erro ao buscar dashboard' }, { status: res.status });
23  } catch (error) {
24    console.error('Erro na API dashboard admin:', error);
25    return NextResponse.json({ error: 'Erro interno' }, { status: 500 });
26  }
27 }
28
```

```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function GET() {
6   try {
7     const res = await fetch(`${API_URL}/api/admin/produtos`);
8     if (res.ok) {
9       const data = await res.json();
10      return NextResponse.json(data);
11    }
12    return NextResponse.json([], { status: res.status });
13  } catch (error) {
14    console.error('Erro ao buscar produtos:', error);
15    return NextResponse.json([], { status: 500 });
16  }
17 }
18
```

```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function POST(request: Request) {
6   try {
7     const { email, password } = await request.json();
8
9     const res = await fetch(`${API_URL}/api/auth/cliente/login`, {
10       method: 'POST',
11       headers: { 'Content-Type': 'application/json' },
12       body: JSON.stringify({ email, senha: password }),
13     });
14
15     if (res.ok) {
16       const data = await res.json();
17       return NextResponse.json(data);
18     }
19
20     const err = await res.json().catch(() => ({ detail: 'Erro no login' }));
21     return NextResponse.json(
22       { error: err.detail || 'Credenciais inválidas' },
23       { status: res.status }
24     );
25   } catch (error) {
26     console.error('Erro no Login:', error);
27     return NextResponse.json(
28       { error: 'Erro ao conectar com o servidor. Verifique se o backend está rodando.' },
29       { status: 500 }
30     );
31   }
32 }
33
```

```
1 import { NextResponse } from 'next/server';
2
3 const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8080';
4
5 export async function GET(request: Request) {
6   try {
7     const authHeader = request.headers.get('authorization') || '';
8     const token = authHeader.replace('Bearer ', '');
9
10    if (!token) {
11      return NextResponse.json({ error: 'Não autorizado' }, { status: 401 });
12    }
13
14    const res = await fetch(`${API_URL}/api/painel/produtos`, {
15      headers: { Authorization: `Bearer ${token}` },
16    });
17
18    if (res.ok) {
19      const data = await res.json();
20      return NextResponse.json(data);
21    }
22
23    return NextResponse.json(
24      { error: 'Erro ao buscar produtos' },
25      { status: res.status }
26    );
27  } catch (error) {
28    console.error('Erro na API do painel:', error);
29    return NextResponse.json(
30      { error: 'Erro interno' },
31      { status: 500 }
32    );
33  }
34 }
35
```

```
1 import { NextResponse } from 'next/server';
2 import { Pool } from 'pg';
3 import bcrypt from 'bcryptjs';
4
5 const pool = new Pool({
6   connectionString: "postgresql://neondb_owner:npg_uvBP7hjZdC9M@ep-weathered-thunder-ap2gc8n9-pooler.c-7.us-east-1.aws.neon.tech/neondb?sslmode=require"
7 });
8
9 export async function GET() {
10   try {
11     const email = 'admin@corumbasistemas.com.br';
12     const senhaPura = 'admin123';
13
14     // 1. Gera o hash seguro da senha usando o bcryptjs
15     const salt = await bcrypt.genSalt(10);
16     const senhaHash = await bcrypt.hash(senhaPura, salt);
17
18     // 2. Verifica se o admin já existe para não duplicar
19     const checkUser = await pool.query('SELECT id FROM usuarios WHERE username = $1', [email]);
20
21     if (checkUser.rows.length > 0) {
22       return NextResponse.json({ message: 'O usuário administrador já existe no banco de dados.' });
23     }
24
25     // 3. Insere o primeiro administrador master do sistema
26     await pool.query(
27       `INSERT INTO usuarios (nome, username, senha_hash, papel, ativo)
28       VALUES ($1, $2, $3, $4, $5)`,
29       ['Admin Corumbá', email, senhaHash, 'ADMIN', true]
30     );
31
32     return NextResponse.json({
33       success: true,
34       message: 'Primeiro acesso criado com sucesso!',
35       usuario: email,
36       senha: textToStars(senhaPura)
37     });
38   } catch (error) {
39     console.error(error);
40     return NextResponse.json({ error: 'Erro ao criar conta master: ' + error }, { status: 500 });
41   }
42 }
43
44 function textToStars(text: string) {
45   return ' '.repeat(text.length);
46 }
47 }
```

```

1 import { NextResponse } from 'next/server';
2 import { Pool } from 'pg';
3 import bcrypt from 'bcryptjs';
4
5 const pool = new Pool({
6   connectionString: "postgresql://neondb_owner:npg_n0pgcEzU2JM7@ep-cold-field-aqxfpb3m-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require"
7 });
8
9 export async function GET(request: Request) {
10   try {
11     const { searchParams } = new URL(request.url);
12     const documento = searchParams.get('documento');
13
14     if (!documento) {
15       return NextResponse.json({ error: 'O CNPJ ou CPF da empresa é obrigatório.' }, { status: 400 });
16     }
17
18     const { rows } = await pool.query(
19       'SELECT id, nome, username as login, papel, ativo FROM usuarios WHERE empresa_documento = $1 ORDER BY id ASC',
20       [documento]
21     );
22
23     return NextResponse.json(rows);
24   } catch (error) {
25     return NextResponse.json({ error: 'Erro ao buscar usuários' }, { status: 500 });
26   }
27 }
28
29 export async function POST(request: Request) {
30   try {
31     const body = await request.json();
32     const { nome, login, senha, papel, empresa_documento } = body;
33
34     // 1. TRAVA DE SEGURANÇA: Verifica se o frontend enviou tudo
35     if (!nome || !login || !senha || !empresa_documento) {
36       return NextResponse.json({
37         error: 'Dados incompletos! Recebido -> Nome: ${nome}, Login: ${login}, Doc: ${empresa_documento}'
38       }, { status: 400 });
39     }
40
41     // 2. Verifica se o login já existe na loja
42     const check = await pool.query('SELECT id FROM usuarios WHERE username = $1 AND empresa_documento = $2', [login, empresa_documento]);
43     if (check.rows.length > 0) {
44       return NextResponse.json({ error: 'Esse login já está em uso na sua loja.' }, { status: 400 });
45     }
46
47     // 3. Criptografa a senha para o PDV
48     const salt = await bcrypt.genSalt(10);
49     const senhaHash = await bcrypt.hash(senha, salt);
50
51     // 4. Insere no banco
52     await pool.query(
53       'INSERT INTO usuarios (nome, username, senha_hash, papel, ativo, empresa_documento) VALUES ($1, $2, $3, $4, $5, $6)',
54       [nome, login, senhaHash, papel, true, empresa_documento]
55     );
56
57     return NextResponse.json({ success: true });
58   } catch (error: any) {
59     // @ AGORA ELE MOSTRA O ERRO EXATO DO BANCO DE DADOS
60     console.error("ERRO DETALHADO NO BANCO:", error);
61     return NextResponse.json({ error: 'Erro no Postgres: ${error.message}' }, { status: 500 });
62   }
63 }
64
65 export async function DELETE(request: Request) {
66   try {
67     const { searchParams } = new URL(request.url);
68     const id = searchParams.get('id');
69
70     await pool.query('DELETE FROM usuarios WHERE id = $1', [id]);
71     return NextResponse.json({ success: true });
72   } catch (error) {
73     return NextResponse.json({ error: 'Erro ao deletar usuário' }, { status: 500 });
74   }
75 }

```

```

1 'use client';
2
3 import { useEffect, useState } from 'react';
4 import { useRouter } from 'next/navigation';
5
6 interface Produto {
7   id: number;
8   nome: string;
9   descricao: string;
10  preco_mensal: number;
11  preco_anual: number | null;
12  icone: string;
13  status: 'adquirido' | 'bloqueado' | 'inadimplente';
14  data_aquisicao: string | null;
15  data_expiracao: string | null;
16 }
17
18 export default function DashboardPage() {
19   const router = useRouter();
20   const [produtos, setProdutos] = useState<Produto[]>([]);
21   const [cliente, setCliente] = useState<{ nome: string; email: string } | null>(null);
22   const [carregando, setCarregando] = useState(true);
23   const [produtoSelecionado, setProdutoSelecionado] = useState<Produto | null>(null);
24
25   useEffect(() => {
26     const token = localStorage.getItem('corumba_token');
27     if (!token) {
28       router.push('/login');
29       return;
30     }
31
32     const clienteData = localStorage.getItem('corumba_cliente');
33     if (clienteData) {
34       setCliente(JSON.parse(clienteData));
35     }
36
37     // Buscar produtos do painel
38     fetch('/api/painel/produtos', {
39       headers: { Authorization: `Bearer ${token}` },
40     })
41       .then((res) => {
42         if (res.ok) return res.json();
43         throw new Error('Erro ao carregar produtos');
44       })
45       .then((data) => setProdutos(data))
46       .catch((err) => console.error(err))
47       .finally(() => setCarregando(false));
48   }, [router]);
49
50   const handleLogout = () => {
51     localStorage.removeItem('corumba_token');
52     localStorage.removeItem('corumba_cliente');
53     router.push('/login');
54   };
55
56   const adquiridos = produtos.filter((p) => p.status === 'adquirido');
57   const bloqueados = produtos.filter((p) => p.status === 'bloqueado');
58   const inadimplentes = produtos.filter((p) => p.status === 'inadimplente');
59
60   if (carregando) {
61     return (
62       <div className="min-h-screen flex items-center justify-center bg-[#f8fafc]">
63         <p className="text-gray-500">Carregando...</p>
64       </div>
65     );
66   }
67
68   return (
69     <div className="min-h-screen bg-[#f8fafc]">
70       {/* Header */}
71       <header className="bg-white border-b border-gray-200 px-8 py-5">
72         <div className="max-w-6xl mx-auto flex justify-between items-center">
73           
78           <div className="flex items-center gap-4">
79             {cliente && (
80               <div className="text-right">
81                 <p className="text-sm font-semibold text-[#0b1a30]">{cliente.nome}</p>
82                 <p className="text-xs text-gray-500">{cliente.email}</p>
83               </div>
84             )}
85             <button
86               onClick={handleLogout}
87               className="text-sm text-gray-500 hover:text-red-600 transition-colors"
88             >
89               Sair
90             </button>
91           </div>
92         </div>
93       </header>
94
95       {/* Conteúdo */}
96       <main className="max-w-6xl mx-auto px-8 py-10">
97         {/* Título */}
98         <div className="mb-8">
99           <h2 className="text-3xl font-bold text-[#0b1a30]">Nossas Soluções</h2>
100           <p className="mt-2 text-gray-500">Gerencie seus produtos e adquira novos serviços</p>
101         </div>
102
103         {/* Produtos Adquiridos */}
104         {adquiridos.length > 0 && (
105           <div className="mb-10">
106             <h3 className="text-lg font-bold text-[#0b1a30] mb-4 flex items-center gap-2">
107               <span className="w-3 h-3 rounded-full bg-green-500"></span>
108               Seus Produtos
109             </h3>
110             <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
111               {adquiridos.map((p) => (
112                 <div
113                   key={p.id}
114                   className="bg-white p-6 rounded-xl shadow-sm border border-green-200 flex flex-col"
115                 >
116                   <div className="flex items-start justify-between mb-4">
117                     <span className="text-4xl">{p.icone}</span>
118                     <span className="bg-green-100 text-green-700 text-xs font-bold px-3 py-1 rounded-full uppercase">Ativo</span>
119                   </div>
120                   <h4 className="text-xl font-bold text-[#0b1a30] mb-2">{p.nome}</h4>
121                   <p className="text-gray-500 text-sm mb-4 flex-grow line-clamp-3">{p.descricao}</p>
122                   <div className="flex items-center justify-between mt-auto pt-4 border-t border-gray-100">
123                     <span className="text-lg font-bold text-[#0b1a30]">
124                       R$ {p.preco_mensal.toFixed(2).replace('.', ',')}
125                     <span className="text-xs text-gray-400 font-normal">/mês</span>
126                   </div>
127                   <button
128                     onClick={() => {

```

```

135         if (p.nome === 'SaltoGestão') {
136             router.push('/dashboard/salto-gestao');
137         } else if (p.nome === 'MesaPronta') {
138             alert('MesaPronta é um aplicativo desktop. Faça o download na área de downloads.');
```

139 } else if (p.nome === 'ConnectFood') {

140 alert('ConnectFood é um módulo integrado ao MesaPronta.\n\nAcesse pelo botão "🚚 Delivery" dentro do MesaPronta.\n\nReceba pedidos do iFood e 99Food direto no seu painel!');

```

141         }
142     })
143     className="bg-[#84cc16] text-[#0b1a30] font-bold text-sm py-2 px-4 rounded-md hover:bg-[#65a30d] hover:text-white transition-colors"
144     >
145     Acessar
146     </button>
147 </div>
148 </div>
149 })}
150 </div>
151 </div>
152 })
153
154 /* Produtos Bloqueados */
155 {bloqueados.length > 0 && (
156 <div className="mb-10">
157     <h3 className="text-lg font-bold text-[#0b1a30] mb-4 flex items-center gap-2">
158         <span className="w-3 h-3 rounded-full bg-gray-400"></span>
159         Disponíveis para Aquisição
160     </h3>
161     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
162         {bloqueados.map(p => (
163             <div
164                 key={p.id}
165                 className="bg-white p-6 rounded-xl shadow-sm border border-gray-200 flex flex-col opacity-80 hover:opacity-100 transition-opacity"
166             >
167                 <div className="flex items-start justify-between mb-4">
168                     <span className="text-4xl grayscale">{p.icone}</span>
169                     <span className="bg-gray-100 text-gray-500 text-xs font-bold px-3 py-1 rounded-full uppercase">
170                         Bloqueado
171                     </span>
172                 </div>
173                 <h4 className="text-xl font-bold text-[#0b1a30] mb-2">{p.nome}</h4>
174                 <p className="text-gray-400 text-sm mb-4 flex-grow line-clamp-3">
175                     {p.descricao}
176                 </p>
177                 <div className="flex items-center justify-between mt-auto pt-4 border-t border-gray-100">
178                     <span className="text-lg font-bold text-gray-400">
179                         R$ {p.preco_mensal.toFixed(2).replace('.', ',')}
180                     <span className="text-xs text-gray-400 font-normal">/mês</span>
181                 </div>
182                 <button
183                     onClick={() => setProdutoSelecioneado(p)}
184                     className="bg-[#0b1a30] text-white font-bold text-sm py-2 px-4 rounded-md hover:bg-slate-800 transition-colors"
185                 >
186                     Ver Detalhes
187                 </button>
188             </div>
189         ))}
190     </div>
191 </div>
192 </div>
193 })
194
195 /* Inadimplentes */
196 {inadimplentes.length > 0 && (
197 <div className="mb-10">
198     <h3 className="text-lg font-bold text-red-600 mb-4 flex items-center gap-2">
199         <span className="w-3 h-3 rounded-full bg-red-500"></span>
200         Pagamento Pendente
201     </h3>
202     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
203         {inadimplentes.map(p => (
204             <div
205                 key={p.id}
206                 className="bg-white p-6 rounded-xl shadow-sm border border-red-200 flex flex-col"
207             >
208                 <div className="flex items-start justify-between mb-4">
209                     <span className="text-4xl">{p.icone}</span>
210                     <span className="bg-red-100 text-red-700 text-xs font-bold px-3 py-1 rounded-full uppercase">
211                         Inadimplente
212                     </span>
213                 </div>
214                 <h4 className="text-xl font-bold text-[#0b1a30] mb-2">{p.nome}</h4>
215                 <p className="text-gray-500 text-sm mb-4 flex-grow">
216                     Acesso bloqueado por inadimplência. Entre em contato com a Corumbá Sistemas para regularizar.
217                 </p>
218                 <button
219                     onClick={() => setProdutoSelecioneado(p)}
220                     className="w-full bg-red-600 text-white font-bold text-sm py-2 px-4 rounded-md hover:bg-red-700 transition-colors mt-4"
221                 >
222                     Regularizar Acesso
223                 </button>
224             </div>
225         ))}
226     </div>
227 </div>
228 </div>
229 </main>
230
231 /* Modal de Detalhes do Produto */
232 {produtoSelecioneado && (
233 <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
234     <div className="bg-white rounded-2xl shadow-2xl max-w-lg w-full p-8">
235         <div className="flex items-start justify-between mb-6">
236             <div className="flex items-center gap-4">
237                 <span className="text-5xl">{produtoSelecioneado.icone}</span>
238             </div>
239             <h3 className="text-2xl font-bold text-[#0b1a30]">
240                 {produtoSelecioneado.nome}
241             </h3>
242             <p className="text-sm text-gray-500">Corumbá Sistemas</p>
243         </div>
244         <button
245             onClick={() => setProdutoSelecioneado(null)}
246             className="text-gray-400 hover:text-gray-600 text-2xl"
247         >
248             x
249         </button>
250     </div>
251 </div>
252
253 <p className="text-gray-600 mb-6 leading-relaxed">
254     {produtoSelecioneado.descricao}
255 </p>
256
257 <div className="bg-gray-50 rounded-lg p-4 mb-6">
258     <div className="flex justify-between items-center mb-2">
259         <span className="text-gray-500">Mensalidade</span>
260         <span className="text-2xl font-bold text-[#0b1a30]">
261             R$ {produtoSelecioneado.preco_mensal.toFixed(2).replace('.', ',')}
262         </span>
263     </div>
264     {produtoSelecioneado.preco_anual && (
265         <div className="flex justify-between items-center">
266             <span className="text-gray-500">Plano Anual (com desconto)</span>
267             <span className="text-lg font-bold text-green-600">
268                 R$ {produtoSelecioneado.preco_anual.toFixed(2).replace('.', ',')}
269             <span className="text-xs text-gray-400 font-normal">/ano</span>
270         </div>
271     )}
272 </div>

```



```

272     })
273   </div>
274
275   <div className="flex gap-3">
276     <button
277       onClick={() => setProdutoSelecionado(null)}
278       className="flex-1 py-3 border border-gray-300 text-gray-700 font-bold rounded-md hover:bg-gray-50 transition-colors"
279     >
280       Fechar
281     </button>
282     <button
283       onClick={() => {
284         alert('Para adquirir este produto, entre em contato com a Corumbá Sistemas:\n\ncontato@corumbasistemas.com.br\n(62) 99999-9999');
285       }}
286       className="flex-1 py-3 bg-[#84cc16] text-[#0b1a30] font-bold rounded-md hover:bg-[#65a30d] hover:text-white transition-colors"
287     >
288       Solicitar Contratação
289     </button>
290   </div>
291 </div>
292 </div>
293 })
294
295 /* Footer */
296 <footer className="bg-[#0b1a30] text-white py-8 text-center mt-auto">
297   <h4 className="font-bold mb-2">Corumbá Sistemas</h4>
298   <p className="text-sm text-gray-400">contato@corumbasistemas.com.br</p>
299   <p className="text-sm text-gray-400">Goiânia, GO</p>
300 </footer>
301 </div>
302   );
303 }
304

```

```
1 import { NextResponse } from 'next/server';
2 import { Pool } from 'pg';
3
4 // ⚠️ ATENÇÃO: Coloque aqui a URL de conexão do banco de dados SALTO GESTAO do Neon
5 // (Diferente da URL do CorumbaCore que usamos nos usuários)
6 const pool = new Pool({
7   connectionString: "postgresql://neondb_owner:neondb_owner@ep-weathered-thunder-ap2gc8n9-pooler.c-7.us-east-1.aws.neon.tech/salto_gestao?sslmode=require"
8 });
9
10 export async function GET(request: Request) {
11   try {
12     const { searchParams } = new URL(request.url);
13     const documento = searchParams.get('documento');
14
15     if (!documento) {
16       return NextResponse.json({ error: 'CNPJ não informado.' }, { status: 400 });
17     }
18
19     // 1. Busca as Vendas de HOJE desta empresa
20     // Ajuste o nome da coluna de data (ex: data_venda, criado_em) de acordo com o seu Python
21     const vendasResult = await pool.query(`
22       SELECT
23         COALESCE(SUM(total), 0) as faturamento_hoje,
24         COUNT(id) as pedidos_hoje
25     FROM vendas
26     WHERE empresa_documento = $1
27     AND DATE(data_venda) = CURRENT_DATE
28     `, [documento]);
29
30     // 2. Busca o total de Produtos cadastrados por esta empresa
31     const produtosResult = await pool.query(`
32       SELECT COUNT(id) as total_produtos
33     FROM produtos
34     WHERE empresa_documento = $1
35     `, [documento]);
36
37     const metricas = {
38       faturamentoHoje: Number(vendasResult.rows[0].faturamento_hoje),
39       pedidosHoje: Number(vendasResult.rows[0].pedidos_hoje),
40       produtosCadastrados: Number(produtosResult.rows[0].total_produtos)
41     };
42
43     return NextResponse.json(metricas);
44   } catch (error) {
45     console.error("Erro ao buscar métricas:", error);
46     return NextResponse.json({ error: 'Erro ao buscar métricas do Salto Gestão' }, { status: 500 });
47   }
48 }
49 }
```

```

1 'use client';
2
3 import { useState, useEffect, useCallback } from 'react';
4 import { useRouter } from 'next/navigation';
5 import Image from 'next/image';
6
7 export default function SaltoGestaoDashboard() {
8   const router = useRouter();
9   const [activeTab, setActiveTab] = useState('visao-geral');
10
11   // Identificação do Inquilino (Multi-tenant)
12   const [documentoCliente, setDocumentoCliente] = useState('00.000.000/0001-00');
13
14   // Estados de dados de Usuários (agora utilizando 'login' ao invés de 'email')
15   const [usuarios, setUsuarios] = useState<any[]>([]);
16   const [novoUsuario, setNovoUsuario] = useState({ nome: '', login: '', senha: '', papel: 'VENDEDOR' });
17   const [isSaving, setIsSaving] = useState(false);
18   const [isLoadingUsers, setIsLoadingUsers] = useState(false);
19
20   // Estados para as Métricas da Visão Geral
21   const [metricas, setMetricas] = useState({ faturamentoHoje: 0, pedidosHoje: 0, produtosCadastrados: 0 });
22   const [carregandoMetricas, setCarregandoMetricas] = useState(false);
23
24   // Verificação de autenticação ao montar a página
25   useEffect(() => {
26     const token = localStorage.getItem('corumba_token');
27     if (!token) {
28       router.push('/login');
29       return;
30     }
31
32     const cnpjSalvo = localStorage.getItem('cnpj_cliente');
33     if (cnpjSalvo) {
34       setDocumentoCliente(cnpjSalvo);
35     }
36   }, [router]);
37
38   // Função para buscar métricas reais do banco SaltoGestao
39   const carregarMetricas = useCallback(async () => {
40     setCarregandoMetricas(true);
41     try {
42       const res = await fetch(`/api/salto-gestao/metricas?documento=${documentoCliente}`);
43       if (res.ok) {
44         const data = await res.json();
45         setMetricas(data);
46       }
47     } catch (error) {
48       console.error("Erro ao carregar métricas da loja:", error);
49     } finally {
50       setCarregandoMetricas(false);
51     }
52   }, [documentoCliente]);
53
54   // Carrega as métricas sempre que a aba Visão Geral for ativada
55   useEffect(() => {
56     if (activeTab === 'visao-geral') {
57       carregarMetricas();
58     }
59   }, [activeTab, carregarMetricas]);
60
61   // Função para buscar usuários amarrados ao documento do cliente
62   const carregarUsuarios = useCallback(async () => {
63     setIsLoadingUsers(true);
64     try {
65       const res = await fetch(`/api/usuarios?documento=${documentoCliente}`);
66       if (res.ok) {
67         const data = await res.json();
68         setUsuarios(data);
69       }
70     } catch (error) {
71       console.error("Erro ao carregar equipe:", error);
72     } finally {
73       setIsLoadingUsers(false);
74     }
75   }, [documentoCliente]);
76
77   // Carrega a listagem sempre que a aba Configurações for ativada
78   useEffect(() => {
79     if (activeTab === 'configuracoes') {
80       carregarUsuarios();
81     }
82   }, [activeTab, carregarUsuarios]);
83
84   // Handler para criar novo usuário/operador
85   const handleCriarUsuario = async (e: React.FormEvent) => {
86     e.preventDefault();
87     setIsSaving(true);
88     try {
89       const payload = {
90         ...novoUsuario,
91         empresa_documento: documentoCliente
92       };
93
94       const res = await fetch('/api/usuarios', {
95         method: 'POST',
96         headers: { 'Content-Type': 'application/json' },
97         body: JSON.stringify(payload)
98       });
99
100      if (res.ok) {
101        alert("Operador cadastrado com sucesso!");
102        // Limpa o formulário mantendo a estrutura de 'login'
103        setNovoUsuario({ nome: '', login: '', senha: '', papel: 'VENDEDOR' });
104        carregarUsuarios();
105      } else {
106        const err = await res.json();
107        alert(err.error || "Erro ao salvar usuário.");
108      }
109    } catch (error) {
110      alert("Falha na comunicação com o banco de dados.");
111    } finally {
112      setIsSaving(false);
113    }
114  };
115
116   // Handler para remover usuário/operador
117   const handleDeletarUsuario = async (id: number, nome: string) => {
118     if (!confirm('Deseja remover o acesso de ${nome}? Esta ação bloqueará o login dele imediatamente no PDV Desktop.')) {
119       return;
120     }
121
122     try {
123       const res = await fetch(`/api/usuarios?id=${id}`, { method: 'DELETE' });
124       if (res.ok) {
125         carregarUsuarios();
126       } else {
127         alert("Não foi possível remover o usuário.");
128       }
129     } catch (error) {
130       alert("Erro ao conectar com o servidor.");
131     }
132   };
133
134   return (

```

```
135 <div className="flex h-screen bg-[#f8fafc] font-sans text-slate-800 overflow-hidden">
136
137   {/* Sidebar Lateral Oficial */}
138   <aside className="w-64 bg-[#0b1a30] text-white flex flex-col justify-between shadow-xl shrink-0">
139     <div>
140       {/* Logo Container */}
141       <div className="p-6 border-b border-slate-800">
142         <div className="relative w-full h-16">
143           
150         </div>
151         <p className="text-[10px] text-emerald-400 font-bold uppercase tracking-widest mt-2 px-1">
152           Paine! de Gest!o
153         </p>
154       </div>
155
156       {/* Navega!o por Abas */}
157       <nav className="px-4 py-6 space-y-1.5">
158         <button
159           onClick={() => setActiveTab('visao-geral')}
160           className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
161             activeTab === 'visao-geral'
162               ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
163               : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
164             }`
165         >
166           <span className="text-lg">🏠</span> Vis!o Geral
167         </button>
168
169         <button
170           onClick={() => setActiveTab('vendas')}
171           className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
172             activeTab === 'vendas'
173               ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
174               : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
175             }`
176         >
177           <span className="text-lg">📈</span> Vendas
178         </button>
179
180         <button
181           onClick={() => setActiveTab('produtos')}
182           className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
183             activeTab === 'produtos'
184               ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
185               : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
186             }`
187         >
188           <span className="text-lg">📦</span> Produtos
189         </button>
190
191         <button
192           onClick={() => setActiveTab('configuracoes')}
193           className={`w-full flex items-center gap-3 px-4 py-3 rounded-lg text-sm font-semibold transition-all ${
194             activeTab === 'configuracoes'
195               ? 'bg-[#84cc16]/10 text-[#84cc16] border-l-4 border-[#84cc16]'
196               : 'text-slate-300 hover:bg-slate-800/50 hover:text-white'
197             }`
198         >
199           <span className="text-lg">⚙️</span> Configura!oes
200         </button>
201       </nav>
202     </div>
203
204     {/* Rodap! da Sidebar */}
205     <div className="p-4 border-t border-slate-800">
206       <button
207         onClick={() => router.push('/dashboard')}
208         className="w-full flex items-center gap-2 px-4 py-3 text-sm text-slate-400 hover:text-white font-medium transition-colors"
209       >
210         <span>🏠</span> Voltar ao Hub
211       </button>
212     </div>
213   </aside>
214
215   {/* !rea de Conte!do Din!mico */}
216   <div className="flex-1 flex flex-col overflow-hidden">
217
218     {/* Header Principal Superior */}
219     <header className="bg-white border-b border-gray-200 px-8 py-5 flex justify-between items-center shrink-0">
220       <h2 className="text-xl font-bold text-[#0b1a30] capitalize">
221         {activeTab === 'visao-geral' ? 'Vis!o Geral da Loja' : activeTab}
222       </h2>
223       <div className="flex items-center gap-2 text-xs font-bold text-[#65a30d] bg-[#84cc16]/10 px-4 py-2 rounded-full">
224         <span className="w-2 h-2 rounded-full bg-[#84cc16] animate-pulse"></span>
225         Licen!a Ativa
226       </div>
227     </header>
228
229     {/* Visualiza!o de Telas */}
230     <main className="flex-1 overflow-y-auto p-8">
231
232       {/* TELA 1: VIS!O GERAL */}
233       {activeTab === 'visao-geral' && (
234         <div className="space-y-6 md:w-6xl mx-auto">
235           {/* Banner de Download */}
236           <div className="bg-[#0b1a30] rounded-xl p-8 flex flex-col md:flex-row justify-between items-start md:items-center gap-6 shadow-md border border-slate-800">
237             <div className="text-white max-w-2xl">
238               <h3 className="text-xl font-bold mb-2">Instale o Salto PDV na sua m!quina</h3>
239               <p className="text-slate-300 text-sm leading-relaxed">
240                 Para operar o seu caixa com velocidade m!xima, emitir cupons fiscais e usar atalhos de teclado r!pidos, fa!a o download da vers!o Desktop estruturada para o seu
241                 neg!cio.
242               </p>
243             </div>
244             <button className="bg-[#84cc16] hover:bg-[#65a30d] text-[#0b1a30] font-bold py-3.5 px-6 rounded-md transition-all shadow-lg shrink-0 flex items-center gap-2">
245               <span>📄</span> Download para Windows
246             </button>
247           </div>
248
249           {/* Cards de Indicadores */}
250           <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
251             <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
252               <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Vendas Hoje</p>
253               <p className="text-3xl font-extrabold text-[#0b1a30]">
254                 {carregandoMetricas ? '...' : new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' }).format(metricas.faturamentoHoje)}
255               </p>
256             </div>
257             <div className="text-xs text-emerald-600 font-semibold mt-1 inline-block">M!tricas em tempo real</span>
258             </div>
259
260             <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
261               <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Pedidos Fechados</p>
262               <p className="text-3xl font-extrabold text-[#0b1a30]">
263                 {carregandoMetricas ? '...' : metricas.pedidosHoje}
264               </p>
265             </div>
266             <div className="text-xs text-gray-500 font-medium mt-1 inline-block">
267               Ticket m!dio: {metricas.pedidosHoje > 0 ? new Intl.NumberFormat('pt-BR', { style: 'currency', currency: 'BRL' }).format(metricas.faturamentoHoje /
268                 metricas.pedidosHoje) : 'R$ 0,00'}
269             </div>
270
271             <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
272               <p className="text-xs font-bold text-gray-400 uppercase tracking-wider mb-1">Produtos Cadastrados</p>
273               <p className="text-3xl font-extrabold text-[#0b1a30]">
```

```
271         {carregandoMetricas ? '...' : metricas.produtosCadastrados}
272     </p>
273     <span className="text-xs text-blue-600 font-semibold mt-1 inline-block">Banco de Dados Sincronizado</span>
274 </div>
275 </div>
276
277     /* Área do Gráfico */
278     <div className="bg-white rounded-xl border border-gray-200 p-12 text-center shadow-sm flex flex-col items-center justify-center min-h-[300px]">
279     <div className="text-4xl mb-4 opacity-40">=</div>
280     <h4 className="text-lg font-bold text-{@0b1a30} mb-1">Gráfico de Faturamento Semanal</h4>
281     <p className="text-sm text-gray-400 max-w-sm">
282     Os dados operacionais coletados no PDV aparecerão estruturados aqui assim que o caixa realizar a primeira sincronização de fechamento.
283     </p>
284 </div>
285 </div>
286
287 ))
288
289 /* TELA 2: VENDAS (PLACEHOLDER) */
290 {activeTab === 'vendas' && (
291     <div className="max-w-6xl mx-auto bg-white rounded-xl border border-gray-200 p-8 text-center shadow-sm">
292     <p className="text-gray-500">Módulo de Histórico e Relatório Analítico de Vendas em desenvolvimento.</p>
293 </div>
294 )}
295
296 /* TELA 3: PRODUTOS (PLACEHOLDER) */
297 {activeTab === 'produtos' && (
298     <div className="max-w-6xl mx-auto bg-white rounded-xl border border-gray-200 p-8 text-center shadow-sm">
299     <p className="text-gray-500">Módulo de Sincronização e ajuste de grade de Produtos em desenvolvimento.</p>
300 </div>
301 )}
302
303 /* TELA 4: CONFIGURAÇÕES (GESTÃO REAL DE USUÁRIOS) */
304 {activeTab === 'configuracoes' && (
305     <div className="max-w-6xl mx-auto space-y-8">
306
307         /* Box de Criação */
308         <div className="bg-white rounded-xl border border-gray-200 shadow-sm p-6">
309         <div className="mb-4">
310             <h3 className="text-lg font-bold text-{@0b1a30}">Controle de Usuários da Loja</h3>
311             <p className="text-xs text-gray-400">Cadastre os colaboradores para liberação de acessos individuais vinculados ao CNPJ {documentoCliente}.</p>
312         </div>
313
314         <form onSubmit={handleCriarUsuario} className="grid grid-cols-1 md:grid-cols-4 gap-4 items-end">
315         <div>
316             <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Nome Completo</label>
317             <input
318                 type="text"
319                 required
320                 value={novoUsuario.nome}
321                 onChange={(e) => setNovoUsuario({...novoUsuario, nome: e.target.value})}
322                 className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-{@84cc16} focus:border-transparent outline-none transition-all
323             </div>
324             <div>
325                 <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Login do PDV</label>
326                 <input
327                     type="text"
328                     required
329                     value={novoUsuario.login}
330                     onChange={(e) => setNovoUsuario({...novoUsuario, login: e.target.value})}
331                     className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-{@84cc16} focus:border-transparent outline-none transition-all
332                 </div>
333                 <div>
334                     <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Senha de Acesso</label>
335                     <input
336                         type="password"
337                         required
338                         value={novoUsuario.senha}
339                         onChange={(e) => setNovoUsuario({...novoUsuario, senha: e.target.value})}
340                         className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-{@84cc16} focus:border-transparent outline-none transition-all
341                     </div>
342                     <div>
343                         <label className="block text-xs font-bold text-gray-500 uppercase mb-1.5">Atribuição / Função</label>
344                         <select
345                             value={novoUsuario.papel}
346                             onChange={(e) => setNovoUsuario({...novoUsuario, papel: e.target.value})}
347                             className="w-full border border-gray-300 rounded-md px-3 py-2 text-sm focus:ring-2 focus:ring-{@84cc16} focus:border-transparent outline-none bg-white
348                             font-medium"
349                         >
350                             <option value="ADMIN">Admin</option>
351                             <option value="CAIXA">Caixa</option>
352                             <option value="VENDEDOR">Vendedor</option>
353                         </select>
354                     </div>
355                     <div className="md:col-span-4 flex justify-end mt-2">
356                     <button
357                         disabled={isSaving}
358                         type="submit"
359                         className="bg-{@0b1a30} hover:bg-slate-800 text-white font-bold text-sm py-2.5 px-6 rounded-md transition-colors shadow-sm disabled:opacity-50"
360                     >
361                         {isSaving ? 'Salvando o banco...' : 'Adicionar Usuário'}
362                     </button>
363                     </div>
364                 </form>
365 </div>
366
367 /* Tabela Grid List */
368 <div className="bg-white rounded-xl border border-gray-200 shadow-sm overflow-hidden">
369 <div className="px-6 py-4 border-b border-gray-200 bg-gray-50/70">
370     <h4 className="text-xs font-bold text-{@0b1a30} uppercase tracking-wider">Usuários e Atribuições Ativas</h4>
371 </div>
372
373 <div className="overflow-x-auto">
374     <table className="w-full text-left border-collapse">
375     <thead>
376     <tr className="border-b border-gray-200 text-xs text-gray-400 uppercase bg-white">
377     <th className="p-4 font-bold">Colaborador</th>
378     <th className="p-4 font-bold">Usuário de Login</th>
379     <th className="p-4 font-bold">Nível de Acesso</th>
380     <th className="p-4 font-bold text-center">Ação</th>
381     </tr>
382 </thead>
383 <tbody className="divide-y divide-gray-100">
384     {isLoadingUsers ? (
385     <tr>
386     <td colspan={4} className="p-8 text-center text-sm text-gray-400">Consultando tabelas Neon...</td>
387     </tr>
388     ) : usuarios.map(user) => (
389     <tr key={user.id} className="hover:bg-gray-50/50 transition-colors">
390     <td className="p-4 text-sm font-bold text-{@0b1a30}">{user.nome}</td>
391     <td className="p-4 text-sm text-gray-600">{user.login}</td>
392     <td className="p-4 text-sm">
393         <span className={`px-3 py-1 text-xs font-extrabold rounded-full ${
394             user.papel === 'ADMIN' ? 'bg-purple-50 text-purple-700 border border-purple-200' :
395             user.papel === 'CAIXA' ? 'bg-blue-50 text-blue-700 border border-blue-200' :
396             'bg-amber-50 text-amber-700 border border-amber-200'
397         }`}>
398             {user.papel}
399         </span>
400     </td>
401     </tr>
402 </tbody>
403 </table>
404
```

```
405         <td className="p-4 text-center">
406             <button
407                 onClick={() => handleDeletarUsuario(user.id, user.nome)}
408                 className="text-red-500 hover:text-red-700 font-bold text-xs transition-colors"
409             >
410                 Remover Acesso
411             </button>
412         </td>
413     </tr>
414 }}
415 {isLoadingUsers && usuarios.length === 0 && (
416     <tr>
417         <td colspan={4} className="p-8 text-center text-sm text-gray-400">Nenhum funcionário cadastrado sob este CNPJ.</td>
418     </tr>
419 )}
420 </tbody>
421 </table>
422 </div>
423 </div>
424
425 </div>
426 )}
427
428 </main>
429 </div>
430 </div>
431 );
432 }
```

```
1 @import "tailwindcss";
2
3 :root {
4   --background: #ffffff;
5   --foreground: #0b1a30;
6 }
7
8 @theme inline {
9   --color-background: var(--background);
10  --color-foreground: var(--foreground);
11  --font-sans: var(--font-geist-sans);
12  --font-mono: var(--font-geist-mono);
13 }
14
15 /* Forçar light mode - não usar dark mode */
16 @media (prefers-color-scheme: dark) {
17   :root {
18     --background: #ffffff;
19     --foreground: #0b1a30;
20   }
21 }
22
23 body {
24   background: #ffffff;
25   color: #0b1a30;
26   font-family: Arial, Helvetica, sans-serif;
27 }
28
29 /* Garantir que inputs sempre tenham texto escuro */
30 input, select, textarea {
31   color: #0b1a30 !important;
32   background-color: #ffffff !important;
33 }
34
35 /* Garantir labels visíveis */
36 label {
37   color: #0b1a30 !important;
38 }
39
```

```
1 import type { Metadata } from "next";
2 import { Geist, Geist_Mono } from "next/font/google";
3 import "./globals.css";
4
5 const geistSans = Geist({
6   variable: "--font-geist-sans",
7   subsets: ["latin"],
8 });
9
10 const geistMono = Geist_Mono({
11   variable: "--font-geist-mono",
12   subsets: ["latin"],
13 });
14
15 export const metadata: Metadata = {
16   title: "Create Next App",
17   description: "Generated by create next app",
18 };
19
20 export default function RootLayout({
21   children,
22 }: Readonly<{
23   children: React.ReactNode;
24 }>) {
25   return (
26     <html
27       lang="en"
28       className={` ${geistSans.variable} ${geistMono.variable} h-full antialiased`}
29     >
30     <body className="min-h-full flex flex-col">{children}</body>
31     </html>
32   );
33 }
34
```



```

1 'use client';
2
3 import { useState } from 'react';
4 import { useRouter } from 'next/navigation';
5
6 export default function LoginPage() {
7   const router = useRouter();
8   const [email, setEmail] = useState('');
9   const [password, setPassword] = useState('');
10  const [isLoading, setIsLoading] = useState(false);
11  const [error, setError] = useState('');
12
13  const handleLogin = async (e: React.FormEvent<HTMLFormElement>) => {
14    e.preventDefault();
15    setIsLoading(true);
16    setError('');
17
18    try {
19      const response = await fetch('/api/login', {
20        method: 'POST',
21        headers: { 'Content-Type': 'application/json' },
22        body: JSON.stringify({ email, password }),
23      });
24
25      if (response.ok) {
26        const data = await response.json();
27        localStorage.setItem('corumba_token', data.access_token);
28        localStorage.setItem('corumba_cliente', JSON.stringify(data.cliente));
29        window.location.href = '/dashboard';
30      } else {
31        const errorData = await response.json();
32        setError(errorData.error || 'Erro no login.');

```

```
1 export default function Home() {
2   return (
3     <div className="flex min-h-screen items-center justify-center bg-gray-100">
4       <h1 className="text-4xl font-bold text-blue-600">
5         Salto Gestão 2.0 - Front-end Iniciado
6       </h1>
7     </div>
8   );
9 }
```

```
1 /// <reference types="next" />
2 /// <reference types="next/image-types/global" />
3 import "../next/dev/types/routes.d.ts";
4
5 // NOTE: This file should not be edited
6 // see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
7
```

```
1 import type { NextConfig } from "next";
2
3 const nextConfig: NextConfig = {
4   allowedDevOrigins: ['191.252.210.235', 'localhost'],
5   images: {
6     unoptimized: true,
7   }
8 };
9
10 export default nextConfig;
```

```
1 {
2   "name": "frontend-web",
3   "version": "0.1.0",
4   "private": true,
5   "scripts": {
6     "dev": "next dev",
7     "build": "next build",
8     "start": "next start",
9     "lint": "eslint"
10  },
11  "dependencies": {
12    "bcryptjs": "^3.0.3",
13    "jsonwebtoken": "^9.0.3",
14    "next": "16.2.6",
15    "pg": "^8.21.0",
16    "react": "19.2.4",
17    "react-dom": "19.2.4"
18  },
19  "devDependencies": {
20    "@tailwindcss/postcss": "^4",
21    "@types/bcryptjs": "^2.4.6",
22    "@types/jsonwebtoken": "^9.0.10",
23    "@types/node": "^20",
24    "@types/pg": "^8.20.0",
25    "@types/react": "^19",
26    "@types/react-dom": "^19",
27    "eslint": "^9",
28    "eslint-config-next": "16.2.6",
29    "tailwindcss": "^4",
30    "typescript": "^5"
31  }
32 }
33
```

```
1 {
2   "compilerOptions": {
3     "target": "ES2017",
4     "lib": [
5       "dom",
6       "dom.iterable",
7       "esnext"
8     ],
9     "allowJs": true,
10    "skipLibCheck": true,
11    "strict": false,
12    "noEmit": true,
13    "incremental": true,
14    "module": "esnext",
15    "esModuleInterop": true,
16    "moduleResolution": "bundler",
17    "resolveJsonModule": true,
18    "isolatedModules": true,
19    "jsx": "react-jsx",
20    "plugins": [
21      {
22        "name": "next"
23      }
24    ]
25  },
26  "include": [
27    "next-env.d.ts",
28    ".next/types/**/*.ts",
29    ".next/dev/types/**/*.ts",
30    "**/*.mts",
31    "**/*.ts",
32    "**/*.tsx"
33  ],
34  "exclude": [
35    "node_modules"
36  ]
37 }
38
```

iniciar.sh

```
1 #!/bin/bash
2
3 echo "% Iniciando o ecossistema Corumbá Sistemas (Salto Gestão)..."
4
5 # Função inteligente: se você apertar CTRL+C, ele desliga o Python e o React juntos
6 cleanup() {
7     echo ""
8     echo "🛑 Desligando os motores..."
9     kill $BACKEND_PID
10    kill $FRONTEND_PID
11    exit
12 }
13 trap cleanup SIGINT
14
15 # 1. Ligar o motor Backend (Python) em segundo plano
16 echo "% Ligando o Backend (FastAPI na porta 8080)..."
17 cd ~/Documentos/projects/sistema-loja/backendPy
18 source venv/bin/activate
19 uvicorn main:app --reload --port 8080 &
20 BACKEND_PID=$!
21
22 # 2. Ligar a interface Frontend (Next.js) em segundo plano
23 echo "% Ligando o Frontend (Next.js na porta 3000)..."
24 cd ~/Documentos/projects/sistema-loja/frontend-corumba
25 npm run dev &
26 FRONTEND_PID=$!
27
28 echo "=====
29 echo "🌐 Todos os sistemas online!"
30 echo "👉 Acesse o painel: http://localhost:3000/login"
31 echo "👉 Para desligar tudo de uma vez, pressione CTRL+C."
32 echo "=====
33
34 # Mantém o terminal aberto e mostrando os logs dos dois sistemas
35 wait
```

```
1 """
2 licenca.py - Gerenciamento de licencas do Corumbá Sistemas
3
4 Fluxo de licenciamento:
5 1. Admin recebe chave de licenca (ex: CORUMBA-A1B2-C3D4-E5F6)
6 2. Na primeira execucao, sistema pede a chave
7 3. Chave validada -> licenca ATIVA -> acesso total sem senha
8 4. Em cada login, verifica se licenca ainda esta ativa
9 5. Se EXPIRADA/SUSPENSA -> pede nova chave para reativar
10 6. Isolamento: cada CNPJ tem sua propria licenca e seus proprios dados
11 """
12 import hashlib
13 import secrets
14 import string
15 from datetime import datetime, timedelta
16 from typing import Optional
17
18
19 def gerar_chave_licenca() -> str:
20     """Gera uma chave de licenca unica no formato CORUMBA-XXXX-XXXX-XXXX."""
21     partes = []
22     for _ in range(3):
23         parte = ''.join(secrets.choice(string.ascii_uppercase + string.digits) for _ in range(4))
24         partes.append(parte)
25     return f"CORUMBA-{'-'.join(partes)}"
26
27
28 def gerar_hash_chave(chave: str) -> str:
29     """Gera hash da chave para armazenamento seguro (nao guarda a chave pura)."""
30     return hashlib.sha256(chave.encode()).hexdigest()
31
32
33 def validar_formato_chave(chave: str) -> bool:
34     """Valida se a chave esta no formato correto: CORUMBA-XXXX-XXXX-XXXX."""
35     if not chave or not chave.startswith("CORUMBA-"):
36         return False
37     partes = chave.split("-")
38     if len(partes) != 4:
39         return False
40     for parte in partes[1:]:
41         if len(parte) != 4 or not all(c in string.ascii_uppercase + string.digits for c in parte):
42             return False
43     return True
44
45
46 def verificar_licenca_ativa(status: str) -> bool:
47     """Verifica se a licenca permite acesso."""
48     return status == "ATIVA"
49
50
51 def licenca_expirou(data_expiracao: Optional[datetime]) -> bool:
52     """Verifica se a licenca expirou."""
53     if data_expiracao is None:
54         return False # Sem expiracao = valida para sempre
55     return datetime.utcnow() > data_expiracao
56
```



```

1 """
2 routers/produtos.py - CRUD de produtos + gestao de estoque
3 Multi-tenant: todas as operacoes filtram por empresa_documento (via JWT)
4 """
5 from fastapi import APIRouter, Depends, HTTPException, Query
6 from sqlalchemy.orm import Session
7 from sqlalchemy import func
8 from typing import Optional
9
10 from database import get_db_salto, get_db_core
11 from models import Produto, EstoqueMovimentacao
12 from schemas import ProdutoCreate, ProdutoUpdate, ProdutoResponse, EstoqueMovimentacaoCreate
13 from models_core import Usuario
14 from security import get_empresa_from_user, get_current_user, require_role, verificar_senha, hash_senha
15
16 router = APIRouter(prefix="/produtos", tags=["Produtos"])
17
18
19 def _get_empresa(current_user):
20     return current_user.empresa_documento
21
22
23 @router.post("/", response_model=ProdutoResponse)
24 def criar_produto(
25     prod: ProdutoCreate,
26     empresa_documento: str = Depends(get_empresa_from_user),
27     current_user: Usuario = Depends(get_current_user),
28     db: Session = Depends(get_db_salto)
29 ):
30     produto = Produto(empresa_documento=empresa_documento, **prod.model_dump())
31     db.add(produto)
32     db.commit()
33     db.refresh(produto)
34
35     # Registrar movimentacao de estoque inicial se > 0
36     if produto.estoque_atual > 0:
37         mov = EstoqueMovimentacao(
38             empresa_documento=empresa_documento,
39             produto_id=produto.id,
40             tipo="ENTRADA",
41             quantidade=produto.estoque_atual,
42             motivo="Cadastro inicial",
43             usuario=current_user.nome
44         )
45         db.add(mov)
46         db.commit()
47     return produto
48
49
50 @router.get("/", response_model=list[ProdutoResponse])
51 def listar_produtos(
52     empresa_documento: str = Depends(get_empresa_from_user),
53     busca: Optional[str] = None,
54     ativo: Optional[bool] = True,
55     estoque_baixo: Optional[bool] = None,
56     current_user: Usuario = Depends(get_current_user),
57     db: Session = Depends(get_db_salto)
58 ):
59     query = db.query(Produto).filter(Produto.empresa_documento == empresa_documento)
60     if ativo is not None:
61         query = query.filter(Produto.ativo == ativo)
62     if busca:
63         query = query.filter(
64             Produto.nome.ilike(f"%{busca}%") |
65             Produto.sku.ilike(f"%{busca}%") |
66             Produto.codigo_barras.ilike(f"%{busca}%")
67         )
68     if estoque_baixo:
69         query = query.filter(Produto.estoque_atual <= Produto.estoque_minimo)
70     return query.order_by(Produto.nome).all()
71
72
73 @router.get("/{produto_id}", response_model=ProdutoResponse)
74 def get_produto(
75     produto_id: int,
76     empresa_documento: str = Depends(get_empresa_from_user),
77     current_user: Usuario = Depends(get_current_user),
78     db: Session = Depends(get_db_salto)
79 ):
80     produto = db.query(Produto).filter(
81         Produto.id == produto_id,
82         Produto.empresa_documento == empresa_documento
83     ).first()
84     if not produto:
85         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
86     return produto
87
88
89 @router.put("/{produto_id}", response_model=ProdutoResponse)
90 def atualizar_produto(
91     produto_id: int,
92     dados: ProdutoUpdate,
93     empresa_documento: str = Depends(get_empresa_from_user),
94     current_user: Usuario = Depends(get_current_user),
95     db: Session = Depends(get_db_salto)
96 ):
97     produto = db.query(Produto).filter(
98         Produto.id == produto_id,
99         Produto.empresa_documento == empresa_documento
100     ).first()
101     if not produto:
102         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
103
104     update_data = dados.model_dump(exclude_unset=True)
105     for key, value in update_data.items():
106         setattr(produto, key, value)
107     db.commit()
108     db.refresh(produto)
109     return produto
110
111
112 @router.delete("/{produto_id}")
113 def desativar_produto(
114     produto_id: int,
115     empresa_documento: str = Depends(get_empresa_from_user),
116     current_user: Usuario = Depends(require_role("ADMIN", "GERENTE")),
117     db: Session = Depends(get_db_salto)
118 ):
119     produto = db.query(Produto).filter(
120         Produto.id == produto_id,
121         Produto.empresa_documento == empresa_documento
122     ).first()
123     if not produto:
124         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
125     produto.ativo = False
126     db.commit()
127     return {"status": "Produto desativado"}
128
129
130 # =====
131 # MOVIMENTACAO DE ESTOQUE
132 # =====
133
134 @router.post("/movimentacao")

```

```

135 def movimentar_estoque(
136     mov: EstoqueMovimentacaoCreate,
137     empresa_documento: str = Depends(get_empresa_from_user),
138     current_user: Usuario = Depends(get_current_user),
139     db: Session = Depends(get_db_salto)
140 ):
141     produto = db.query(Produto).filter(
142         Produto.id == mov.produto_id,
143         Produto.empresa_documento == empresa_documento
144     ).first()
145     if not produto:
146         raise HTTPException(status_code=404, detail="Produto nao encontrado.")
147
148     if mov.tipo == "SAIDA" and produto.estoque_atual < mov.quantidade:
149         raise HTTPException(status_code=400, detail="Estoque insuficiente.")
150
151     # Atualizar estoque
152     if mov.tipo == "ENTRADA":
153         produto.estoque_atual += mov.quantidade
154     elif mov.tipo == "SAIDA":
155         produto.estoque_atual -= mov.quantidade
156     elif mov.tipo == "AJUSTE":
157         produto.estoque_atual = mov.quantidade # Ajuste define o valor absoluto
158
159     # Registrar movimentacao
160     reg = EstoqueMovimentacao(
161         empresa_documento=empresa_documento,
162         produto_id=mov.produto_id,
163         tipo=mov.tipo,
164         quantidade=mov.quantidade,
165         motivo=mov.motivo,
166         usuario=current_user.nome
167     )
168     db.add(reg)
169     db.commit()
170     return {"status": "Movimentacao registrada", "estoque_atual": float(produto.estoque_atual)}
171
172
173 @router.get("/{produto_id}/movimentacoes")
174 def historico_movimentacoes(
175     produto_id: int,
176     empresa_documento: str = Depends(get_empresa_from_user),
177     current_user: Usuario = Depends(get_current_user),
178     db: Session = Depends(get_db_salto)
179 ):
180     return db.query(EstoqueMovimentacao).filter(
181         EstoqueMovimentacao.produto_id == produto_id,
182         EstoqueMovimentacao.empresa_documento == empresa_documento
183     ).order_by(EstoqueMovimentacao.data_movimentacao.desc()).limit(50).all()
184
185
186 @router.get("/estoque/alertas")
187 def alertas_estoque(
188     empresa_documento: str = Depends(get_empresa_from_user),
189     current_user: Usuario = Depends(get_current_user),
190     db: Session = Depends(get_db_salto)
191 ):
192     """Retorna produtos com estoque abaixo do minimo."""
193     return db.query(Produto).filter(
194         Produto.empresa_documento == empresa_documento,
195         Produto.ativo == True,
196         Produto.estoque_atual <= Produto.estoque_minimo
197     ).all()
198

```

rodar-backend.bat

```
1 @echo off
2 chcp 65001 >nul 2>&1
3 title SaltoGestao - Backend
4 color 0A
5
6 echo =====
7 echo     SALTO GESTAO - Backend FastAPI
8 echo =====
9 echo.
10
11 cd /d "%~dp0\backendPy"
12
13 echo [1] Instalando dependências Python...
14 pip install -r requirements.txt 2>nul
15 echo.
16
17 echo [2] Iniciando servidor...
18 echo.
19 echo     API: http://localhost:8080
20 echo     Docs: http://localhost:8080/docs
21 echo.
22 echo     Para parar: Ctrl+C
23 echo.
24
25 python dev.py
26
27 pause
28
```

```
1 # @ Sites
2
3 Esta pasta armazena **sites completos** criados para o Corumbá Sistemas / SaltoGestão.
4
5 ## O que vai aqui
6
7 - **Landing pages** – páginas de vendas, captação de leads
8 - **Sites institucionais** – páginas para barbearias, clientes, etc.
9 - **Páginas de demonstração** – demos, portfolios, Catálogos online
10
11 ## Estrutura por site
12
13 Cada site fica em sua própria subpasta:
14
15 ```
16 sites/
17 └─ barbearia-template-1/
18 └─ barbearia-template-2/
19 └─ landing-saltdgestao/
20   ...
21 ```
22
23 Cada subpasta é um site independente com seus próprios arquivos HTML, CSS, JS, imagens, etc.
24
25 > ⚠️ **Nota:** O frontend principal do SaltoGestão fica em `frontend-web/`. Esta pasta é para **sites separados** criados sob demanda.
26
```

```

1 """
2 =====
3 Teste de 100 Clientes - SaltoGestão Backend
4 =====
5 Insere 100 clientes via API operacional (empresa CNPJ: 12.345.678/0001-90)
6 Pré-requisitos:
7   1. Backend rodando em localhost:8080
8   2. Usuário colaborador criado na empresa
9 Uso: python3 testar_100_clientes.py
10 """
11 import requests
12 import json
13 import time
14 import sys
15
16 # =====
17 # CONFIGURAÇÃO - API do SaltoGestão
18 # =====
19 API_BASE = "http://localhost:8080"
20 CNPJ_EMPRESA = "12.345.678/0001-90" # CNPJ da empresa de teste
21
22 # =====
23 # ETAPA 1: CRIAR EMPRESA E USUÁRIO
24 # =====
25 def preparar_ambiente():
26     """Cria empresa e usuário colaborador se não existirem."""
27     # Cria um admin primeiro para gerar token de cadastro
28     # Usa o endpoint de login operacional
29     try:
30         # Tenta criar um usuário colaborador via seed direto no banco
31         import os
32         os.chdir("/opt/projects/SaltoGestao/backendPy")
33         sys.path.insert(0, "/opt/projects/SaltoGestao/backendPy")
34
35         from database import SessionLocal, engine
36         from models_core import BaseCore, Usuario, AdminCorumba, Empresa
37         from security import hash_senha
38
39         BaseCore.metadata.create_all(bind=engine)
40         db = SessionLocal()
41
42         # Cria empresa se não existe
43         empresa = db.query(Empresa).filter(Empresa.documento == CNPJ_EMPRESA).first()
44         if not empresa:
45             empresa = Empresa(
46                 razao_social="Empresa Teste LTDA",
47                 nome_fantasia="Empresa Teste",
48                 documento=CNPJ_EMPRESA,
49                 email="contato@empresateste.com",
50                 telefone="62999990000",
51                 ativo=True
52             )
53             db.add(empresa)
54             db.commit()
55             print(f"✓ Empresa criada: {empresa.nome_fantasia}")
56
57         # Cria usuário admin da empresa se não existe
58         user = db.query(Usuario).filter(
59             Usuario.username == "admin_empresa",
60             Usuario.empresa_documento == CNPJ_EMPRESA
61         ).first()
62         if not user:
63             user = Usuario(
64                 nome="Admin Empresa",
65                 username="admin_empresa",
66                 email="admin@empresateste.com",
67                 senha_hash=hash_senha("empresa123"),
68                 papel="ADMIN",
69                 empresa_documento=CNPJ_EMPRESA,
70                 ativo=True
71             )
72             db.add(user)
73             db.commit()
74             print(f"✓ Usuário criado: admin_empresa / empresa123 (ADMIN)")
75
76         # Se não existe admin Corumbá, cria um
77         admin = db.query(AdminCorumba).filter(AdminCorumba.email == "admin@corumba.com").first()
78         if not admin:
79             admin = AdminCorumba(
80                 nome="Admin Corumbá",
81                 email="admin@corumba.com",
82                 senha_hash=hash_senha("corumba123"),
83                 papel="ADMIN",
84                 ativo=True
85             )
86             db.add(admin)
87             db.commit()
88             print(f"✓ Admin Corumbá criado: admin@corumba.com / corumba123")
89
90         db.close()
91         return True
92     except Exception as e:
93         print(f"✗ Erro ao preparar ambiente: {e}")
94         return False
95
96 # =====
97 # ETAPA 2: LOGIN
98 # =====
99
100 def fazer_login():
101     """Faz login como colaborador da empresa."""
102     credenciais = [
103         {"username": "admin_empresa", "senha": "empresa123"},
104         {"email": "admin@corumbasistemas.com.br", "senha": "corumba2026"},
105         {"email": "vendedor@corumbasistemas.com.br", "senha": "vendedor123"},
106     ]
107     for cred in credenciais:
108         try:
109             r = requests.post(f"{API_BASE}/api/login", json=cred, timeout=5)
110             if r.status_code == 200:
111                 data = r.json()
112                 token = data.get("access_token") or data.get("token")
113                 if token:
114                     print(f"✓ Login OK: {cred}")
115                     return token, data.get("usuario"), data.get("admin", {})
116             except Exception:
117                 continue
118     print("✗ Falha no login. Verifique se o backend está rodando.")
119     return None, None
120
121 # =====
122 # GERA DADOS DE 100 CLIENTES
123 # =====
124
125 def gerar_clientes_100():
126     """Gera 100 clientes com dados variados."""
127     import random
128     random.seed(42)
129
130     nomes_base = [
131         "João", "Maria", "José", "Ana", "Pedro", "Carla", "Lucas", "Fernanda",
132         "Marcos", "Juliana", "Rafael", "Patricia", "Bruno", "Camila", "Diego",
133         "Amanda", "Gustavo", "Larissa", "Thiago", "Beatriz", "Felipe", "Mariana",
134         "Rodrigo", "Letícia", "André", "Vanessa", "Marcelo", "Tatiana", "Leonardo",

```

```

135     "Priscila", "Eduardo", "Sabrina", "Ricardo", "Isabela", "Fábio", "Natália",
136     "Alexandre", "Daniela", "Guilherme", "Bianca", "Vinícius", "Lorena",
137     "Matheus", "Gabriela", "Samuel", "Helena", "Isaac", "Valentina", "Daniel",
138     "Laura", "Henrique", "Manuela", "Caio", "Sophia", "Enzo", "Heloísa",
139     "Miguel", "Alice", "Arthur", "Júlia", "Bernardo", "Luísa", "Heitor",
140     "Isabella", "Davi", "Lara", "Lorenzo", "Cecília", "Theo", "Eloá",
141     "Gabriel", "Yasmin", "Noah", "Rafaela", "Benjamin", "Giovanna", "Anthony",
142     "Clara", "Joaquim", "Melissa", "Nicolas", "Rebeca", "Lucca", "Agatha",
143     "Ryan", "Esther", "Ian", "Nicole", "David", "Sarah", "Pedro", "Vitória"
144 ]
145
146 sobrenomes = [
147     "Silva", "Santos", "Oliveira", "Souza", "Rodrigues", "Ferreira",
148     "Alves", "Pereira", "Lima", "Gomes", "Costa", "Ribeiro", "Martins",
149     "Carvalho", "Araújo", "Melo", "Barbosa", "Rocha", "Dias", "Nascimento"
150 ]
151
152 clientes = []
153 usados_docs = set()
154
155 for i in range(100):
156     nome = nomes_base[i % len(nomes_base)]
157     sobrenome = sobrenomes[i % len(sobrenomes)]
158     nome_completo = f"{nome} {sobrenome} {sobrenomes[(i*3+7) % len(sobrenomes)]}"
159
160     # Gera CPF único
161     while True:
162         cpf = f"{random.randint(100,999)}{random.randint(100,999)}{random.randint(100,999)}{random.randint(0,99):02d}"
163         if cpf not in usados_docs:
164             usados_docs.add(cpf)
165             break
166
167     bairro = random.choice([
168         "Centro", "Jardim América", "Setor Oeste", "Setor Bueno",
169         "Jardim Goiás", "Campinas", "Vila Aurora", "Jardim Europa"
170     ])
171
172     cidade = random.choice(["Aparecida de Goiânia", "Goiânia", "Anápolis"])
173     estado = "GO"
174     cep = f"{random.randint(10000,99999)}-{random.randint(100,999)}"
175
176     ddd = random.choice(["62", "11", "21", "31", "41", "51"])
177     telefone = f"{ddd}9{random.randint(8000,9999)}-{random.randint(1000,9999)}"
178
179     email = f"{nome.lower()}+{i+1}@email.com"
180
181     cliente = {
182         "nome": nome_completo.strip(),
183         "telefone": telefone,
184         "email": email,
185         "cpf_cnpj": cpf,
186         "endereço": f"Rua {random.choice(['Flores', 'Brasil', 'São João', 'Goiás', 'Palmeiras'])} {random.randint(1,999)}",
187         "cidade": cidade,
188         "estado": estado,
189         "cep": cep,
190         "data_nascimento": f"{random.randint(1960,2005)}-{random.randint(1,12):02d}-{random.randint(1,28):02d}"
191     }
192     clientes.append(cliente)
193
194 return clientes
195
196
197 # =====
198 # TESTE PRINCIPAL
199 # =====
200 def main():
201     print("\n * 60)
202     print("\n TESTE: 100 CLIENTES - SALTOSTGESTÃO (API Operacional)")
203     print("\n * 60)
204
205     # 1. Prepara ambiente
206     print("\n * ETAPA 1: Preparando ambiente...")
207     preparar_ambiente()
208
209     # 2. Login
210     print("\n * ETAPA 2: Login")
211     token, usuario = fazer_login()
212     if not token:
213         print("\n * Não foi possível fazer login. Abortando.")
214         return
215
216     headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}
217     print(f"\n * Usuário: {usuario.get('nome', '?')} | Papel: {usuario.get('apel', '?')}")
218     empresa_doc = usuario.get('empresa_documento', CNPJ_EMPRESA)
219     print(f"\n * Empresa: {empresa_doc}")
220
221     # 3. Gera 100 clientes
222     print("\n * ETAPA 3: Gerando 100 clientes")
223     clientes = gerar_clientes_100()
224     print(f"\n * {len(clientes)} clientes gerados")
225
226     # 4. Insere todos via POST /api/clientes/
227     print("\n * ETAPA 4: Inserindo clientes via POST /api/clientes/")
228     inseridos = 0
229     falhas = 0
230     erros_detalhados = []
231     ids_inseridos = []
232
233     inicio = time.time()
234     for i, cliente in enumerate(clientes):
235         try:
236             r = requests.post(
237                 f"{API_BASE}/api/clientes/",
238                 json=cliente,
239                 headers=headers,
240                 timeout=10
241             )
242             if r.status_code in [200, 201]:
243                 inseridos += 1
244                 resp_data = r.json()
245                 cid = resp_data.get("id")
246                 if cid:
247                     ids_inseridos.append(cid)
248             else:
249                 falhas += 1
250                 if falhas <= 5:
251                     erros_detalhados.append(f"#{i+1} {cliente['nome']}[:20]: HTTP {r.status_code} - {r.text[:80]}")
252         except Exception as e:
253             falhas += 1
254             if falhas <= 5:
255                 erros_detalhados.append(f"#{i+1} {cliente['nome']}[:20]: {str(e)[:80]}")
256
257     if (i + 1) % 20 == 0:
258         print(f"\n * {i+1}/100 inseridos... ({inseridos} OK, {falhas} falhas)")
259
260     tempo = time.time() - inicio
261     print(f"\n * Inseridos: {inseridos}/100")
262     if falhas > 0:
263         print(f"\n * Falhas: {falhas}")
264         for err in erros_detalhados:
265             print(f"\n * {err}")
266     print(f"\n * Tempo: {tempo:.1f}s ({tempo/max(inseridos,1)*1000:.0f}ms/cliente)")
267
268     # 5. Validação: Busca todos
269     print("\n * ETAPA 5: Validação - GET /api/clientes/")
270     try:
271         r = requests.get(
272             f"{API_BASE}/api/clientes/?ativo=true&ordem=nome",

```

```

273     headers=headers, timeout=10
274 )
275 if r.status_code == 200:
276     todos = r.json()
277     print(f"    ✓ Total na API: {len(todos)} clientes")
278
279     # Mostra 5 primeiros
280     print("\n    ✎ Primeiros 5 clientes:")
281     for c in todos[:5]:
282         print(f"        ✓ ID:{c.get('id')} | {c.get('nome','?')[:30]} | {c.get('cpf_cnpj','?')}")
283
284     print(f"\n    ✎ Últimos 5 clientes:")
285     for c in todos[-5:]:
286         print(f"        ✓ ID:{c.get('id')} | {c.get('nome','?')[:30]} | {c.get('cpf_cnpj','?')}")
287
288     else:
289         print(f"    ✕ Erro: HTTP {r.status_code} - {r.text[:100]}")
290 except Exception as e:
291     print(f"    ✕ Erro: {e}")
292
293 # 6. Teste de busca por ID
294 if ids_inseridos:
295     print("\n✎ ETAPA 6: Teste de busca por ID")
296     test_id = ids_inseridos[0]
297     try:
298         r = requests.get(f"{API_BASE}/api/clientes/{test_id}", headers=headers, timeout=5)
299         if r.status_code == 200:
300             c = r.json()
301             print(f"    ✓ GET /api/clientes/{test_id}: {c.get('nome','?')[:40]}")
302         else:
303             print(f"    ✕ Falha: HTTP {r.status_code}")
304     except Exception as e:
305         print(f"    ✕ Erro: {e}")
306
307 # 7. Teste de atualização
308 if len(ids_inseridos) > 1:
309     print("\n✎ ETAPA 7: Teste de atualização (PUT)")
310     test_id = ids_inseridos[1]
311     try:
312         r = requests.put(
313             f"{API_BASE}/api/clientes/{test_id}",
314             json={"telefone": "6299999-8888"},
315             headers=headers,
316             timeout=5
317         )
318         if r.status_code in [200, 201]:
319             c = r.json()
320             print(f"    ✓ PUT /api/clientes/{test_id}: tel={c.get('telefone','?')}")
321         else:
322             print(f"    ✕ Falha: HTTP {r.status_code}")
323     except Exception as e:
324         print(f"    ✕ Erro: {e}")
325
326 # 8. Teste de desativação
327 if len(ids_inseridos) > 2:
328     print("\n✎ ETAPA 8: Teste de desativação (DELETE)")
329     test_id = ids_inseridos[2]
330     try:
331         r = requests.delete(f"{API_BASE}/api/clientes/{test_id}", headers=headers, timeout=5)
332         if r.status_code in [200, 204]:
333             print(f"    ✓ DELETE /api/clientes/{test_id}: desativado")
334             # Verifica se está inativo
335             r2 = requests.get(
336                 f"{API_BASE}/api/clientes/?ativo=false",
337                 headers=headers, timeout=5
338             )
339             if r2.status_code == 200:
340                 inativos = r2.json()
341                 print(f"    ✓ Clientes inativos: {len(inativos)}")
342             else:
343                 print(f"    ✕ Falha: HTTP {r2.status_code}")
344     except Exception as e:
345         print(f"    ✕ Erro: {e}")
346
347 # 9. RESULTADO FINAL
348 print("\n" + "=" * 60)
349 print("📊 RESULTADO FINAL")
350 print("=" * 60)
351 print(f"API: {API_BASE}")
352 print(f"Empresa: {empresa_doc}")
353 print(f>Inseridos: {inseridos}/100")
354 print(f>Falhas: {falhas}")
355 print(f>Tempo total: {tempo:.1f}s")
356 print(f>Performance: {tempo/max(inseridos,1)*1000:.0f}ms/cliente")
357
358 if inseridos == 100:
359     print("\n    ✎ PASSOU! Todos os 100 clientes inseridos com sucesso!")
360     print("    ✓ API operacional SaltoGestão funcionando corretamente!")
361 elif inseridos > 90:
362     print(f"\n    ✓ QUASE LÁ! {inseridos}/100 clientes inseridos.")
363 else:
364     print(f"\n    ⚠ PROBLEMA: {falhas} clientes falharam")
365
366 print("\n🏁 Teste concluído!")
367
368 if __name__ == "__main__":
369     main()
370

```

```
1 #!/bin/bash
2 # =====
3 # update.sh - Atualiza todos os projetos Corumbá Sistemas
4 # =====
5 # Uso: bash update.sh
6 # =====
7
8 set -e
9
10 GREEN='\033[0;32m'
11 YELLOW='\033[1;33m'
12 RED='\033[0;31m'
13 NC='\033[0m'
14
15 log() { echo -e "${GREEN}✓ $1${NC}"; }
16 warn() { echo -e "${YELLOW}⚠ $1${NC}"; }
17 err() { echo -e "${RED}✗ $1${NC}"; }
18
19 SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
20
21 echo "=====
22 echo "🔄 Atualizando Corumbá Sistemas"
23 echo "=====
24 echo ""
25
26 # 1) SaltoGestão (branch: main)
27 echo "🔄 [1/3] SaltoGestão..."
28 cd "$SCRIPT_DIR/SaltoGestao"
29 if git pull origin main 2>/dev/null; then
30     log "SaltoGestão atualizado"
31 else
32     warn "SaltoGestão: sem alterações ou erro"
33 fi
34
35 echo ""
36
37 # 2) ConsultaAgenda (branch: master)
38 echo "🔄 [2/3] ConsultaAgenda..."
39 cd "$SCRIPT_DIR/consultaagenda"
40 if git pull origin master 2>/dev/null; then
41     log "ConsultaAgenda atualizado"
42 else
43     warn "ConsultaAgenda: sem alterações ou erro"
44 fi
45
46 echo ""
47
48 # 3) MesaPronta (branch: main)
49 echo "🔄 [3/3] MesaPronta..."
50 cd "$SCRIPT_DIR/coruba-food"
51 if git pull origin main 2>/dev/null; then
52     log "MesaPronta atualizado"
53 else
54     warn "MesaPronta: sem alterações ou erro"
55 fi
56
57 echo ""
58 echo "=====
59 log "Tudo atualizado!"
60 echo "=====
61
```



```

1 """
2 web/app.py - Paine Web SaltoGestão (alternativa ao desktop tkinter)
3 Serve HTML/CSS/JS que conecta à API FastAPI via fetch()
4 """
5 from fastapi import FastAPI, Request
6 from fastapi.staticfiles import StaticFiles
7 from fastapi.responses import HTMLResponse
8 from fastapi.templating import Jinja2Templates
9 from fastapi.middleware.cors import CORSMiddleware
10 import os
11
12 app = FastAPI(title="SaltoGestão Web", version="1.0.0")
13
14 app.add_middleware(
15     CORSMiddleware,
16     allow_origins=["*"],
17     allow_credentials=True,
18     allow_methods=["*"],
19     allow_headers=["*"],
20 )
21
22 # Caminho para pasta static
23 STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "static")
24 os.makedirs(STATIC_DIR, exist_ok=True)
25 app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")
26
27
28 @app.get("/", response_class=HTMLResponse)
29 def painel(request: Request):
30     """Painel web principal - SaltoGestão"""
31     return HTMLResponse(content=PAGE_HTML)
32
33
34 # =====
35 # PÁGINA HTML COMPLETA (single-page app)
36 # =====
37 PAGE_HTML = """
38 <!DOCTYPE html>
39 <html lang="pt-BR">
40 <head>
41 <meta charset="UTF-8">
42 <meta name="viewport" content="width=device-width, initial-scale=1.0">
43 <title> SaltoGestão - Painel Administrativo</title>
44 <style>
45 :root {
46     --bg: #0b1a30; --bg2: #0f2044; --bg3: #142a55;
47     --card: #112240; --border: #1e3a5f;
48     --primary: #3b82f6; --primary-hover: #2563eb;
49     --success: #10b981; --warning: #f59e0b; --danger: #ef4444;
50     --text: #e2e8f0; --text2: #94a3b8; --text3: #64748b;
51 }
52 * { margin:0; padding:0; box-sizing:border-box; }
53 body { font-family: 'Segoe UI', system-ui, sans-serif; background: var(--bg); color: var(--text); min-height:100vh; }
54
55 /* LOGIN */
56 .login-bg { display:flex; align-items:center; justify-content:center; min-height:100vh; background: linear-gradient(135deg, #0b1a30 0%, #1a1a4e 100%); }
57 .login-box { background: var(--card); border:1px solid var(--border); border-radius:16px; padding:40px; width:420px; box-shadow: 0 20px 60px rgba(0,0,0,0.3); }
58 .login-box h1 { font-size:28px; color:var(--primary); text-align:center; margin-bottom:4px; }
59 .login-box .sub { color:var(--text3); text-align:center; margin-bottom:30px; font-size:14px; }
60 .form-group { margin-bottom:16px; }
61 .form-group label { display:block; color:var(--text2); font-size:13px; margin-bottom:6px; }
62 .form-group input, .form-group select {
63     width:100%; padding:12px 16px; border-radius:8px; border:1px solid var(--border);
64     background:var(--bg); color:var(--text); font-size:14px; outline:none; transition:.2s;
65 }
66 .form-group input:focus, .form-group select:focus { border-color:var(--primary); }
67 .btn { width:100%; padding:14px; border:none; border-radius:10px; font-size:16px; font-weight:600; cursor:pointer; transition:.2s; }
68 .btn-primary { background:var(--primary); color:#fff; }
69 .btn-primary:hover { background:var(--primary-hover); }
70 .btn-success { background:var(--success); color:#fff; }
71 .btn-danger { background:var(--danger); color:#fff; }
72 .btn-sm { padding:8px 14px; font-size:13px; width:auto; }
73 .error-msg { color:var(--danger); font-size:13px; margin-top:12px; text-align:center; }
74 .success-msg { color:var(--success); font-size:13px; margin-top:12px; text-align:center; }
75
76 /* LAYOUT PRINCIPAL */
77 .app { display:none; }
78 .sidebar { position:fixed; left:0; top:0; width:240px; height:100vh; background:var(--card); border-right:1px solid var(--border); padding:24px 0; z-index:100; }
79 .sidebar h2 { color:var(--primary); font-size:20px; padding:0 24px; margin-bottom:8px; }
80 .sidebar .user-info { color:var(--text3); font-size:12px; padding:0 24px; margin-bottom:30px; }
81 .sidebar nav a { display:flex; align-items:center; gap:10px; padding:12px 24px; color:var(--text2); text-decoration:none; font-size:14px; transition:.2s; border-left:3px solid transparent; }
82 .sidebar nav a:hover, .sidebar nav a.active { background:var(--bg2); color:var(--text); border-left-color:var(--primary); }
83 .main { margin-left:240px; padding:24px; min-height:100vh; }
84
85 /* HEADER */
86 .header { display:flex; justify-content:space-between; align-items:center; margin-bottom:24px; }
87 .header h1 { font-size:24px; }
88 .header .stats { display:flex; gap:16px; }
89 .stat-card { background:var(--card); border:1px solid var(--border); border-radius:12px; padding:16px 24px; text-align:center; }
90 .stat-card .num { font-size:28px; font-weight:700; color:var(--primary); }
91 .stat-card .label { font-size:12px; color:var(--text3); margin-top:4px; }
92
93 /* TABLE */
94 .table-wrap { background:var(--card); border:1px solid var(--border); border-radius:12px; overflow:hidden; }
95 .table-wrap .table-header { display:flex; justify-content:space-between; align-items:center; padding:16px 20px; border-bottom:1px solid var(--border); }
96 .table-wrap .table-header h3 { font-size:16px; }
97 .search-box { display:flex; gap:8px; }
98 .search-box input { padding:8px 14px; border-radius:8px; border:1px solid var(--border); background:var(--bg); color:var(--text); font-size:13px; width:240px; outline:none; }
99 table { width:100%; border-collapse:collapse; }
100 th { text-align:left; padding:12px 16px; font-size:12px; color:var(--text3); text-transform:uppercase; border-bottom:1px solid var(--border); background:var(--bg2); }
101 td { padding:12px 16px; font-size:13px; border-bottom:1px solid rgba(30,58,95,0.5); }
102 tr:hover { background:var(--bg2); }
103 .badge { display:inline-block; padding:3px 10px; border-radius:20px; font-size:11px; font-weight:600; }
104 .badge-success { background:rgba(16,185,129,0.15); color:var(--success); }
105 .badge-danger { background:rgba(239,68,68,0.15); color:var(--danger); }
106 .badge-info { background:rgba(59,130,246,0.15); color:var(--primary); }
107 .actions { display:flex; gap:6px; }
108
109 /* MODAL */
110 .modal-overlay { display:none; position:fixed; inset:0; background:rgba(0,0,0,0.6); z-index:200; align-items:center; justify-content:center; }
111 .modal-overlay.open { display:flex; }
112 .modal { background:var(--card); border:1px solid var(--border); border-radius:16px; padding:32px; width:520px; max-height:80vh; overflow-y:auto; }
113 .modal h3 { margin-bottom:20px; font-size:18px; }
114 .modal .form-group { margin-bottom:14px; }
115 .modal .actions { justify-content:flex-end; margin-top:20px; gap:8px; }
116
117 /* PAGINATION */
118 .pagination { display:flex; justify-content:center; gap:6px; margin-top:16px; }
119 .pagination button { padding:8px 14px; border:1px solid var(--border); background:var(--bg); color:var(--text); border-radius:8px; cursor:pointer; font-size:13px; }
120 .pagination button.active { background:var(--primary); border-color:var(--primary); }
121 .pagination button:hover:not(.active) { background:var(--bg2); }
122
123 /* NOTIFICATION */
124 .toast { position:fixed; top:20px; right:20px; padding:14px 24px; border-radius:10px; font-size:14px; z-index:300; animation: slideIn .3s ease; }
125 .toast-success { background:var(--success); color:#fff; }
126 .toast-error { background:var(--danger); color:#fff; }
127 @keyframes slideIn { from { transform:translateX(100%); opacity:0; } to { transform:translateX(0); opacity:1; } }
128 </style>
129 </head>
130 <body>
131
132 <!-- LOGIN -->
133 <div id="loginPage" class="login-bg">

```

```
134 <div class="login-box">
135 <h1>■ SaltoGestão</h1>
136 <div class="sub">Sistema de Gestão Comercial</div>
137
138 <div class="form-group">
139 <label>Servidor da API:</label>
140 <input type="text" id="apiUrl" value="http://191.252.210.235:8080" placeholder="http://localhost:8080">
141 </div>
142
143 <div class="form-group">
144 <label>Usuário (username):</label>
145 <input type="text" id="username" placeholder="admin_empresa" value="admin_empresa">
146 </div>
147
148 <div class="form-group">
149 <label>Senha:</label>
150 <input type="password" id="senha" placeholder="*****" value="empresa123">
151 </div>
152
153 <button class="btn btn-primary" onclick="fazerLogin()">⬆ Entrar</button>
154
155 <div id="loginError" class="error-msg"></div>
156 </div>
157 </div>
158
159 <!-- APP -->
160 <div id="app" class="app">
161 <!-- SIDEBAR -->
162 <div class="sidebar">
163 <h2>■ SaltoGestão</h2>
164 <div class="user-info" id="userInfo">Carregando...</div>
165 <nav>
166 <a href="#" class="active" onclick="showPage('dashboard', event)">🏠 Dashboard</a>
167 <a href="#" onclick="showPage('clientes', event)">👤 Clientes</a>
168 <a href="#" onclick="showPage('produtos', event)">📦 Produtos</a>
169 <a href="#" onclick="showPage('vendas', event)">💰 Vendas</a>
170 <a href="#" onclick="showPage('fornecedores', event)">🏢 Fornecedores</a>
171 <a href="#" onclick="showPage('relatorios', event)">📊 Relatórios</a>
172 <a href="#" onclick="sair()" style="margin-top:30px; color:var(--danger);">🚪 Sair</a>
173 </nav>
174 </div>
175
176 <!-- MAIN CONTENT -->
177 <div class="main">
178
179 <!-- DASHBOARD -->
180 <div id="page-dashboard" class="page">
181 <div class="header">
182 <h1>🏠 Dashboard</h1>
183 </div>
184 <div class="stats">
185 <div class="stat-card"><div class="num" id="stClientes">0</div><div class="label">Clientes</div></div>
186 <div class="stat-card"><div class="num" id="stProdutos">0</div><div class="label">Produtos</div></div>
187 <div class="stat-card"><div class="num" id="stVendas">0</div><div class="label">Vendas</div></div>
188 <div class="stat-card"><div class="num" id="stFornecedores">0</div><div class="label">Fornecedores</div></div>
189 </div>
190 <div class="table-wrap" style="margin-top:24px;">
191 <div class="table-header"><h3>Últimos clientes cadastrados</h3></div>
192 <table>
193 <thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>CPF/CNPJ</th><th>Status</th></tr></thead>
194 <tbody id="dashboardClientes"></tbody>
195 </table>
196 </div>
197 </div>
198
199 <!-- CLIENTES -->
200 <div id="page-clientes" class="page" style="display:none;">
201 <div class="header">
202 <h1>👤 Clientes</h1>
203 </div>
204 <div class="table-wrap">
205 <div class="table-header">
206 <div class="search-box">
207 <input type="text" id="searchClientes" placeholder="🔍 Buscar cliente..." oninput="buscarClientes()" />
208 <select id="filterClientes" onchange="buscarClientes()">
209 <option value="true">Ativos</option>
210 <option value="false">Inativos</option>
211 <option value="">Todos</option>
212 </select>
213 </div>
214 <button class="btn btn-success btn-sm" onclick="abrirModalCliente()">➕ Novo Cliente</button>
215 </div>
216 <table>
217 <thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>E-mail</th><th>CPF/CNPJ</th><th>Cidade</th><th>Pontos</th><th>Status</th><th>Ações</th></tr></thead>
218 <tbody id="tbodyClientes"></tbody>
219 </table>
220 <div class="pagination" id="pagClientes"></div>
221 </div>
222 </div>
223
224 <!-- PRODUTOS -->
225 <div id="page-produtos" class="page" style="display:none;">
226 <div class="header"><h1>📦 Produtos</h1></div>
227 <div class="table-wrap">
228 <div class="table-header">
229 <div class="search-box">
230 <input type="text" id="searchProdutos" placeholder="🔍 Buscar produto..." oninput="buscarProdutos()" />
231 </div>
232 <button class="btn btn-success btn-sm" onclick="abrirModalProduto()">➕ Novo Produto</button>
233 </div>
234 <table>
235 <thead><tr><th>ID</th><th>Nome</th><th>SKU</th><th>Preço Venda</th><th>Estoque</th><th>Unidade</th><th>Status</th><th>Ações</th></tr></thead>
236 <tbody id="tbodyProdutos"></tbody>
237 </table>
238 </div>
239 </div>
240
241 <!-- VENDAS -->
242 <div id="page-vendas" class="page" style="display:none;">
243 <div class="header"><h1>💰 Vendas</h1></div>
244 <div class="table-wrap">
245 <div class="table-header">
246 <div class="search-box">
247 <input type="text" placeholder="🔍 Buscar venda..." />
248 </div>
249 <button class="btn btn-success btn-sm" onclick="abrirModalVenda()">➕ Nova Venda</button>
250 </div>
251 <table>
252 <thead><tr><th>ID</th><th>Data</th><th>Cliente</th><th>Total</th><th>Pagamento</th><th>Status</th><th>Ações</th></tr></thead>
253 <tbody id="tbodyVendas"></tbody>
254 </table>
255 </div>
256 </div>
257
258 <!-- FORNECEDORES -->
259 <div id="page-fornecedores" class="page" style="display:none;">
260 <div class="header"><h1>🏢 Fornecedores</h1></div>
261 <div class="table-wrap">
262 <div class="table-header">
263 <div class="search-box">
264 <input type="text" placeholder="🔍 Buscar fornecedor..." />
265 </div>
266 <button class="btn btn-success btn-sm" onclick="abrirModalFornecedor()">➕ Novo Fornecedor</button>
267 </div>
268 <table>
269 <thead><tr><th>ID</th><th>Nome Fantasia</th><th>CNPJ</th><th>Telefone</th><th>E-mail</th><th>Status</th><th>Ações</th></tr></thead>
270 <tbody id="tbodyFornecedores"></tbody>
271 </table>
```

```

272 </div>
273 </div>
274
275 <!-- RELATÓRIOS -->
276 <div id="page-relatorios" class="page" style="display:none;">
277   <div class="header"><h1> Relatórios</h1></div>
278   <div class="stats">
279     <div class="stat-card"><div class="num" id="rClientes">0</div><div class="label">Total Clientes</div></div>
280     <div class="stat-card"><div class="num" id="rProdutos">0</div><div class="label">Total Produtos</div></div>
281     <div class="stat-card"><div class="num" id="rVendas">0</div><div class="label">Total Vendas</div></div>
282     <div class="stat-card"><div class="num" id="rFornecedores">0</div><div class="label">Total Fornecedores</div></div>
283   </div>
284   <div class="table-wrap" style="margin-top:24px;">
285     <div class="table-header"><h3>Top 10 Clientes por Fidelidade</h3></div>
286     <table>
287       <thead><tr><th>#</th><th>Cliente</th><th>Pontos</th><th>Total Gasto</th></tr></thead>
288       <tbody id="tbodyRanking"></tbody>
289     </table>
290   </div>
291 </div>
292 </div>
293 </div>
294
295 <!-- MODAL CLIENTE -->
296 <div id="modalCliente" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalCliente')">
297   <div class="modal">
298     <h3 id="modalClienteTitulo">Novo Cliente</h3>
299     <input type="hidden" id="cId" />
300     <div class="form-group"><label>Nome:*</label><input type="text" id="cNome" placeholder="Nome completo"></div>
301     <div class="form-group"><label>Telefone:</label><input type="text" id="cTelefone" placeholder="(62) 99999-0000"></div>
302     <div class="form-group"><label>E-mail:</label><input type="email" id="cEmail" placeholder="email@exemplo.com"></div>
303     <div class="form-group"><label>CPF/CNPJ:</label><input type="text" id="cCpfCnpj" placeholder="000.000.000-00"></div>
304     <div class="form-group"><label>Endereço:</label><input type="text" id="cEndereco" placeholder="Rua, número"></div>
305     <div class="form-group"><label>Cidade:</label><input type="text" id="cCidade" placeholder="Cidade"></div>
306     <div class="form-group"><label>Estado:</label><input type="text" id="cEstado" placeholder="GO" maxlength="2"></div>
307     <div class="form-group"><label>CEP:</label><input type="text" id="cCep" placeholder="00000-000"></div>
308     <div class="actions">
309       <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalCliente')">Cancelar</button>
310       <button class="btn btn-success btn-sm" onclick="salvarCliente()"> Salvar</button>
311     </div>
312   </div>
313 </div>
314
315 <!-- MODAL PRODUTO -->
316 <div id="modalProduto" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalProduto')">
317   <div class="modal">
318     <h3 id="modalProdutoTitulo">Novo Produto</h3>
319     <input type="hidden" id="pId" />
320     <div class="form-group"><label>Nome:*</label><input type="text" id="pNome" placeholder="Nome do produto"></div>
321     <div class="form-group"><label>Descrição:</label><input type="text" id="pDescricao" placeholder="Descrição"></div>
322     <div class="form-group"><label>SKU:</label><input type="text" id="pSku" placeholder="SKU-001"></div>
323     <div class="form-group"><label>Preço Venda:*</label><input type="number" id="pPrecoVenda" step="0.01" placeholder="0.00"></div>
324     <div class="form-group"><label>Preço Custo:</label><input type="number" id="pPrecoCusto" step="0.01" placeholder="0.00"></div>
325     <div class="form-group"><label>Estoque Atual:</label><input type="number" id="pEstoque" step="1" placeholder="0"></div>
326     <div class="form-group"><label>Estoque Mínimo:</label><input type="number" id="pEstoqueMin" step="1" placeholder="0"></div>
327     <div class="form-group"><label>Unidade:</label>
328       <select id="pUnidade">
329         <option value="UN">UN</option><option value="KG">KG</option>
330         <option value="LT">LT</option><option value="CX">CX</option>
331       </select>
332     </div>
333     <div class="actions">
334       <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalProduto')">Cancelar</button>
335       <button class="btn btn-success btn-sm" onclick="salvarProduto()"> Salvar</button>
336     </div>
337   </div>
338 </div>
339
340 <!-- MODAL FORNECEDOR -->
341 <div id="modalFornecedor" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalFornecedor')">
342   <div class="modal">
343     <h3>Novo Fornecedor</h3>
344     <input type="hidden" id="fId" />
345     <div class="form-group"><label>Nome Fantasia:*</label><input type="text" id="fNome" placeholder="Nome do fornecedor"></div>
346     <div class="form-group"><label>Razão Social:</label><input type="text" id="fRazao" placeholder="Razão social"></div>
347     <div class="form-group"><label>CNPJ/CPF:</label><input type="text" id="fCnpj" placeholder="00.000.000/0000-00"></div>
348     <div class="form-group"><label>Telefone:</label><input type="text" id="fTelefone" placeholder="(62) 99999-0000"></div>
349     <div class="form-group"><label>E-mail:</label><input type="email" id="fEmail" placeholder="email@exemplo.com"></div>
350     <div class="form-group"><label>Endereço:</label><input type="text" id="fEndereco" placeholder="Rua, número"></div>
351     <div class="actions">
352       <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalFornecedor')">Cancelar</button>
353       <button class="btn btn-success btn-sm" onclick="salvarFornecedor()"> Salvar</button>
354     </div>
355   </div>
356 </div>
357
358 <script>
359 // =====
360 // ESTADO GLOBAL
361 // =====
362 const S = {
363   token: localStorage.getItem('token') || null,
364   usuario: JSON.parse(localStorage.getItem('usuario')) || 'null',
365   apiUrl: localStorage.getItem('apiUrl') || 'http://191.252.210.235:8080',
366   clientes: [],
367   produtos: [],
368   vendas: [],
369   fornecedores: [],
370   page: 1,
371   perPage: 20,
372   totalClientes: 0
373 };
374
375 // =====
376 // API HELPER
377 // =====
378 async function api(method, path, body) {
379   const opts = {
380     method,
381     headers: { 'Content-Type': 'application/json' }
382   };
383   if (S.token) opts.headers['Authorization'] = 'Bearer ' + S.token;
384   if (body) opts.body = JSON.stringify(body);
385   const r = await fetch(S.apiUrl + path, opts);
386   if (r.status === 401) { sair(); return null; }
387   const t = await r.text();
388   return t ? JSON.parse(t) : null;
389 }
390
391 function toast(msg, type) {
392   const d = document.createElement('div');
393   d.className = 'toast toast-' + (type || 'success');
394   d.textContent = msg;
395   document.body.appendChild(d);
396   setTimeout(() => d.remove(), 3000);
397 }
398
399 // =====
400 // LOGIN
401 // =====
402 function fazerLogin() {
403   const apiUrl = document.getElementById('apiUrl').value.trim();
404   const username = document.getElementById('username').value.trim();
405   const senha = document.getElementById('senha').value.trim();
406   const err = document.getElementById('loginError');
407   err.textContent = '';
408
409   S.apiUrl = apiUrl;

```

```

410 localStorage.setItem('apiUrl', apiUrl);
411
412 const body = username.includes('@') ? { email: username, senha } : { username, senha };
413
414 fetch(apiUrl + '/api/login', {
415   method: 'POST',
416   headers: { 'Content-Type': 'application/json' },
417   body: JSON.stringify(body)
418 })
419 .then(r => r.json())
420 .then(d => {
421   if (d.access token) {
422     S.token = d.access token;
423     S.usuario = d.usuario || d.admin;
424     localStorage.setItem('token', S.token);
425     localStorage.setItem('usuario', JSON.stringify(S.usuario));
426     entrar();
427   } else {
428     err.textContent = 'x' + (d.detail || 'Falha no login');
429   }
430 })
431 .catch(e => { err.textContent = 'x Erro de conexão: ' + e.message; });
432 }
433
434 function entrar() {
435   document.getElementById('loginPage').style.display = 'none';
436   document.getElementById('app').style.display = 'block';
437   document.getElementById('userInfo').textContent = S.usuario.nome + ' (' + S.usuario.papel + ')';
438   carregarDashboard();
439 }
440
441 function sair() {
442   S.token = null; S.usuario = null;
443   localStorage.removeItem('token'); localStorage.removeItem('usuario');
444   document.getElementById('app').style.display = 'none';
445   document.getElementById('loginPage').style.display = 'flex';
446 }
447
448 // =====
449 // NAVEGAÇÃO
450 // =====
451 function showPage(page, e) {
452   if (e) e.preventDefault();
453   document.querySelectorAll('.page').forEach(p => p.style.display = 'none');
454   document.querySelectorAll('.sidebar nav a').forEach(a => a.classList.remove('active'));
455   document.getElementById('page-' + page).style.display = 'block';
456   if (e) e.target.classList.add('active');
457
458   if (page === 'dashboard') carregarDashboard();
459   if (page === 'clientes') carregarClientes();
460   if (page === 'produtos') carregarProdutos();
461   if (page === 'vendas') carregarVendas();
462   if (page === 'fornecedores') carregarFornecedores();
463   if (page === 'relatorios') carregarRelatorios();
464 }
465
466 // =====
467 // DASHBOARD
468 // =====
469 async function carregarDashboard() {
470   const [c, p, v, f] = await Promise.all([
471     api('GET', '/api/clientes/?ativo=true&ordem=recente&limite=5'),
472     api('GET', '/api/produtos/'),
473     api('GET', '/api/vendas/'),
474     api('GET', '/api/fornecedores/')
475   ]);
476
477   if (c) {
478     document.getElementById('stClientes').textContent = c.length;
479     const tbody = document.getElementById('dashboardClientes');
480     tbody.innerHTML = c.slice(0, 5).map(x => `<tr>
481       <td>${x.id}</td><td>${x.nome || '-'}</td><td>${x.telefone || '-'}</td>
482       <td>${x.cpf_cnpj || '-'}</td>
483       <td><span class="badge ${x.ativo ? 'badge-success' : 'badge-danger'}">${x.ativo ? 'Ativo' : 'Inativo'}</span></td>
484     `).join('');
485   }
486   if (p) document.getElementById('stProdutos').textContent = p.length;
487   if (v) document.getElementById('stVendas').textContent = v.length;
488   if (f) document.getElementById('stFornecedores').textContent = f.length;
489 }
490
491 // =====
492 // CLIENTES
493 // =====
494 async function carregarClientes() {
495   const busca = document.getElementById('searchClientes').value;
496   const ativo = document.getElementById('filterClientes').value;
497   let url = '/api/clientes/?';
498   if (ativo) url += 'ativo=' + ativo + '&';
499   if (busca) url += 'busca=' + encodeURIComponent(busca) + '&';
500   url += 'ordem=nome&limite=100';
501
502   S.clientes = await api('GET', url) || [];
503   renderClientes(S.clientes);
504 }
505
506 function renderClientes(lista) {
507   const tbody = document.getElementById('tbodyClientes');
508   if (!lista.length) { tbody.innerHTML = `<tr><td colspan="9" style="text-align:center;color:var(--text3);padding:40px;">Nenhum cliente encontrado</td></tr>`; return; }
509   tbody.innerHTML = lista.map(c => `<tr>
510     <td>${c.id}</td>
511     <td>${c.nome || '-'}</td>
512     <td>${c.telefone || '-'}</td>
513     <td>${c.email || '-'}</td>
514     <td>${c.cpf_cnpj || '-'}</td>
515     <td>${c.cidade || '-'}</td>
516     <td><span class="badge badge-info">${c.pontos_fidelidade || 0}</span></td>
517     <td><span class="badge ${c.ativo ? 'badge-success' : 'badge-danger'}">${c.ativo ? 'Ativo' : 'Inativo'}</span></td>
518     <td>
519       <button class="btn btn-sm btn-primary" onclick="editarCliente(${c.id})"><=</button>
520       <button class="btn btn-sm btn-danger" onclick="excluirCliente(${c.id})">⌵</button>
521     </td>
522   `).join('');
523 }
524
525 function buscarClientes() { carregarClientes(); }
526
527 function abrirModalCliente() {
528   document.getElementById('modalClienteTitulo').textContent = 'Novo Cliente';
529   ['cId', 'cNome', 'cTelefone', 'cEmail', 'cCpfCnpj', 'cEndereco', 'cCidade', 'cEstado', 'cCep'].forEach(id => document.getElementById(id).value = '');
530   document.getElementById('modalCliente').classList.add('open');
531 }
532
533 async function editarCliente(id) {
534   const c = S.clientes.find(x => x.id === id);
535   if (!c) return;
536   document.getElementById('modalClienteTitulo').textContent = 'Editar Cliente #' + id;
537   for (const k of ['Id', 'Nome', 'Telefone', 'Email', 'CpfCnpj', 'Endereco', 'Cidade', 'Estado', 'Cep']) {
538     const el = document.getElementById(c[k]);
539     if (el) el.value = c[k.toLowerCase()] || '';
540   }
541   document.getElementById('modalCliente').classList.add('open');
542 }
543
544 async function salvarCliente() {
545   const btn = document.querySelector('#modalCliente .btn-success');
546   btn.disabled = true; btn.textContent = '⌵ Salvando...';
547

```

```

548 const dados = {
549   nome: document.getElementById('Nome').value.trim(),
550   telefone: document.getElementById('cTelefone').value.trim() || null,
551   email: document.getElementById('cEmail').value.trim() || null,
552   cpf_cnpj: document.getElementById('cCpfCnpj').value.trim() || null,
553   endereco: document.getElementById('cEndereco').value.trim() || null,
554   cidade: document.getElementById('cCidade').value.trim() || null,
555   estado: document.getElementById('cEstado').value.trim() || null,
556   cep: document.getElementById('cCep').value.trim() || null
557 };
558 if (!dados.nome) { toast('Nome é obrigatório!', 'error'); btn.disabled = false; btn.textContent = '✎ Salvar'; return; }
559
560 const id = document.getElementById('cId').value;
561 if (id) {
562   await api('PUT', '/api/clientes/' + id, dados);
563   toast('Cliente atualizado! 🎉');
564 } else {
565   await api('POST', '/api/clientes/', dados);
566   toast('Cliente cadastrado! 🎉');
567 }
568 fecharModal('modalCliente');
569 carregarClientes();
570 btn.disabled = false; btn.textContent = '✎ Salvar';
571 }
572
573 async function excluirCliente(id) {
574   if (!confirm('Deseja realmente excluir o cliente #' + id + '?')) return;
575   await api('DELETE', '/api/clientes/' + id);
576   toast('Cliente desativado! 🎉');
577   carregarClientes();
578 }
579
580 // =====
581 // PRODUTOS
582 // =====
583 async function carregarProdutos() {
584   S.produtos = await api('GET', '/api/produtos/') || [];
585   renderProdutos(S.produtos);
586 }
587
588 function renderProdutos(lista) {
589   const tbody = document.getElementById('tbodyProdutos');
590   if (!lista.length) { tbody.innerHTML = '<tr><td colspan="8" style="text-align:center;color:var(--text3);padding:40px;">Nenhum produto</td></tr>'; return; }
591   tbody.innerHTML = lista.map(p => `<tr>
592     <td>${p.id}</td><td>${p.nome || '-'}</td><td>${p.sku || '-'}</td>
593     <td>R$ ${parseFloat(p.preco_venda).toFixed(2)}</td>
594     <td>${p.estoque_atual || 0}</td><td>${p.unidade || 'UN'}</td>
595     <td><span class="badge ${p.ativo ? 'badge-success' : 'badge-danger'}">${p.ativo ? 'Ativo' : 'Inativo'}</span></td>
596     <td>
597       <button class="btn btn-sm btn-primary"></button>
598       <button class="btn btn-sm btn-danger"></button>
599     </td>
600   `).join('');
601 }
602
603 function buscarProdutos() { carregarProdutos(); }
604
605 function abrirModalProduto() {
606   document.getElementById('modalProdutoTitulo').textContent = 'Novo Produto';
607   ['Id', 'pNome', 'pDescricao', 'pSku', 'pPrecoVenda', 'pPrecoCusto', 'pEstoque', 'pEstoqueMin'].forEach(id => document.getElementById(id).value = '');
608   document.getElementById('pUnidade').value = 'UN';
609   document.getElementById('modalProduto').classList.add('open');
610 }
611
612 async function salvarProduto() {
613   const dados = {
614     nome: document.getElementById('pNome').value.trim(),
615     descricao: document.getElementById('pDescricao').value.trim() || null,
616     sku: document.getElementById('pSku').value.trim() || null,
617     preco_venda: parseFloat(document.getElementById('pPrecoVenda').value) || 0,
618     preco_custo: parseFloat(document.getElementById('pPrecoCusto').value) || 0,
619     estoque_atual: parseFloat(document.getElementById('pEstoque').value) || 0,
620     estoque_minimo: parseFloat(document.getElementById('pEstoqueMin').value) || 0,
621     unidade: document.getElementById('pUnidade').value
622   };
623   if (!dados.nome) { toast('Nome é obrigatório!', 'error'); return; }
624   await api('POST', '/api/produtos/', dados);
625   toast('Produto cadastrado! 🎉');
626   fecharModal('modalProduto');
627   carregarProdutos();
628 }
629
630 // =====
631 // VENDAS
632 // =====
633 async function carregarVendas() {
634   S.vendas = await api('GET', '/api/vendas/') || [];
635   const tbody = document.getElementById('tbodyVendas');
636   if (!S.vendas.length) { tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;color:var(--text3);padding:40px;">Nenhuma venda</td></tr>'; return; }
637   tbody.innerHTML = S.vendas.map(v => `<tr>
638     <td>${v.id}</td><td>${v.data_venda ? v.data_venda.substring(0,10) : '-'}</td>
639     <td>${v.cliente_nome || '-'}</td><td>R$ ${parseFloat(v.total || 0).toFixed(2)}</td>
640     <td>${v.forma_pagamento || '-'}</td>
641     <td><span class="badge ${v.status === 'CONCLUIDA' ? 'badge-success' : 'badge-danger'}">${v.status}</span></td>
642     <td><button class="btn btn-sm btn-primary"></button></td>
643   `).join('');
644 }
645
646 // =====
647 // FORNECEDORES
648 // =====
649 async function carregarFornecedores() {
650   S.fornecedores = await api('GET', '/api/fornecedores/') || [];
651   const tbody = document.getElementById('tbodyFornecedores');
652   if (!S.fornecedores.length) { tbody.innerHTML = '<tr><td colspan="7" style="text-align:center;color:var(--text3);padding:40px;">Nenhum fornecedor</td></tr>'; return; }
653   tbody.innerHTML = S.fornecedores.map(f => `<tr>
654     <td>${f.id}</td><td>${f.nome_fantasia || '-'}</td><td>${f.cnpj_cpf || '-'}</td>
655     <td>${f.telefone || '-'}</td><td>${f.email || '-'}</td>
656     <td><span class="badge ${f.ativo ? 'badge-success' : 'badge-danger'}">${f.ativo ? 'Ativo' : 'Inativo'}</span></td>
657     <td><button class="btn btn-sm btn-primary"></button><button class="btn btn-sm btn-danger"></button></td>
658   `).join('');
659 }
660
661 function abrirModalFornecedor() {
662   ['fId', 'fNome', 'fRazao', 'fCnpj', 'fTelefone', 'fEmail', 'fEndereco'].forEach(id => document.getElementById(id).value = '');
663   document.getElementById('modalFornecedor').classList.add('open');
664 }
665
666 async function salvarFornecedor() {
667   const dados = {
668     nome_fantasia: document.getElementById('fNome').value.trim(),
669     razao_social: document.getElementById('fRazao').value.trim() || null,
670     cnpj_cpf: document.getElementById('fCnpj').value.trim() || null,
671     telefone: document.getElementById('fTelefone').value.trim() || null,
672     email: document.getElementById('fEmail').value.trim() || null,
673     endereco: document.getElementById('fEndereco').value.trim() || null
674   };
675   if (!dados.nome_fantasia) { toast('Nome é obrigatório!', 'error'); return; }
676   await api('POST', '/api/fornecedores/', dados);
677   toast('Fornecedor cadastrado! 🎉');
678   fecharModal('modalFornecedor');
679   carregarFornecedores();
680 }
681
682 // =====
683 // RELATÓRIOS
684 // =====
685 async function carregarRelatorios() {

```

```

686 const [c, p, v, f, ranking] = await Promise.all([
687   api('GET', '/api/clientes/?limite=10000'),
688   api('GET', '/api/produtos/'),
689   api('GET', '/api/vendas/'),
690   api('GET', '/api/fornecedores/'),
691   api('GET', '/api/clientes/fidelidade/ranking?limite=10')
692 ]);
693 document.getElementById('rClientes').textContent = c ? c.length : 0;
694 document.getElementById('rProdutos').textContent = p ? p.length : 0;
695 document.getElementById('rVendas').textContent = v ? v.length : 0;
696 document.getElementById('rFornecedores').textContent = f ? f.length : 0;
697
698 const tbody = document.getElementById('tbodyRanking');
699 if (ranking && ranking.length) {
700   tbody.innerHTML = ranking.map((r, i) => `<tr>
701     <td>${i + 1}</td><td>${r.nome} || '-'</td>
702     <td><span class="badge badge-info">${r.pontos_fidalidade} || 0</span></td>
703     <td>-</td>
704   `).join('');
705 } else {
706   tbody.innerHTML = `<tr><td colspan="4" style="text-align:center;color:var(--text3);padding:20px;">Nenhum dado de fidelidade</td></tr>`;
707 }
708 }
709
710 // =====
711 // HELPERS
712 // =====
713 function fecharModal(id) { document.getElementById(id).classList.remove('open'); }
714
715 // Auto-login se tem token
716 if (S.token && S.usuario) entrar();
717 </script>
718 </body>
719 </html>
720 ""
721

```

```

1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>■ SaltoGestão - Paine! Administrativo</title>
7 <style>
8 :root {
9   --bg: #0b1a30; --bg2: #0f2044; --bg3: #142a55;
10  --card: #112240; --border: #1e3a5f;
11  --primary: #3b82f6; --primary-hover: #2563eb;
12  --success: #10b981; --warning: #f59e0b; --danger: #ef4444;
13  --text: #e2e8f0; --text2: #94a3b8; --text3: #64748b;
14 }
15 * { margin:0; padding:0; box-sizing:border-box; }
16 body { font-family: 'Segoe UI', system-ui, sans-serif; background: var(--bg); color: var(--text); min-height:100vh; }
17
18 /* LOGIN */
19 .login-bg { display:flex; align-items:center; justify-content:center; min-height:100vh; background: linear-gradient(135deg, #0b1a30 0%, #1a1a4e 100%); }
20 .login-box { background: var(--card); border:1px solid var(--border); border-radius:16px; padding:40px; width:420px; box-shadow: 0 20px 60px rgba(0,0,0,0.3); }
21 .login-box h1 { font-size:28px; color:var(--primary); text-align:center; margin-bottom:4px; }
22 .login-box .sub { color:var(--text3); text-align:center; margin-bottom:30px; font-size:14px; }
23 .form-group { margin-bottom:16px; }
24 .form-group label { display:block; color:var(--text2); font-size:13px; margin-bottom:6px; }
25 .form-group input, .form-group select {
26   width:100%; padding:12px 16px; border-radius:8px; border:1px solid var(--border);
27   background:var(--bg); color:var(--text); font-size:14px; outline:none; transition:.2s;
28 }
29 .form-group input:focus, .form-group select:focus { border-color:var(--primary); }
30 .btn { width:100%; padding:14px; border:none; border-radius:10px; font-size:16px; font-weight:600; cursor:pointer; transition:.2s; }
31 .btn-primary { background:var(--primary); color:#fff; }
32 .btn-primary:hover { background:var(--primary-hover); }
33 .btn-success { background:var(--success); color:#fff; }
34 .btn-danger { background:var(--danger); color:#fff; }
35 .btn-sm { padding:8px 14px; font-size:13px; width:auto; }
36 .error-msg { color:var(--danger); font-size:13px; margin-top:12px; text-align:center; }
37
38 /* LAYOUT PRINCIPAL */
39 .app { display:none; }
40 .sidebar { position:fixed; left:0; top:0; width:240px; height:100vh; background:var(--card); border-right:1px solid var(--border); padding:24px 0; z-index:100; }
41 .sidebar h2 { color:var(--primary); font-size:20px; padding:0 24px; margin-bottom:8px; }
42 .sidebar .user-info { color:var(--text3); font-size:12px; padding:0 24px; margin-bottom:30px; }
43 .sidebar nav a { display:flex; align-items:center; gap:10px; padding:12px 24px; color:var(--text2); text-decoration:none; font-size:14px; transition:.2s; border-left:3px solid
transparent; }
44 .sidebar nav a:hover, .sidebar nav a.active { background:var(--bg2); color:var(--text); border-left-color:var(--primary); }
45 .main { margin-left:240px; padding:24px; min-height:100vh; }
46
47 /* HEADER */
48 .header { display:flex; justify-content:space-between; align-items:center; margin-bottom:24px; }
49 .header h1 { font-size:24px; }
50 .stats { display:flex; gap:16px; flex-wrap:wrap; }
51 .stat-card { background:var(--card); border:1px solid var(--border); border-radius:12px; padding:16px 24px; text-align:center; }
52 .stat-card .num { font-size:28px; font-weight:700; color:var(--primary); }
53 .stat-card .label { font-size:12px; color:var(--text3); margin-top:4px; }
54
55 /* TABLE */
56 .table-wrap { background:var(--card); border:1px solid var(--border); border-radius:12px; overflow:hidden; }
57 .table-wrap .table-header { display:flex; justify-content:space-between; align-items:center; padding:16px 20px; border-bottom:1px solid var(--border); flex-wrap:wrap; gap:10px; }
58 .table-wrap .table-header h3 { font-size:16px; }
59 .search-box { display:flex; gap:8px; flex-wrap:wrap; }
60 .search-box input { padding:8px 14px; border-radius:8px; border:1px solid var(--border); background:var(--bg); color:var(--text); font-size:13px; width:220px; outline:none; }
61 table { width:100%; border-collapse:collapse; }
62 th { text-align:left; padding:12px 16px; font-size:12px; color:var(--text3); text-transform:uppercase; border-bottom:1px solid var(--border); background:var(--bg2); }
63 td { padding:12px 16px; font-size:13px; border-bottom:1px solid rgba(30,58,95,0.5); }
64 tr:hover { background:var(--bg2); }
65 .badge { display:inline-block; padding:3px 10px; border-radius:20px; font-size:11px; font-weight:600; }
66 .badge-success { background:rgba(16,185,129,0.15); color:var(--success); }
67 .badge-danger { background:rgba(239,68,68,0.15); color:var(--danger); }
68 .badge-info { background:rgba(59,130,246,0.15); color:var(--primary); }
69 .actions { display:flex; gap:6px; }
70
71 /* MODAL */
72 .modal-overlay { display:none; position:fixed; inset:0; background:rgba(0,0,0,0.6); z-index:200; align-items:center; justify-content:center; }
73 .modal-overlay.open { display:flex; }
74 .modal { background:var(--card); border:1px solid var(--border); border-radius:16px; padding:32px; width:520px; max-height:80vh; overflow-y:auto; }
75 .modal h3 { margin-bottom:20px; font-size:18px; }
76 .modal .actions { display:flex; justify-content:flex-end; gap:8px; margin-top:20px; }
77
78 /* PAGINATION */
79 .pagination { display:flex; justify-content:center; gap:6px; margin-top:16px; flex-wrap:wrap; }
80 .pagination button { padding:8px 14px; border:1px solid var(--border); background:var(--bg); color:var(--text); border-radius:8px; cursor:pointer; font-size:13px; }
81 .pagination button.active { background:var(--primary); border-color:var(--primary); }
82 .pagination button:hover:not(.active) { background:var(--bg2); }
83
84 /* TOAST */
85 .toast { position:fixed; top:20px; right:20px; padding:14px 24px; border-radius:10px; font-size:14px; z-index:300; animation: slideIn .3s ease; }
86 .toast-success { background:var(--success); color:#fff; }
87 .toast-error { background:var(--danger); color:#fff; }
88 @keyframes slideIn { from { transform:translateX(100%); opacity:0; } to { transform:translateX(0); opacity:1; } }
89 </style>
90 </head>
91 <body>
92
93 <!-- LOGIN -->
94 <div id="loginPage" class="login-bg">
95   <div class="login-box">
96     <h1>■ SaltoGestão</h1>
97     <div class="sub">Sistema de Gestão Comercial - Paine! Web</div>
98
99     <div class="form-group">
100       <label>Servidor da API</label>
101       <input type="text" id="apiUrl" value="http://191.252.210.235:8080" placeholder="http://localhost:8080">
102     </div>
103
104     <div class="form-group">
105       <label>Usuário (username ou email)</label>
106       <input type="text" id="username" placeholder="admin_empresa" value="admin_empresa">
107     </div>
108
109     <div class="form-group">
110       <label>Senha:</label>
111       <input type="password" id="senha" placeholder="*****" value="empresa123">
112     </div>
113
114     <button class="btn btn-primary" onclick="fazerLogin()">■ Entrar</button>
115
116     <div id="loginError" class="error-msg"></div>
117   </div>
118 </div>
119
120 <!-- APP -->
121 <div id="app" class="app">
122   <div class="sidebar">
123     <h2>■ SaltoGestão</h2>
124     <div class="user-info" id="userInfo">Carregando...</div>
125     <nav>
126       <a href="#" class="active" onclick="showPage('dashboard', event)">■ Dashboard</a>
127       <a href="#" onclick="showPage('clientes', event)">■ Clientes</a>
128       <a href="#" onclick="showPage('produtos', event)">■ Produtos</a>
129       <a href="#" onclick="showPage('vendas', event)">■ Vendas</a>
130       <a href="#" onclick="showPage('fornecedores', event)">■ Fornecedores</a>
131       <a href="#" onclick="showPage('relatorios', event)">■ Relatórios</a>
132       <a href="#" onclick="showPage('testar100', event)">■ Teste 100 Clientes</a>
133       <a href="#" onclick="sair()" style="margin-top:30px; color:var(--danger);">■ Sair</a>

```



```
134 </nav>
135 </div>
136
137 <div class="main">
138
139 <!-- DASHBOARD -->
140 <div id="page-dashboard" class="page">
141 <div class="header"><h1>Dashboard</h1></div>
142 <div class="stats">
143 <div class="stat-card"><div class="num" id="stClientes">0</div><div class="label">Clientes</div></div>
144 <div class="stat-card"><div class="num" id="stProdutos">0</div><div class="label">Produtos</div></div>
145 <div class="stat-card"><div class="num" id="stVendas">0</div><div class="label">Vendas</div></div>
146 <div class="stat-card"><div class="num" id="stFornecedores">0</div><div class="label">Fornecedores</div></div>
147 </div>
148 <div class="table-wrap" style="margin-top:24px;">
149 <div class="table-header"><h3>Últimos clientes cadastrados</h3></div>
150 <table><thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>CPF/CNPJ</th><th>Status</th><th></tr></thead>
151 <tbody id="dashboardClientes"></tbody></table>
152 </div>
153 </div>
154
155 <!-- CLIENTES -->
156 <div id="page-clientes" class="page" style="display:none;">
157 <div class="header"><h1>Clientes</h1></div>
158 <div class="table-wrap">
159 <div class="table-header">
160 <div class="search-box">
161 <input type="text" id="searchClientes" placeholder="🔍 Buscar..." oninput="buscarClientes()">
162 <select id="filterClientes" onchange="buscarClientes()">
163 <option value="true">Ativos</option>
164 <option value="false">Inativos</option>
165 <option value="">Todos</option>
166 </select>
167 </div>
168 <button class="btn btn-success btn-sm" onclick="abrirModalCliente()">+ Novo Cliente</button>
169 </div>
170 <table><thead><tr><th>ID</th><th>Nome</th><th>Telefone</th><th>E-mail</th><th>CPF/CNPJ</th><th>Cidade</th><th>Pontos</th><th>Status</th><th>Ações</th></tr></thead>
171 <tbody id="tbodyClientes"></tbody></table>
172 </div>
173 </div>
174
175 <!-- PRODUTOS -->
176 <div id="page-produtos" class="page" style="display:none;">
177 <div class="header"><h1>Produtos</h1></div>
178 <div class="table-wrap">
179 <div class="table-header">
180 <div class="search-box"><input type="text" placeholder="🔍 Buscar..."></div>
181 <button class="btn btn-success btn-sm">+ Novo Produto</button>
182 </div>
183 <table><thead><tr><th>ID</th><th>Nome</th><th>SKU</th><th>Preço</th><th>Estoque</th><th>Status</th></tr></thead>
184 <tbody id="tbodyProdutos"></tbody></table>
185 </div>
186 </div>
187
188 <!-- VENDAS -->
189 <div id="page-vendas" class="page" style="display:none;">
190 <div class="header"><h1>Vendas</h1></div>
191 <div class="table-wrap">
192 <div class="table-header"><div class="search-box"><input type="text" placeholder="🔍 Buscar..."></div><button class="btn btn-success btn-sm">+ Nova Venda</button></div>
193 <table><thead><tr><th>ID</th><th>Data</th><th>Cliente</th><th>Total</th><th>Status</th></tr></thead>
194 <tbody id="tbodyVendas"></tbody></table>
195 </div>
196 </div>
197
198 <!-- FORNECEDORES -->
199 <div id="page-fornecedores" class="page" style="display:none;">
200 <div class="header"><h1>Fornecedores</h1></div>
201 <div class="table-wrap">
202 <div class="table-header"><div class="search-box"><input type="text" placeholder="🔍 Buscar..."></div><button class="btn btn-success btn-sm">+ Novo Fornecedor</button></div>
203 <table><thead><tr><th>ID</th><th>Nome</th><th>CPF/CNPJ</th><th>Telefone</th><th>Status</th></tr></thead>
204 <tbody id="tbodyFornecedores"></tbody></table>
205 </div>
206 </div>
207
208 <!-- RELATÓRIOS -->
209 <div id="page-relatorios" class="page" style="display:none;">
210 <div class="header"><h1>Relatórios</h1></div>
211 <div class="stats">
212 <div class="stat-card"><div class="num" id="rClientes">0</div><div class="label">Total Clientes</div></div>
213 <div class="stat-card"><div class="num" id="rProdutos">0</div><div class="label">Total Produtos</div></div>
214 <div class="stat-card"><div class="num" id="rVendas">0</div><div class="label">Total Vendas</div></div>
215 <div class="stat-card"><div class="num" id="rFornecedores">0</div><div class="label">Total Fornecedores</div></div>
216 </div>
217 </div>
218
219 <!-- TESTE 100 CLIENTES -->
220 <div id="page-testar100" class="page" style="display:none;">
221 <div class="header"><h1>Teste de 100 Clientes</h1></div>
222 <div class="table-wrap">
223 <div class="table-header">
224 <div>
225 <p style="color:var(--text3);font-size:13px;">Insere 100 clientes de teste via API para validar o sistema.</p>
226 </div>
227 <button class="btn btn-success btn-sm" onclick="executarTeste100()">▶ Executar Testes</button>
228 </div>
229 <div style="padding:24px;">
230 <div id="testeResultado" style="font-family:monospace;font-size:13px;color:var(--text2);">Clique em "Executar Teste" para iniciar...</div>
231 </div>
232 </div>
233 </div>
234
235 </div>
236 </div>
237
238 <!-- MODAL CLIENTE -->
239 <div id="modalCliente" class="modal-overlay" onclick="if(event.target===this)fecharModal('modalCliente')">
240 <div class="modal">
241 <h3 id="modalClienteTitulo">Novo Cliente</h3>
242 <input type="hidden" id="cId">
243 <div class="form-group"><label>Nome:</label><input type="text" id="cNome"></div>
244 <div class="form-group"><label>Telefone:</label><input type="text" id="cTelefone"></div>
245 <div class="form-group"><label>E-mail:</label><input type="email" id="cEmail"></div>
246 <div class="form-group"><label>CPF/CNPJ:</label><input type="text" id="cCpfCnpj"></div>
247 <div class="form-group"><label>Endereço:</label><input type="text" id="cEndereco"></div>
248 <div class="form-group"><label>Cidade:</label><input type="text" id="cCidade"></div>
249 <div class="form-group"><label>Estado:</label><input type="text" id="cEstado" maxlength="2"></div>
250 <div class="form-group"><label>CEP:</label><input type="text" id="cCep"></div>
251 <div class="actions">
252 <button class="btn btn-sm" style="background:var(--bg2);color:var(--text)" onclick="fecharModal('modalCliente')">Cancelar</button>
253 <button class="btn btn-success btn-sm" onclick="salvarCliente()">▶ Salvar</button>
254 </div>
255 </div>
256 </div>
257
258 <script>
259 const S = {
260 token: localStorage.getItem('token') || null,
261 usuario: JSON.parse(localStorage.getItem('usuario')) || 'null',
262 apiUrl: localStorage.getItem('apiUrl') || 'http://191.252.210.235:8080',
263 clientes: [],
264 produtos: [],
265 vendas: [],
266 fornecedores: []
267 };
268
269 async function api(method, path, body) {
270 const opts = { method, headers: {'Content-Type':'application/json'} };
271 if (S.token) opts.headers['Authorization'] = 'Bearer ' + S.token;
```



```

272 if (body) opts.body = JSON.stringify(body);
273 const r = await fetch(S.apiUrl + path, opts);
274 if (r.status === 401) { sair(); return null; }
275 const t = await r.text();
276 return t ? JSON.parse(t) : null;
277 }
278
279 function toast(msg, type) {
280   const d = document.createElement('div');
281   d.className = 'toast toast-' + (type || 'success');
282   d.textContent = msg;
283   document.body.appendChild(d);
284   setTimeout(() => d.remove(), 3000);
285 }
286
287 function fazerLogin() {
288   S.apiUrl = document.getElementById('apiUrl').value.trim();
289   localStorage.setItem('apiUrl', S.apiUrl);
290   const username = document.getElementById('username').value.trim();
291   const senha = document.getElementById('senha').value.trim();
292   const body = username.includes('@') ? {email:username, senha: {username, senha};
293   fetch(S.apiUrl + '/api/login', {method:'POST', headers:{'Content-Type':'application/json'}, body:JSON.stringify(body)})
294   .then(r=>r.json()).then(d=>{
295     if (d.access_token) { S.token=d.access_token; S.usuario=d.usuario||d.admin; localStorage.setItem('token',S.token); localStorage.setItem('usuario',JSON.stringify(S.usuario));
entrar(); }
296     else document.getElementById('loginError').textContent = 'x ' + (d.detail||'Falha no login');
297   }).catch(e=>{ document.getElementById('loginError').textContent = 'x Erro: ' + e.message; });
298 }
299
300 function entrar() {
301   document.getElementById('loginPage').style.display='none';
302   document.getElementById('app').style.display='block';
303   document.getElementById('userInfo').textContent = S.usuario.nome + ' (' + S.usuario.papel + ')';
304   carregarDashboard();
305 }
306
307 function sair() {
308   S.token=S.usuario=null; localStorage.removeItem('token'); localStorage.removeItem('usuario');
309   document.getElementById('app').style.display='none';
310   document.getElementById('loginPage').style.display='flex';
311 }
312
313 function showPage(page, e) {
314   if (e) e.preventDefault();
315   document.querySelectorAll('.page').forEach(p=>p.style.display='none');
316   document.querySelectorAll('.sidebar nav a').forEach(a=>a.classList.remove('active'));
317   document.getElementById('page-' + page).style.display='block';
318   if (e && e.target) e.target.classList.add('active');
319   if (page==='dashboard') carregarDashboard();
320   if (page==='clientes') carregarClientes();
321   if (page==='produtos') carregarProdutos();
322   if (page==='vendas') carregarVendas();
323   if (page==='fornecedores') carregarFornecedores();
324   if (page==='relatorios') carregarRelatorios();
325 }
326
327 async function carregarDashboard() {
328   const [c,p,v,f] = await Promise.all([
329     api('GET','/api/clientes/?ativo=true&ordem=recente&limite=5'),
330     api('GET','/api/produtos/'),
331     api('GET','/api/vendas/'),
332     api('GET','/api/fornecedores/')
333   ]);
334   if(c) document.getElementById('stClientes').textContent=c.length;
335   document.getElementById('dashboardClientes').innerHTML = c.slice(0,5).map(x=><tr><td>${x.id}</td><td>${x.nome||'-'}</td><td>${x.telefone||'-'}</td><td>${x.cpf_cnpj||'-'}</td><td>
<span class="badge ${x.ativo?'badge-success':'badge-danger'}">${x.ativo?'Ativo':'Inativo'}</span></td></tr>`).join('');
336 }
337 if(p) document.getElementById('stProdutos').textContent=p.length;
338 if(v) document.getElementById('stVendas').textContent=v.length;
339 if(f) document.getElementById('stFornecedores').textContent=f.length;
340 }
341
342 async function carregarClientes() {
343   const busca = document.getElementById('searchClientes').value;
344   const ativo = document.getElementById('filterClientes').value;
345   let url = '/api/clientes/?';
346   if(ativo) url+=`ativo=${ativo}&`;
347   if(busca) url+=`busca=${encodeURIComponent(busca)}&`;
348   url+=`ordem=nome&limite=100`;
349   S.clientes = await api('GET',url) || [];
350   renderClientes(S.clientes);
351 }
352
353 function renderClientes(list) {
354   const t = document.getElementById('tbodyClientes');
355   if(!list.length){ t.innerHTML=<tr><td colspan="9" style="text-align:center;color:var(--text3);padding:40px;">Nenhum cliente encontrado</td></tr>; return; }
356   t.innerHTML = list.map(c=><tr><td>${c.id}</td><td>${c.nome||'-'}</td><td>${c.telefone||'-'}</td><td>${c.email||'-'}</td><td>${c.cpf_cnpj||'-'}</td><td>${c.cidade||'-'}</td><td>
<span class="badge badge-info">${c.pontos_fidelidade}</td><td><span class="badge ${c.ativo?'badge-success':'badge-danger'}">${c.ativo?'Ativo':'Inativo'}</span></td><td><div
class="actions"><button class="btn btn-sm btn-primary" onclick="editarCliente(${c.id})">✎</button><button class="btn btn-sm btn-danger" onclick="excluirCliente(${c.id})">✖</button></div>
</td></tr>`).join('');
357 }
358
359 function buscarClientes(){ carregarClientes(); }
360
361 function abrirModalCliente() {
362   document.getElementById('modalClienteTitulo').textContent='Novo Cliente';
363   ['Id','Nome','Telefone','Email','CpfCnpj','Endereco','Cidade','Estado','Cep'].forEach(id=>document.getElementById(id).value='');
364   document.getElementById('modalCliente').classList.add('open');
365 }
366
367 async function editarCliente(id) {
368   const c=S.clientes.find(x=>x.id===id); if(!c) return;
369   document.getElementById('modalClienteTitulo').textContent='Editar #' +id;
370   for(const k of ['Id','Nome','Telefone','Email','CpfCnpj','Endereco','Cidade','Estado','Cep']){ const el=document.getElementById('c'+k); if(el) el.value=c[k.toLowerCase()]||''; }
371   document.getElementById('modalCliente').classList.add('open');
372 }
373
374 async function salvarCliente() {
375   const dados = {
376     nome:document.getElementById('cNome').value.trim(),
377     telefone:document.getElementById('cTelefone').value.trim()||null,
378     email:document.getElementById('cEmail').value.trim()||null,
379     cpf_cnpj:document.getElementById('cCpfCnpj').value.trim()||null,
380     endereco:document.getElementById('cEndereco').value.trim()||null,
381     cidade:document.getElementById('cCidade').value.trim()||null,
382     estado:document.getElementById('cEstado').value.trim()||null,
383     cep:document.getElementById('cCep').value.trim()||null
384   };
385   if(!dados.nome){toast('Nome obrigatório!','error');return;}
386   const id=document.getElementById('cId').value;
387   if(id){ await api('PUT','/api/clientes/'+id,dados); toast('Atualizado! 🎉'); }
388   else{ await api('POST','/api/clientes/',dados); toast('Cadastrado! 🎉'); }
389   fecharModal('modalCliente');
390   carregarClientes();
391 }
392
393 async function excluirCliente(id){ if(!confirm('Excluir cliente #' +id+'?')) return; await api('DELETE','/api/clientes/'+id); toast('Desativado! 🗑️'); carregarClientes(); }
394
395 async function carregarProdutos(){ S.produtos=await api('GET','/api/produtos/'); const t=document.getElementById('tbodyProdutos'); if(!S.produtos.length){t.innerHTML=<tr><td colspan="7" style="text-align:center;color:var(--text3);padding:40px;">Nenhum produto</td></tr>;return;} t.innerHTML=S.produtos.map(p=><tr><td>${p.id}</td><td>${p.nome||'-'}</td>
<td>${p.sku||'-'}</td><td>R$${parseFloat(p.preco_venda).toFixed(2)}</td><td>${p.estoque_atual}</td><td><span class="badge ${p.ativo?'badge-success':'badge-danger'}">${p.ativo?'Ativo':'Inativo'}</span></td></tr>`).join(''); }
396
397 async function carregarVendas(){ S.vendas=await api('GET','/api/vendas/'); const t=document.getElementById('tbodyVendas'); if(!S.vendas.length){t.innerHTML=<tr><td colspan="6" style="text-align:center;color:var(--text3);padding:40px;">Nenhuma venda</td></tr>;return;} t.innerHTML=S.vendas.map(v=><tr><td>${v.id}</td><td>${v.data_venda?
v.data_venda.substring(0,10):'-'}</td><td>${v.cliente_nome||'-'}</td><td>R$${parseFloat(v.total)</td><td><span class="badge ${v.status==='CONCLUIDA'? 'badge-success':'badge-danger'}">${v.status}</span></td></tr>`).join(''); }
398

```

```

399 async function carregarFornecedores(){ S.fornecedores=await api('GET','/api/fornecedores/')||[]; const t=document.getElementById('tbodyFornecedores'); if(!S.fornecedores.length)
{t.innerHTML=<tr><td colspan="6" style="text-align:center;color:var(--text3);padding:40px;">Nenhum fornecedor</td></tr>; return; } t.innerHTML=S.fornecedores.map(f=><tr><td>${f.id}</td>
<td>${f.nome.fantasia}|-'|</td><td>${f.cnpj_cpf}|-'|</td><td>${f.telefone}|-'|</td><td><span class="badge ${f.ativo?'badge-success':'badge-danger'}">${f.ativo?'Ativo':'Inativo'}</span>
</td></tr>').join(''); }
400
401 async function carregarRelatorios(){
402   const [c,p,v,f]=await Promise.all([api('GET','/api/clientes/?limite=10000'),api('GET','/api/produtos/'),api('GET','/api/vendas/'),api('GET','/api/fornecedores/')]);
403   if(c)document.getElementById('rClientes').textContent=c.length;
404   if(p)document.getElementById('rProdutos').textContent=p.length;
405   if(v)document.getElementById('rVendas').textContent=v.length;
406   if(f)document.getElementById('rFornecedores').textContent=f.length;
407 }
408
409 // =====
410 // TESTE 100 CLIENTES
411 // =====
412 async function executarTeste100() {
413   const out = document.getElementById('testeResultado');
414   out.innerHTML = '🚧 Executando teste...<br>';
415
416   const nomes =
['João','Maria','José','Ana','Pedro','Carla','Lucas','Fernanda','Marcos','Juliana','Rafael','Patricia','Bruno','Camila','Diego','Amanda','Gustavo','Larissa','Thiago','Beatriz','Felipe','Maria
na','Rodrigo','Letícia','André','Vanessa','Marcelo','Tatiana','Leonardo','Priscila','Eduardo','Sabrina','Ricardo','Isabela','Fábio','Natália','Alexandre','Daniela','Guilherme','Bianca','Viníc
ius','Lorena','Matheus','Gabriela','Samuel','Helena','Isaac','Valentina','Daniel','Laura','Henrique','Manuela','Caio','Sophia','Enzo','Heloísa','Miguel','Alice','Arthur','Júlia','Bernardo','L
uisa','Heitor','Isabella','Davi','Lara','Lorenzo','Cecília','Theo','Eloá','Gabriel','Yasmin','Noah','Rafaela','Benjamin','Giovanna','Anthony','Clara','Joaquim','Melissa','Nicolas','Rebeca','L
ucca','Agatha','Ryan','Esther','Ian','Nicole','David','Sarah','Pedro H.','Vitória','Bruno H.','Luna','Cauã','Marina','Kaïque','Valentin','Yuri','Antonella'];
417   const sobren =
['Silva','Santos','Oliveira','Souza','Rodrigues','Ferreira','Alves','Pereira','Lima','Gomes','Costa','Ribeiro','Martins','Carvalho','Araújo','Melo','Barbosa','Rocha','Dias','Nascimento'];
418   const bairros = ['Centro','Jardim América','Setor Oeste','Setor Bueno','Jardim Goiás','Campinas','Vila Aurora','Jardim Europa'];
419   const cidades = ['Aparecida de Goiânia','Goiânia','Anápolis'];
420   const usados = new Set();
421
422   let ok=0, fail=0;
423   for(let i=0;i<100;i++){
424     const nome = nomes[i%nomes.length]+' '+sobren[i%sobren.length]+' '+sobren[(i*3+7)%sobren.length];
425     let cpf; do{ cpf = String(Math.floor(1e10*Math.random()*9e10)); }while(usados.has(cpf.substring(0,9)));
426     usados.add(cpf.substring(0,9));
427     const dados = {
428       nome: nome.trim(),
429       telefone: '629'+String(Math.floor(1e7+Math.random()*9e7)),
430       email: nome.toLowerCase().replace(/ /g,'')+(i+1)+'@email.com',
431       cpf_cnpj: cpf,
432       endereco: 'Rua '+ (i+1) +', '+ (i*10),
433       cidade: cidades[i%cidades.length],
434       estado: 'GO',
435       cep: String(Math.floor(1e8+Math.random()*9e8))
436     };
437     try{
438       const r = await api('POST','/api/clientes/',dados);
439       if(r && r.id){ ok++; }else{ fail++; }
440     } catch(e){ fail++; }
441     if((i+1)%20===0) out.innerHTML += `📊 ${i+1}/100... (-${ok} x${fail})<br>`;
442   }
443   out.innerHTML += `<br>🏁 <b>Resultado: ${ok}/100 OK, ${fail} falhas</b>`;
444   if(ok===100){ out.innerHTML += `<br>🎉 <b style="color:#10b981">PASSOU! Sistema operacional!</b>`; }
445   toast(ok+' /100 clientes inseridos!', ok===100?'success':'error');
446   // Recarregar dashboard
447   carregarDashboard();
448 }
449
450 function fecharModal(id){ document.getElementById(id).classList.remove('open'); }
451 if(S.token && S.usuario) entrar();
452 </script>
453 </body>
454 </html>

```